

**IMPLEMENTASI *GRAPH*
RETRIEVAL-AUGMENTED GENERATION UNTUK
MENINGKATKAN PRESISI *RETRIEVAL* DAN
VALIDITAS SINTAKSIS PADA KONFIGURASI
KUBERNETES**

Laporan Tugas Akhir

Oleh

**Jihan Aurelia
18222001**



**PROGRAM STUDI SISTEM DAN TEKNOLOGI INFORMASI
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG**

Juni 2026

LEMBAR PENGESAHAN

IMPLEMENTASI *GRAPH RETRIEVAL AUGMENTED GENERATION* UNTUK MENINGKATKAN PRESISI RETRIEVAL DAN VALIDITAS SINTAKSIS PADA KONFIGURASI KUBERNETES

Laporan Tugas Akhir

Oleh

**Jihan Aurelia
18222001**

Program Studi Sistem dan Teknologi Informasi
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

Laporan Tugas Akhir ini telah disetujui dan disahkan
di Bandung, pada tanggal 6 Juni 2026

Pembimbing

Dr. Ir. Dimitri Mahayana, M.Eng.

NIP. 196808091991021001

PERNYATAAN ORISINALITAS

Saya yang bertanda tangan di bawah ini menyatakan dengan sesungguhnya bahwa:

1. Tugas Akhir ini adalah hasil karya saya sendiri yang dibuat dengan sebenar-benarnya sesuai dengan kaidah ilmiah dan etika akademik.
2. Semua sumber rujukan dan data yang saya gunakan dalam penyusunan laporan tugas akhir ini, baik yang berupa kutipan langsung maupun tidak langsung, telah saya cantumkan sumbernya dengan baik dan benar sesuai dengan ketentuan penulisan laporan tugas akhir.
3. Isi laporan ini belum pernah diajukan pada program pendidikan di institusi mana pun.

Apabila di kemudian hari terbukti bahwa laporan ini melanggar hal-hal di atas, saya bersedia menerima sanksi sesuai dengan peraturan yang berlaku di Institut Teknologi Bandung.

Bandung, 6 Juni 2026



Jihan Aurelia

NIM: 18222001

PERNYATAAN PENGGUNAAN AI

Saya yang bertanda tangan di bawah ini menyatakan bahwa dalam penulisan dokumen Tugas Akhir ini, saya menggunakan alat bantu kecerdasan artifisial (AI) untuk beberapa aspek tertentu di dalam proses penulisan, seperti yang tercantum dalam tabel berikut.

No.	Jenis	Alat Bantu	Tingkat	Tempat
1	Memeriksa ejaan dan tata bahasa	Claude (Anthropic)	Sedang	Semua bab
2	Pembuatan teks	Claude (Anthropic)	Sedang	Bab II, IV–V
3	Pembuatan grafik, gambar, atau ilustrasi	Tidak ada	Tidak ada	Tidak ada
4	Pencarian informasi atau referensi	Gemini	Rendah	Bab II
5	Penyusunan bahan diskusi	Tidak ada	Tidak ada	Tidak ada
6	Penyusunan kode program	Claude (Anthropic)	Sedang	Bab IV–V, Lampiran A
7	Penyusunan formula atau perhitungan matematis	Tidak ada	Tidak ada	Tidak ada
8	Penyusunan konsep desain atau algoritma	Tidak ada	Tidak ada	Tidak ada
9	Penyusunan analisis data	Tidak ada	Tidak ada	Tidak ada
10	Penyusunan kesimpulan atau rekomendasi	Tidak ada	Tidak ada	Tidak ada

Bandung, 6 Juni 2026



Jihan Aurelia

NIM: 18222001

ABSTRAK

Implementasi *Graph Retrieval-Augmented Generation* untuk Meningkatkan Presisi *Retrieval* dan Validitas Sintaksis pada Konfigurasi Kubernetes

oleh

Jihan Aurelia

18222001

Kubernetes menjadi standar orkestrasi kontainer *cloud-native*, namun kompleksitas konfigurasi YAML mendorong penggunaan LLM yang rentan menghasilkan halusinasi, yaitu konfigurasi yang keliru secara semantik atau tidak valid secara sintaksis. Pendekatan *Retrieval-Augmented Generation* (RAG) berbasis vektor terbukti tidak memadai karena gagal menangkap relasi hierarkis domain Kubernetes, dengan akurasi hanya mencapai 63,4% *exact match* dan 69,8% *context precision*.

Penelitian ini membangun sistem *Graph Retrieval-Augmented Generation* (GraphRAG) dari spesifikasi OpenAPI Kubernetes v1.30, menghasilkan *knowledge graph* dengan 725 *node*, 18 jenis *edge*, dan 7 kategori relasi. Mekanisme *retrieval* menggabungkan pencocokan nama eksak, *fallback* kemiripan vektor, dan *multi-hop graph traversal* dengan kedalaman adaptif berbasis *intent* ($d = 2$ atau $d = 3$). Validasi YAML diimplementasikan dalam tiga lapis berbasis *knowledge graph*. Pipeline dibangun menggunakan LangGraph dengan GPT-4o-mini sebagai *Thinker* dan *Speaker*, Neo4j, dan SQLite.

Penelitian ini membandingkan GraphRAG terhadap Vector RAG dan Vanilla LLM menggunakan 97 *fixture* uji tervalidasi pakar. AnsQ setara antar-sistem ($p > 0,05$). Keunggulan GraphRAG teridentifikasi pada dimensi *retrieval* dan penalaran: RetQ 0,7259 vs. 0,4255 (Vector RAG) dan 0,0241 (Vanilla LLM); ReaQ 0,5554 vs. 0,3307 dan 0,3351; *Path Coverage* 0,8515 ($\Delta = +0,310$ vs. Vector RAG); RGA 0,4536 vs. 0,2784 dan 0,0206. Seluruh keunggulan signifikan ($p < 0,001$) dikonfirmasi melalui uji Wilcoxon + *bootstrap* 95% CI dengan koreksi Holm-Bonferroni.

Kata kunci: *Graph Retrieval-Augmented Generation*, *knowledge graph*, Kubernetes, YAML, halusinasi LLM, *intent-adaptive depth traversal*, *Path Coverage*, *Hop Accuracy*.

KATA PENGANTAR

Puji syukur saya panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya, saya dapat menyelesaikan penulisan Laporan Tugas Akhir yang berjudul “Implementasi *Graph Retrieval-Augmented Generation* untuk Meningkatkan Presisi *Retrieval* dan Validitas Sintaksis pada Konfigurasi Kubernetes”. Laporan ini disusun sebagai salah satu syarat untuk menyelesaikan program Sarjana di Sekolah Teknik Elektro dan Informatika, Institut Teknologi Bandung.

Selama proses penyusunan Tugas Akhir ini, saya banyak mendapatkan bimbingan, bantuan, dan dukungan dari berbagai pihak. Oleh karena itu, saya menyampaikan ucapan terima kasih yang sebesar-besarnya kepada:

1. Bapak Dr. Ir. Dimitri Mahayana, M.Eng., selaku dosen pembimbing yang telah memberikan arahan, masukan, dan dukungan yang sangat berharga sepanjang pelaksanaan penelitian ini.
2. Seluruh dosen penguji yang telah memberikan kritik dan saran konstruktif demi penyempurnaan penelitian ini.
3. Kedua orang tua dan keluarga yang senantiasa memberikan doa, semangat, dan dukungan tanpa henti selama saya menempuh pendidikan di ITB.
4. Para narasumber praktisi DevOps dan *Site Reliability Engineering* yang bersedia meluangkan waktunya untuk wawancara dan validasi dalam penelitian ini.
5. Rekan-rekan mahasiswa Program Studi Sistem dan Teknologi Informasi angkatan 2022 atas kebersamaan, diskusi, dan dukungan selama masa studi.

Saya menyadari bahwa laporan ini masih jauh dari sempurna. Oleh karena itu, saran dan kritik yang membangun sangat saya harapkan demi perbaikan di masa mendatang. Semoga penelitian ini dapat memberikan manfaat bagi pengembangan ilmu pengetahuan dan praktik rekayasa perangkat lunak berbasis *cloud-native*.

Bandung, 6 Juni 2026

Penulis



Jihan Aurelia

DAFTAR ISI

LEMBAR PENGESAHAN	iii
PERNYATAAN ORISINALITAS	v
PERNYATAAN PENGGUNAAN AI	vii
ABSTRAK	ix
KATA PENGANTAR	xi
DAFTAR ISI	xiii
DAFTAR GAMBAR	xvii
DAFTAR TABEL	xix
DAFTAR PERSAMAAN	xxi
DAFTAR <i>LISTING</i>	xxiv
DAFTAR SIMBOL	xxv
DAFTAR SINGKATAN	xxvii
I PENDAHULUAN	1
I.1 Latar Belakang	1
I.2 Rumusan Masalah	4
I.3 Tujuan	4
I.4 Batasan Masalah	4
I.5 Metodologi	5
I.6 Sistematika Penulisan	7
II STUDI LITERATUR	9
II.1 Arsitektur <i>Cloud</i> Modern dan Tantangan Kompleksitas	9
II.2 Kubernetes: Platform Orkestrasi Kontainer untuk Aplikasi <i>Cloud-Native</i>	9
II.2.1 Gambaran Umum Arsitektur	10
II.2.2 <i>Resource Model</i> dan Hierarki	11
II.2.3 Manifes YAML dan <i>Infrastructure as Code</i>	12
II.3 <i>Large Language Models</i> (LLM)	12
II.3.1 Definisi dan Kapabilitas	12
II.3.2 <i>Prompt Engineering</i>	13
II.4 <i>Retrieval-Augmented Generation</i> (RAG)	13
II.4.1 Arsitektur <i>Retrieval-Augmented Generation</i>	14
II.4.2 Relevansi RAG terhadap Dokumentasi Kubernetes API	15
II.5 <i>Knowledge Graph</i> dan GraphRAG	16
II.5.1 Definisi dan Struktur	16
II.5.2 GraphRAG: Integrasi <i>Knowledge Graph</i> dan <i>Large Language Models</i>	18
II.6 Metrik Evaluasi Sistem Informasi Retrieval	19
II.6.1 <i>Answer Quality</i>	19
II.6.2 <i>Retrieval Quality</i>	21
II.6.3 <i>Reasoning Quality</i>	21
II.6.4 Metrik Evaluasi Berbasis <i>Graph Traversal</i>	22
II.7 Penelitian Terkait	23

II.7.1	GraphRAG untuk Platform IoT Domain-Spesifik	23
II.7.2	RAG Hibrida KG-Vektor untuk Manufaktur Pintar	24
II.7.3	HSG-RAG: Konstruksi Basis Pengetahuan Hierarkis untuk Domain Teknis	24
III	ANALISIS MASALAH	27
III.1	Analisis Kondisi Saat Ini	27
III.1.1	Pemahaman Masalah Bisnis (<i>Business Understanding</i>) . . .	27
III.1.2	Pemahaman Data (<i>Data Understanding</i>)	29
III.1.2.1	Sumber dan Spesifikasi Data (<i>Data Source & Spe-</i> <i>cifications</i>)	29
III.1.2.2	Analisis Struktur Dokumen	29
III.1.2.3	Analisis Data Eksploratif Spesifikasi OpenAPI . . .	30
III.1.2.4	Analisis Keterhubungan Data	31
III.2	Analisis Kebutuhan	33
III.2.1	Kebutuhan Fungsional	35
III.2.2	Kebutuhan Non Fungsional	38
III.2.3	Pemetaan Kebutuhan Bisnis terhadap Kerangka Evaluasi . .	39
III.3	Alternatif Solusi	40
III.3.1	Hybrid KG–Vector RAG	40
III.3.2	<i>GNN-augmented GraphRAG</i> (G-Retriever)	41
III.3.3	<i>Hybrid Neural-Symbolic</i> dengan <i>Relational DB</i> (NeuSym- RAG)	41
III.4	Analisis Pemilihan Solusi	42
III.4.1	Rubrik Evaluasi Solusi	42
III.4.2	Matriks Keputusan & Justifikasi Penilaian	42
III.4.3	Analisis Pemilihan dan Penyesuaian (Selection & Adjustment)	44
IV	PERANCANGAN SISTEM <i>GRAPH</i>RAG BERBASIS <i>KNOWLEDGE</i>	
	<i>GRAPH</i> KUBERNETES	47
IV.1	Rancangan Solusi Secara Garis Besar	47
IV.2	Pemetaan Metodologi terhadap Kebutuhan Sistem	47
IV.3	Perancangan <i>Knowledge Graph</i> Kubernetes	52
IV.3.1	Perancangan <i>Pipeline Ingestion</i> dan Representasi Objek . .	53
IV.3.2	Perancangan Taksonomi Relasi <i>Knowledge Graph</i>	53
IV.3.3	Perancangan Pembentukan <i>Embedding</i> Semantik	54
IV.4	Perancangan Mekanisme <i>GraphRAG</i>	54
IV.4.1	Perancangan Klasifikasi <i>Intent</i> dan Manajemen Memori . .	54
IV.4.2	Perancangan Strategi <i>Retrieval</i> dan Konstruksi Konteks . .	55
IV.4.3	Perancangan Generasi dan Validasi YAML	56
IV.4.4	Perancangan Antarmuka <i>Web</i> Interaktif	57
IV.5	Perancangan Sistem Pembandingan	58
IV.5.1	Evolusi Rancangan Ketiga Sistem	58
IV.5.2	Rancangan Vanilla LLM	59
IV.5.3	Rancangan Vector RAG	59
IV.5.4	Rangkuman Perbandingan Komponen Teknis	60

V	IMPLEMENTASI SISTEM <i>GRAPH</i>RAG BERBASIS <i>KNOWLEDGE</i>	61
	<i>GRAPH</i> KUBERNETES	61
V.1	Lingkungan Implementasi	61
V.2	Implementasi <i>Pipeline</i> Ekstraksi dan <i>Knowledge Graph</i>	62
V.2.1	Implementasi <i>Pipeline Ingestion</i> dan Representasi Objek	62
V.2.2	Implementasi Taksonomi Relasi <i>Knowledge Graph</i>	64
V.2.3	Implementasi Pembentukan <i>Embedding</i> Semantik	69
V.3	Implementasi Mekanisme <i>GraphRAG</i>	70
V.3.1	Implementasi Klasifikasi <i>Intent</i> dan Manajemen Memori	72
V.3.2	Implementasi Strategi <i>Retrieval</i> dan Konstruksi Konteks	74
V.3.3	Implementasi Generasi dan Validasi YAML	79
V.3.4	Implementasi Antarmuka <i>Web</i> Interaktif	80
V.4	Implementasi Sistem Pembandingan	81
V.4.1	Implementasi Vanilla LLM	82
V.4.2	Implementasi Vector RAG	83
VI	EVALUASI	85
VI.1	Metode Evaluasi	85
VI.1.1	Dataset Uji dan Validasi Pakar	85
VI.1.2	Metrik dan Dimensi Evaluasi	87
VI.1.3	Protokol Eksperimen dan Perbandingan Tiga Sistem LLM	87
VI.2	Hasil Evaluasi	88
VI.2.1	<i>Answer Quality</i> (AnsQ)	89
VI.2.2	<i>Retrieval Quality</i> (RetQ)	90
VI.2.3	<i>Reasoning Quality</i> (ReaQ)	91
VI.2.4	Metrik Domain Kubernetes	92
VI.3	Analisis Evaluasi	93
VI.3.1	Analisis Internal Sistem <i>GraphRAG</i>	94
VI.3.2	Perbandingan Tiga Sistem	95
VI.3.3	Faktor Keunggulan <i>GraphRAG</i>	96
VI.4	Validasi Pakar atas Jawaban Sistem	101
VII	PENUTUP	105
VII.1	Kesimpulan	105
VII.2	Keterbatasan	106
VII.3	Saran	107
Lampiran A	KODE PROGRAM	113
A.1	Manajemen Memori Percakapan (<i>ZepMemoryStore</i>)	113
A.2	<i>Pipeline Ingestion</i> Data (<i>SwaggerGraphBuilder</i>)	116
A.3	<i>LangGraph Agent Pipeline</i>	125
A.4	Mekanisme <i>Retrieval</i> Bertingkat (<i>StatefulK8sRetriever</i>)	129
A.5	Validasi YAML Tiga Lapis (<i>YAMLValidator</i>)	134
A.6	Query Cypher Neo4j	135
A.7	Fungsi Metrik Evaluasi	138
Lampiran B	FIXTURE EVALUASI PAKAR	147

B.1	Fixture B.1 — deployment_basic (Konseptual)	148
B.2	Fixture B.2 — service_types (Konseptual)	149
B.3	Fixture B.3 — scale_existing_deployment (Lanjutan)	150
B.4	Fixture B.4 — ingress_service_pod (Relasional)	151
B.5	Fixture B.5 — hpa_deployment_pod (Relasional)	152
B.6	Fixture B.6 — cronjob_backup (Generasi YAML)	154
B.7	Fixture B.7 — networkpolicy_deny_all (Generasi YAML)	155
B.8	Fixture B.8 — statefulset_with_pvc (Generasi YAML)	156
B.9	Fixture B.9 — pi_want_to_reject_all_docker (Skenario Nyata)	158
B.10	Fixture B.10 — kubectl_find_pods_with_env (Perintah)	159
Lampiran C HASIL EVALUASI KUANTITATIF PER <i>FIXTURE</i>		161
C.1	Metrik <i>Answer Quality</i> (AnsQ)	161
C.2	Metrik <i>Retrieval Quality</i> (RetQ)	166
C.3	Metrik <i>Reasoning Quality</i> (ReaQ)	171
C.4	Metrik Domain Kubernetes	177
Lampiran D PANDUAN DAN HASIL WAWANCARA PRAKTISI		183
D.1	Panduan Wawancara	183
D.2	Ringkasan Hasil Wawancara	184
D.2.1	Konteks dan Frekuensi Penggunaan	184
D.2.2	Tipe Pertanyaan dan Konteks yang Dilampirkan	184
D.2.3	Keterbatasan LLM yang Dirasakan	184
D.2.4	Harapan terhadap Sistem Ideal	184
Lampiran E LEMBAR VALIDASI PAKAR		185
E.1	Lembar Validasi E.1 — Ahmad Afzalulhaq	185
E.2	Lembar Validasi E.2 — Raden Dizi Assyafadi	187
E.3	Lembar Validasi E.3 — Zaky Hassani	189
E.4	Lembar Validasi E.4 — Muhamad Iqbal Ramadhan	191
E.5	Lembar Validasi E.5 — Muhammad Faiq Dhiya Ul Haq	192

DAFTAR GAMBAR

I.1	Tahapan model referensi CRISP-DM (Niakšu 2015)	6
II.1	Arsitektur klaster Kubernetes yang terdiri atas <i>Control Plane</i> dan <i>Worker Nodes</i> , dihubungkan melalui <i>kube-apiserver</i> (The Kubernetes Authors 2024b)	10
II.2	Arsitektur <i>Knowledge Graph</i> (J. Yu dkk. 2021)	16
II.3	Taksonomi pada pembangunan <i>Knowledge Graph</i> (Abu-Salih 2021)	17
III.1	Proporsi objek Kubernetes <i>root</i> vs komponen pendukung dalam spesifikasi OpenAPI Kubernetes	31
IV.1	Rancangan solusi berdasarkan metodologi CRISP-DM	47
IV.2	<i>Architecture Diagram</i> Sistem <i>GraphRAG</i> Kubernetes	51
IV.3	Diagram Urutan (<i>Sequence Diagram</i>) Alur Kerja Sistem <i>GraphRAG</i>	52
IV.4	<i>Pipeline Ingestion</i> dari <i>swagger.json</i> ke Neo4j <i>Knowledge Graph</i>	53
IV.5	<i>Use Case Diagram</i> Interaksi Pengguna dengan Sistem <i>GraphRAG</i>	57
IV.6	Evolusi Rancangan: Vanilla LLM → Vector RAG → <i>GraphRAG</i>	59
V.1	Alur fase 1: Pembuatan <i>Node Definition</i> dari <i>swagger.json</i>	63
V.2	Alur fase 2: Pembuatan <i>Edge</i> Struktural dari Referensi <i>\$ref</i>	65
V.3	Alur fase 2.5: Pembuatan <i>Edge</i> Hierarki dan Tipe Alternatif	66
V.4	Alur fase 3: Pembuatan <i>Edge</i> Semantik dari Pola Domain Kubernetes	67
V.5	Alur fase 1.5: Pembuatan Vektor <i>Embedding</i> untuk Seluruh <i>Node</i>	69
V.6	Diagram interaksi lima modul <i>agent LangGraph</i>	72
V.7	Alur <i>cascading retrieval</i> tiga tahap	74
V.8	Pengaruh kedalaman <i>traversal</i> terhadap RetQ per kategori <i>fixture</i>	76
V.9	<i>Precision@k</i> , <i>Recall@k</i> , dan <i>NDCG@k</i> terhadap kedalaman <i>traversal</i>	77
V.10	Distribusi panjang <i>graph context</i> pada 97 <i>fixture</i>	78
V.11	Skor evaluasi per-faktor terhadap nilai batas karakter	78
V.12	Alur validasi YAML tiga lapis berbasis <i>knowledge graph</i>	80
V.13	Perbandingan alur eksekusi ketiga sistem yang dievaluasi: Vanilla LLM, Vector RAG, dan <i>GraphRAG</i>	82

VI.1 Rata-rata <i>RetQ-gain</i> (GraphRAG – Vector RAG) per tipe <i>fixture</i> . . .	97
VI.2 <i>Scatter plot</i> jumlah <i>hop</i> yang dilalui vs. <i>RetQ-gain</i> per <i>fixture</i> . Ko- relasi Spearman $\rho = +0,290$ ($p = 0,004$, $n = 97$).	99
VI.3 <i>Scatter plot</i> derajat node primer vs. <i>RetQ-gain</i>	100

DAFTAR TABEL

III.1	<i>Technical Gap Analysis</i> pada Domain Kubernetes	34
III.2	Pemetaan Kebutuhan Bisnis terhadap <i>Technical Gap</i> dan Kebutuhan Fungsional	36
III.3	Kebutuhan Fungsional Sistem	36
III.4	Kebutuhan Non-Fungsional Sistem	39
III.5	Pemetaan kebutuhan bisnis terhadap dimensi dan metrik evaluasi . .	40
III.6	Rubrik Penilaian Kriteria Evaluasi Solusi	43
III.7	Matriks Perbandingan Solusi Berdasarkan Rubrik Evaluasi	43
IV.1	Pemetaan Tahap Metodologi	48
IV.2	Strategi Pembentukan Relasi dalam <i>Knowledge Graph</i> Kubernetes .	53
IV.3	Distribusi karakteristik <i>node</i> berdasarkan kedalaman penelusuran graf	56
IV.4	Fungsi Tiga Lapis Validasi YAML	57
IV.5	Perbandingan Komponen Teknis Ketiga Sistem	60
V.1	Dependensi Utama Implementasi Sistem GraphRAG Kubernetes . .	62
V.2	Tujuh Kategori Relasi dalam <i>Knowledge Graph</i> Kubernetes	64
V.3	Taksonomi 18 Jenis <i>Edge</i> dalam <i>Knowledge Graph</i> Kubernetes . . .	67
V.4	Statistik <i>Knowledge Graph</i> Kubernetes Hasil Konstruksi	70
V.5	Lima Modul <i>Agent</i> LangGraph Pipeline	71
V.6	Definisi Tipe <i>Intent</i> beserta Contoh Pertanyaan	73
V.7	Pemetaan <i>Intent Type</i> terhadap Kedalaman Traversal Graf	75
VI.1	Profil Narasumber Wawancara Validasi Dataset	86
VI.2	Penilaian Realisme Sampel <i>Fixture</i> oleh Pakar (Skala 1–5)	86
VI.3	Perbandingan Skor per Faktor: Vanilla LLM vs. Vector RAG vs. GraphRAG (97 <i>Fixture</i> , 95% CI <i>bootstrap</i>)	88
VI.4	Sub-Metrik <i>Answer Quality</i> (AnsQ) Sistem GraphRAG (97 <i>Fixture</i>)	90
VI.5	Sub-Metrik <i>Retrieval Quality</i> (RetQ) Sistem GraphRAG (97 <i>Fixture</i>)	91
VI.6	Sub-Metrik <i>Reasoning Quality</i> (ReaQ) Sistem GraphRAG (97 <i>Fixture</i>)	92
VI.7	Skor Evaluasi Sistem GraphRAG per Kategori <i>Fixture</i>	93

VI.8 <i>Ablation study</i> : selisih skor terhadap <i>baseline</i> GraphRAG per faktor	94
VI.9 Signifikansi Statistik <i>Ablation Study</i> per Faktor (Wilcoxon + <i>Boots-trap</i> , <i>p-value</i>)	94
VI.10 Uji Signifikansi Statistik: GraphRAG vs. <i>Baseline</i> per Faktor ($n = 97$, Holm-Bonferroni)	96
VI.11 Rata-rata <i>RetQ-gain</i> (GraphRAG – Vector RAG) per Tipe <i>Fixture</i> .	98
VI.12 Korelasi Spearman: Faktor Struktural vs. <i>RetQ-gain</i>	101
VI.13 Penilaian Pakar atas Jawaban Sistem GraphRAG (Skala 1–5, $n = 4$ Pakar)	102
B.1 Ringkasan 10 <i>Fixture</i> Evaluasi Pakar	147
C.1 Nilai Metrik AnsQ per <i>Fixture</i> (97 <i>Fixture</i>)	161
C.2 Nilai Metrik RetQ per <i>Fixture</i> (97 <i>Fixture</i>)	166
C.3 Nilai Metrik ReaQ per <i>Fixture</i> (97 <i>Fixture</i>)	172
C.4 Metrik Domain Kubernetes per <i>Fixture</i> (Path Coverage, Hop Accuracy, RGA, YAML, <i>Intent</i>)	177
E.1 Peran Lima Pakar dalam Proses Validasi	185

DAFTAR PERSAMAAN

II.1	Model kondisional GraphRAG berbasis <i>knowledge graph</i>	18
II.2	Metrik <i>Faithfulness</i> : proporsi node kunci dalam jawaban	20
II.3	Metrik <i>Answer Relevance</i> berbasis kemiripan kosinus <i>embedding</i> . . .	20
II.4	Metrik <i>Syntactic Validity</i> keluaran YAML	20
II.5	Metrik <i>Schema Compliance</i> terhadap spesifikasi OpenAPI Kubernetes	20
II.6	Metrik <i>Precision@k</i> dan <i>Recall@k</i> pada <i>retrieval</i>	21
II.7	Metrik <i>F1-Score@k</i> : rata-rata harmonik presisi dan kelengkapan . . .	21
II.8	Metrik <i>NDCG@k</i> : peringkat hasil <i>retrieval</i> bernormalisasi	21
II.9	Metrik <i>Grounding Score</i> : proporsi istilah Kubernetes berbasis konteks	22
II.10	Metrik <i>Hop Accuracy</i> : ketepatan kedalaman <i>traversal</i> berbasis <i>intent</i> .	22
II.11	Metrik <i>Path Coverage</i> : kelengkapan konteks relasional berbasis <i>edge</i>	23
II.12	Metrik <i>Retrieval-Grounded Answer</i> (RGA): jawaban benar berbasis <i>retrieval</i>	23
VI.1	<i>RetQ-gain</i> per <i>fixture</i> sebagai ukuran keunggulan relatif	97

DAFTAR LISTING

III.1	Skema referensi objek pada <code>swagger.json</code>	32
III.2	Wujud YAML objek <i>LabelSelector</i>	32
III.3	Skema referensi daftar pada <code>swagger.json</code>	32
III.4	Wujud YAML daftar objek <i>Container</i>	32
III.5	Skema referensi pemetaan pada <code>swagger.json</code>	33
III.6	Wujud YAML pemetaan objek <i>Quantity</i>	33
V.1	Kamus kedalaman <i>traversal</i> per tipe <i>intent</i> (<code>_DEPTH_BY_INTENT</code>)	75
V.2	Pseudokode <i>invoker</i> <code>invoke_mode</code> untuk mode Vector RAG dan Vanna LLM	83
A.1	<code>src/memory/zep_store.py</code> — Manajemen Memori Percakapan	113
A.2	<code>src/ingestion/parser.py</code> — Pipeline Ingestion Data (Swagger-GraphBuilder)	116
A.3	<code>src/chatbot/graph_agent.py</code> — LangGraph Agent Pipeline	125
A.4	<code>src/chatbot/custom_retriever.py</code> — Mekanisme Retrieval Ber-tingkat	129
A.5	<code>src/validation/yaml_validator.py</code> — Validasi YAML Tiga Lapis	134
A.6	<code>src/graph/queries.py</code> — Query Cypher Neo4j	136
A.7	<code>scripts/evaluate.py</code> — Fungsi Metrik Evaluasi (AnsQ, RetQ, ReaQ)	138
B.1	<code>fixture-deployment-basic.json</code> — Konseptual: Pembaruan Image Deployment	148
B.2	<code>fixture-service-types.json</code> — Konseptual: Service LoadBalancer vs. Ingress	149
B.3	<code>fixture-scale-existing-deployment.json</code> — Lanjutan: Skala-lakan Replika Deployment	150
B.4	<code>fixture-ingress-service-pod.json</code> — Relasional: Alur Traffic Ingress→Service→Pod	151
B.5	<code>fixture-hpa-deployment-pod.json</code> — Relasional: HPA dan Penskalaan Otomatis	152

B.6	<code>fixture-cronjob-backup.json</code> — Generasi YAML: CronJob Backup Database	154
B.7	<code>fixture-networkpolicy-deny-all.json</code> — Generasi YAML: NetworkPolicy Deny-All	155
B.8	<code>fixture-statefulset-with-pvc.json</code> — Generasi YAML: StatefulSet dengan PVC	156
B.9	<code>fixture-pi-reject-docker-registry.json</code> — Skenario Nyata: Kebijakan Registry Docker	158
B.10	<code>fixture-kubectrl-find-pods-with-env.json</code> — Perintah: Pencarian Environment Variable	159

DAFTAR SIMBOL

Notasi	Deskripsi	Pemakaian pertama kali
Huruf Yunani		
ρ	Koefisien korelasi Spearman	99
Δ	Selisih skor antar sistem	85
Variabel Skalar		
d	Kedalaman <i>traversal</i> graf	47
f	<i>Fixture</i> evaluasi individual	22
i	Indeks posisi atau indeks <i>fixture</i>	21
k	Parameter top- k <i>retrieval</i>	21
n	Jumlah sampel atau <i>fixture</i>	99
p	p -value uji signifikansi statistik	99
rel_i	Relevansi biner node pada posisi ke- i	21
Himpunan		
$E_{\text{retrieved}}$	Himpunan <i>edge</i> pada subgraf hasil penelusuran	23
E_{relevant}	Himpunan <i>edge</i> relevan menurut <i>ground truth</i>	23
F	Himpunan seluruh <i>fixture</i> evaluasi	22
$N_{\text{retrieved}}$	Himpunan <i>node</i> yang diambil saat <i>retrieval</i>	21
N_{relevant}	Himpunan <i>node</i> relevan menurut <i>ground truth</i>	21
N_{kunci}	Himpunan <i>node</i> kunci pada <i>ground truth</i>	20
N_{jawaban}	Himpunan <i>node</i> yang disebut dalam jawaban sistem	20
T_{grounded}	Himpunan istilah Kubernetes yang terlacak ke konteks	22
T_{jawaban}	Himpunan istilah Kubernetes dalam jawaban sistem	22
Vektor		
$\mathbf{e}_a$	Vektor <i>embedding</i> jawaban sistem	20
$\mathbf{e}_g$	Vektor <i>embedding</i> jawaban acuan (<i>ground truth</i>)	20
Notasi Graf		
$\mathcal{K}$	<i>Knowledge graph</i> sebagai himpunan <i>triples</i> (h, r, t)	18

Notasi	Deskripsi	Pemakaian pertama kali
$\mathcal{Z}(x, \mathcal{K})$	Himpunan subgraf relevan yang ditarik dari $\mathcal{K}$ untuk kueri x	18
Fungsi dan Metrik		
$d_{\text{pred}}(f)$	Kedalaman <i>traversal</i> yang dieksekusi sistem untuk <i>fixture</i> f	22
$d_{\text{gt}}(f)$	Kedalaman yang dibutuhkan menurut anotasi <i>ground truth</i> fixture f	22
RetQ-gain_i	Selisih RetQ GraphRAG dan Vector RAG untuk <i>fixture</i> ke- i	97

DAFTAR SINGKATAN

Singkatan	Kepanjangan	Pemakaian pertama kali
AI	<i>Artificial Intelligence</i>	1
AnsQ	<i>Answer Quality</i>	19
API	<i>Application Programming Interface</i>	1
CI	<i>Confidence Interval</i>	85
CNCF	<i>Cloud Native Computing Foundation</i>	1
CRD	<i>Custom Resource Definition</i>	61
CRISP-DM	<i>Cross-Industry Standard Process for Data Mining</i>	1
DAG	<i>Directed Acyclic Graph</i>	61
EDA	<i>Exploratory Data Analysis</i>	27
GNN	<i>Graph Neural Network</i>	27
GPT	<i>Generative Pre-trained Transformer</i>	1
GraphRAG	<i>Graph Retrieval-Augmented Generation</i>	1
GVK	<i>Group-Version-Kind</i>	27
HPA	<i>HorizontalPodAutoscaler</i>	9
IaC	<i>Infrastructure as Code</i>	9
IR	<i>Information Retrieval</i>	19
JSON	<i>JavaScript Object Notation</i>	27
K8s	Kubernetes	1
KG	<i>Knowledge Graph</i>	1
LLM	<i>Large Language Model</i>	1
NDCG	<i>Normalized Discounted Cumulative Gain</i>	19
NLP	<i>Natural Language Processing</i>	9
OpenAPI	<i>Open Application Programming Interface Specification</i>	1
PCST	<i>Prize-Collecting Steiner Tree</i>	27
PVC	<i>PersistentVolumeClaim</i>	9
RAG	<i>Retrieval-Augmented Generation</i>	1
RBAC	<i>Role-Based Access Control</i>	9
ReaQ	<i>Reasoning Quality</i>	19
RetQ	<i>Retrieval Quality</i>	19
RGA	<i>Retrieval-Grounded Answer</i>	19
SRE	<i>Site Reliability Engineering</i>	85
UUID	<i>Universally Unique Identifier</i>	47

Singkatan	Kepanjangan	Pemakaian pertama kali
YAML	<i>YAML Ain't Markup Language</i>	1

BAB I

PENDAHULUAN

I.1 Latar Belakang

Transformasi industri teknologi informasi menuju arsitektur *cloud-native* kini menjadi kebutuhan fundamental, bukan sekadar tren. Gartner (2021) memproyeksikan bahwa pada tahun 2025, lebih dari 95% beban kerja digital akan di-*deploy* pada platform *cloud-native*. Angka ini meningkat tajam dari hanya 30% pada tahun 2021. Fondasi utama dari arsitektur *cloud-native* ini adalah penggunaan teknologi kontainer yang memungkinkan aplikasi dikemas dan dijalankan secara efisien. Namun, seiring dengan peningkatan skala sistem, pengelolaan ribuan kontainer secara manual tidak lagi memungkinkan sehingga dibutuhkan sebuah alat otomatisasi. Untuk menjembatani kebutuhan kompleks inilah, Kubernetes (K8s) hadir di pusat ekosistem dan mengukuhkan posisinya sebagai standar industri *de facto* untuk orkestrasi kontainer.

Berdasarkan laporan survei *Cloud Native Computing Foundation* (CNCF) tahun 2023, Kubernetes mendominasi pangsa pasar global karena ekosistemnya yang matang, dukungan komunitas yang luas, serta fleksibilitas dalam mengelola infrastruktur berskala besar secara deklaratif (Cloud Native Computing Foundation 2023). Pada tahun 2022, sekitar 58% pengguna telah menjalankan Kubernetes di lingkungan produksi dan 23% lainnya masih dalam tahap evaluasi (total 81%). Pada tahun 2023, angka tersebut meningkat menjadi 66% pengguna di produksi dengan 18% dalam evaluasi (total 84%). Peningkatan signifikan ini menunjukkan bahwa Kubernetes semakin menjadi fondasi utama dalam pengelolaan layanan *cloud-native* modern.

Meskipun Kubernetes mendominasi lanskap orkestrasi kontainer, platform ini menghadirkan tantangan operasional yang signifikan berupa kompleksitas konfigurasi berbasis berkas YAML. Paradigma deklaratif Kubernetes menuntut pengembang untuk menulis ratusan baris kode yang verbos dan hierarkis demi mendefinisikan *desired state* dari sebuah aplikasi yang mencakup berbagai spesifikasi parameter dari Kubernetes. Kompleksitas ini diperparah oleh interdependensi antarobjek yang

sangat ketat namun sering kali tidak terlihat secara eksplisit (*implicit dependencies*) yang mengakibatkan kesalahan kecil seperti ketidakcocokan *label selector* dapat memicu kegagalan sistem berantai. Akibat beban kognitif yang tinggi ini, kerumitan YAML tidak hanya menjadi hambatan skalabilitas utama bagi 65% penggunanya, tetapi juga menjadi sumber utama miskonfigurasi operasional dan keamanan; tercatat sekitar 80% insiden operasional berakar pada kompleksitas ini, dan 18% berkas konfigurasi *open-source* teridentifikasi mengandung kerentanan keamanan yang serius (Rahman dkk. 2023).

Situasi tersebut berdampak langsung pada pemanfaatan *Large Language Models* (LLM) untuk menghasilkan kode maupun konfigurasi teknis Kubernetes. Walaupun model seperti GPT-4 menawarkan kemudahan dalam memahami dokumentasi API, kemampuan tersebut tidak selalu sejalan dengan ketepatan semantik. Studi evaluatif oleh Liu dkk. (2024) menunjukkan bahwa LLM dapat menghasilkan kode yang secara sintaksis tampak benar, tetapi keliru secara semantis. Dalam lingkungan *production-grade*, kesalahan semantik semacam ini tidak hanya bersifat minor, tetapi berpotensi memicu hilangnya layanan, gangguan orkestrasi, hingga *downtime* berskala besar.

Upaya mitigasi telah dilakukan melalui *Retrieval-Augmented Generation* (RAG) berbasis vektor untuk menambahkan konteks dokumentasi saat LLM menjawab pertanyaan teknis. Namun, penelitian oleh Wan dkk. (2025) mengungkapkan bahwa pendekatan vektor RAG hanya mengutamakan kesamaan *embedding* sehingga gagal menangkap relasi kausal, hierarkis, dan berbasis aturan teknis yang diperlukan untuk memahami konfigurasi domain Kubernetes. Pendekatan vektor RAG hanya mencapai 63,4% *exact match accuracy* dan 69,8% *context precision*, menunjukkan rendahnya ketepatan semantik dalam jawaban sistem. *Embedding* yang dilatih dari korpus umum juga mengabaikan terminologi teknis spesifik sehingga menghasilkan jawaban yang tampak relevan secara teks, tetapi tidak tepat secara domain.

Sebagai solusi, Wan dkk. (2025) mengusulkan penggabungan representasi pengetahuan terstruktur melalui *Knowledge Graph* (KG) untuk memberikan fondasi penalaran relasional yang eksplisit. Integrasi KG dalam sistem RAG menghasilkan peningkatan kinerja yang signifikan, mencapai 77,8% *exact match accuracy* dan 76,5% *context precision*, yaitu peningkatan lebih dari 14,4 poin persentase dibanding pendekatan vektor RAG. KG tidak hanya meningkatkan presisi pencarian informasi, tetapi juga memungkinkan *semantic alignment* melalui pembatasan ontologis sehingga *retrieval* tidak bergantung semata pada kemiripan *embedding*, melainkan

pada struktur relasional domain.

Dengan demikian, pendekatan Graph-RAG menawarkan fondasi penalaran teknis yang lebih dapat dipertanggungjawabkan dan mampu menurunkan risiko halusinasi semantik berbasis domain, khususnya pada sistem *cloud-native* yang sensitif terhadap kesalahan konfigurasi. Berdasarkan urgensi tersebut, penelitian ini bertujuan untuk membangun sistem *Retrieval-Augmented Generation* (RAG) berbasis *Knowledge Graph* yang dikhususkan untuk dokumentasi Kubernetes. Pendekatan ini diharapkan mampu meningkatkan akurasi *retrieval* terhadap pertanyaan yang membutuhkan penalaran struktural dengan mengekspresikan relasi ontologis antar objek konfigurasi Kubernetes. Selain menghasilkan jawaban teknis berupa penjelasan dan konfigurasi YAML yang valid, sistem juga dirancang untuk memberikan jejak penalaran graf sebagai mekanisme pendeteksi halusinasi. Hal ini memungkinkan setiap keluaran tidak hanya relevan secara tekstual, tetapi juga dapat diverifikasi kesesuaiannya terhadap skema objek Kubernetes.

Namun, penerapan pendekatan GraphRAG pada domain Kubernetes masih menyisakan tiga celah teknis yang belum terjawab. Pertama, mayoritas pendekatan ekstraksi *knowledge graph* pada literatur GraphRAG memanfaatkan LLM sehingga graf hasil ekstraksi bervariasi antareksekusi dan sulit dijamin konsistensinya. Kedua, mekanisme *traversal* pada GraphRAG yang ada umumnya menggunakan kedalaman tetap, padahal kebutuhan konteks berbeda antar jenis kueri sehingga kedalaman seragam berisiko menurunkan presisi maupun kelengkapan. Ketiga, belum tersedia mekanisme validasi struktural YAML yang memanfaatkan representasi graf sebagai alternatif terhadap *dry-run* pada kluster Kubernetes aktif.

Berdasarkan ketiga celah tersebut, penelitian ini mengajukan tiga kontribusi teknis. **Pertama**, *pipeline* ekstraksi *knowledge graph* yang bersifat deterministik berbasis skema OpenAPI (*schema-derived deterministic KG construction*). Graf dibangun langsung dari `swagger.json` melalui aturan transformasi yang eksplisit sehingga hasil ekstraksi konsisten antareksekusi, berbeda dengan pendekatan ekstraksi berbasis LLM yang menghasilkan struktur graf bervariasi karena bersifat stokastik (Panda dkk. 2024; Wan dkk. 2025). **Kedua**, mekanisme *traversal* dengan kedalaman yang disesuaikan per tipe *intent* (*intent-adaptive depth traversal*): alih-alih menggunakan kedalaman tetap yang seragam sebagaimana pada GraphRAG yang ada (Wan dkk. 2025), sistem menetapkan kedalaman berdasarkan karakteristik struktural tiap jenis pertanyaan pengguna. **Ketiga**, validasi struktural YAML tiga lapis berbasis *knowledge graph* (*KG-grounded structural validation*) yang berfungsi sebagai alternatif

statis terhadap mekanisme *dry-run* pada kluster Kubernetes aktif.

I.2 Rumusan Masalah

Penelitian ini bertujuan untuk menjawab pertanyaan-pertanyaan berikut,

1. Bagaimana membangun *knowledge graph* dari spesifikasi Kubernetes guna merepresentasikan struktur dan relasi antarobjek konfigurasi secara komprehensif?
2. Bagaimana merancang mekanisme GraphRAG yang memanfaatkan representasi struktural dalam *knowledge graph* untuk meningkatkan kualitas *retrieval* sekaligus memvalidasi struktur YAML berbasis *knowledge graph* pada konfigurasi Kubernetes?
3. Bagaimana perbandingan kinerja antara sistem GraphRAG, Vector RAG, dan Vanilla LLM dalam melakukan *retrieval* informasi?

I.3 Tujuan

Tujuan dari penelitian ini adalah sebagai berikut:

1. Membangun *knowledge graph* dari spesifikasi OpenAPI Kubernetes (swagger.json) melalui *pipeline* ekstraksi guna memetakan objek Kubernetes beserta relasi hierarkis dan referensial antarobjek Kubernetes.
2. Mengembangkan sistem GraphRAG yang menelusuri jalur relasional (*intent-adaptive depth traversal*) berdasarkan kueri pengguna untuk menghasilkan jawaban yang konsisten dengan logika domain Kubernetes, dilengkapi mekanisme validasi struktural YAML tiga lapis berbasis *knowledge graph* yang memverifikasi *required fields* langsung dari representasi graf.
3. Membandingkan kinerja sistem GraphRAG dengan pendekatan Vector RAG dan Vanilla LLM untuk memetakan area keunggulan *knowledge graph* sebagai basis pencarian (*retrieval*).

I.4 Batasan Masalah

Untuk menjaga fokus dan kelayakan penelitian dalam lingkup Tugas Akhir, batasan masalah berikut ditetapkan:

1. Data penelitian bersumber secara eksklusif dari spesifikasi resmi Kubernetes OpenAPI (swagger.json) versi v1.30 tanpa melakukan *web scraping* dokumentasi HTML. Ekstraksi data dibatasi murni pada blok *definitions* dengan mengecualikan blok operasional protokol API (seperti *paths*, *parameters*, dan *responses*) guna mencegah *context noise* pada LLM.
2. *Knowledge graph* yang dibangun hanya merepresentasikan struktur skema (*schema structure*) dan tidak mencakup perilaku operasional (*runtime beha-*

vior) maupun logika validasi dengan fokus utama penelitian untuk menguji validitas struktur sintaksis YAML.

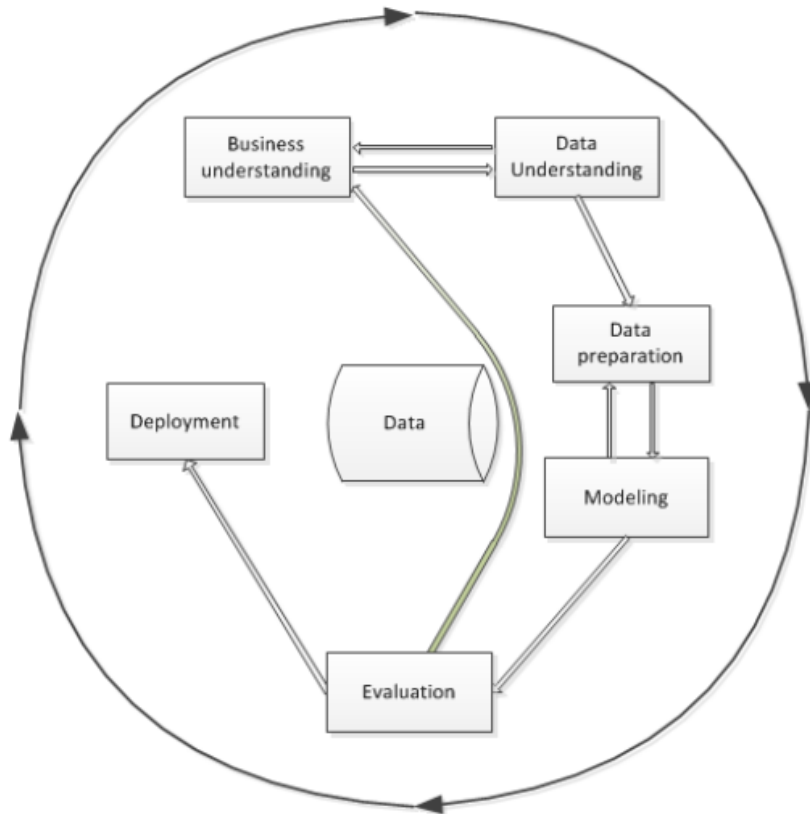
3. Luaran penelitian berupa jawaban teknis dalam dua bentuk, yaitu penjelasan naratif konsep Kubernetes dan manifes YAML murni (*plain YAML*) berdasarkan *OpenAPI Specification*. Setiap luaran dilengkapi jejak penalaran graf (*graph reasoning path*) guna mendeteksi halusinasi LLM.
4. Luaran YAML tidak diuji langsung pada klaster Kubernetes nyata (*actual cluster*). Validasi dilakukan secara statis menggunakan pustaka *PyYAML* untuk memeriksa kebenaran sintaksis dan pustaka *kubernetes-validate* untuk memeriksa kesesuaian terhadap skema Kubernetes, dilengkapi evaluasi kinerja berdasarkan dataset uji.
5. Model LLM yang digunakan berasal dari *pre-trained models* yang diakses melalui API, yaitu **GPT-4o-mini** untuk peran *thinker* maupun *speaker*, tanpa proses *fine-tuning*.
6. Kedalaman *graph traversal* ditentukan secara adaptif berdasarkan tipe *intent* kueri, yaitu kedalaman 2 untuk *intent explain* dan *follow-up*, serta kedalaman 3 untuk *intent* lainnya (*generate_yaml*, *trace_relationship*, *planning*). Batas atas kedalaman 3 dipilih sebagai titik optimal antara kelengkapan dan presisi konteks karena traversal yang lebih dalam mulai didominasi tipe utilitas generik yang menurunkan presisi. Justifikasi distribusi kedalaman yang mendasari keputusan ini diuraikan lebih lanjut pada Bab IV.
7. Seluruh proses pengembangan, pengolahan data, dan eksperimen dilakukan pada lingkungan **Visual Studio Code**.
8. Kinerja komputasi dan hasil eksperimen didasarkan pada perangkat keras spesifik (PC dengan CPU AMD Ryzen 5, GPU AMD Radeon RX 6750XT, RAM 16 GB) dan mungkin bervariasi pada konfigurasi lain.

I.5 Metodologi

Pelaksanaan Tugas Akhir ini mengadopsi kerangka kerja *Cross-Industry Standard Process for Data Mining* (CRISP-DM) yang dapat dilihat pada Gambar I.1 sebagai metodologi utama. CRISP-DM dipilih karena memberikan proses terstruktur, iteratif, dan dapat diadaptasi untuk pengembangan sistem berbasis analisis data dan representasi pengetahuan (Niakšu 2015). Tahapan metodologi yang digunakan terdiri dari enam fase berikut,

1. Pemahaman Bisnis (*Business Understanding*)

Tahap ini bertujuan untuk mengidentifikasi kebutuhan serta masalah inti yang ingin diselesaikan melalui pemanfaatan data. Dalam konteks penelitian ini, permasalahan yang diangkat adalah ketidaktepatan jawaban teknis yang diha-



Gambar I.1 Tahapan model referensi CRISP-DM (Niakšu 2015)

silkan *Large Language Models* (LLM) ketika merujuk dokumentasi Kubernetes, terutama pada konfigurasi YAML yang bersifat struktural dan rentan mengandung halusinasi. Oleh karena itu, kebutuhan penelitian diarahkan pada peningkatan akurasi penalaran struktural pada proses *retrieval* serta mitigasi halusinasi konfigurasi pada sistem *cloud-native*.

2. Pemahaman Data (*Data Understanding*)

Fase ini berfokus pada pengumpulan dan eksplorasi sumber data untuk memahami struktur, kualitas, serta batasannya. Pada penelitian ini, data diperoleh dari spesifikasi resmi Kubernetes OpenAPI yang dianalisis untuk memahami jenis objek Kubernetes, atribut yang dimiliki, serta relasi hierarkis dan referensial antarobjek Kubernetes. Pemahaman ini menentukan objek Kubernetes yang akan dimodelkan dalam *knowledge graph*.

3. Persiapan Data (*Data Preparation*)

Tahapan ini mencakup pembersihan dan transformasi data agar siap digunakan dalam proses pemodelan. Dalam penelitian ini, dilakukan ekstraksi berbasis *script* dari spesifikasi OpenAPI Kubernetes (JSON) untuk membentuk representasi entitas, yaitu relasi yang dapat dimodelkan sebagai *Knowledge Graph*. Proses ini mencakup identifikasi objek, pemetaan hubungan antarob-

jek, serta penghubungan *cross-resource references*. Hasil akhirnya berupa struktur graf yang siap digunakan pada mekanisme *retrieval*.

4. **Pemodelan (*Modeling*)**

Fase ini bertujuan untuk menerapkan algoritma yang sesuai guna menyelesaikan masalah yang telah dirumuskan. Pada penelitian ini, dibangun mekanisme *graph-guided retrieval* yang menelusuri graf menggunakan *graph traversal* berdasarkan kueri pengguna dan menghasilkan *reasoning path*. Jalur penalaran tersebut kemudian diinjeksikan sebagai konteks pada proses RAG untuk memandu LLM dalam menghasilkan jawaban serta konfigurasi YAML yang sesuai dengan logika sistem Kubernetes.

5. **Evaluasi (*Evaluation*)**

Tahap evaluasi bertujuan menilai kualitas model dan membandingkannya dengan pendekatan alternatif menggunakan metrik yang terukur. Dalam penelitian ini, kinerja sistem dievaluasi menggunakan metrik *answer quality*, *retrieval quality*, dan *reasoning quality*, kemudian dibandingkan dengan dua *baseline*, yaitu *Vanilla LLM* dan *Vector-based RAG*.

6. **Penyebaran (*Deployment*)**

Fase ini berkaitan dengan penyajian hasil implementasi dalam bentuk prototipe dan dokumentasi. Pada penelitian ini, penyebaran diwujudkan dalam bentuk antarmuka web interaktif berbasis Streamlit yang menampilkan jawaban, konfigurasi YAML, serta *reasoning path* sebagai bukti penalaran graf. Selain itu, dokumentasi lengkap disusun sebagai bentuk penyebaran akademik yang memuat rancangan sistem, hasil evaluasi, serta rekomendasi penelitian selanjutnya.

I.6 **Sistematika Penulisan**

Dokumen Tugas Akhir ini disusun dalam tujuh bab dengan sistematika sebagai berikut.

1. **Bab I Pendahuluan**

Bab ini memaparkan latar belakang permasalahan, rumusan masalah, tujuan penelitian, batasan masalah, metodologi pengerjaan, dan sistematika penulisan.

2. **Bab II Studi Literatur**

Bab ini menguraikan dasar teori yang mendasari penelitian, meliputi arsitektur Kubernetes dan kompleksitasnya, *Large Language Models*, pendekatan *Retrieval-Augmented Generation*, *knowledge graph*, serta kerangka evaluasi dan seluruh definisi metrik yang digunakan, dan kajian penelitian terkait.

3. **Bab III Analisis Masalah**

Bab ini menyajikan analisis terhadap permasalahan yang dihadapi berdasarkan kebutuhan bisnis (B01–B03), eksplorasi data spesifikasi OpenAPI Kubernetes, analisis kebutuhan fungsional dan non-fungsional sistem, pemetaan kebutuhan bisnis terhadap dimensi evaluasi, serta justifikasi pemilihan pendekatan GraphRAG sebagai solusi.

4. **Bab IV Perancangan**

Bab ini merinci rancangan arsitektur dan komponen sistem GraphRAG, mencakup *pipeline* ekstraksi *knowledge graph* deterministik beserta taksonomi *edge* (T1); mekanisme GraphRAG dengan *intent-adaptive depth traversal* yang dilengkapi validasi struktural YAML tiga lapis berbasis *knowledge graph* (T2); serta rancangan sistem pembandingan Vector RAG dan Vanilla LLM sebagai dasar perbandingan (T3).

5. **Bab V Implementasi**

Bab ini menguraikan detail implementasi teknis setiap komponen sistem, mencakup implementasi *pipeline* ekstraksi *knowledge graph* (T1), mekanisme GraphRAG dan validasi YAML (T2), serta implementasi sistem pembandingan (T3).

6. **Bab VI Evaluasi**

Bab ini menyajikan metodologi evaluasi dan hasil perbandingan kinerja per-faktor antara sistem GraphRAG, Vector RAG, dan Vanilla LLM terhadap skenario uji. Pembahasan mencakup perbandingan pada dimensi kualitas jawaban, kualitas *retrieval*, dan kualitas penalaran, *ablation study*, analisis sensitivitas kedalaman, uji signifikansi statistik, serta analisis kondisi keunggulan sistem.

7. **Bab VII Penutup**

Bab ini menyimpulkan capaian penelitian terhadap ketiga tujuan (T1, T2, dan T3) berdasarkan temuan empiris pada Bab VI, memaparkan keterbatasan penelitian, serta menyajikan saran pengembangan lanjutan.

BAB II

STUDI LITERATUR

II.1 Arsitektur *Cloud* Modern dan Tantangan Kompleksitas

Aplikasi *cloud-native* adalah perangkat lunak yang dirancang sejak awal untuk berjalan di lingkungan *cloud* dengan memanfaatkan skalabilitas, resiliensi, dan elastisitas infrastruktur modern (Soldani, Tamburri, dan Van Den Heuvel 2018). Pergeseran paradigma dari arsitektur monolitik menuju layanan mikro (*microservices*) kini telah menjadi fondasi utama dalam pengembangan aplikasi *cloud-native* modern. Berbeda dengan pendekatan arsitektur tradisional, gaya arsitektur layanan mikro mendistribusikan logika bisnis secara masif ke dalam puluhan hingga ratusan layanan kecil independen. Distribusi fungsionalitas ini memunculkan kebutuhan akan mekanisme *deployment* independen. Sebagai solusi, layanan mikro secara alami terintegrasi dengan platform berbasis kontainer karena setiap layanan dapat dikemas dan didistribusikan di dalam kontainernya masing-masing (Soldani, Tamburri, dan Van Den Heuvel 2018). Walaupun memberikan keuntungan operasional yang signifikan berupa portabilitas lintas *cloud*, skalabilitas tingkat tinggi, dan kebebasan pemilihan teknologi bagi pengembang, ekosistem kontainer ini juga menghadirkan tantangan kompleksitas (Soldani, Tamburri, dan Van Den Heuvel 2018).

Dalam lingkungan terdistribusi ini, setiap layanan berkomunikasi melalui antarmuka API yang bertindak sebagai kontrak standar komunikasi antarlayanan (Soldani, Tamburri, dan Van Den Heuvel 2018). Kesalahan interpretasi terhadap arsitektur dan kontrak API ini sering kali berujung pada kegagalan operasional; kegagalan satu layanan dapat memicu rentetan kegagalan layanan lain yang bergantung padanya (*cascading failures*) (Soldani, Tamburri, dan Van Den Heuvel 2018). Akibatnya, dokumentasi API dan manifes konfigurasi orkestrasi menjadi semakin kompleks, yang menjadi tantangan utama pengembang *cloud-native*.

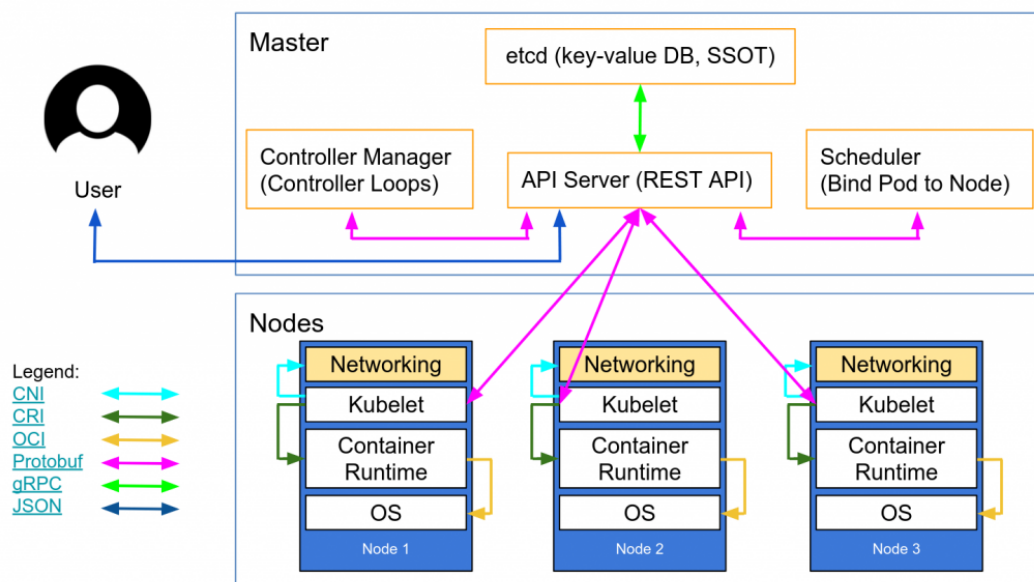
II.2 Kubernetes: Platform Orkestrasi Kontainer untuk Aplikasi *Cloud-Native*

Kubernetes merupakan platform orkestrasi kontainer yang dirancang untuk mengotomatisasi proses *deployment*, *scaling*, dan manajemen aplikasi berbasis kontainer

dalam lingkungan *cloud-native* (The Kubernetes Authors 2024b). Platform ini menyediakan mekanisme deklaratif berbasis konfigurasi yang memungkinkan pengguna mendefinisikan keadaan sistem yang diinginkan (*desired state*) melalui berkas konfigurasi, sementara Kubernetes menjaga agar keadaan tersebut tetap terpenuhi melalui rangkaian komponen pengendali (*controller loops*).

II.2.1 Gambaran Umum Arsitektur

Arsitektur Kubernetes dibangun atas konsep kluster yang terdiri dari dua kelompok komponen utama, yaitu *Control Plane* dan *Worker Nodes*. *Control Plane* berfungsi sebagai pengendali utama yang membuat keputusan bisnis aplikasi, misalnya penjadwalan, pemantauan, serta konsistensi status. *Worker Nodes* merupakan mesin (server) fisik atau virtual yang menjalankan kontainer sebagai satuan aplikasi. Kedua komponen ini berkomunikasi secara internal melalui API server yang menjadi pintu utama seluruh permintaan sistem.



Gambar II.1 Arsitektur kluster Kubernetes yang terdiri atas *Control Plane* dan *Worker Nodes*, dihubungkan melalui *kube-apiserver* (The Kubernetes Authors 2024b)

1. *Control Plane*

Control Plane bertanggung jawab mengelola keadaan (*state*) kluster secara keseluruhan. Komponen utamanya meliputi,

- kube-apiserver** sebagai gerbang utama untuk mengelola objek Kubernetes melalui API,
- etcd** sebagai basis data *key-value* yang menyimpan informasi konfigurasi dan status,

- c. **kube-scheduler** yang menentukan Node penempatan Pod berdasarkan ketersediaan sumber daya,
- d. **kube-controller-manager** yang berisi sekumpulan kontroler untuk menjaga kesesuaian antara kondisi aktual dan *desired state*. Melalui mekanisme deklaratif ini, Kubernetes secara otomatis memperbaiki status objek yang tidak sesuai dan memastikan setiap aplikasi berjalan sesuai aturan konfigurasi yang ditentukan.

2. **Worker Nodes**

Worker Nodes merupakan lapisan eksekusi tempat aplikasi berjalan dalam bentuk Pod. Node terdiri atas tiga komponen inti:

- a. **kubelet** yang bertugas mengeksekusi Pod sesuai *specification* yang diterima dari *Control Plane*,
- b. **kube-proxy** yang menyediakan layanan *network forwarding* dan *load balancing* sederhana antar Pod dalam klaster,
- c. **container runtime** yang bertanggung jawab menjalankan kontainer. Dengan demikian, Node menjalankan komputasi terdistribusi yang dikontrol oleh *Control Plane* melalui komunikasi berbasis API.

II.2.2 **Resource Model dan Hierarki**

Dalam *resource model* Kubernetes, terdapat beberapa objek inti pembangun aplikasi yang paling sering digunakan:

- a. **Pod**: Unit komputasi terkecil sebagai pembungkus kontainer aplikasi
- b. **Deployment**: Pengatur replika dan proses pembaruan versi untuk mencapai *high availability*
- c. **Service**: Penyedia titik akses jaringan yang stabil untuk aplikasi di dalam *Pod*
- d. **Ingress**: Pengatur rute lalu lintas eksternal masuk ke dalam klaster
- e. **ConfigMap**: Penyimpan variabel lingkungan dan konfigurasi non-sensitif
- f. **Secret**: Pengelola data sensitif seperti kredensial dan token
- g. **PersistentVolumeClaim**: *Persistent storage* untuk data aplikasi yang bersifat *stateful*
- h. **ServiceAccount**: Identitas yang digunakan oleh *Pod* untuk berinteraksi dengan API server
- i. **Role** dan **RoleBinding**: Kontrol akses berbasis peran (RBAC)
- j. **HorizontalPodAutoscaler**: Pengatur skala otomatis berdasarkan metrik penggunaan

Antarobjek ini saling terikat melalui berbagai jenis relasi semantik yang mencakup relasi kepemilikan (*ownership*), ketergantungan fungsional (*dependencies*), dan relasi konfigurasi.

II.2.3 Manifes YAML dan *Infrastructure as Code*

Pengembang berinteraksi dengan model sumber daya Kubernetes melalui pendekatan *Infrastructure as Code* (IaC) (The Kubernetes Authors 2024a). Pendekatan ini mewajibkan pengguna untuk mendeklarasikan wujud infrastruktur yang diinginkan (*desired state*) ke dalam bentuk dokumen teks berformat YAML. Setiap manifes YAML standar wajib memiliki empat atribut struktural utama agar dapat diterima oleh API Kubernetes:

1. `apiVersion`: Menentukan versi skema API pembungkus objek.
2. `kind`: Mendefinisikan jenis objek Kubernetes.
3. `metadata`: Memuat data identifikasi unik seperti nama, label pengelompokan, dan *namespace*.
4. `spec`: Mendefinisikan *desired state* atau kondisi yang diharapkan dari sebuah objek Kubernetes.

Penyusunan manifes YAML menuntut pemahaman terkait batasan skema. Pengembang harus bisa membedakan secara presisi antara atribut wajib (*required*) penentu validitas dan atribut opsional (*optional*) penambah fitur khusus. Praktik terbaik dalam penulisan IaC menyarankan modularitas dokumen, konsistensi penggunaan label, dan pemisahan logika aplikasi dari data konfigurasi.

II.3 *Large Language Models* (LLM)

II.3.1 Definisi dan Kapabilitas

Large Language Model (LLM) merupakan model kecerdasan buatan berbasis jaringan saraf berskala besar yang dilatih pada korpus teks *multi-domain* untuk mempelajari representasi bahasa alami secara mendalam (Wan dkk. 2025). Melalui proses pelatihan tersebut, LLM mampu menginterpretasikan konteks linguistik yang kompleks serta menghasilkan teks yang koheren, kontekstual, dan adaptif terhadap berbagai jenis tugas seperti *question answering*, penalaran, serta peringkasan teks.

Meskipun unggul dalam pemahaman bahasa alami, LLM memiliki keterbatasan faktual yang signifikan ketika diterapkan pada domain teknis. Beberapa keterbatasan yang disorot oleh Wan dkk. (2025) meliputi,

- a. *Hallucination*, yaitu kemampuan menghasilkan jawaban yang secara linguistik meyakinkan tetapi tidak memiliki landasan fakta yang jelas sehingga berisiko menyesatkan proses pengambilan keputusan teknis.
- b. Keterbatasan cakupan pengetahuan, LLM hanya mengandalkan data yang ada di internet sehingga tidak selalu mencakup konteks domain spesifik.
- c. Tidak adanya referensi sumber bukti, yaitu LLM tidak dapat memberikan jus-

tifikasi atau referensi eksplisit terhadap jawaban yang dihasilkan.

II.3.2 *Prompt Engineering*

Prompt engineering merupakan teknik perancangan instruksi masukan bagi LLM guna mengendalikan keluaran model berdasarkan tujuan tugas, cakupan konteks, serta batasan informasi yang ditetapkan oleh pengembang (Wan dkk. 2025). Dalam domain teknis, teknik ini tidak hanya berperan sebagai instruksi linguistik, tetapi juga sebagai representasi pengetahuan eksplisit yang mendefinisikan cara LLM memanfaatkan sumber informasi domain.

Efektivitas keluaran LLM dipengaruhi secara signifikan oleh strategi *prompt engineering* berikut,

- a. Spesifikasi tugas yang eksplisit, termasuk definisi peran model dan batasan prosedural, misalnya instruksi untuk membatasi jawaban pada proses manufaktur tertentu.
- b. Penyediaan konteks terstruktur, melalui penyisipan entitas domain, hubungan semantik, serta cuplikan dokumen terkait yang relevan dengan pertanyaan pengguna.
- c. Pemberian batasan (*constraint injection*) yang membatasi ruang solusi secara eksplisit pada dokumen tertentu yang valid.
- d. Penyusunan format jawaban, misalnya dalam bentuk tabel, daftar berhierarki, atau penjelasan berbasis pembuktian prosedural.

Strategi tersebut tidak hanya mengontrol keluaran, tetapi juga mencegah LLM menafsirkan konteks di luar pengetahuan domain sehingga berperan sebagai mekanisme mitigasi terhadap fenomena *hallucination*.

II.4 *Retrieval-Augmented Generation (RAG)*

Retrieval-Augmented Generation (RAG) merupakan arsitektur pemrosesan bahasa alami yang mengombinasikan mekanisme *Information Retrieval (IR)* dengan kemampuan generatif *Large Language Models (LLM)*. Pendekatan ini dikembangkan untuk mengatasi keterbatasan memori parametris pada model bahasa, khususnya dalam menangani tugas yang bersifat *knowledge-intensive*, yaitu model rentan menghasilkan informasi yang keliru atau tidak berbasis fakta (*hallucination*) (Moreno-Cediel dkk. 2025).

Berbeda dengan metode generatif konvensional yang hanya mengandalkan pengetahuan tersimpan selama proses pelatihan, RAG memanfaatkan basis pengetahuan non-parametris melalui proses pencarian (*retrieval*) sehingga jawaban yang dihasilkan lebih akurat dan relevan terhadap konteks masukan pengguna.

II.4.1 Arsitektur *Retrieval-Augmented Generation*

Arsitektur *Retrieval-Augmented Generation* (RAG) merupakan pendekatan hibrida yang menggabungkan kemampuan penalaran *Large Language Models* (LLM) dengan basis pengetahuan eksternal melalui proses pencarian semantik. Struktur ini dirancang untuk mengurangi *hallucination*, meningkatkan akurasi fakta, serta mendukung pembaruan pengetahuan secara dinamis (Gao dkk. 2024). Secara umum, alur kerja RAG terdiri atas empat tahapan teknis utama yang berurutan dan saling bergantung, yaitu *Data Ingestion & Indexing*, *Retrieval*, *Augmentation*, dan *Generation*.

1. *Data Ingestion & Indexing*

Tahapan ini bersifat *offline* dan mencakup lima langkah berurutan: (a) *akuisisi data* dari sumber dokumentasi teknis atau spesifikasi API, (b) *preprocessing* berupa normalisasi format dan ekstraksi metadata, (c) *chunking* untuk memecah dokumen menjadi unit 200–500 token, (d) *embedding* menggunakan model seperti MiniLM atau BERT untuk mengubah setiap *chunk* menjadi representasi vektor, serta (e) *indexing* ke *vector store* (misalnya FAISS atau Chroma) agar pencarian berbasis kedekatan semantik dapat dilakukan secara efisien.

2. *Retrieval*

Proses *retrieval* dilakukan ketika pengguna mengajukan kueri. Sistem akan mencari konteks paling relevan melalui mekanisme berikut,

a. *Embedding Kueri*

Kueri pengguna diubah menjadi vektor menggunakan model *embedding* yang sama dengan proses *indexing*.

b. *Similarity Search*

Dilakukan pencocokan vektor menggunakan metrik jarak seperti *cosine similarity*, *dot-product*, atau *Euclidean distance*.

c. *Top-k Selection*

Sistem memilih sejumlah dokumen teratas (umumnya $k = 3$ atau $k = 5$) yang memiliki skor kemiripan tertinggi dan mengandung konteks yang berpotensi menjawab pertanyaan.

Kualitas *retrieval* yang baik menjadi faktor kunci dalam menurunkan risiko *hallucination* pada LLM karena jawaban dibatasi oleh fakta yang relevan.

3. *Augmentation*

Tahap *augmentation* bertugas menggabungkan konteks hasil *retrieval* ke dalam *prompt* secara sistematis sehingga dapat dipahami oleh LLM. Prosesnya

mencakup tahapan berikut,

a. *Concatenation*

Sistem menggabungkan pertanyaan pengguna dengan konten dokumen yang ditemukan.

b. *Formatting*

Struktur informasi dapat diberikan dalam format teks baku, tabel, *JSON schema*, *instruction-style*, atau bentuk terstruktur lainnya.

c. *Constraint Injection*

Sistem memberikan batasan eksplisit, misalnya “*jawab hanya menggunakan konteks yang diberikan, jangan menambah informasi baru*”, untuk mencegah LLM berhalusinasi.

Meskipun efektif untuk skenario umum, RAG tradisional yang berbasis *vector similarity* memiliki keterbatasan mendasar: dokumen dipecah menjadi potongan teks (*chunks*) independen sehingga relasi struktural dan hierarki antarentitas sering kali hilang dalam proses *chunking* (Gao dkk. 2024).

II.4.2 Relevansi RAG terhadap Dokumentasi Kubernetes API

Dokumentasi Web API, termasuk Kubernetes, memiliki karakteristik unik berupa kombinasi deskripsi sintaks terstruktur (misalnya skema JSON/YAML) dan penjelasan semantik dalam bentuk teks alami. Penelitian Kotstein dan Decker (2024) menunjukkan bahwa pemrosesan semantik terhadap dokumentasi API dapat dilakukan tanpa ontologi formal dengan memetakan makna berdasarkan nama parameter, struktur hierarkis, dan deskripsi bahasa alami.

Dalam konteks Kubernetes, komponen API seperti *Pod*, *Service*, dan *Deployment* direpresentasikan melalui skema bertingkat yang bersifat kompleks dan memiliki ketergantungan semantik. Oleh karena itu, implementasi RAG pada domain ini membutuhkan,

1. *Schema-aware semantic chunking*

Teknik ini memastikan bahwa pemotongan dokumen (*chunking*) dilakukan mengikuti struktur skema objek API. Setiap potongan teks harus mewakili satu entitas atau blok atribut yang utuh sehingga relasi antar*field* tidak terpisah atau kehilangan konteks. Dengan demikian, representasi vektor yang dihasilkan tetap mempertahankan hierarki objek.

2. *Path-based serialization*

Teknik ini menyimpan setiap elemen API beserta jalur lengkapnya dalam struktur objek sehingga posisi semantiknya tetap terpelihara.

3. *Hybrid index retrieval*

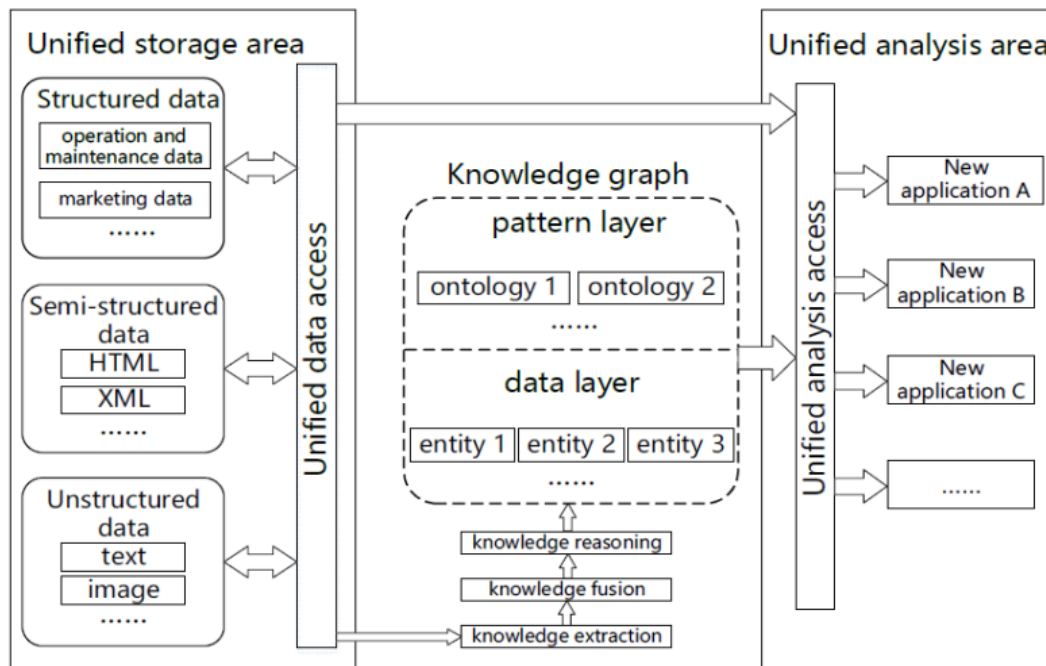
Pendekatan ini mengombinasikan pencarian berbasis kesamaan semantik teks dengan pembobotan struktur sintaksis dari objek API. Selain memanfaatkan *embedding* untuk menemukan kemiripan makna antarteks, sistem juga memberikan bobot lebih tinggi pada elemen struktural kunci (misalnya *field* wajib atau relasi antarobjek). Hasilnya, mekanisme *retrieval* menghasilkan konteks yang tidak hanya relevan secara linguistik, tetapi juga benar secara hierarki dan fungsi API.

Dengan demikian, penerapan RAG pada dokumentasi Kubernetes tidak hanya berfungsi sebagai mesin pencarian semantik, tetapi juga menjadi pendekatan mitigasi *hallucination*.

II.5 Knowledge Graph dan GraphRAG

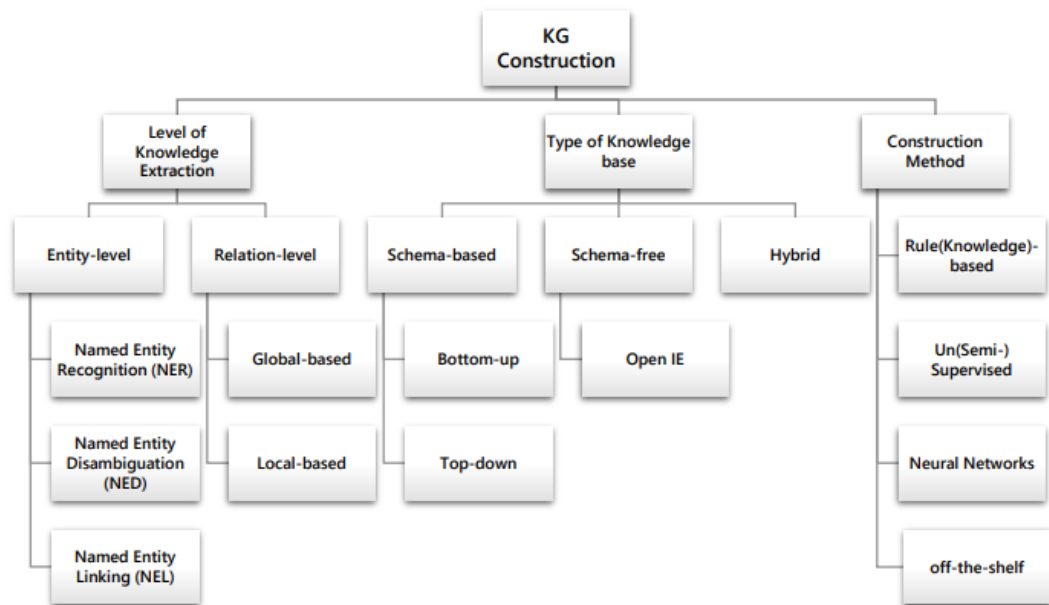
II.5.1 Definisi dan Struktur

Knowledge Graph (KG) merupakan representasi pengetahuan dalam bentuk graf yang tersusun dari entitas (*node*) dan relasi (*edge*) yang memiliki makna semantik (Hofer dkk. 2024). Basis data konvensional umumnya mengandalkan skema yang statis. Sebaliknya, KG bersifat *schema-flexible* sehingga lebih mudah untuk mengintegrasikan berbagai jenis data, baik yang terstruktur, semi-terstruktur, maupun tidak terstruktur, seperti yang ditunjukkan pada Gambar II.2. Struktur KG umumnya dinyatakan melalui *triples* berbentuk (subjek–predikat–objek).



Gambar II.2 Arsitektur *Knowledge Graph* (J. Yu dkk. 2021)

Abu-Salih (2021) mengklasifikasikan konstruksi KG ke dalam dua dimensi utama seperti yang bisa dilihat pada Gambar II.3. Dimensi pertama membagi KG berdasarkan jenis basis pengetahuan yang digunakan, yaitu *schema-based*, *schema-free*, dan *hybrid*. Pendekatan *schema-based* membangun KG berdasarkan ontologi atau skema yang telah didefinisikan sebelumnya; pendekatan *schema-free* menggunakan teknik ekstraksi terbuka (*Open Information Extraction*) tanpa panduan ontologi; sedangkan *hybrid* menggabungkan keduanya. Dimensi kedua mengklasifikasikan metode konstruksi berdasarkan teknik yang digunakan, yaitu *rule/knowledge-based* yang mengandalkan aturan eksplisit yang dirancang oleh pakar domain, *learning-based* yang memanfaatkan teknik statistik dan supervised learning, pendekatan berbasis jaringan saraf (*neural networks*), serta pemanfaatan alat NLP siap pakai (*off-the-shelf tools*).



Gambar II.3 Taksonomi pada pembangunan *Knowledge Graph* (Abu-Salih 2021)

Posisi penelitian ini dalam taksonomi tersebut terletak pada kombinasi *schema-based* dengan metode *rule/knowledge-based*: sumber pengetahuan berupa spesifikasi OpenAPI Kubernetes yang bersifat formal dan terstruktur, sementara konstruksi KG dilakukan melalui aturan transformasi deterministik yang dirancang secara eksplisit berdasarkan semantik domain. Berbeda dengan pendekatan *schema-free* berbasis LLM yang bersifat stokastik dan menghasilkan struktur graf yang bervariasi antar-eksekusi (Pan dkk. 2024), pendekatan *rule-based* menghasilkan representasi yang konsisten dan dapat direproduksi selama sumber skemanya tidak berubah.

Sifat deterministik KG berbasis skema ini membuka peluang penggunaan ganda:

selain sebagai basis *retrieval*, graf yang sama dapat dimanfaatkan sebagai validator struktural. Rahman dkk. (2023) menemukan bahwa sekitar 80% miskonfigurasi Kubernetes bersifat struktural sehingga dapat dideteksi melalui analisis statis (*static analysis*) tanpa menjalankan kluster aktif. Temuan ini memvalidasi prinsip pendekatan validasi berbasis skema: jika spesifikasi *required fields* setiap objek Kubernetes sudah terencode dalam representasi graf, pemeriksaan kelengkapan atribut wajib dapat dilakukan langsung terhadap graf tersebut, tanpa memerlukan mekanisme *dry-run* yang bergantung pada infrastruktur nyata.

II.5.2 GraphRAG: Integrasi *Knowledge Graph* dan *Large Language Models*

Large Language Models (LLM) dan *Knowledge Graphs* (KG) merupakan dua paradigma representasi pengetahuan yang saling melengkapi. LLM unggul dalam pemrosesan bahasa alami serta generalisasi lintas tugas tetapi rentan terhadap halusinasi fakta. Sebaliknya, KG menawarkan representasi pengetahuan faktual dalam bentuk struktur graf eksplisit yang kuat untuk penalaran simbolik namun kurang mampu menangani pemahaman bahasa alami yang kompleks (Pan dkk. 2024).

Untuk mengatasi kelemahan LLM tersebut, pendekatan *Retrieval-Augmented Generation* (RAG) konvensional sering digunakan. Namun, RAG tradisional bekerja dengan cara memecah dokumen menjadi potongan teks (*chunks*) independen sehingga sering kali menghilangkan informasi struktural dan relasi esensial antarentitas (Ma dkk. 2025). Sebagai solusi komprehensif atas keterbatasan tersebut, muncullah konsep *Graph Retrieval-Augmented Generation* (GraphRAG).

Berbeda dengan pencarian semantik biasa berbasis vektor teks, GraphRAG memanfaatkan mekanisme penelusuran graf (*graph traversal*) untuk mengambil konteks terstruktur secara utuh sebelum diberikan ke LLM (Han dkk. 2024). Mekanisme ini secara langsung mengekstraksi pengetahuan relevan dari data graf dengan merangkai subgraf atau jalur relasional berdasarkan kueri pengguna (Ma dkk. 2025).

Secara formal, integrasi generatif ini dimodelkan sebagai fungsi yang bersyarat terhadap pengetahuan graf:

$$p_{\theta}(y \mid x, \mathcal{K}) = \sum_{z \in \mathcal{Z}(x, \mathcal{K})} p_{\theta}(y \mid x, z) p(z \mid x, \mathcal{K}), \quad (\text{II.1})$$

dengan $\mathcal{K} = \{(h, r, t) \subseteq \mathcal{E} \times \mathcal{R} \times \mathcal{E}\}$ merepresentasikan KG sebagai himpunan *triples*, dan $\mathcal{Z}(x, \mathcal{K})$ merupakan himpunan subgraf atau fakta terstruktur yang ditarik dari graf berdasarkan kueri x . Dalam skema ini, KG bertindak sebagai ruang pengetahuan non-parametrik penyedia panduan penalaran (*reasoning guidelines*),

sedangkan LLM bertindak sebagai generator parametrik (Ma dkk. 2025).

Pendekatan GraphRAG ini menjadi sangat krusial dalam domain berskala kompleks seperti manajemen konfigurasi Kubernetes. Dengan memetakan hierarki objek dan referensi silang ke dalam graf, algoritma *traversal* dapat merangkai jejak penalaran (*reasoning path*) yang kohesif. Namun demikian, kedalaman *traversal* menjadi faktor kritis yang memengaruhi kualitas konteks. Jika penelusuran yang dilakukan terlalu dalam, maka akan memicu *information overload* karena graf yang berkembang secara eksponensial memungkinkan masuknya informasi yang tidak relevan (Han dkk. 2024), sedangkan penelusuran yang terlalu dangkal menyebabkan *under-retrieval* sehingga konteks relasional yang dibutuhkan untuk penalaran *multi-hop* tidak terjangkau saat proses penelusuran graf.

Solusi atas dilema kedalaman ini adalah mekanisme *intent-adaptive retrieval*, yaitu strategi yang menyesuaikan kedalaman atau strategi *traversal* berdasarkan klasifikasi maksud (*intent*) dari kueri pengguna. Literatur NLP dan IR menunjukkan bahwa mengklasifikasi tipe kueri sebelum *retrieval* meningkatkan presisi konteks secara signifikan (Zhao dkk. 2024): kueri definitif (*explain*) memerlukan konteks yang lebih dangkal dan terfokus, sedangkan kueri generatif atau relasional (*generate_yaml*, *trace_relationship*) memerlukan penelusuran yang lebih dalam untuk menangkap dependensi multi-tingkat antarobjek. Dengan demikian, klasifikasi *intent* bukan sekadar pra-pemrosesan kueri, melainkan mekanisme kendali yang menentukan batas eksplorasi kontekstual dalam ruang graf.

II.6 Metrik Evaluasi Sistem Informasi Retrieval

Evaluasi sistem GraphRAG membutuhkan landasan metrik yang bersumber dari literatur *Information Retrieval* (IR) dan NLP. Bagian ini membahas konsep metrik standar yang menjadi acuan beserta formulasi perhitungannya. Selain metrik standar tersebut, tiga metrik *graph traversal* (*Hop Accuracy*, *Path Coverage*, dan *Retrieval-Grounded Answer*) diadaptasi dan dioperasionalisasikan ke konteks grafis Kubernetes; definisi lengkapnya disajikan pada Subbab II.6.4. Seluruh metrik dalam penelitian ini dinormalisasi ke rentang $[0, 1]$ dengan orientasi seragam sehingga nilai yang lebih tinggi menandakan kualitas yang lebih baik.

II.6.1 Answer Quality

Evaluasi kualitas jawaban sistem RAG mencakup metrik kesesuaian isi jawaban maupun validitas struktural keluaran. Untuk *fixture* bertipe generatif (*yaml_gen*), dimensi ini juga mencakup validitas sintaksis dan kesesuaian skema keluaran YAML.

1. Faithfulness

Mengukur sejauh mana jawaban yang dihasilkan oleh LLM bersumber dari konteks yang diambil, bukan dari pengetahuan parametrisnya sendiri (Es dkk. 2023). Metrik ini secara langsung mengukur risiko halusinasi: jawaban yang tinggi *faithfulness*-nya berarti setiap klaim dapat ditelusuri ke konteks yang disediakan. Dalam penelitian ini, formulasi RAGAS diadaptasi ke level node graf, sehingga *faithfulness* dihitung sebagai proporsi node kunci pada *ground truth* yang disebutkan secara eksplisit dalam jawaban:

$$\text{Faithfulness} = \frac{|N_{\text{kunci}} \cap N_{\text{jawaban}}|}{|N_{\text{kunci}}|}, \quad (\text{II.2})$$

dengan N_{kunci} adalah himpunan node kunci pada *ground truth* dan N_{jawaban} adalah himpunan node yang disebutkan dalam jawaban.

2. *Answer Relevance*

Mengukur kesesuaian semantik antara jawaban yang dihasilkan dan jawaban acuan (*ground truth*) (Es dkk. 2023). Implementasinya berbasis kemiripan kosinus antara representasi vektor *embedding* jawaban sistem $\mathbf{e}_a$ dan *embedding* jawaban acuan $\mathbf{e}_g$:

$$\text{Answer Relevance} = \cos(\mathbf{e}_a, \mathbf{e}_g) = \frac{\mathbf{e}_a \cdot \mathbf{e}_g}{\|\mathbf{e}_a\| \|\mathbf{e}_g\|}. \quad (\text{II.3})$$

3. *Syntactic Validity*

Mengukur proporsi keluaran YAML yang dapat diurai tanpa galat menggunakan *parser* standar seperti `pyyaml`. Metrik ini bersifat biner pada level *fixture*, yaitu bernilai 1 jika dokumen YAML valid dan 0 jika tidak (Moreno-Cediel dkk. 2025):

$$\text{Syntactic Validity} = \frac{|\text{YAML lolos parsing}|}{|\text{total fixture yaml_gen}|}. \quad (\text{II.4})$$

4. *Schema Compliance*

Mengukur kesesuaian YAML terhadap spesifikasi skema yang berlaku, dalam konteks ini spesifikasi OpenAPI Kubernetes v1.30 (The Kubernetes Authors 2024b). Sebagaimana *syntactic validity*, metrik ini bersifat biner pada level *fixture*:

$$\text{Schema Compliance} = \frac{|\text{YAML lolos validasi skema}|}{|\text{total fixture yaml_gen}|}. \quad (\text{II.5})$$

Metrik ketiga dan keempat ini hanya relevan untuk *fixture* bertipe *yaml_gen*.

II.6.2 Retrieval Quality

Evaluasi komponen *retrieval* menggunakan metrik standar IR yang mengukur ketepatan dan kelengkapan node yang diambil selama *traversal* graf. Misalkan $N_{\text{retrieved}}$ adalah himpunan node yang diambil dan N_{relevant} adalah himpunan node relevan menurut *ground truth*.

1. Precision@k dan Recall@k

Precision@k mengukur proporsi node relevan di antara node yang diambil, sedangkan *Recall@k* mengukur proporsi node relevan yang berhasil ditemukan dari seluruh node relevan yang ada (Zhao dkk. 2024):

$$\text{Precision@k} = \frac{|N_{\text{retrieved}} \cap N_{\text{relevant}}|}{|N_{\text{retrieved}}|}, \quad \text{Recall@k} = \frac{|N_{\text{retrieved}} \cap N_{\text{relevant}}|}{|N_{\text{relevant}}|}. \quad (\text{II.6})$$

2. F1-Score@k

Merupakan rata-rata harmonik antara *Precision@k* dan *Recall@k*, memberikan gambaran seimbang tentang *trade-off* antara ketepatan dan kelengkapan (Zhao dkk. 2024):

$$\text{F1@k} = 2 \times \frac{\text{Precision@k} \times \text{Recall@k}}{\text{Precision@k} + \text{Recall@k}}. \quad (\text{II.7})$$

3. NDCG@k (Normalized Discounted Cumulative Gain)

Mengevaluasi kualitas peringkat hasil *retrieval* dengan memberikan bobot lebih tinggi pada node relevan yang muncul di posisi atas (Es dkk. 2023). Nilai *Discounted Cumulative Gain* (DCG) dinormalisasi terhadap nilai idealnya (IDCG):

$$\text{NDCG@k} = \frac{\text{DCG@k}}{\text{IDCG@k}}, \quad \text{DCG@k} = \sum_{i=1}^k \frac{rel_i}{\log_2(i+1)}, \quad (\text{II.8})$$

dengan rel_i bernilai 1 jika node pada posisi ke- i relevan dan 0 jika tidak. Metrik ini penting dalam konteks RAG karena node relevan yang terpotong akibat batasan token LLM tidak memberikan manfaat apapun.

II.6.3 Reasoning Quality

Evaluasi dimensi penalaran mengukur sejauh mana jawaban sistem dapat diverifikasi melalui konteks yang diambil, bukan semata dari pengetahuan parametrik LLM. Metrik standar yang digunakan adalah *Grounding Score*:

1. Grounding Score

Mengukur proporsi istilah teknis Kubernetes dalam jawaban yang dapat dite-

lusuri secara langsung ke konteks yang diambil, bukan berasal dari pengetahuan bawaan model (Es dkk. 2023). Nilai tinggi menunjukkan bahwa jawaban berbasis konteks graf, bukan halusinasi:

$$\text{Grounding Score} = \frac{|T_{\text{grounded}}|}{|T_{\text{jawaban}}|}, \quad (\text{II.9})$$

dengan T_{jawaban} adalah himpunan istilah Kubernetes dalam jawaban dan T_{grounded} adalah himpunan istilah yang dapat ditelusuri ke konteks yang di-*retrieve*.

Kerangka evaluasi tiga dimensi (AnsQ, RetQ, ReaQ) ini diperluas dengan tiga metrik yang diadaptasi untuk konteks *graph traversal* Kubernetes, sebagaimana diuraikan pada Subbab II.6.4.

II.6.4 Metrik Evaluasi Berbasis *Graph Traversal*

Metrik standar IR tidak memadai untuk mengevaluasi *GraphRAG* karena mengabaikan kelengkapan jalur relasional (*edge*) dan validitas konteks graf. Sebagai solusi, penelitian ini mengoperasionalkan tiga metrik *graph traversal* khusus Kubernetes: *Hop Accuracy* dan *Path Coverage*, serta *Retrieval-Grounded Answer* (RGA) yang diadaptasi dari *AR-metric* (Han dkk. 2024).

1. *Hop Accuracy*

Mengukur ketepatan kedalaman penelusuran (*traversal*) yang dilakukan sistem terhadap kedalaman yang dibutuhkan menurut *ground truth*. Metrik ini memvalidasi apakah mekanisme *intent-adaptive depth* berhasil memetakan tipe kueri ke kedalaman yang tepat:

$$\text{Hop Accuracy} = \frac{|\{f \mid d_{\text{pred}}(f) = d_{\text{gt}}(f)\}|}{|F|}, \quad (\text{II.10})$$

dengan F adalah himpunan seluruh *fixture*, $d_{\text{pred}}(f)$ adalah kedalaman penelusuran yang dieksekusi sistem untuk *fixture* f (operasionalisasi: jumlah langkah dalam *reasoning_path* yang dikembalikan), dan $d_{\text{gt}}(f)$ adalah kedalaman yang dibutuhkan menurut anotasi *ground truth* (operasionalisasi: jumlah *edge* dalam *expected_path* pada *fixture*). Mekanisme *intent-adaptive depth* (T2) diverifikasi jika $d_{\text{pred}}(f) = d_{\text{gt}}(f)$ untuk sebagian besar *fixture*.

2. *Path Coverage*

Mengukur kelengkapan konteks relasional yang diambil dengan mempertimbangkan baik node maupun *edge* yang menghubungkannya. Formulasi berbasis *edge* ini lebih diskriminatif daripada cakupan berbasis node semata, karena

sebuah node dapat diambil tanpa relasi penghubung yang tepat:

$$\text{Path Coverage} = \frac{|E_{\text{retrieved}} \cap E_{\text{relevant}}|}{|E_{\text{relevant}}|}, \quad (\text{II.11})$$

dengan $E_{\text{retrieved}}$ adalah himpunan *edge* pada subgraf hasil penelusuran dan E_{relevant} adalah himpunan *edge* relevan menurut *ground truth*.

3. **Retrieval-Grounded Answer (RGA)**

Mengukur proporsi jawaban yang benar *dan* didukung oleh konteks yang berhasil diambil. Metrik ini diadaptasi dari *Answer Recall metric* (AR-metric) (Han dkk. 2024): sebuah jawaban yang benar tetapi tidak didukung oleh konteks graf yang relevan diklasifikasikan sebagai tebakan parametrik, bukan bukti keunggulan sistem:

$$\text{RGA} = \frac{|\{f \mid \text{correct}(f) = 1 \wedge \text{grounded}(f) = 1\}|}{|F|}, \quad (\text{II.12})$$

dengan $\text{correct}(f) = 1$ jika jawaban sistem untuk *fixture* f dinilai benar berdasarkan *faithfulness* dan *answer relevance*, dan $\text{grounded}(f) = 1$ jika *path coverage* sistem untuk *fixture* tersebut melebihi ambang batas minimal yang ditetapkan ($\geq 0,5$).

Ketiga metrik ini bersifat komplementer terhadap metrik standar IR: *Hop Accuracy* memvalidasi mekanisme *intent-adaptive depth* (T2), *Path Coverage* memvalidasi kelengkapan struktural KG (T1), dan RGA memvalidasi bahwa keunggulan sistem bukan sekadar artefak dari kemampuan generatif LLM (T2). Dimensi RetQ mencakup F1@k, NDCG@k, dan *Path Coverage*; dimensi ReaQ mencakup *Hop Accuracy* dan *Grounding Score*.

II.7 Penelitian Terkait

Bagian ini merangkum tiga penelitian terdahulu yang paling relevan terhadap pendekatan yang dikembangkan dalam penelitian ini. Ketiga penelitian tersebut dipilih karena secara bersama-sama merepresentasikan perkembangan mutakhir penerapan *Knowledge Graph* dan RAG pada domain teknis spesifik, sehingga membentuk pijakan perbandingan langsung terhadap kontribusi yang diajukan.

II.7.1 GraphRAG untuk Platform IoT Domain-Spesifik

H.-H. Yu dkk. (2025) mengembangkan mesin dialog tanya jawab berbasis GraphRAG untuk membantu pengguna mengambil informasi secara presisi dari *Civil IoT Taiwan Data Service Platform*. Penelitian ini membangun *Knowledge Graph* secara semi-otomatis menggunakan GPT-4.1 untuk mengekstraksi entitas dan rela-

si dari dokumen platform, kemudian menggunakan model Llama-3-TAIDE-LX-8B untuk menerjemahkan pertanyaan bahasa alami pengguna ke jalur kueri graf. Sistem berhasil melampaui nilai F1 sebesar 0,7 pada kueri multi-entitas dan menunjukkan efektivitas dalam menghindari halusinasi pada kasus data yang tidak tersedia.

Keterbatasan utamanya adalah *Knowledge Graph* yang bersifat statis tanpa mekanisme pembaruan dinamis, serta biaya adaptasi yang tinggi untuk platform lain. Perbedaan kunci dengan penelitian ini terletak pada sumber data dan topologi graf: penelitian H.-H. Yu dkk. (2025) membangun graf dari narasi dokumen tidak terstruktur dengan entitas IoT yang beragam, sedangkan penelitian ini membangun graf dari skema OpenAPI Kubernetes yang bersifat deterministik dengan taksonomi relasi yang didefinisikan secara eksplisit.

II.7.2 RAG Hibrida KG-Vektor untuk Manufaktur Pintar

Wan dkk. (2025) mengusulkan kerangka kerja Q&A berbasis RAG hibrida yang menggabungkan *Knowledge Graph* Neo4j dengan pencarian vektor untuk domain *Design for Additive Manufacturing* (DfAM). Konstruksi KG dilakukan berdasarkan metadata dari 69 publikasi teknis, diperkuat dengan *Semantic Alignment* dan *Prompt Enhancement* (PE) berbasis konstrain domain. Kombinasi pendekatan hibrida tersebut dengan GPT-4o menghasilkan akurasi 77,8% *Exact Match* dan 76,5% *Context Precision*, melampaui pendekatan RAG vektor atau graf secara mandiri.

Keterbatasan utamanya adalah latensi tinggi akibat proses pencarian hibrida yang kompleks, serta KG yang belum mendukung pembaruan data secara dinamis. Penelitian ini mengadopsi konsep hibrida KG-vektor yang sama, namun memperluas mekanisme *retrieval* dengan pencocokan eksak berbasis nama *resource* dan penelusuran multi-hop mengikuti taksonomi 18 jenis relasi Kubernetes yang dirancang secara domain-spesifik.

II.7.3 HSG-RAG: Konstruksi Basis Pengetahuan Hierarkis untuk Domain Teknis

Lu dkk. (October 2025) mengembangkan HSG-RAG (*Hierarchical Semantic Graph Retrieval Augmented Generation*), sebuah metode konstruksi dan pencarian basis pengetahuan bertingkat untuk mendukung pengembangan sistem *embedded*. Graf dibangun dari korpus dokumen teknis SDK dan *datasheet* tiga platform perangkat keras (ESP32S3, STM32F103, JL701) menggunakan LLM, mencakup dua lapisan relasi: jarak dekat antarparagraf dan jarak jauh berbasis kemiripan semantik. Strategi pencarian *top-down* memecah kueri pengguna menjadi sub-kueri yang ditelusuri secara bertahap di dalam graf, menghasilkan jawaban yang lebih spesifik, ringkas,

dan lengkap dibandingkan VanillaRAG maupun GraphRAG konvensional.

Keterbatasan utamanya adalah belum adanya mekanisme verifikasi otomatis kebenaran kode yang dihasilkan, serta cakupan evaluasi yang masih terbatas pada beberapa platform perangkat keras. Penelitian ini mengadopsi konsep dekomposisi kueri dan penelusuran bertingkat yang serupa, namun menerapkannya pada skema Kubernetes yang memiliki hierarki objek lebih terstruktur dan jenis relasi antar *resource* yang terdefinisi secara formal dalam spesifikasi OpenAPI.

Ketiga penelitian terdahulu secara konsisten memvalidasi efektivitas pendekatan GraphRAG dalam domain teknis spesifik, sekaligus mengidentifikasi dua pola keterbatasan yang berulang: (1) KG bersifat statis tanpa mekanisme pembaruan dinamis, dan (2) taksonomi relasi yang belum disesuaikan dengan karakteristik skema target secara mendalam. Penelitian ini merespons kedua pola keterbatasan tersebut secara domain-spesifik untuk Kubernetes.

Berdasarkan kajian literatur yang telah dipaparkan, teridentifikasi dua kesenjangan utama yang menjadi landasan penelitian ini. Pertama, pendekatan RAG berbasis vektor konvensional menunjukkan keterbatasan signifikan untuk domain Kubernetes karena gagal menangkap relasi hierarkis dan referensial yang menentukan validitas konfigurasi (Wan dkk. 2025). Kedua, meskipun integrasi *Knowledge Graph* dalam RAG telah menunjukkan peningkatan signifikan pada domain umum, belum ada rancangan taksonomi relasi dan mekanisme *traversal* yang secara spesifik disesuaikan untuk skema OpenAPI Kubernetes. Bab selanjutnya menganalisis karakteristik data, kebutuhan sistem, dan alternatif pendekatan solusi untuk menjawab kedua kesenjangan tersebut.

BAB III

ANALISIS MASALAH

III.1 Analisis Kondisi Saat Ini

Bagian ini menganalisis permasalahan yang muncul dalam proses pengembangan sistem *cloud-native* berbasis Kubernetes, ditinjau dari dua sudut utama yaitu pemahaman masalah bisnis (*Business Understanding*) dan kondisi data yang menjadi sumber permasalahan (*Data Understanding*). Untuk memastikan permasalahan yang dirumuskan bersifat valid dan bukan sekadar asumsi teoritis, analisis ini dibangun atas triangulasi tiga jenis bukti, yaitu data sekunder berupa survei industri dan studi empiris terpublikasi, data primer berupa wawancara praktisi, serta analisis langsung terhadap spesifikasi OpenAPI Kubernetes.

III.1.1 Pemahaman Masalah Bisnis (*Business Understanding*)

Transformasi digital di berbagai industri mendorong adopsi arsitektur *cloud-native* berbasis *microservices* yang dikelola melalui Kubernetes. Dalam praktiknya, Kubernetes tidak hanya berfungsi sebagai alat operasional, tetapi juga sebagai aset strategis yang menentukan kecepatan dan stabilitas siklus pengembangan perangkat lunak (*software development lifecycle*). Kompleksitas konfigurasi yang tinggi, kurva pembelajaran yang curam, serta kebutuhan untuk melakukan *onboarding* bagi *engineer* baru menjadikan Kubernetes sebuah titik krusial dalam kapasitas operasional.

Permasalahan terkait pengelolaan konfigurasi Kubernetes tidak hanya bersifat teknis, tetapi menimbulkan dampak bisnis berupa biaya operasional dan risiko produksi. Beberapa permasalahan utama yang diidentifikasi adalah sebagai berikut:

1. B01 - Inefisiensi Waktu (*Operational Cost*)

Dokumentasi Kubernetes bersifat luas dan terfragmentasi sehingga *developer/DevOps* membutuhkan waktu yang cukup lama untuk menelusuri relasi antarobjek Kubernetes. Survei industri mencatat bahwa kerumitan konfigurasi YAML menjadi hambatan skalabilitas utama bagi mayoritas pengguna Kubernetes (Cloud Native Computing Foundation 2023). Beban ini bersifat struktural, sebagaimana ditunjukkan oleh analisis spesifikasi pada Subbab III.1.2.3

yang mengungkapkan bahwa hampir 60% definisi skema saling bergantung melalui referensi silang. Temuan ini diperkuat wawancara praktisi yang melaporkan penggunaan asisten LLM secara intensif dalam pekerjaan harian, sebagian mencapai 15–20 kueri per hari, untuk menelusuri relasi dan mencari konteks konfigurasi. Waktu pencarian informasi yang berlebih menjadi pemborosan sumber daya, terutama pada posisi *engineer* dengan biaya tenaga kerja tinggi.

2. B02 - Kesenjangan Pengetahuan (*Knowledge Gap*)

Dokumentasi statis tidak sepenuhnya mendukung cara berpikir asosiatif manusia ketika memahami struktur spesifikasi. Sistem generatif berbasis LLM pun masih sering menghasilkan *hallucinated fields* karena tidak memiliki pemahaman skematik. Halusinasi merupakan fenomena yang terdokumentasi dan terukur pada model generatif (Ji dkk. 2023), dan studi empiris terhadap pembuatan kode oleh LLM menunjukkan tingkat ketidaktepatan yang signifikan pada keluaran teknis (Liu dkk. 2024). Wawancara praktisi mengonfirmasi pola ini secara konsisten, yaitu jawaban LLM yang gagal dijalankan, berbeda antarmodel, serta membuat asumsi yang tidak sesuai konteks pengguna. Hal ini bisa memperburuk risiko kesalahan konfigurasi.

3. B03 - Risiko Kesalahan Konfigurasi (*Operational Risk*)

Kesalahan sintaksis atau penempatan atribut berkas konfigurasi YAML yang tidak sesuai spesifikasi dapat menyebabkan *deployment failure* hingga *downtime*. Studi empiris terhadap manifes Kubernetes *open-source* menemukan bahwa mayoritas miskonfigurasi bersifat struktural sehingga dapat dideteksi melalui analisis statis (Rahman dkk. 2023). *Downtime* produksi berimplikasi langsung pada kerugian finansial, reputasi layanan, dan penundaan proses bisnis. Praktisi mengonfirmasi risiko ini melalui pengalaman keluaran YAML yang tidak konsisten serta konfigurasi yang tidak memperbarui variabel terbaru.

Ketiga butir *business understanding* tersebut menjadi dasar dalam penyusunan rumusan masalah penelitian. Poin B01 mengenai inefisiensi waktu akibat dokumentasi yang terfragmentasi menunjukkan adanya kebutuhan terhadap representasi pengetahuan yang lebih terstruktur. Hal ini dirumuskan menjadi Rumusan Masalah 1, yaitu pembangunan *knowledge graph* dari spesifikasi OpenAPI Kubernetes guna memetakan objek dan relasi hierarkis antarobjek konfigurasi secara komprehensif. Sementara itu, poin B02 yang menyoroti kesenjangan pemahaman skematik dan risiko *hallucination* pada LLM, serta poin B03 mengenai risiko kesalahan konfigurasi akibat ketidaklengkapan atribut wajib YAML, secara bersama-sama mendasari Rumusan Masalah 2. Rumusan masalah ini difokuskan pada perancangan mekanisme

GraphRAG yang memanfaatkan representasi struktural *knowledge graph* melalui penelusuran jalur relasional (*intent-adaptive depth traversal*) untuk meningkatkan kualitas *retrieval*. Selanjutnya, penelitian ini akan mengidentifikasi keunggulan GraphRAG dibandingkan dengan Vector RAG dan Vanilla LLM dalam menyelesaikan berbagai persoalan yang telah diidentifikasi pada B01–B03.

III.1.2 Pemahaman Data (*Data Understanding*)

Bagian ini menjelaskan karakteristik data dokumentasi API Kubernetes yang menjadi sumber utama dalam proses konstruksi *Graph-based Retrieval Augmentation*. Analisis mencakup identitas data, struktur, pola hubungan antarobjek Kubernetes, keterbatasan kualitas, serta seleksi data relevan yang digunakan dalam penelitian.

III.1.2.1 Sumber dan Spesifikasi Data (*Data Source & Specifications*)

Data penelitian diperoleh dari repositori resmi Kubernetes pada GitHub `kubernetes/kubernetes`. Data diambil dalam bentuk berkas `swagger.json` yang mengikuti standar *OpenAPI Specification* (OAS). Penelitian ini menggunakan versi Kubernetes **v1.30** sebagai acuan. Berkas `swagger.json` memiliki ukuran rata-rata 3,757 MB dan memuat 2.500–3.500 definisi API. Kompleksitas tersebut menjustifikasi penggunaan metode ekstraksi otomatis karena pendekatan manual berpotensi menghasilkan kesalahan interpretasi struktur dan kehilangan relasi semantik antarobjek Kubernetes.

III.1.2.2 Analisis Struktur Dokumen

Berdasarkan analisis terhadap struktur `swagger.json`, terdapat dua komponen utama beserta karakteristik hierarkisnya:

1. **Definitions (atau Schema Resources):** Mendeskripsikan spesifikasi objek Kubernetes seperti *Pod*, *Service*, dan *Deployment*. Hampir seluruh objek Kubernetes mematuhi pola struktural wajib yang terdiri atas `apiVersion`, `kind`, `metadata`, `spec`, dan `status`. Namun, isi dari atribut `spec` sangat bervariasi antarkomponen di dalam teks YAML. Akibatnya, waktu pembuatan konfigurasi meningkat dan pengembang sangat bergantung pada panduan eksternal. Secara spesifik, setiap definisi skema di bagian ini memuat:
 - a. `properties`: Merepresentasikan *field* aktual pada berkas konfigurasi YAML.
 - b. `type`: Menentukan format tipe data (seperti *object*, *array*, *string*, *integer*, *boolean*).
 - c. `required`: Mendefinisikan daftar *field* konfigurasi yang wajib diisi.
 - d. `description`: Menyediakan dokumentasi tekstual mengenai fungsio-

nalitas *field*.

- e. `$ref`: Bertindak sebagai referensi silang ke definisi skema bersarang (*nested schema*). Ciri utama struktur dokumen ini adalah ketergantungan referensial bersarang antardefinisi skema sehingga satu *field* YAML dapat mengarah ke objek skema lain, struktur daftar (*array*), maupun struktur pemetaan (*map*). Ketiga bentuk referensi ini dianalisis lebih lanjut pada Subbab III.1.2.4.
2. **Paths**: Mendeskripsikan *endpoint* API, misalnya `GET /api/v1/ pods`, sebagai antarmuka interaksi objek Kubernetes dalam ekosistem kluster. Setiap *endpoint* memiliki atribut penentu interaksi komunikasi yang memuat:
- a. Metode HTTP fungsional (*GET, POST, PUT, PATCH, DELETE*).
 - b. Parameter injeksi pada jalur (*path*) dan kueri (*query*).
 - c. Skema respons (*response schema*) dari *server*.
 - d. Relasi struktural pembentuk fungsionalitas yang mengarah kembali ke blok *definitions*.

III.1.2.3 Analisis Data Eksploratif Spesifikasi OpenAPI

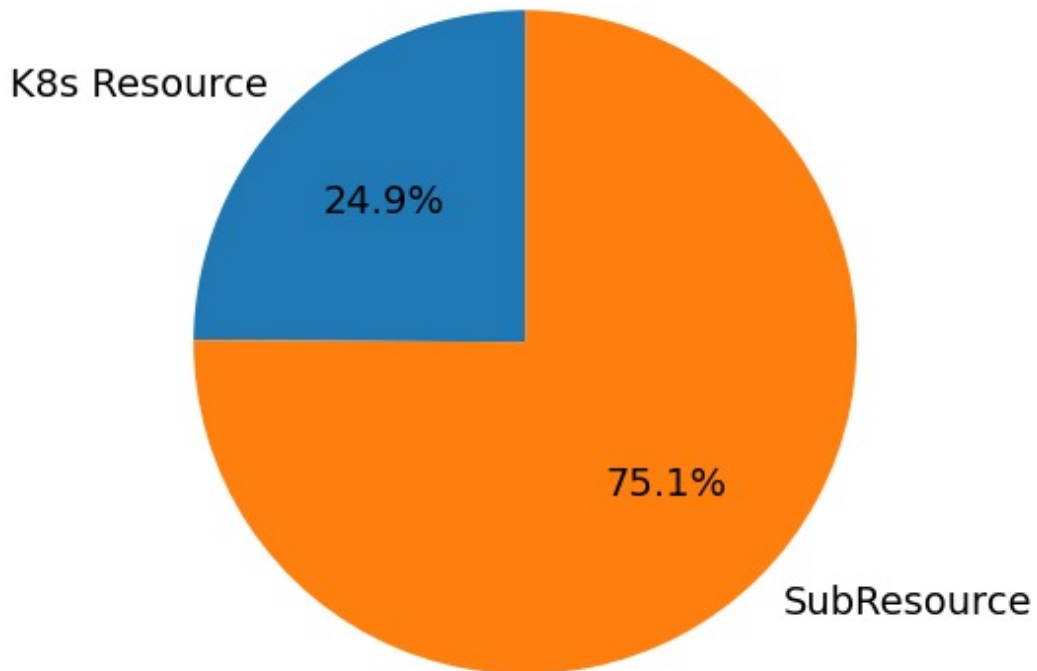
Sebelum merancang skema *knowledge graph*, analisis data eksploratif (*Exploratory Data Analysis* / EDA) dilakukan terhadap berkas `kubernetes_swagger.json` berukuran 3,757 MB guna memetakan skala, kompleksitas struktural, dan sebaran definisi skema. Proses ekstraksi awal berhasil mengidentifikasi 735 definisi skema. Sebanyak 5 entri diabaikan karena teridentifikasi sebagai *noise*, menyisakan 730 definisi skema valid untuk dievaluasi lebih lanjut.

Temuan utama dari proses EDA ini dikelompokkan ke dalam dua metrik utama:

1. Distribusi objek Kubernetes *root* vs komponen pendukung

Klasifikasi definisi skema menunjukkan ketimpangan proporsi yang sangat signifikan antara objek utama dan objek pendukung. Dari total 730 definisi, hanya 183 (24,9%) terklasifikasi sebagai objek Kubernetes *root* (*root resources*). Objek *root* ini merupakan objek yang memiliki atribut *GroupVersionKind* (GVK) sehingga dapat diterapkan secara langsung ke dalam kluster. Sementara itu, sebagian besar definisi (547 definisi atau 75,1%) merupakan skema pendukung (*sub-resources*) yang menyusun komponen *root* tersebut. Ketimpangan ini menunjukkan besarnya dependensi struktural di dalam Kubernetes.

Proporsi Entitas Root vs Komponen



Gambar III.1 Proporsi objek Kubernetes *root* vs komponen pendukung dalam spesifikasi OpenAPI Kubernetes

2. Kepadatan Properti dan Intensitas Referensi Silang

Inspeksi terhadap struktur properti mengungkapkan bahwa setiap definisi skema rata-rata memuat 4,1 atribut konfigurasi (*field*) dengan objek Kubernetes paling kompleks menampung hingga 44 atribut. Selain itu, ditemukan bahwa 437 dari 730 definisi (59,9%) tercatat memuat setidaknya satu referensi ke skema lain melalui mekanisme `$ref`. Temuan ini mengindikasikan bahwa pemahaman terhadap satu objek Kubernetes menuntut penelusuran ke definisi skema lain sehingga penulisan konfigurasi secara manual menjadi semakin kompleks.

III.1.2.4 Analisis Keterhubungan Data

Secara struktural, keterhubungan antardefinisi pada `swagger.json` dinyatakan melalui mekanisme referensi silang (`$ref`). Sebuah properti pada satu definisi skema dapat merujuk definisi lain dalam tiga bentuk, yaitu referensi langsung ke objek skema lain, referensi di dalam struktur daftar (*array*), serta referensi di dalam struktur pemetaan (*map*). Ketiga bentuk inilah yang menjadi sumber deterministik bagi pembentukan *edge* struktural pada graf.

a. Referensi ke objek skema lain

Properti merujuk satu objek skema lain secara langsung. Sebagai contoh, properti `selector` pada *DaemonSetSpec* merujuk definisi *LabelSelector* sehingga isi di bawah `selector` mengikuti struktur *LabelSelector*, sebagaimana Kode III.1 dan III.2.

Listing III.1 Skema referensi objek pada `swagger.json`

```
1 "selector": {
2   "$ref": "#/definitions/io.k8s.apimachinery.pkg.apis.meta.v1
   .LabelSelector"
3 }
```

Listing III.2 Wujud YAML objek *LabelSelector*

```
1 selector:                # nilai berbentuk objek LabelSelector
2   matchLabels:           # field milik LabelSelector
3     app: nginx
```

b. Referensi di dalam struktur daftar (*array*)

Properti bertipe array merujuk definisi skema melalui atribut `items`. Sebagai contoh, properti `containers` pada *PodSpec* memuat daftar objek *Container* dengan tiap elemen bertipe *Container* dan dibedakan oleh urutannya, sebagaimana Kode III.3 dan III.4.

Listing III.3 Skema referensi daftar pada `swagger.json`

```
1 "containers": {
2   "type": "array",
3   "items": {
4     "$ref": "#/definitions/io.k8s.api.core.v1.Container"
5   }
6 }
```

Listing III.4 Wujud YAML daftar objek *Container*

```
1 containers:              # array; tiap elemen objek Container
2   - name: app             # Container ke-1 (name = field
   Container)
3     image: nginx:1.25
4   - name: sidecar         # Container ke-2, dibedakan oleh
   urutan
5     image: envoy:v1.31
```

c. Referensi di dalam struktur pemetaan (*map*)

Properti bertipe object merujuk definisi skema melalui atribut `additionalProperties`. Sebagai contoh, properti `limits` pada *ResourceRequirements*

memetakan setiap kunci ke objek *Quantity* dengan nama kunci dipilih bebas oleh pengguna, sebagaimana Kode III.5 dan III.6.

Listing III.5 Skema referensi pemetaan pada swagger . json

```
1 "limits": {
2   "type": "object",
3   "additionalProperties": {
4     "$ref": "#/definitions/io.k8s.apimachinery.pkg.api.
       resource.Quantity"
5   }
6 }
```

Listing III.6 Wujud YAML pemetaan objek *Quantity*

```
1 limits:                                # map; tiap nilai objek Quantity
2   cpu: "500m"                          # kunci 'cpu' (bebas), nilai Quantity
3   memory: "1Gi"                        # kunci 'memory' (bebas), nilai
       Quantity
```

III.2 Analisis Kebutuhan

Tahap analisis kebutuhan berfokus pada identifikasi kapabilitas yang harus dimiliki sistem guna menjawab tujuan penelitian dan kebutuhan bisnis. Untuk memastikan kebutuhan yang dirumuskan relevan dengan masalah yang dihadapi, penelitian ini menganalisis kesenjangan teknis (*technical gap*) pendekatan *Retrieval-Augmented Generation* saat ini, termasuk pendekatan GraphRAG yang ada saat ini pada domain teknis serupa (H.-H. Yu dkk. 2025; Lu dkk. October 2025). Hasil analisis disajikan pada Tabel III.1.

Tabel III.1 *Technical Gap Analysis* pada Domain Kubernetes

Aspek Kesenjangan Teknis	Kondisi Saat Ini (<i>Existing Approaches</i>)	Kesenjangan di Domain Kubernetes
T01 - Representasi Struktur Data	Menggunakan pemecahan teks secara <i>linear chunking</i> ataupun konstruksi <i>knowledge graph</i> berbasis ekstraksi entitas generik dari teks, tanpa mendefinisikan taksonomi jenis relasi yang spesifik terhadap semantik domain target (Cao dkk. 2025; H.-H. Yu dkk. 2025; Lu dkk. October 2025).	Terjadi kehilangan konteks mendalam (<i>loss of deep context</i>) dan ambiguitas semantik relasi. Sistem gagal memahami kedalaman hierarki bersarang (misalnya properti <i>limits</i> sebagai turunan dari <i>Deployment</i>) sekaligus tidak dapat membedakan jenis relasi antarobjek Kubernetes.
T02 - Mekanisme Validasi	Validasi bergantung pada model LLM atau aturan manual berbasis <i>hard constraints</i> yang harus ditulis satu per satu (Wan dkk. 2025).	Risiko terjadinya <i>syntactic hallucination</i> tinggi (LLM mengarang <i>field</i>), atau beban pemeliharaan menjadi besar saat aturan manual harus diperbarui (Gao dkk. 2024).
T03 - Ambiguitas Entitas	Mengandalkan frekuensi kemunculan kata kunci (<i>keyword frequency</i>) atau kedekatan vektor semata (Gao dkk. 2024).	Terdapat ambiguitas pada pola <i>loose coupling</i> karena <i>keywords</i> (misal: <i>labels</i> , <i>ports</i>) muncul berulang di banyak objek berbeda. Hal ini menyebabkan tercampurnya konteks antarobjek (<i>contextual mixing</i>) (Kotstein dan Decker 2024).

bersambung ke halaman berikutnya

Tabel III.1 *Technical Gap Analysis* pada Domain Kubernetes (lanjutan)

Aspek Kesenjangan Teknis	Kondisi Saat Ini (<i>Existing Approaches</i>)	Kesenjangan di Domain Kubernetes
T04 - Pemeliharaan Pengetahuan	Membutuhkan <i>fine-tuning</i> ulang model atau pembaruan aturan manual ketika terjadi perubahan versi API (Gao dkk. 2024). Penelitian GraphRAG terkini pada domain teknis serupa pun masih menghadapi keterbatasan yang sama berupa <i>knowledge graph</i> yang bersifat statis tanpa mekanisme pembaruan dinamis (H.-H. Yu dkk. 2025; Lu dkk. October 2025).	Tidak efisien, mudah menjadi <i>obsolete</i> , dan sensitif terhadap perubahan versi Kubernetes yang dirilis secara berkala.
T05 - Akurasi Jawaban	Sistem cenderung menghasilkan jawaban naratif dan tidak memberikan jaminan presisi sintaksis konfigurasi YAML (Liu dkk. 2024).	Tidak mampu menghasilkan artefak konfigurasi yang sesuai karena sering ditemukan kesalahan format atau struktur (Rahman dkk. 2023).

III.2.1 Kebutuhan Fungsional

Berdasarkan *Business Understanding* (B01–B03), diidentifikasi sejumlah urgensi bisnis yang dianalisis lebih lanjut hingga menghasilkan beberapa kesenjangan teknis (T01–T05) yang menjadi dasar perumusan kebutuhan sistem. Setiap kesenjangan teknis kemudian dijawab melalui perancangan kebutuhan fungsional yang akan diimplementasikan untuk menyelesaikan permasalahan bisnis yang ada. Tabel III.2 berikut memetakan hubungan antara kebutuhan bisnis, kesenjangan teknis, serta kebutuhan fungsional yang dirumuskan untuk mengatasinya.

Tabel III.2 Pemetaan Kebutuhan Bisnis terhadap *Technical Gap* dan Kebutuhan Fungsional

Business Requirement (B)	Technical GAP (T)	Functional Requirement (F)
B01 – Inefisiensi waktu	T01 – Representasi struktur data	F-01 <i>Structured Object Representation</i>
		F-02 <i>Structured Relation Retrieval</i>
	T03 – Ambiguitas entitas	F-03 <i>Intent Classification</i>
		F-04 <i>Context Construction & Injection</i>
		F-05 <i>Semantic Embedding</i>
B02 – Kesenjangan pengetahuan	T02 – Mekanisme validasi	F-07 <i>Conditional YAML Generation Constraint</i>
	T04 – Pemeliharaan pengetahuan	F-01 <i>Structured Object Representation</i>
B03 – Risiko kesalahan konfigurasi	T05 – Akurasi jawaban	F-06 <i>Web Interaction Interface</i>
		F-07 <i>Conditional YAML Generation Constraint</i>

Berdasarkan hasil pemetaan pada Tabel III.2, berikut adalah kebutuhan fungsional yang dibutuhkan oleh sistem,

Tabel III.3 Kebutuhan Fungsional Sistem

Kode	Parameter	Deskripsi Spesifikasi
F-01	<i>Structured Object Representation</i>	Sistem harus mengubah skema objek ke dalam suatu representasi terstruktur yang membedakan setiap komponen, <i>field</i> , dan relasi antarobjek sehingga dapat diakses kembali secara terprogram pada tahap <i>retrieval</i> .

bersambung ke halaman berikutnya

Tabel III.3 Kebutuhan Fungsional Sistem (lanjutan)

Kode	Parameter	Deskripsi Spesifikasi
F-02	<i>Structured Relation Retrieval</i>	Sistem harus menyediakan mekanisme <i>retrieval</i> yang mampu mengambil himpunan objek dan <i>field</i> yang saling berkaitan (misalnya hubungan antara <i>Deployment</i> , <i>Pod</i> , dan <i>Service</i>) berdasarkan entitas awal dan <i>intent</i> pengguna sebagaimana didefinisikan pada F-03.
F-03	<i>Intent Classification</i>	Sistem harus mampu (1) mendeteksi entitas Kubernetes yang dirujuk pengguna (misalnya <i>Service</i> , <i>Pod</i>), dan (2) mengklasifikasikan maksud pengguna ke dalam beberapa kategori kueri yang berbeda sehingga strategi <i>retrieval</i> dan format keluaran dapat disesuaikan dengan karakteristik tiap jenis kueri.
F-04	<i>Context Construction & Injection</i>	Sistem harus mengubah hasil F-02 menjadi konteks terstruktur yang dapat digunakan sebagai basis generasi jawaban.
F-05	<i>Semantic Embedding</i>	Sistem harus menghasilkan representasi vektor (<i>embedding</i>) untuk setiap node dalam representasi terstruktur dan menyimpannya ke dalam indeks vektor, sehingga pencarian berbasis kemiripan semantik dapat digunakan sebagai mekanisme <i>fall-back</i> pada tahap <i>retrieval</i> .

bersambung ke halaman berikutnya

Tabel III.3 Kebutuhan Fungsional Sistem (lanjutan)

Kode	Parameter	Deskripsi Spesifikasi
F-06	<i>Web Interaction Interface</i>	Sistem harus menyediakan antarmuka web interaktif yang memungkinkan pengguna memasukkan pertanyaan dalam bahasa alami, menampilkan jawaban beserta konfigurasi YAML (jika relevan), serta menyajikan jejak penalaran (<i>reasoning path</i>) secara visual disertai pesan kesalahan yang informatif.
F-07	<i>Conditional YAML Generation Constraint</i>	Sistem harus secara kondisional menghasilkan keluaran berupa blok kode YAML yang valid secara sintaksis apabila pengguna meminta pembuatan konfigurasi, dan menghasilkan jawaban naratif untuk jenis kueri lainnya yang tidak bersifat generatif.

III.2.2 Kebutuhan Non Fungsional

Kebutuhan non-fungsional mendefinisikan karakteristik kualitas yang harus dimiliki oleh sistem untuk memastikan kinerja, keandalan, dan keamanan operasional. Kebutuhan ini penting untuk menjamin bahwa sistem tidak hanya berfungsi dengan benar, tetapi juga memenuhi standar kualitas yang diharapkan oleh pengguna. Berikut adalah kebutuhan non-fungsional utama yang diidentifikasi:

Tabel III.4 Kebutuhan Non-Fungsional Sistem

Kode	Parameter	Deskripsi Spesifikasi
NF-01 (Reliability)	<i>Fail-Safe Mechanism</i>	Sistem harus mencegah keluaran halusina-si melalui mekanisme <i>error handling</i> , ya-itu (1) Jika entitas tidak ditemukan, sis-tem harus memberi respons “Entity not found”. (2) Jika YAML terdeteksi ti-dak valid oleh validator internal, sistem harus menampilkan peringatan “Warning: Potential Syntax Error” pada antarmuka pengguna.
NF-02 (Performance)	<i>Retrieval–Generation Latency</i>	Waktu total yang dibutuhkan untuk memp-roses satu permintaan pengguna (<i>end-to-end latency</i>) tidak boleh melebihi 60 detik pada lingkungan pengujian standar.
NF-03 (Usability)	<i>Web UI Usability Stand-ard</i>	Antarmuka web interaktif sistem ha-rus menyediakan pengalaman pengguna-an yang konsisten, mudah dipahami, dan informatif, mencakup mekanisme input pertanyaan, panel tampilan jawaban, ser-ta visualisasi jejak penalaran.
NF-04 (Accessibility)	<i>Public Accessibility</i>	Sistem harus dapat diakses oleh pengguna umum melalui antarmuka web yang terse-dia secara daring, tanpa memerlukan in-stalasi perangkat lunak tambahan di sisi pengguna.

III.2.3 Pemetaan Kebutuhan Bisnis terhadap Kerangka Evaluasi

Untuk memastikan bahwa setiap dimensi evaluasi sistem memiliki dasar bisnis yang jelas, Tabel III.5 memetakan hubungan antara kebutuhan bisnis (B01–B03), dimensi evaluasi, dan metrik yang digunakan.

Tabel III.5 Pemetaan kebutuhan bisnis terhadap dimensi dan metrik evaluasi

Kebutuhan Bisnis	Pertanyaan Evaluasi	Dimensi	Metrik Utama
B01 – Inefisiensi Waktu	Apakah hasil <i>retrieval</i> memiliki nilai dengan presisi tinggi dan mencakup seluruh relasi yang dibutuhkan dalam satu kueri sehingga meminimalisir waktu penelusuran relasi secara manual?	RetQ	F1@k, NDCG@k, <i>Path Coverage</i>
B02 – Kesenjangan Pengetahuan	Apakah jawaban yang dihasilkan dapat ditelusuri ke konteks graf yang relevan, bukan berasal dari pengetahuan parametrik model?	ReaQ	<i>Hop Accuracy, Grounding Score, RGA</i>
B03 – Risiko Konfigurasi	Apakah keluaran YAML valid dan relevan terhadap kueri?	AnsQ	<i>Syntactic Validity, Schema Compliance, Answer Relevance, Faithfulness</i>

Pemetaan ini mempertegas bahwa dimensi evaluasi bukan sekadar metrik teknis, melainkan representasi langsung dari dampak bisnis yang ingin dimitigasi oleh sistem GraphRAG.

III.3 Alternatif Solusi

Bagian ini membahas berbagai alternatif solusi yang dapat diimplementasikan untuk memenuhi kebutuhan sistem yang telah diidentifikasi sebelumnya.

III.3.1 Hybrid KG–Vector RAG

Pendekatan *hybrid KG–Vector RAG* menggabungkan dua jalur pencarian, yakni *vector-based RAG* untuk pencarian semantik dan *knowledge graph* (KG) berbasis metadata untuk penalaran relasional (Wan dkk. 2025). Kueri pengguna diproses melalui ekstraksi entitas untuk menarik subgraf relevan dari KG sekaligus mengambil potongan teks melalui pencarian kemiripan vektor; kedua konteks terstruktur dan semantik ini

kemudian digabungkan sebagai basis generasi bagi LLM.

Hasil evaluasi oleh Wan dkk. (2025) pada dataset benchmark menunjukkan peningkatan yang signifikan dibanding *vector-only RAG: Exact Match* meningkat dari 63,4% menjadi 77,8% (+14,4 poin persentase) dan *Context Precision* dari 69,8% menjadi 76,5% (+6,7 poin persentase). Konsekuensi dari penggabungan dua jalur ini adalah lonjakan latensi akibat *traversal overhead* graf dan kompleksitas *prompt engineering*.

III.3.2 GNN-augmented GraphRAG (G-Retriever)

Arsitektur G-Retriever dirancang untuk menangani pemahaman graf tekstual berskala besar dengan memadukan *Graph Neural Networks* (GNN) dan LLM (He dkk. 2024). Sistem ini mereduksi ukuran konteks kueri melalui algoritma *Prize-Collecting Steiner Tree* (PCST) guna membangun subgraf koheren berisi *node* dan *edge* paling informatif sebelum disuntikkan ke dalam LLM.

Secara kuantitatif, pendekatan ini mampu memangkas penggunaan token hingga 99% dibanding RAG konvensional, sekaligus menaikkan validitas *node* dari 31% menjadi 77% (+46 poin persentase) dan validitas *edge* dari 12% menjadi 76% tanpa memerlukan proses *fine-tuning* pada LLM (He dkk. 2024). Namun, metode PCST memiliki kompleksitas komputasi $O(|V| + |E|)$ yang semakin meningkat seiring ukuran graf, dan ketergantungan terhadap GNN menyebabkan *indexing* harus diulang setiap versi Kubernetes berubah.

III.3.3 Hybrid Neural-Symbolic dengan Relational DB (NeuSym-RAG)

Metode NeuSym-RAG memadukan model neural dengan basis pengetahuan simbolik berbasis aturan di dalam basis data relasional (Cao dkk. 2025). Pendekatan ini menyimpan entitas dalam tabel terstruktur dan mengeksekusi aturan simbolik untuk memvalidasi hasil inferensi LLM secara deterministik dengan akurasi validasi mencapai lebih dari 90% pada dataset *question answering* terstruktur (Cao dkk. 2025).

Namun, skema penyimpanan relasional memiliki kelemahan struktural jika diaplikasikan pada domain Kubernetes. Dengan 730 definisi skema dan rata-rata 4,1 *field* per definisi, menelusuri relasi hierarkis bersarang (*multi-hop*) membutuhkan operasi *join* SQL yang kompleks. Selain itu, aturan simbolik harus didefinisikan secara manual untuk setiap relasi dan diperbarui setiap siklus rilis Kubernetes (rata-rata setiap 4 bulan), menjadikan pemeliharaan tidak praktis dalam skala produksi.

III.4 Analisis Pemilihan Solusi

Bagian ini bertujuan menentukan pendekatan pemecahan masalah yang paling sesuai untuk membangun sistem *Graph-based Retrieval* pada dokumentasi API Kubernetes. Evaluasi dilakukan dengan membandingkan beberapa alternatif solusi berdasarkan kriteria yang disesuaikan dengan karakteristik konfigurasi *cloud-native* yang bersifat deklaratif, terstruktur, dan saling bergantung secara hierarkis.

III.4.1 Rubrik Evaluasi Solusi

Evaluasi dilakukan menggunakan skala 1 (Kurang Baik) hingga 5 (Sangat Baik), berdasarkan empat kriteria utama sebagai berikut:

K1. Kesesuaian Representasi Data

Mengukur kemampuan metode dalam menangani data konfigurasi yang bersifat hierarkis dan memiliki *cross-reference* antarobjek Kubernetes (misalnya *Service* $\rightarrow$ *Pod* melalui *selector*).

K2. Kemampuan Generatif

Menilai kemampuan sistem menghasilkan berkas konfigurasi YAML yang baru dan valid, bukan sekadar menemukan lokasi informasi pada dokumentasi.

K3. Fleksibilitas Pemeliharaan

Mengukur kemudahan sistem dalam melakukan pembaruan pengetahuan ketika spesifikasi API Kubernetes berubah versi tanpa memerlukan *fine-tuning* ulang.

K4. Presisi Konteks

Menilai kemampuan sistem menekan halusinasi melalui pembatasan struktural yang tegas, terutama pada relasi antarobjek Kubernetes.

Tabel III.6 menunjukkan kategori penilaian untuk setiap skor pada rubrik evaluasi tersebut.

III.4.2 Matriks Keputusan & Justifikasi Penilaian

Penilaian dilakukan terhadap tiga pendekatan alternatif sebagai kandidat solusi menggunakan metode *Weighted Sum Model* (WSM) dengan bobot setara ($w = 0,25$ per kriteria, $\sum w = 1,0$). Alternatif 1 (Hybrid KG–Vector RAG) merupakan pendekatan yang diusulkan dalam penelitian ini. Hasil evaluasi ditampilkan pada Tabel III.7.

Tabel III.6 Rubrik Penilaian Kriteria Evaluasi Solusi

Skor	Kategori	Deskripsi
1	Kurang Baik	Tidak memenuhi kebutuhan domain Kubernetes secara fungsional maupun struktural.
2	Cukup Buruk	Hanya relevan sebagian dan berisiko tinggi menghasilkan kesalahan konfigurasi.
3	Cukup Baik	Dapat digunakan, tetapi memerlukan banyak penyesuaian untuk konteks Kubernetes.
4	Baik	Umumnya sesuai dan dapat diterapkan dengan penyesuaian minor.
5	Sangat Baik	Sangat sesuai dengan karakteristik masalah dan hampir tidak memerlukan modifikasi.

Tabel III.7 Matriks Perbandingan Solusi Berdasarkan Rubrik Evaluasi

Metode	K1	K2	K3	K4	Total
Hybrid KG–Vector RAG (Alt 1)	5	4	5	5	19
GNN-augmented GraphRAG (Alt 2)	5	3	4	5	17
NeuSym-RAG (Alt 3)	3	2	3	4	12

Berdasarkan hasil penilaian kuantitatif pada Tabel III.7, diperlukan penjelasan lebih lanjut mengenai dasar pemberian skor pada setiap kriteria. Berikut ini justifikasi kuantitatif terhadap nilai yang diberikan untuk setiap metode.

1. Hybrid KG–Vector RAG

- K1 = 5:** Mampu menggabungkan struktur relasional (KG) dan teks tidak terstruktur (*vector store*). Cocok untuk OpenAPI Kubernetes yang bersifat hierarkis dan memiliki *cross-reference*.
- K2 = 4:** Menghasilkan konteks yang lengkap sehingga LLM dapat menyusun berkas konfigurasi YAML baru dengan lebih tepat, meskipun tidak memiliki mekanisme pembatasan struktural seketat GNN.
- K3 = 5:** KG dapat diperbarui secara *incremental*, sedangkan *vector store* hanya memerlukan *re-embedding* teks tanpa melatih ulang model.
- K4 = 5:** Kombinasi *hard constraint*, *schema constraint*, dan *vector similarity* memberikan presisi konteks, serta menunjukkan peningkatan EM dan CP pada studi.

2. GNN-augmented GraphRAG (G-Retriever)

- a. **K1 = 5**: Representasi graf Kubernetes sangat sesuai diproses oleh GNN yang sensitif terhadap struktur topologi dan relasi *multi-hop* (*Deployment* → *PodTemplate* → *Container*).
- b. **K2 = 3**: Kemampuan generatif bergantung pada subgraf hasil PCST. LLM menghasilkan jawaban berdasarkan subgraf, tetapi tidak menyusun struktur baru secara fleksibel.
- c. **K3 = 4**: Meski tidak memerlukan *fine-tuning* ulang, proses *indexing* dan pembentukan *embedding* graf harus diulang ketika versi API berubah.
- d. **K4 = 5**: Eksperimen menunjukkan validitas *node* meningkat dari 31% menjadi 77%, dan validitas *edge* dari 12% menjadi 76%, menunjukkan kontrol presisi yang sangat tinggi.

3. NeuSym-RAG (Hybrid Neural-Symbolic)

- a. **K1 = 3**: Representasi tabel relasional kurang mampu menampung kedalaman hierarki OpenAPI tanpa melakukan banyak operasi *join* sehingga tidak sebaik KG atau GNN.
- b. **K2 = 2**: Aturan simbolik bersifat validasi, bukan generasi. Kemampuan LLM untuk membuat berkas konfigurasi YAML baru terbatas karena sistem lebih cocok untuk *consistency checking*.
- c. **K3 = 3**: Relational schema stabil, tetapi aturan simbolik harus diperbarui manual saat Kubernetes menambahkan spesifikasi baru.
- d. **K4 = 4**: Aturan deterministik (*symbolic rules*) memberi kontrol kuat terhadap halusinasi, tetapi kelenturannya tidak sekuat *constraint* pada KG–Vector atau pemodelan struktural pada GNN.

III.4.3 Analisis Pemilihan dan Penyesuaian (Selection & Adjustment)

Berdasarkan hasil evaluasi pada Tabel III.7, metode *Hybrid KG–Vector RAG* (Alt 1) memperoleh skor tertinggi di antara alternatif lain dan menjadi landasan utama dalam perancangan sistem. Pendekatan tersebut dipilih karena berhasil menggabungkan representasi relasional yang eksplisit dari KG dengan cakupan semantik luas dari *vector retrieval*.

Namun, arsitektur Wan dkk. (2025) masih memiliki keterbatasan jika diterapkan langsung pada konteks API Kubernetes. Komponen *Semantic Alignment* pada penelitian tersebut sangat bergantung pada aturan manual (*domain constraints*) yang harus dituliskan eksplisit pada *prompt*. Pada Kubernetes, pendekatan ini tidak efisien karena spesifikasi OpenAPI v1.30 memuat 730 definisi skema dengan rata-rata 4,1 *field* per definisi dan 18 jenis relasi semantik. Menuliskan aturan *constraint* secara manual untuk setiap kombinasi relasi tersebut akan menghasilkan ratusan aturan

yang harus diperbarui setiap ada pembaruan versi Kubernetes (rata-rata sebanyak 4 bulan sekali) sehingga pendekatan ini tidak skalabel dalam jangka panjang.

Oleh karena itu, penelitian ini mengusulkan penyesuaian strategi sebagai berikut:

- A1.** Pendekatan Wan dkk. (2025) : Validasi berbasis aturan manual di dalam *prompt* (*Constraint-based Semantic Alignment*).
- A2.** Implementasi solusi Kubernetes : Validasi berbasis struktur Kubernetes melalui penelusuran graf (*Graph-Native Structural Traversal*).

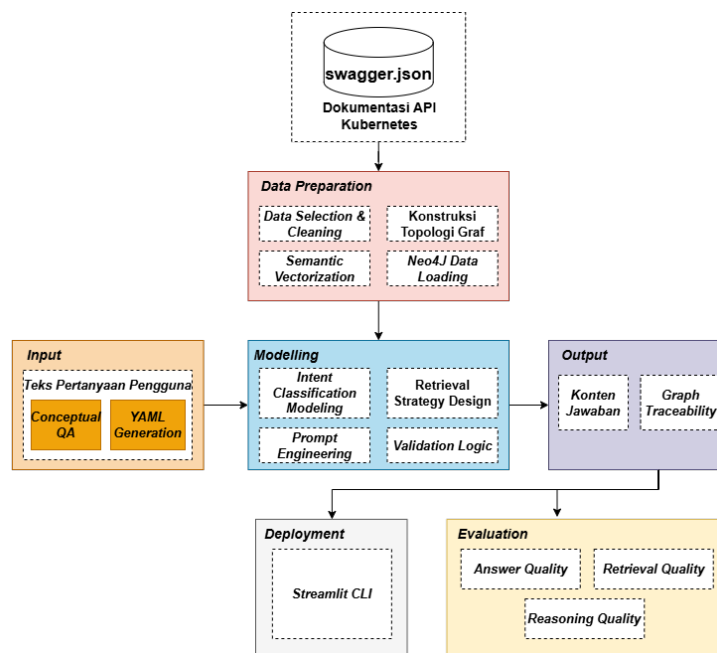
Pendekatan ini menjadikan topologi *knowledge graph* sebagai validator bawaan. Jika relasi antarobjek Kubernetes ditemukan secara eksplisit pada graf (misalnya *Service* → *Deployment* melalui *selector*), maka konfigurasi tersebut dianggap valid tanpa perlu mendefinisikan aturan tambahan.

BAB IV

PERANCANGAN SISTEM *GRAPHRAG* BERBASIS *KNOWLEDGE GRAPH* KUBERNETES

IV.1 Rancangan Solusi Secara Garis Besar

Berdasarkan hasil analisis karakteristik data, pola keterhubungan komponen Kubernetes, serta pemetaan kebutuhan fungsional dan non-fungsional yang telah dijelaskan pada bagian sebelumnya, langkah selanjutnya adalah merumuskan rancangan solusi yang akan dikembangkan dalam penelitian ini. Rancangan solusi ini disusun secara sistematis mengacu pada metodologi *Cross-Industry Standard Process for Data Mining* (CRISP-DM) sebagaimana ditampilkan pada Gambar IV.1.



Gambar IV.1 Rancangan solusi berdasarkan metodologi CRISP-DM

IV.2 Pemetaan Metodologi terhadap Kebutuhan Sistem

Tahapan metodologi penelitian sebagaimana digambarkan pada Gambar IV.1 menjadi landasan utama dalam perumusan kebutuhan sistem. Setiap aktivitas pada ta-

hap *Data Preparation*, *Modelling*, *Input Processing*, *Output Generation*, *Evaluation*, dan *Deployment* dipetakan secara langsung terhadap kebutuhan fungsional yang harus dipenuhi oleh sistem. Pemetaan ini memastikan bahwa rancangan solusi GraphRAG tidak hanya bersifat konseptual, namun juga merefleksikan proses yang dibutuhkan untuk mencapai tujuan penelitian. Hubungan antara tahapan metodologi dan kebutuhan fungsional ditunjukkan pada Tabel IV.1.

Tabel IV.1 Pemetaan Tahap Metodologi

Tahap Metodologi	Aktivitas Utama	Kebutuhan yang Dipe- nuhi (F)
Data Preparation		
<i>Data Selection & Cleaning</i>	Identifikasi objek target dari spesifikasi OpenAPI Kubernetes, seleksi blok <i>definitions</i> , dan pembersihan tipe generik yang tidak relevan untuk pembuatan YAML.	F-01 (<i>Structured Object Representation</i>), F-05 (<i>Semantic Embedding</i>)
Konstruksi Topologi Graf	Transformasi struktur JSON menjadi node-edge dengan relasi referensial dan hierarkis.	F-01 (<i>Structured Object Representation</i>), F-02 (<i>Structured Relation Retrieval</i>), F-05 (<i>Semantic Embedding</i>)
<i>Semantic Vectorization</i>	Ekstraksi deskripsi objek menjadi <i>embedding</i> numerik untuk pencarian semantik.	F-05 (<i>Semantic Embedding</i>)
Neo4j <i>Data Loading</i>	Memuat hasil konstruksi graf ke basis data Neo4j agar dapat ditelusuri kembali saat retrieval.	F-01 (<i>Structured Object Representation</i>)
Modelling		

bersambung ke halaman berikutnya

Tabel IV.1 Pemetaan Tahap Metodologi (lanjutan)

Tahap Metodologi	Aktivitas Utama	Kebutuhan yang Dipe- nuhi (F)
<i>Intent Classification Modeling</i>	Klasifikasi pertanyaan pengguna ke dalam tipe <i>intent</i> untuk menentukan strategi dan kedalaman penelusuran graf.	F-03 (<i>Intent Classification</i>)
<i>Retrieval Strategy Design</i>	Perancangan traversal graf berdasarkan entitas target, relasi, dan kebutuhan pengguna.	F-02 (<i>Structured Relation Retrieval</i>), F-04 (<i>Context Construction & Injection</i>)
<i>Prompt Engineering</i>	Perancangan template <i>prompt</i> untuk dua peran LLM serta injeksi konteks dan pengaturan format keluaran.	F-04 (<i>Context Construction & Injection</i>), F-07 (<i>Conditional YAML Generation Constraint</i>)
<i>Validation Logic</i>	Integrasi prosedur validasi YAML bertingkat sebagai mekanisme <i>fail-safe</i> yang memastikan keluaran memenuhi standar sintaksis, skema, dan kelengkapan <i>field</i> .	F-07 (<i>Conditional YAML Generation Constraint</i>)
<i>Conversation Memory Management</i>	Penyimpanan dan pengambilan riwayat percakapan dalam <i>pipeline</i> LangGraph untuk mendukung konteks <i>multi-turn</i> .	F-03 (<i>Intent Classification</i>), F-04 (<i>Context Construction & Injection</i>)
Input Processing		
Pertanyaan Pengguna	Input teks berupa permintaan teknis Kubernetes dalam bahasa alami yang diajukan pengguna melalui antarmuka percakapan.	F-03 (<i>Intent Classification</i>)

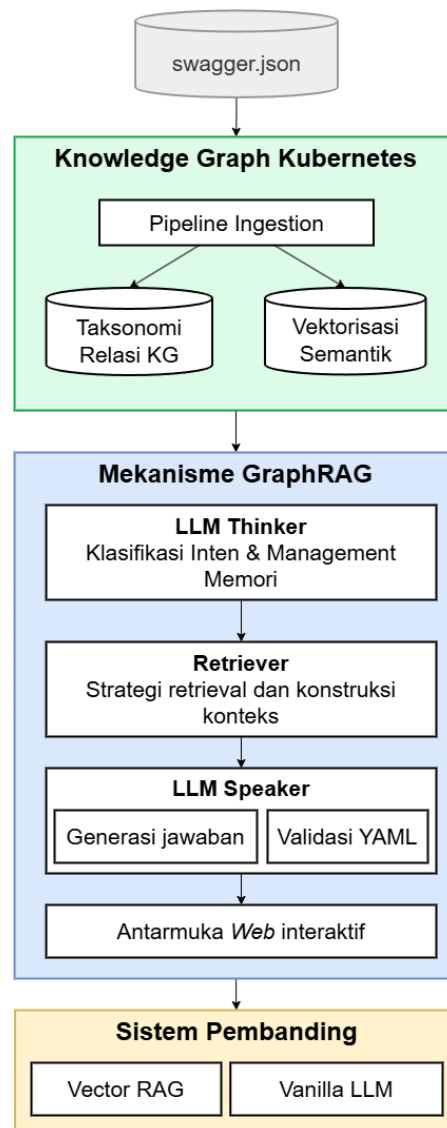
bersambung ke halaman berikutnya

Tabel IV.1 Pemetaan Tahap Metodologi (lanjutan)

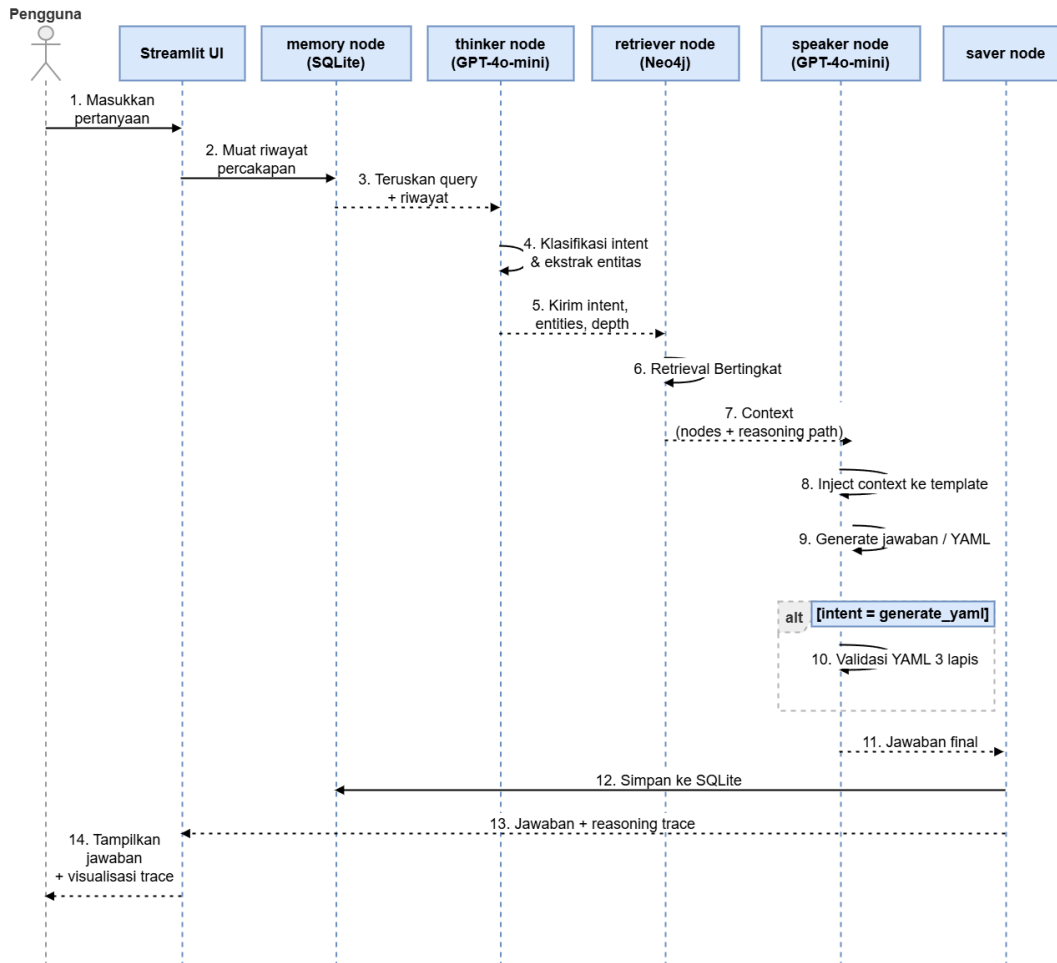
Tahap Metodologi	Aktivitas Utama	Kebutuhan yang Dipe- nuhi (F)
Output Generation		
Konten Jawaban	Keluaran sistem berupa nara- si penjelas atau blok YAML konfigurasi Kubernetes sesu- ai permintaan pengguna.	F-07 (<i>Conditional YAML Generation Constraint</i>)
<i>Graph Traceability</i>	Penelusuran asal-usul <i>field</i> melalui relasi <i>node-edge</i> di dalam graf.	F-02 (<i>Structured Rela- tion Retrieval</i>)
Evaluation		
<i>Answer Quality</i> (An- sQ)	Evaluasi kualitas jawaban (<i>Answer Quality</i>); defini- si metrik lengkap pada Subbab II.6.	– (Metrik domain, buk- an requirement fungsio- nal)
<i>Retrieval Quality</i> (Re- tQ)	Evaluasi kualitas <i>retrieval</i> (<i>Retrieval Quality</i>); defi- nisi metrik lengkap pada Subbab II.6.	– (Metrik domain, buk- an requirement fungsio- nal)
<i>Reasoning Quality</i> (ReaQ)	Evaluasi kualitas penalaran (<i>Reasoning Quality</i>); defini- si metrik lengkap pada Su- bbab II.6.	– (Metrik domain, buk- an requirement fungsio- nal)
Deployment		
Antarmuka Web Inte- raktif (Streamlit)	Penyajian sistem melalui an- tarmuka web berbasis Stream- lit (<code>main.py</code>) yang menam- pilkan <i>chat interface</i> , jawab- an LLM, blok YAML (jika re- levan), serta visualisasi <i>Retri- eval Trace</i> berbasis Graphviz.	F-06 (<i>Web Interaction Interface</i>)

Pemetaan pada Tabel IV.1 menunjukkan bahwa seluruh tahapan metodologis yang digunakan dalam penelitian ini memiliki keterhubungan langsung dengan kebutuhan fungsional yang dirancang. Setiap proses dalam *Data Preparation*, *Modelling*, *Input Processing*, *Output Generation*, *Evaluation*, dan *Deployment* telah diterjemahkan menjadi kapabilitas sistem yang operasional, terukur, dan selaras dengan tujuan penelitian. Rancangan tiap kebutuhan fungsional diuraikan secara terperinci pada Subbab IV.3 hingga Subbab IV.5.

Gambaran menyeluruh sebelum penjelasan detail tiap komponen dapat dilihat pada Gambar IV.2 berisi diagram arsitektur yang menggambarkan komponen sistem beserta keterhubungannya, sedangkan Gambar IV.3 menggambarkan alur interaksi antarkomponen secara kronologis dari masukan pengguna hingga keluaran akhir.



Gambar IV.2 *Architecture Diagram* Sistem *GraphRAG* Kubernetes



Gambar IV.3 Diagram Urutan (*Sequence Diagram*) Alur Kerja Sistem GraphRAG

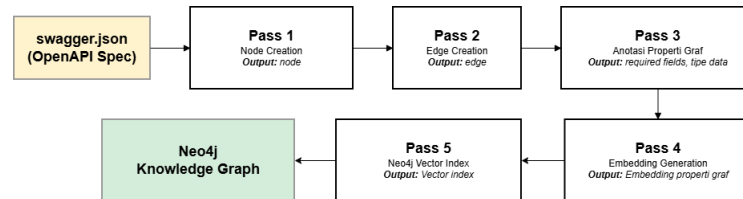
IV.3 Perancangan *Knowledge Graph* Kubernetes

Kontribusi pertama penelitian ini, sebagaimana dirumuskan pada Bab I, adalah merancang *pipeline* konstruksi *knowledge graph* yang bersifat deterministik berbasis skema OpenAPI Kubernetes (*schema-derived deterministic KG construction*). Graf dibangun langsung dari `swagger.json` melalui aturan transformasi yang eksplisit sehingga hasil ekstraksi konsisten antareksekusi, berbeda dengan pendekatan berbasis LLM yang menghasilkan struktur graf bervariasi karena bersifat stokastik. Perancangan *knowledge graph* mencakup tiga aspek yang saling melengkapi:

1. *pipeline ingestion* untuk representasi objek (F-01),
2. strategi pembentukan relasi graf yang menjadi fondasi *retrieval* (F-02), dan
3. pembentukan *embedding* semantik untuk pencarian berbasis kemiripan (F-05).

IV.3.1 Perancangan *Pipeline Ingestion* dan Representasi Objek

Pipeline ingestion bertanggung jawab memenuhi kebutuhan fungsional F-01 (*Structured Object Representation*) dengan mengubah spesifikasi OpenAPI Kubernetes (*swagger.json*) menjadi *knowledge graph* terstruktur yang tersimpan di Neo4j. Proses ini dilaksanakan melalui lima tahapan pemrosesan yang dieksekusi secara berurutan sebagaimana ditunjukkan pada Gambar IV.4.



Gambar IV.4 *Pipeline Ingestion* dari *swagger . json* ke *Neo4j Knowledge Graph*

Urutan *pass* ini bersifat deterministik dan tidak dapat dibalik.

IV.3.2 Perancangan Taksonomi Relasi *Knowledge Graph*

Berdasarkan analisis pola keterhubungan data pada Bab III, penelitian ini merancang strategi pembentukan relasi *knowledge graph* yang menggunakan dua mekanisme komplementer sebagaimana ditunjukkan pada Tabel IV.2.

Tabel IV.2 Strategi Pembentukan Relasi dalam *Knowledge Graph* Kubernetes

Strategi	Sumber Derivasi	Contoh <i>Edge</i>	Sifat
Deterministik	Referensi eksplisit \$ref, allOf, oneOf, anyOf dalam skema OpenAPI	<i>HAS_PROPERTY</i> , <i>EXTENDS</i> , <i>ONE_OF</i> , <i>ANY_OF</i>	Konsisten 100% antareksekusi
Semi-deterministik (berbasis aturan domain)	Aturan transformasi berbasis semantik Kubernetes: pencocokan <i>kind</i> dan pola jalur <i>field</i>	<i>SELECTS_POD</i> , <i>BINDS_ROLE</i> , <i>SCALES_RESOURCE</i> , <i>MOUNTS_VOLUME</i>	Deterministik saat eksekusi.

Strategi pertama bersifat **deterministik**, yaitu setiap referensi \$ref, allOf, dan oneOf, anyOf dalam *swagger . json* diubah menjadi *edge*.

Strategi kedua bersifat **semi-deterministik**, yaitu relasi yang tidak dinyatakan secara eksplisit dalam skema OpenAPI tetapi relevan secara operasional di Kubernetes (seperti relasi antara *Service* dan *Pod*, atau antara *RoleBinding* dan *ServiceAccount*) dibentuk melalui aturan transformasi berdasarkan semantik domain Kubernetes. Aturan ini bersifat deterministik saat dieksekusi (hasil selalu sama untuk input yang sama), tetapi derivasi aturannya memerlukan pengetahuan domain yang didefinisikan secara eksplisit.

IV.3.3 Perancangan Pembentukan *Embedding* Semantik

Pembentukan *embedding* semantik dirancang untuk memenuhi kebutuhan fungsional F-05 (*Semantic Embedding*) dengan menghasilkan representasi vektor dari setiap node dalam *knowledge graph*. Vektor tersebut disimpan langsung sebagai properti setiap node di Neo4j, dan sebuah indeks vektor dibangun agar pencarian kemiripan berbasis semantik dapat dieksekusi langsung dalam graf yang sama tanpa memerlukan *vector store* terpisah.

IV.4 Perancangan Mekanisme *GraphRAG*

Kontribusi kedua penelitian ini, sebagaimana dirumuskan pada Bab I, adalah mengembangkan mekanisme *GraphRAG* dengan kedalaman *traversal* yang disesuaikan per tipe *intent* (*intent-adaptive depth traversal*). Alih-alih menggunakan kedalaman tetap yang seragam, sistem menetapkan kedalaman berdasarkan karakteristik struktural tiap jenis kueri sehingga *retrieval* lebih presisi dan sesuai dengan kebutuhan konteks. Mekanisme ini juga mencakup validasi YAML berbasis *knowledge graph* serta antarmuka pengguna berbasis *web* yang memungkinkan interaksi dalam bahasa alami. Perancangan mekanisme *GraphRAG* terdiri dari empat komponen fungsional:

1. klasifikasi *intent* dan manajemen memori (F-03),
2. strategi *retrieval* bertingkat dan konstruksi konteks (F-02, F-04),
3. generasi dan validasi YAML (F-07), serta
4. antarmuka *web* interaktif (F-06).

IV.4.1 Perancangan Klasifikasi *Intent* dan Manajemen Memori

Node *thinker* bertanggung jawab memenuhi kebutuhan fungsional F-03 (*Intent Classification*) melalui dua fungsi utama: (1) mengklasifikasikan maksud dari kueri pengguna, dan (2) mengekstrak entitas Kubernetes yang disebutkan dalam kueri. Node *thinker* menghasilkan keluaran berformat JSON menggunakan konfigurasi deterministik. Hal ini bertujuan agar klasifikasi *intent* dan ekstraksi entitas tetap konsisten setiap kali kueri yang identik dieksekusi.

Jenis *intent* dibedakan berdasarkan jenis informasi yang dibutuhkan dan kedalaman traversal yang sesuai. Kedalaman yang lebih besar ditetapkan untuk tipe *intent* yang memerlukan konteks relasional lebih luas, sedangkan kedalaman yang lebih kecil ditetapkan untuk kueri yang bersifat spesifik.

Manajemen memori percakapan dirancang menggunakan basis data relasional ringan dengan identifikasi sesi berbasis UUID. Pada setiap interaksi, riwayat percakapan dimuat dan diteruskan ke modul *thinker* bersama kueri baru. Riwayat ini memungkinkan komunikasi 2 arah antara LLM dengan pengguna, contohnya untuk memahami rujukan seperti "hal tersebut" atau "itu" berdasarkan konteks percakapan sebelumnya tanpa memerlukan pengulangan nama entitas secara eksplisit. Setelah respons dihasilkan, pasangan kueri-respons disimpan untuk digunakan pada interaksi berikutnya.

IV.4.2 Perancangan Strategi *Retrieval* dan Konstruksi Konteks

Node *retriever* mengimplementasikan *cascading retrieval* dengan tiga tahap berurutan untuk memenuhi kebutuhan fungsional F-02 (*Structured Relation Retrieval*) dan F-04 (*Context Construction & Injection*).

- 1. Tahap 1: *Exact Name Match* (Prioritas Utama).**

Sistem mencari *node* pada Neo4j yang namanya sama persis dengan entitas hasil ekstraksi *node thinker*. Jika ditemukan, *node* ini akan dijadikan sebagai *seed node* (simpul awal) untuk penelusuran graf. Pendekatan ini memberikan presisi tertinggi karena mengandalkan pencocokan eksak, bukan sekadar perkiraan semantik.

- 2. Tahap 2: *Vector Similarity* (Fallback).**

Tahap ini hanya dieksekusi apabila Tahap 1 tidak menemukan kecocokan. Sistem akan menghasilkan representasi vektor dari kueri pengguna, kemudian mencari *node* dengan tingkat kemiripan tertinggi menggunakan indeks vektor pada Neo4j. Kandidat dengan skor kemiripan terbesar tersebut selanjutnya digunakan sebagai *seed node*.

- 3. Tahap 3: *Multi-hop Graph Traversal*.**

Berawal dari *seed node*, sistem melakukan penelusuran graf pada Neo4j secara rekursif. Batas kedalaman penelusuran disesuaikan dengan jenis *intent* dari kueri. Penentuan kedalaman ini didasarkan pada distribusi karakteristik *node* di tiap-tiap level, sebagaimana dirangkum pada Tabel IV.3.

Tabel IV.3 Distribusi karakteristik *node* berdasarkan kedalaman penelusuran graf

Kedalaman	Karakteristik <i>Node</i>	Contoh	Penggunaan
1–2	Spesifik <i>resource</i> (dibagikan 2–3 <i>resource</i>)	<i>DeploymentSpec</i> , <i>ServiceSpec</i>	Tipe <i>explain</i> , <i>followup</i>
3	<i>Field</i> tingkat kontainer	<i>image</i> , <i>ports</i> , <i>env</i> , <i>resources</i>	Tipe <i>generate_yaml</i> , <i>trace_relationship</i>
4+	Tipe generik (dibagikan 19–136 <i>resource</i>)	<i>Quantity</i> , <i>IntOrString</i> , <i>LocalObjectReference</i>	Tidak digunakan (menurunkan presisi)

Tabel IV.3 menunjukkan bahwa karakteristik *node* berubah secara signifikan seiring bertambahnya kedalaman: *node* pada kedalaman 1–2 bersifat spesifik terhadap *resource*, sedangkan *node* pada kedalaman 4 ke atas merupakan tipe generik yang dibagikan oleh 19–136 *resource* sehingga menurunkan presisi konteks. Kedalaman traversal dibatasi pada rentang 2–3 untuk menghindari injeksi tipe generik yang tidak informatif ke dalam konteks LLM.

Setelah traversal selesai, modul *retriever* membangun *reasoning path* berupa daftar relasi dengan format Parent –[REL]–> Child yang merepresentasikan jalur relasional yang dilalui selama traversal. Konteks yang terbentuk dari *reasoning path* beserta riwayat percakapan kemudian diinjeksikan ke dalam *prompt template* yang diteruskan ke modul *speaker* sebagai bukti penalaran yang dapat dilihat oleh pengguna.

IV.4.3 Perancangan Generasi dan Validasi YAML

Modul *speaker* bertanggung jawab memenuhi kebutuhan fungsional F-07 (*Conditional YAML Generation Constraint*) melalui dua jalur keluaran yang ditentukan secara kondisional berdasarkan tipe *intent*. Apabila *intent* diklasifikasikan sebagai *generate_yaml*, sistem menghasilkan blok kode YAML konfigurasi Kubernetes yang kemudian divalidasi melalui mekanisme validasi tiga lapis. Apabila *intent* bukan *generate_yaml*, sistem menghasilkan jawaban naratif tanpa proses validasi YAML.

Validasi YAML tiga lapis ini dirancang sebagai *KG-grounded structural validation*. Berbeda dengan pendekatan *dry-run* yang memerlukan kluster Kubernetes aktif, va-

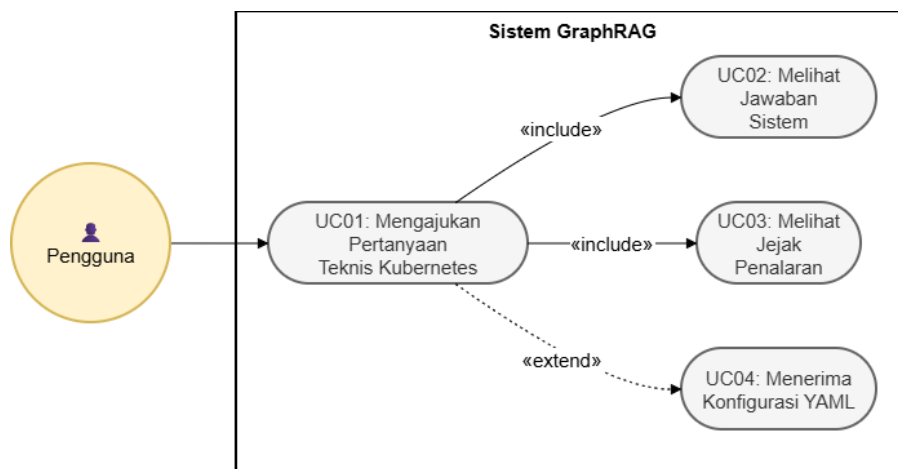
validasi lapis ketiga dilakukan langsung terhadap *knowledge graph* yang telah dibangun pada tahap *ingestion* sehingga verifikasi *required fields* dapat dilakukan tanpa menggunakan infrastruktur Kubernetes langsung. Fungsi tiap lapis validasi diuraikan pada Tabel IV.4.

Tabel IV.4 Fungsi Tiga Lapis Validasi YAML

Lapis	Mekanisme	Fungsi
1	PyYAML <code>safe_load</code>	Memvalidasi sintaksis YAML; kegagalan menghasilkan entri pada daftar <code>syntax_errors</code> .
2	<i>kubernetes-validate</i>	Memvalidasi kesesuaian struktur YAML terhadap skema OpenAPI Kubernetes v1.29; kegagalan menghasilkan entri pada daftar <code>schema_errors</code> .
3	Kueri Neo4j terhadap KG	Memverifikasi kelengkapan <i>required field</i> untuk setiap <i>resource</i> ; <i>field</i> yang absen dikumpulkan pada daftar <code>missing_fields</code> .

IV.4.4 Perancangan Antarmuka *Web* Interaktif

Antarmuka *web* interaktif dirancang untuk memenuhi kebutuhan fungsional F-06 (*Web Interaction Interface*) sebagai *entry point* utama sistem GraphRAG. Gambar IV.5 menggambarkan empat interaksi utama yang dapat dilakukan pengguna dengan sistem dalam bentuk *use case diagram*.



Gambar IV.5 *Use Case Diagram* Interaksi Pengguna dengan Sistem GraphRAG

Antarmuka menyediakan tiga komponen utama. Pertama, *chat interface* sebagai sarana masukan pertanyaan dalam bahasa alami. Kedua, area keluaran yang menampilkan jawaban naratif atau blok YAML konfigurasi beserta peringatan validasi apabila relevan. Ketiga, *Retrieval Trace Expander* yang menyajikan jejak penalaran graf dalam empat tab:

1. visualisasi Graphviz yang menampilkan *reasoning path* secara grafis;
2. tabel relasi *node* yang dilalui selama traversal;
3. referensi resmi *kubernetes.io* untuk setiap entitas yang ditemukan; dan
4. legenda kedalaman traversal.

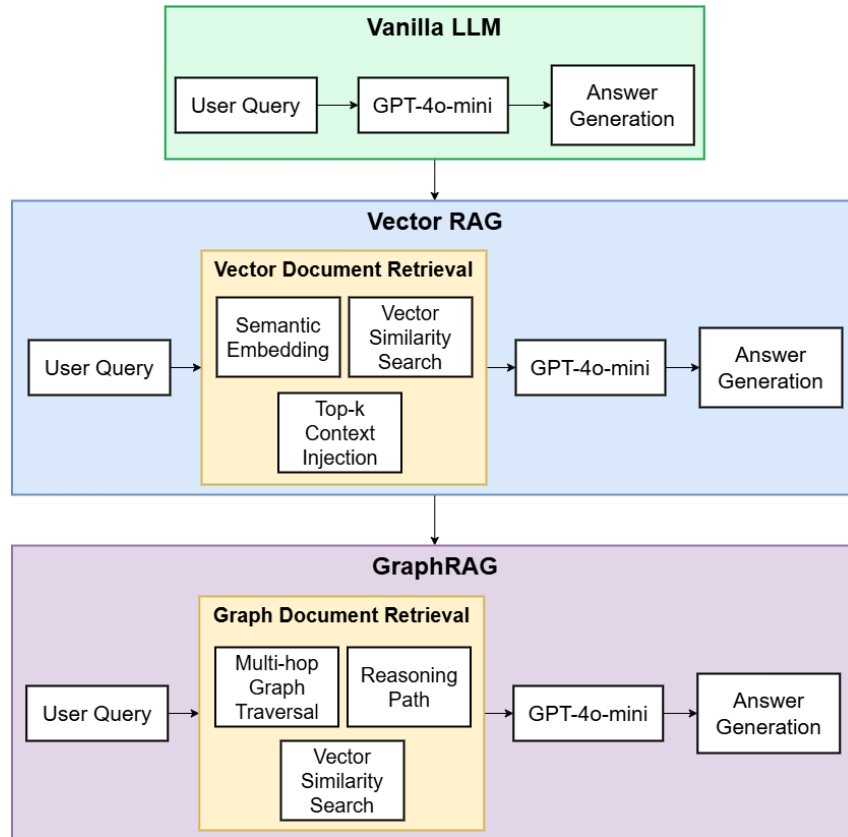
IV.5 Perancangan Sistem Pembandingan

Untuk memenuhi tujuan ketiga penelitian ini (T3), ditetapkan dua sistem pembandingan (*baseline*) yang digunakan dalam evaluasi komparatif pada Bab VI. Penetapan sistem pembandingan yang terdefinisi dengan baik merupakan prasyarat untuk menghasilkan perbandingan yang adil (*fair comparison*): ketiga sistem menggunakan model LLM yang sama (GPT-4o-mini) dan dataset uji yang sama (97 skenario uji atau *fixtue*) sehingga perbedaan kinerja dapat diatribusikan pada mekanisme *retrieval*, bukan pada kapabilitas model.

Ketiga sistem yang dibandingkan membentuk rangkaian evolusi yang progresif: Vector RAG menyempurnakan Vanilla LLM dengan menambahkan mekanisme *retrieval* berbasis kemiripan semantik, sedangkan GraphRAG menyempurnakan Vector RAG dengan menambahkan traversal graf *multi-hop*, klasifikasi *intent*, dan validasi YAML berbasis *knowledge graph*.

IV.5.1 Evolusi Rancangan Ketiga Sistem

Gambar IV.6 menggambarkan evolusi komponen ketiga sistem secara berlapis. Komponen dasar yang digunakan bersama oleh ketiga sistem ditampilkan pada lapisan pertama. Komponen yang ditambahkan oleh Vector RAG ditampilkan pada lapisan kedua, sedangkan komponen tambahan yang hanya dimiliki GraphRAG ditampilkan pada lapisan ketiga.



Gambar IV.6 Evolusi Rancangan: Vanilla LLM → Vector RAG → GraphRAG

IV.5.2 Rancangan Vanilla LLM

Vanilla LLM menggunakan GPT-4o-mini secara langsung tanpa mekanisme *retrieval* apapun. Pertanyaan pengguna diteruskan langsung ke model tanpa konteks tambahan sehingga jawaban bergantung sepenuhnya pada pengetahuan parametrik model. Alur pemrosesan bersifat langsung: kueri pengguna → GPT-4o-mini → jawaban. Sebagai kondisi dasar tanpa proses augmentasi, sistem ini digunakan sebagai acuan batas bawah kinerja untuk evaluasi komparatif antara ketiga sistem.

IV.5.3 Rancangan Vector RAG

Vector RAG menggunakan *retrieval* berbasis kemiripan *embedding* atas indeks vektor Neo4j. Kueri pengguna di-*embed* menggunakan model yang sama dengan GraphRAG, kemudian digunakan untuk mencari *node* paling mirip ($\text{top-}k=5$) dengan ekspansi satu lompatan ke *node* tetangga. Konteks hasil *retrieval* diteruskan ke GPT-4o-mini sebagai *speaker*. Alur pemrosesan adalah: kueri pengguna → *embedding* → pencarian vektor ($\text{top-}k=5$) → ekspansi 1 lompatan → konstruksi konteks → GPT-4o-mini → jawaban. Sistem ini merepresentasikan pendekatan RAG konvensional tanpa *intent-adaptive multi-hop traversal*.

IV.5.4 Rangkuman Perbandingan Komponen Teknis

Tabel IV.5 merangkum perbedaan komponen teknis ketiga sistem yang dibandingkan. Perbedaan komponen ini menjadi dasar interpretasi hasil evaluasi pada Bab VI: peningkatan kinerja yang konsisten dari Vanilla LLM ke Vector RAG dan dari Vector RAG ke GraphRAG mengindikasikan kontribusi tiap-tiap komponen tambahan terhadap aspek kualitas yang diukur.

Tabel IV.5 Perbandingan Komponen Teknis Ketiga Sistem

Aspek	Vanilla LLM	Vector RAG	GraphRAG
Metode <i>retrieval</i>	Tidak ada (pengetahuan parametrik)	Kemiripan vektor (top- $k=5$)	<i>Exact match</i> → vektor → <i>multi-hop traversal</i>
Sumber konteks	—	Neo4j Vector Index (1 lompatan)	Neo4j <i>Knowledge Graph</i> (2–3 lompatan)
Kedalaman traversal	—	Tetap (1 lompatan)	Adaptif per <i>intent</i> (kedalaman 2–3)
Klasifikasi <i>intent</i>	—	—	5 tipe <i>intent</i> (F-03)
Validasi YAML	—	—	Tiga lapis: sintaksis, skema, <i>KG</i> (F-07)
Manajemen memori	—	—	SQLite (riwayat percakapan)
Kebutuhan fungsional	—	F-05 (parsial)	F-01 s.d. F-07 (lengkap)

BAB V

IMPLEMENTASI SISTEM *GRAPH*RAG BERBASIS *KNOWLEDGE GRAPH* KUBERNETES

V.1 Lingkungan Implementasi

Implementasi sistem *GraphRAG* Kubernetes dilaksanakan menggunakan bahasa pemrograman Python 3.11 pada sistem operasi Windows 11. Seluruh dependensi dikelola melalui *virtual environment* dan didefinisikan dalam berkas `requirements.txt`. Tabel V.1 merangkum komponen utama beserta perannya dalam sistem.

Tabel V.1 Dependensi Utama Implementasi Sistem GraphRAG Kubernetes

Komponen	Versi	Fungsi dalam Sistem
LangGraph	1.0.9	Orkestrasi lima <i>AI agent</i> berbasis <i>state machine</i> (lihat Tabel V.5)
LangChain-Core	1.2.15	Penyimpanan riwayat percakapan dalam sesi yang sama melalui format pesan <i>BaseMessage/AIMessage</i>
LangChain-OpenAI	1.1.10	<i>ChatOpenAI</i> untuk node <i>Thinker</i> (temp=0,0) dan <i>Speaker</i> (temp=0,1); <i>OpenAIEmbeddings</i> untuk evaluasi
Neo4j Python Driver	6.1.0	Koneksi dan eksekusi <i>query</i> Cypher ke <i>knowledge graph</i>
OpenAI SDK	2.21.0	Pembuatan <i>embedding text-embedding-3-small</i>
Streamlit	1.54.0	Antarmuka <i>web</i> percakapan berbasis <i>browser</i>
PyYAML	6.0.3	Validasi sintaksis YAML
kubernetes-validate	1.35.0	Validasi terhadap skema Kubernetes v1.29
SQLite	<i>stdlib</i>	Penyimpanan riwayat percakapan antarsesi

V.2 Implementasi *Pipeline* Ekstraksi dan *Knowledge Graph*

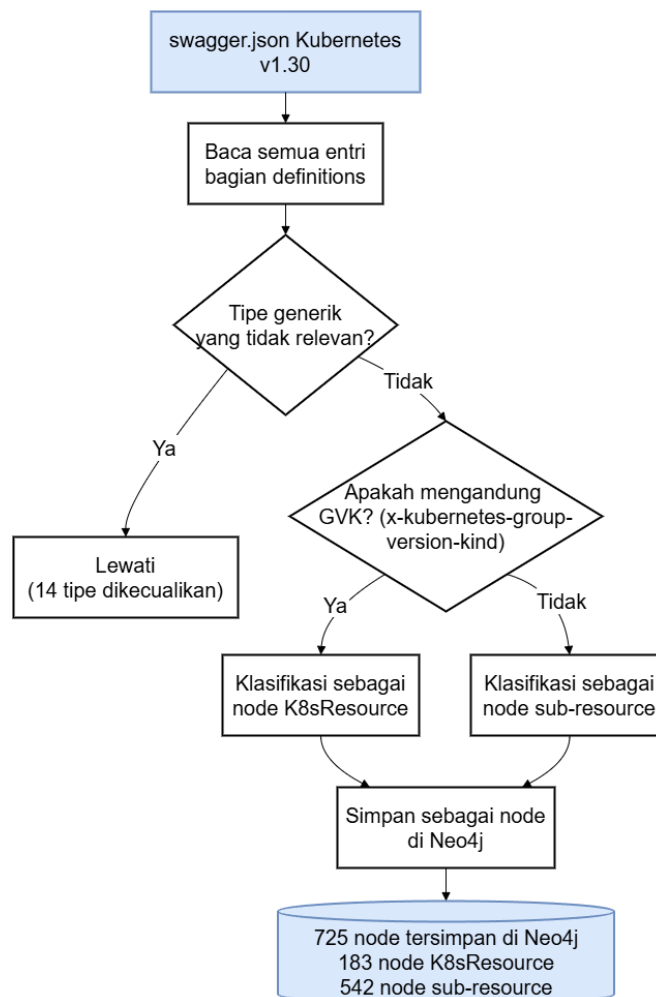
Bagian ini menguraikan realisasi teknis pembangunan *knowledge graph* Kubernetes yang terdiri atas tiga komponen utama sebagaimana dirancang pada Bab IV. Subbab V.2.1 menjelaskan proses ekstraksi dan pembuatan *node* dari spesifikasi OpenAPI. Subbab V.2.2 menjelaskan pembangunan 18 jenis relasi antarnode. Subbab V.2.3 menjelaskan pembuatan representasi vektor semantik untuk mendukung pencarian berbasis kemiripan vektor.

V.2.1 Implementasi *Pipeline Ingestion* dan Representasi Objek

Sistem mengkonstruksi *knowledge graph* dengan mengeksekusi kelas *SwaggerGraphBuilder* yang terdefinisi di `src/ingestion/parser.py` terhadap berkas `swagger.json` Kubernetes versi 1.30, sebagaimana dirancang pada Subbab IV.3.1. Seluruh fase di-

eksekusi secara berurutan.

Fase 1 membuat *node* dari setiap entri bagian *definitions* dalam *swagger.json*. Sistem memeriksa keberadaan *field* *x-kubernetes-group-version-kind* (GVK) pada setiap definisi untuk mengklasifikasikan *node* sebagai *K8sResource* (sumber daya Kubernetes utama yang memiliki GVK) atau *sub-resource* (tipe pendukung tanpa GVK). Sebanyak 5 tipe generik dikecualikan karena tidak relevan untuk pembuatan konfigurasi YAML.



Gambar V.1 Alur fase 1: Pembuatan *Node Definition* dari *swagger.json*

Eksekusi fase 1 menghasilkan 725 *node*, terdiri atas 183 *node* *K8sResource* dan 542 *node* *sub-resource*. Fase selanjutnya, yaitu fase 1.5 hingga fase 3, membangun representasi vektor dan relasi antarnode sebagaimana diuraikan pada Subbab V.2.3 dan Subbab V.2.2.

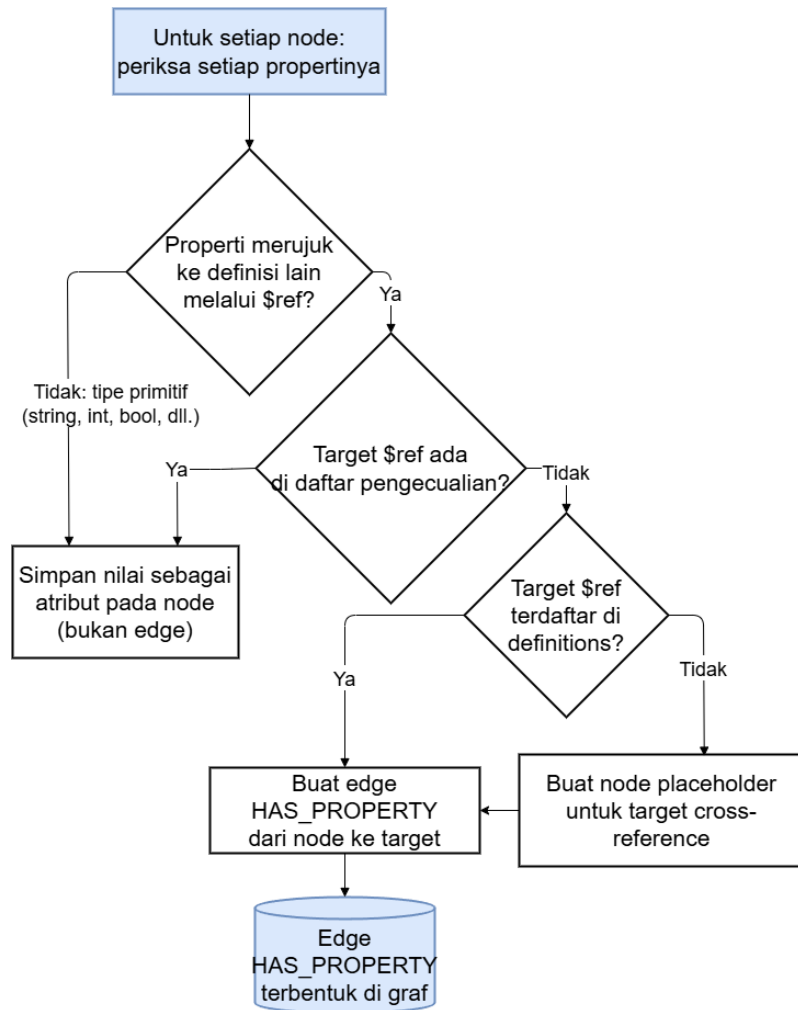
V.2.2 Implementasi Taksonomi Relasi *Knowledge Graph*

Relasi antarnode dibangun melalui dua strategi komplementer sebagaimana dirancang pada Subbab IV.3.2. Strategi deterministik mengubah *field* OpenAPI (\$ref, allOf, oneOf, anyOf) menjadi *edge* secara langsung pada fase 2 dan fase 2.5. Strategi semi-deterministik menerapkan 26 aturan berbasis pengetahuan domain Kubernetes pada fase 3. Jumlah 18 jenis *edge* bukan nilai yang ditetapkan secara apriori, melainkan hasil enumerasi lengkap: empat jenis dari *field* OpenAPI dan 14 jenis dari pola operasional Kubernetes di tujuh domain fungsional. Tujuh kategori relasi beserta definisinya ditampilkan pada Tabel V.2.

Tabel V.2 Tujuh Kategori Relasi dalam *Knowledge Graph* Kubernetes

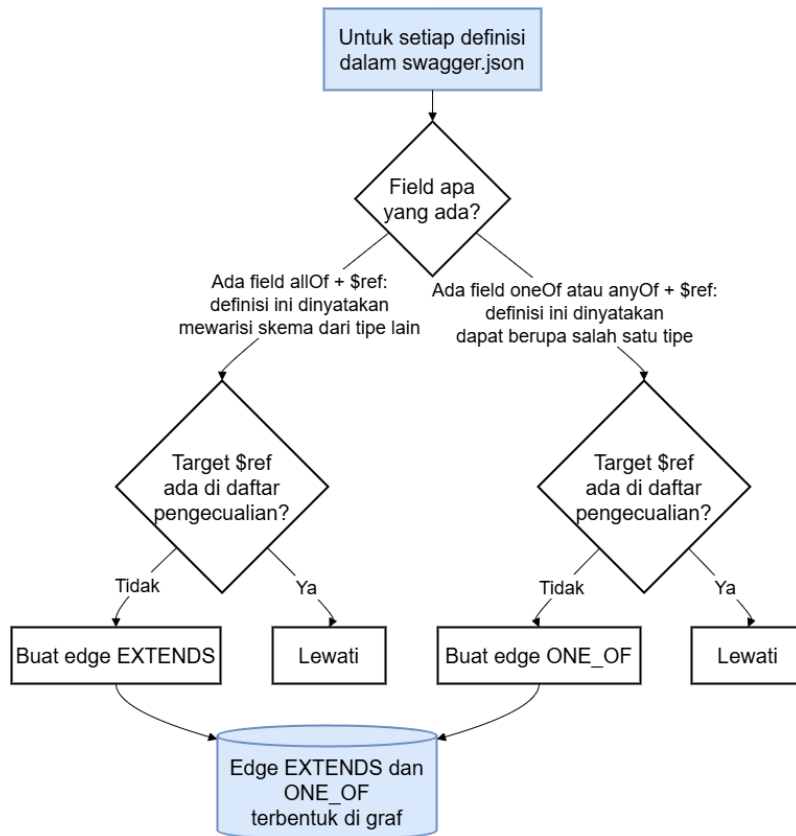
Kategori	Strategi	Definisi Fungsional
Struktural	Deterministik	Relasi langsung dari <i>field</i> OpenAPI (\$ref, allOf, oneOf, anyOf); merepresentasikan hierarki dan komposisi skema
<i>Workload</i>	Semi-deterministik	Relasi komposisi beban kerja; <i>resource</i> tingkat tinggi mengandung dan mengelola <i>Pod/Container</i>
Penyimpanan	Semi-deterministik	Relasi manajemen volume persisten dari klaim PVC hingga pemilihan <i>StorageClass</i>
Konfigurasi	Semi-deterministik	Relasi injeksi konfigurasi <i>runtime</i> ke <i>Container</i> melalui <i>ConfigMap</i> dan <i>Secret</i>
Jaringan	Semi-deterministik	Relasi <i>routing</i> trafik dari <i>Ingress</i> ke <i>Service</i> hingga ke <i>Pod</i> via selektor label
RBAC	Semi-deterministik	Relasi kontrol akses berbasis peran; <i>Role-Binding</i> ke <i>Role</i> dan <i>ServiceAccount</i>
<i>Autoscaling</i>	Semi-deterministik	Relasi target penskalaan otomatis HPA ke <i>resource workload</i>

Fase 2 mengimplementasikan strategi deterministik dengan menelusuri setiap properti pada tiap definisi. Properti bertipe primitif disimpan sebagai atribut *node*, sedangkan properti yang merujuk ke definisi lain melalui *field* \$ref dikonversi menjadi *edge HAS_PROPERTY*.



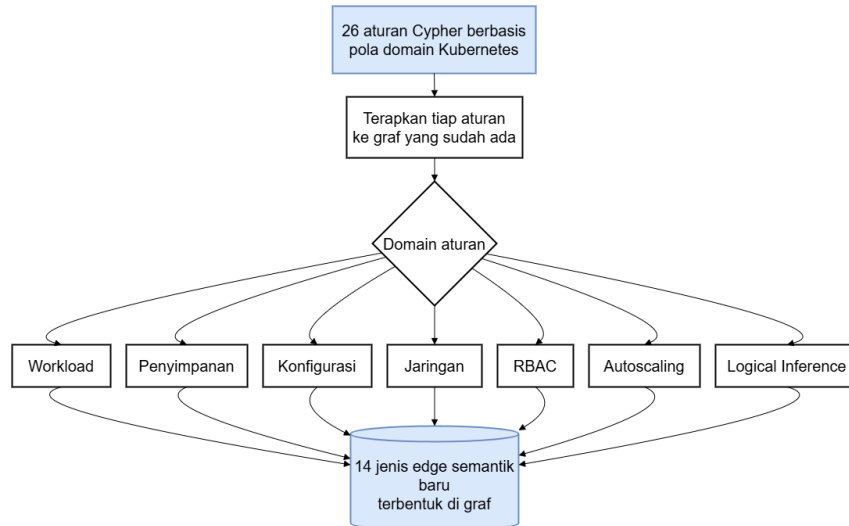
Gambar V.2 Alur fase 2: Pembuatan *Edge* Struktural dari Referensi \$ref

Fase 2 hanya menangani referensi properti langsung. Skema OpenAPI juga mendefinisikan pola pewarisan dan tipe alternatif melalui *field* `allOf`, `oneOf`, dan `anyOf` yang ditangani oleh fase 2.5.



Gambar V.3 Alur fase 2.5: Pembuatan *Edge* Hierarki dan Tipe Alternatif

Keempat jenis *edge* struktural dari fase 2 dan fase 2.5 merepresentasikan hubungan yang dinyatakan secara eksplisit dalam skema OpenAPI. Relasi operasional Kubernetes yang tidak tertulis dalam skema, seperti hubungan antara *Service* dan *Pod* atau antara *RoleBinding* dan *ServiceAccount*, memerlukan pendekatan berbeda. Fase 3 menerapkan 26 aturan Cypher berbasis pengetahuan domain untuk membangun 14 jenis *edge* semantik tambahan.



Gambar V.4 Alur fase 3: Pembuatan *Edge* Semantik dari Pola Domain Kubernetes

Kombinasi ketiga *pass* menghasilkan 18 jenis *edge* yang merepresentasikan hubungan struktural dan operasional Kubernetes secara lengkap. Daftar seluruh jenis *edge* beserta definisi dan sumber ekstraksinya ditampilkan pada Tabel V.3.

Tabel V.3 Taksonomi 18 Jenis *Edge* dalam *Knowledge Graph* Kubernetes

<i>Edge Type</i>	Kategori	Definisi	Sumber
<i>HAS_PROPERTY</i>	Struktural	A memiliki properti ber-tipe B	Fase 2 (\$ref)
<i>EXTENDS</i>	Struktural	A mewarisi semua properti B	Fase 2.5 (allOf)
<i>ONE_OF</i>	Struktural	A bertipe tepat satu alternatif dari B	Fase 2.5 (oneOf)
<i>ANY_OF</i>	Struktural	A dapat bertipe satu atau lebih alternatif	Fase 3 (logical)
<i>CONTAINS_POD_TEMPLATE</i>	<i>Workload</i>	<i>Resource workload</i> mengandung spesifikasi <i>Pod</i> yang dikelolanya	Fase 3
<i>CONTAINS_JOB_TEMPLATE</i>	<i>Workload</i>	<i>CronJob template Job</i> mengandung <i>Job</i> yang dieksekusi periodik	Fase 3

bersambung ke halaman berikutnya

Tabel V.3 Taksonomi 18 Jenis *Edge* (lanjutan)

<i>Edge Type</i>	Kategori	Definisi	Sumber
<i>HAS_CONTAINER</i>	<i>Workload</i>	<i>PodSpec</i> mendefinisikan <i>Container</i> yang dieksekusi	Fase 3
<i>CLAIMS_VOLUME</i>	Penyimpanan	<i>StatefulSet</i> mengklaim PVC untuk penyimpanan persisten	Fase 3
<i>MOUNTS_VOLUME</i>	Penyimpanan	<i>PodSpec</i> memasang <i>Volume</i> ke <i>filesystem Container</i>	Fase 3
<i>USES_STORAGE_CLASS</i>	Penyimpanan	PVC memilih <i>Storage-Class</i> untuk <i>provisioning</i> dinamis	Fase 3
<i>LOADS_CONFIGMAP</i>	Konfigurasi	<i>Container</i> memuat konfigurasi <i>runtime</i> dari <i>ConfigMap</i>	Fase 3
<i>USES_SECRET</i>	Konfigurasi	<i>PodSpec</i> menggunakan <i>Secret</i> untuk data sensitif	Fase 3
<i>SELECTS_POD</i>	Jaringan	<i>Service</i> merutekan trafik ke <i>Pod</i> via selektor label	Fase 3
<i>ROUTES_TO_SERVICE</i>	Jaringan	<i>Ingress</i> meneruskan trafik HTTP/HTTPS ke <i>Service</i>	Fase 3
<i>BINDS_ROLE</i>	RBAC	<i>RoleBinding</i> memberikan izin <i>Role/ClusterRole</i> ke subjek	Fase 3
<i>BINDS_SERVICE_ACCOUNT</i>	RBAC	<i>RoleBinding</i> mengikat <i>ServiceAccount</i> sebagai subjek	Fase 3

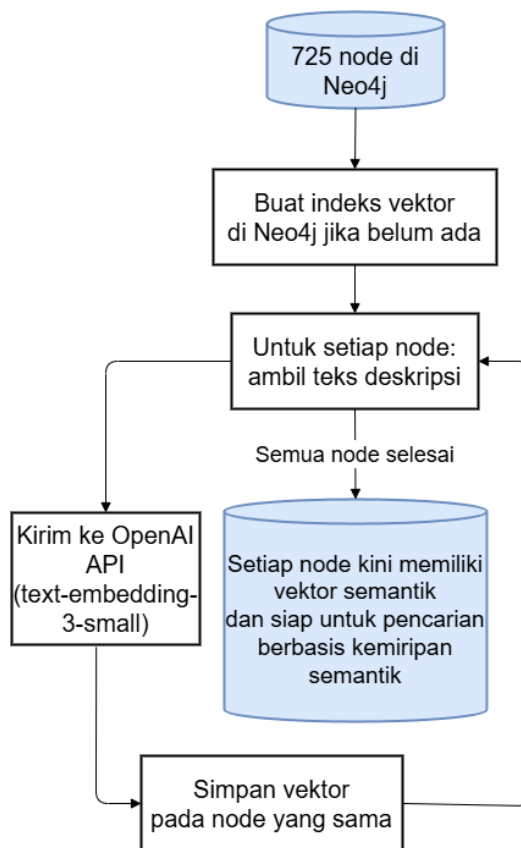
bersambung ke halaman berikutnya

Tabel V.3 Taksonomi 18 Jenis *Edge* (lanjutan)

<i>Edge Type</i>	Kategori	Definisi	Sumber
<i>USES_SERVICE_ACCOUNT</i>	RBAC	<i>PodSpec</i> menjalankan <i>Pod</i> dengan identitas <i>ServiceAccount</i>	Fase 3
<i>SCALES_RESOURCE</i>	Autoscaling	HPA menarget <i>Deployment/StatefulSet</i> untuk penskalaan otomatis	Fase 3

V.2.3 Implementasi Pembentukan *Embedding* Semantik

Pembangunan *embedding* semantik dilakukan pada proses fase 1.5 yang dieksekusi setelah fase 1 selesai dan sebelum pembangunan *edge* dimulai. Tujuannya adalah menghasilkan representasi vektor untuk setiap *node* agar pencarian berbasis kemiripan semantik dapat dilakukan langsung di Neo4j tanpa *vector store* eksternal. Gambar V.5 menunjukkan alur eksekusi fase 1.5 secara keseluruhan.



Gambar V.5 Alur fase 1.5: Pembuatan Vektor *Embedding* untuk Seluruh *Node*

Vektor disimpan sebagai properti *embedding* pada setiap *node* di Neo4j dan diindeks menggunakan *Native Vector Index* dengan fungsi kemiripan *cosine*. Pendekatan ini memungkinkan *vector similarity search* dilakukan dalam satu *query* Cypher tanpa kebutuhan *vector store* eksternal terpisah. Implementasi kode lengkap kelas *SwaggerGraphBuilder* (fase 1 hingga fase 3) disajikan pada Lampiran A.2.

Ketiga subbab di atas secara bersama-sama merealisasikan pembangunan *knowledge graph* Kubernetes. Tabel V.4 merangkum statistik akhir *knowledge graph* yang dihasilkan dan menjadi fondasi mekanisme *GraphRAG* yang diuraikan pada bagian berikutnya.

Tabel V.4 Statistik *Knowledge Graph* Kubernetes Hasil Konstruksi

Properti	Nilai
Sumber data	swagger . json Kubernetes v1.30
Total node <i>Definition</i>	725
Total jenis relasi (<i>edge type</i>)	18
Kategori relasi	7 (Struktural, <i>Workload</i> , Penyimpanan, Konfigurasi, Jaringan, RBAC, <i>Autoscaling</i>)
Model <i>embedding</i>	<i>text-embedding-3-small</i> (OpenAI)
Dimensi vektor <i>embedding</i>	1.536
Penyimpanan <i>vector index</i>	Neo4j Native Vector Index (kemiripan <i>cosine</i>)

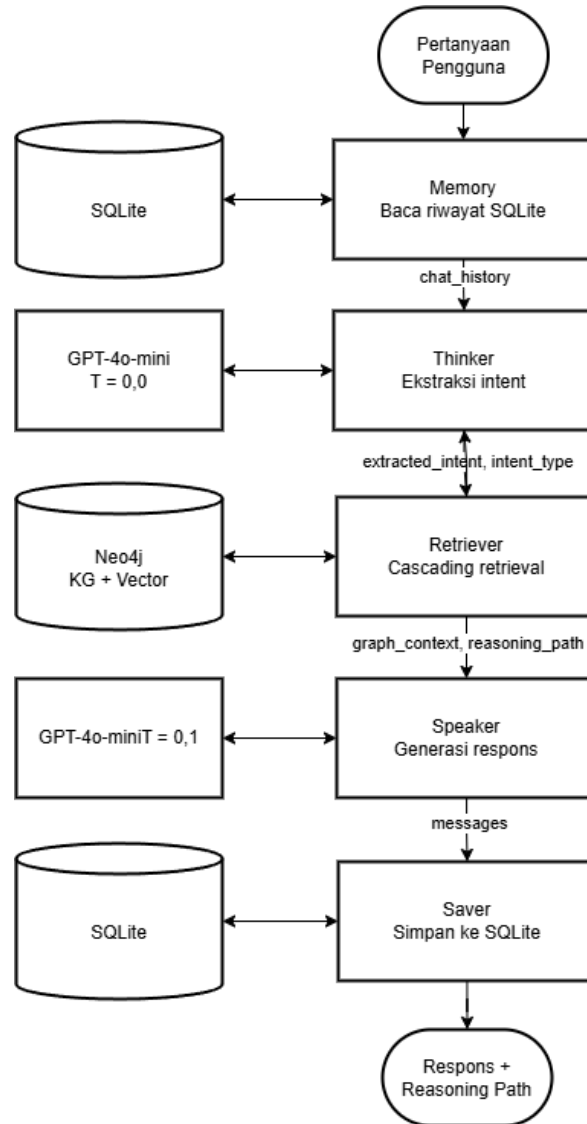
V.3 Implementasi Mekanisme *GraphRAG*

Pipeline chatbot diimplementasikan sebagai *directed acyclic graph* (DAG) menggunakan LangGraph pada berkas `src/chatbot/graph_agent.py`. *State* yang dipertukarkan antarmodul *agent* didefinisikan sebagai *TypedDict* dengan nama `AgentState` yang memuat sembilan *field*: `messages`, `question`, `session_id`, `chat_history`, `extracted_intent`, `graph_context`, `reasoning_path`, `intent_type`, dan `error`. Lima modul *agent* yang dieksekusi secara linear sebagaimana ditunjukkan pada Tabel V.5 diimplementasikan masing-masing sebagai fungsi Python yang menerima dan mengembalikan `AgentState`.

Tabel V.5 Lima Modul *Agent* LangGraph Pipeline

Modul	Fungsi	Aktivitas	Komponen
<i>memory</i>	Ambil riwayat	Membaca 5 giliran percakapan terakhir dari penyimpanan SQLite.	SQLite (<i>SQLite-MemoryStore</i>)
<i>thinker</i>	Ekstraksi <i>intent</i>	Menganalisis pertanyaan + riwayat, mengeluarkan JSON berisi <i>primary_resource</i> , <i>related_concepts</i> , dan <i>intent_type</i> .	GPT-4o-mini
<i>retriever</i>	Eksekusi <i>retrieval</i>	Meneruskan JSON intent ke <i>StatefulK8sRetriever</i> untuk retrieval bertingkat; menghasilkan <i>graph_context</i> dan <i>reasoning_path</i> .	<i>StatefulK8sRetriever</i>
<i>speaker</i>	Generasi respons	Menggunakan konteks graf dan <i>reasoning_path</i> untuk menghasilkan jawaban naratif atau blok YAML.	GPT-4o-mini
<i>saver</i>	Simpan riwayat	Menyimpan pasangan pertanyaan–jawaban ke SQLite untuk giliran percakapan berikutnya.	SQLite (<i>SQLite-MemoryStore</i>)

Interaksi antarmodul *agent* beserta aliran data melalui *AgentState* ditunjukkan pada Gambar V.6. Setiap panah berlabel *field* *AgentState* diteruskan dari satu modul *agent* ke modul *agent* berikutnya sehingga dependensi data antartahap dapat dilacak secara eksplisit.



Gambar V.6 Diagram interaksi lima modul *agent LangGraph*

Arsitektur *dual-LLM* diimplementasikan dengan konfigurasi *temperature* yang berbeda untuk setiap peran: modul *thinker* menggunakan *temperature*=0,0 untuk keluaran JSON yang deterministik, sedangkan modul *speaker* menggunakan *temperature*=0,1 sebagai titik keseimbangan antara *faithfulness* terhadap konteks graf dan variasi naratif respons. Detail implementasi setiap modul *agent* diuraikan pada Subbab V.3.1 hingga Subbab V.3.4.

V.3.1 Implementasi Klasifikasi *Intent* dan Manajemen Memori

Modul *thinker* mengimplementasikan kebutuhan fungsional F-03 (*Intent Classification*) sebagaimana dirancang pada Subbab IV.4.1. Modul ini menggunakan GPT-4o-mini dengan *temperature*=0,0 untuk menghasilkan keluaran JSON yang determi-

nistik tanpa variasi *sampling*. Keluaran JSON memuat dua entitas utama: *intent_type* yang mengklasifikasikan jenis kueri ke dalam lima tipe sebagaimana didefinisikan pada Tabel V.6, serta *primary_resource* dan *related_concepts* yang mengidentifikasi *resource* Kubernetes yang disebutkan dalam kueri. Konsistensi JSON pada setiap eksekusi krusial karena keluaran yang tidak konsisten akan menyebabkan kegagalan *parsing* pada modul *retriever*.

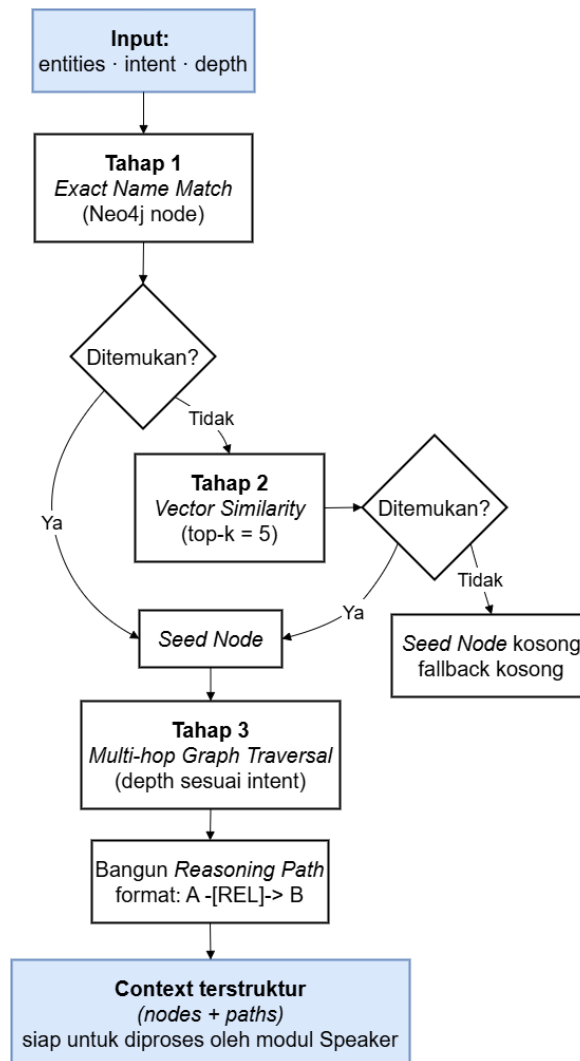
Tabel V.6 Definisi Tipe *Intent* beserta Contoh Pertanyaan

<i>Intent Type</i>	Definisi	Contoh Pertanyaan
<i>explain</i>	Menanyakan definisi, cara kerja, atau tujuan suatu <i>resource</i> Kubernetes.	"Apa fungsi <i>ConfigMap</i> dan bagaimana cara kerjanya?"
<i>generate_yaml</i>	Meminta pembuatan atau konfigurasi manifes YAML untuk <i>resource</i> Kubernetes.	"Buatkan YAML <i>ClusterRole</i> untuk akses <i>read-only</i> ke <i>Pod</i> ."
<i>trace_relationship</i>	Menanyakan hubungan atau interaksi antara dua <i>resource</i> atau lebih.	"Mengapa <i>field</i> port pada <i>Container</i> bisa menerima nilai <i>integer</i> maupun <i>string</i> ?"
<i>followup</i>	Meminta modifikasi atau kelanjutan dari jawaban atau konfigurasi pada giliran sebelumnya.	"Tambahkan <i>environment variable</i> <i>DB_HOST</i> yang nilainya diambil dari <i>ConfigMap</i> <i>app-config</i> ."
<i>planning</i>	Meminta perancangan arsitektur sistem multi-komponen Kubernetes.	"Rancang <i>setup</i> agar aplikasi dapat <i>auto-scale</i> dan tetap tersedia saat diperbarui."

Untuk penyimpanan percakapan, sistem menggunakan SQLite lokal yang diakses melalui kelas *SQLiteMemoryStore*. SQLite dipilih karena memberikan persistensi yang memadai dengan latensi rendah tanpa ketergantungan infrastruktur tambahan. Riwayat percakapan disimpan secara deterministik berdasarkan UUID sesi dan dimuat ulang pada setiap interaksi untuk memungkinkan penyelesaian referensi anafora seperti "hal tersebut" atau "konfigurasi tadi". Hasil klasifikasi *intent* beserta riwayat percakapan ini selanjutnya diteruskan ke modul *retriever* untuk pengambilan konteks.

V.3.2 Implementasi Strategi *Retrieval* dan Konstruksi Konteks

Mekanisme *retrieval* diimplementasikan dalam kelas *StatefulK8sRetriever* pada berkas `src/chatbot/custom_retriever.py` sebagaimana dirancang pada Subbab IV.4.2. *Method* utama `retrieve_context()` menerima parameter `intent_data` (hasil ekstraksi modul *thinker*), `intent_type` (string jenis *intent*), dan `max_depth` (opsional) untuk menghasilkan pasangan (`graph_context_json`, `reasoning_path`).



Gambar V.7 Alur *cascading retrieval* tiga tahap

Tahap 1 mengeksekusi `EXACT_MATCH_QUERY` ke Neo4j menggunakan nama entitas yang diekstraksi modul *thinker*. Apabila Tahap 1 tidak menemukan kecocokan, sistem beralih ke Tahap 2 yang mengeksekusi `HYBRID_VECTOR_GRAPH_QUERY` menggunakan *embedded string* berupa konkatenasi dari `primary_resource`, seluruh elemen `related_concepts`, dan kata kunci “Kubernetes”. Tahap 3 melakukan

multi-hop graph traversal dari *seed node* yang diperoleh dengan kedalaman yang ditetapkan berdasarkan tipe *intent*.

Pemetaan kedalaman berbasis *intent* diimplementasikan sebagai kamus `_DEPTH_BY_INTENT` di tingkat modul sebagaimana ditunjukkan pada Listing V.1. Prioritas resolusi kedalaman adalah: (1) argumen `max_depth` yang diberikan secara eksplisit, (2) nilai dari `_DEPTH_BY_INTENT` berdasarkan `intent_type`, dan (3) nilai *default* tiga lompatan. Pemetaan nilai kedalaman untuk setiap tipe *intent* beserta alasan penetapannya ditunjukkan pada Tabel V.7.

Tabel V.7 Pemetaan *Intent Type* terhadap Kedalaman Traversal Graf

<i>Intent Type</i>	Depth	Alasan
<i>explain</i>	2	Cukup untuk menjawab definisi dan cara kerja <i>resource</i> ; depth 3+ menambahkan <i>noise</i> generik.
<i>followup</i>	2	Pertanyaan lanjutan biasanya merujuk entitas yang sama; konteks depth 2 sudah tersedia dari giliran sebelumnya.
<i>generate_yaml</i>	3	Pembuatan YAML membutuhkan field tingkat kontainer (<i>image</i> , <i>ports</i> , <i>env</i>) yang berada di depth 3.
<i>trace_relationship</i>	3	Penelusuran relasi antar <i>resource</i> menjangkau jembatan lintas- <i>resource</i> pada depth 2–3.
<i>planning</i>	3	Pertanyaan perencanaan arsitektur membutuhkan konteks dari hingga dua <i>related_concepts</i> secara bersamaan; kedalaman 3 diperlukan untuk menjangkau komponen spesifikasi yang dibutuhkan.

```

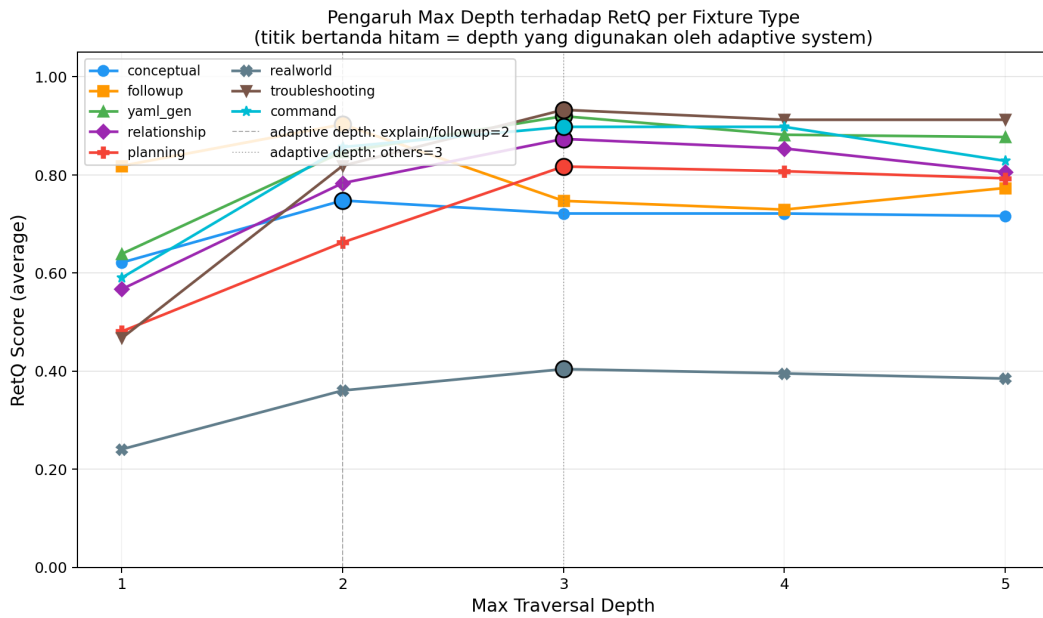
1 _DEPTH_BY_INTENT = {
2     "explain":          2,
3     "followup":         2,
4     "generate_yaml":    3,
5     "trace_relationship": 3,
6     "planning":         3,
7 }

```

```
8 _DEFAULT_DEPTH = 3 # fallback untuk intent yang tidak dikenali
```

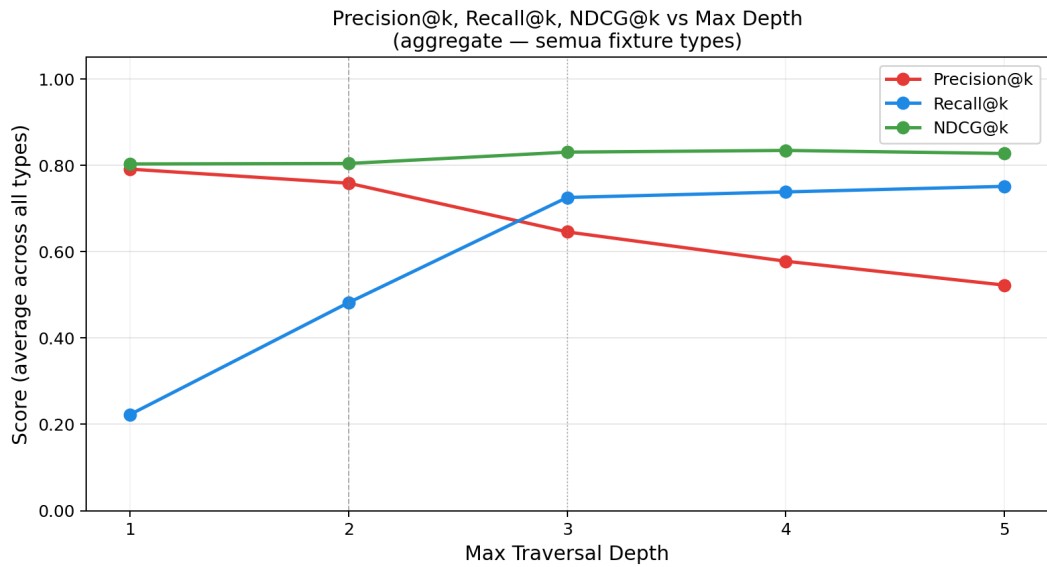
Listing V.1 Kamus kedalaman *traversal* per tipe *intent* (`_DEPTH_BY_INTENT`)

Pemilihan nilai kedalaman per tipe *intent* divalidasi secara empiris dengan mengevaluasi seluruh 97 *fixture* pada lima variasi kedalaman ($d \in \{1, 2, 3, 4, 5\}$). Hasil tiap kedalaman tetap dibandingkan terhadap konfigurasi *intent-adaptive* yang memilih kedalaman berdasarkan `_DEPTH_BY_INTENT` untuk memastikan kedalaman per *intent* berada pada titik optimum tiap kategori. Gambar V.8 menunjukkan pengaruh kedalaman *traversal* terhadap skor RetQ per kategori *fixture*, sedangkan Gambar V.9 menunjukkan tren *Precision@k*, *Recall@k*, dan *NDCG@k* secara agregat.



Gambar V.8 Pengaruh kedalaman *traversal* terhadap RetQ per kategori *fixture*

Titik bertanda hitam menunjukkan kedalaman yang digunakan oleh sistem *adaptive*; garis putus-putus vertikal menandai kedalaman 2 (*conceptual/followup*) dan kedalaman 3 (kategori lainnya).



Gambar V.9 *Precision@k*, *Recall@k*, dan *NDCG@k* terhadap kedalaman *traversal*

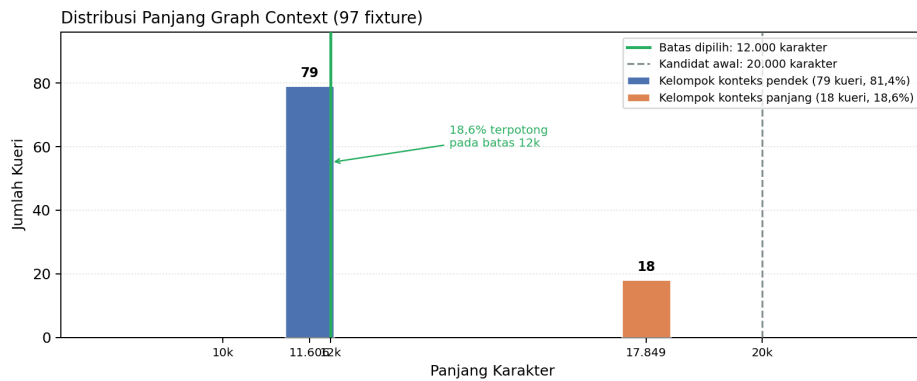
Nilai optimal *Precision@k*, *Recall@k*, dan *NDCG@k* terpusat pada kedalaman 2–3; skor stabil atau menurun pada kedalaman 4 ke atas akibat masuknya *node* generik yang menurunkan presisi konteks.

Untuk *intent* yang melibatkan lebih dari satu *resource*, yaitu *planning*, *trace_relationship*, dan *generate_yaml*, implementasi menambahkan *loop* tambahan yang mengambil konteks dari hingga dua *related_concepts* yang diekstraksi modul *thinker* dan menggabungkan hasilnya, baik *graph_context* maupun *reasoning_path*, dengan hasil dari *primary_resource*. Penanganan ini diperlukan karena pertanyaan relasional dan generasi YAML multi-*resource* membutuhkan konteks skema dari seluruh entitas yang disebutkan. Deduplikasi diterapkan pada *reasoning_path* agar langkah *traversal* yang identik antarsumber daya tidak dicatat lebih dari satu kali.

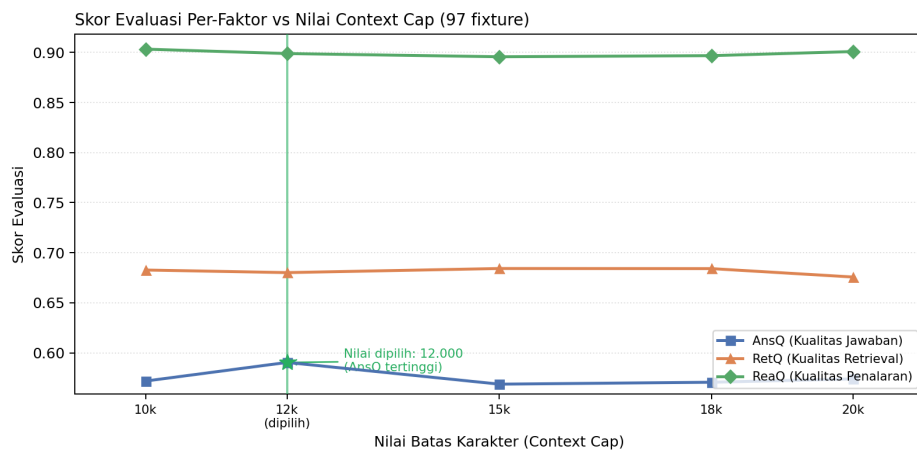
Untuk memenuhi kendala latensi (NF-02), *graph context* yang disampaikan ke modul *speaker* dibatasi maksimum 12.000 karakter. Nilai ini dipilih melalui dua tahap analisis yang saling melengkapi, sebagaimana ditunjukkan pada Gambar V.10 dan Gambar V.11.

Tahap pertama adalah analisis distribusi panjang *graph context* terhadap seluruh 97 *fixture* evaluasi menggunakan mekanisme *dry-run retriever*. Analisis tersebut mengungkap distribusi bimodal: 79 kueri (81,4%) menghasilkan konteks sepanjang 11.606 karakter dan 18 kueri (18,6%) menghasilkan 17.849 karakter, sesuai dengan kedalaman *traversal intent-adaptive* yang berbeda (Gambar V.10).

Tahap kedua adalah evaluasi empiris terhadap lima nilai batas karakter yang berbeda, yaitu 10.000, 12.000, 15.000, 18.000, dan 20.000. Pengujian ini dilakukan dengan mengeksekusi seluruh 97 *fixture* evaluasi pada tiap-tiap nilai batas (Gambar V.11). Hasil evaluasi per-faktor menunjukkan bahwa nilai 12.000 karakter menghasilkan AnsQ tertinggi di antara semua kandidat. Batas yang lebih besar (15k–20k) mengakibatkan penurunan *faithfulness* pada submetrik AnsQ karena konteks yang terlalu panjang sehingga model *speaker* kehilangan fokus pada dokumen referensi. Sebaliknya, batas 10.000 menghasilkan ReaQ yang lebih tinggi tetapi AnsQ yang lebih rendah karena konteks yang tidak memadai untuk generasi jawaban yang lengkap. GPT-4o-mini mendukung jendela konteks 128.000 token sehingga batas ini bukan kendala kapasitas, melainkan pengendali kualitas, biaya, dan latensi.



Gambar V.10 Distribusi panjang *graph context* pada 97 *fixture*



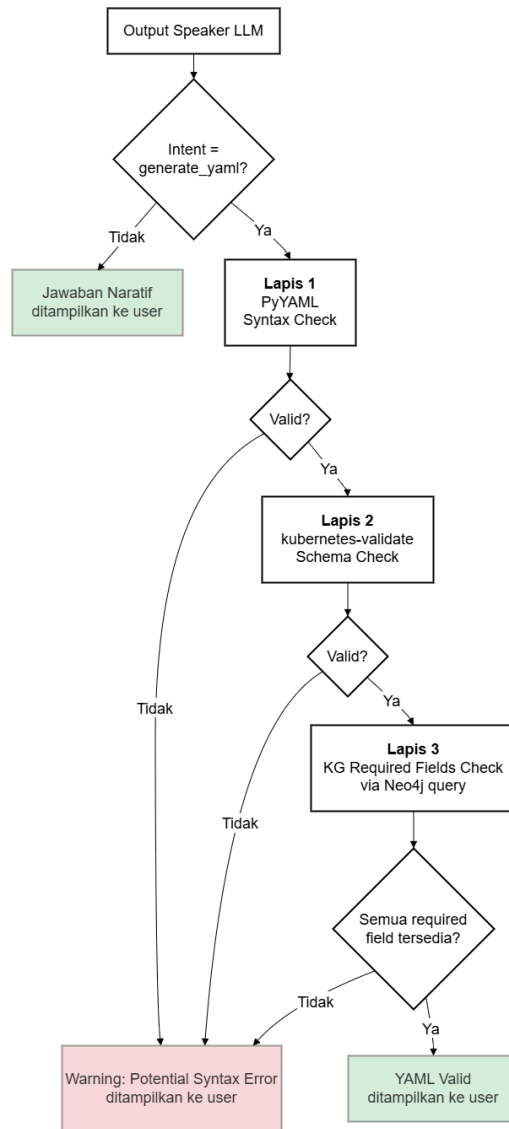
Gambar V.11 Skor evaluasi per-faktor terhadap nilai batas karakter

Konteks terstruktur yang telah dibatasi selanjutnya diteruskan ke modul *speaker* bersama *reasoning path* untuk proses generasi respons.

V.3.3 Implementasi Generasi dan Validasi YAML

Modul *speaker* mengimplementasikan dua jalur keluaran berdasarkan tipe *intent* sebagaimana dirancang pada Subbab IV.4.3. Apabila *intent* diklasifikasikan sebagai *generate_yaml*, *speaker* menghasilkan blok kode YAML konfigurasi Kubernetes yang kemudian divalidasi. Apabila *intent* bukan *generate_yaml*, *speaker* menghasilkan jawaban naratif tanpa proses validasi YAML.

Validasi YAML diimplementasikan dalam kelas *YAMLValidator* pada berkas `src/validation/yaml_validator.py`. Tiga lapisan validasi dieksekusi secara berurutan sebagaimana ditunjukkan pada Gambar V.12. Hasil validasi berupa objek *YAMLValidationResult* dengan empat *field*: *valid* (*Boolean*), *syntax_errors* (daftar kesalahan sintaksis PyYAML), *schema_errors* (daftar pelanggaran skema Kubernetes), dan *missing_fields* (daftar *required field* yang tidak ditemukan dalam YAML yang dihasilkan). Ketika salah satu lapisan mendeteksi kegagalan, antarmuka menampilkan peringatan `Warning: Potential Syntax Error` kepada pengguna.



Gambar V.12 Alur validasi YAML tiga lapis berbasis *knowledge graph*

V.3.4 Implementasi Antarmuka *Web* Interaktif

Antarmuka pengguna diimplementasikan menggunakan *Streamlit* pada berkas `main.py` sebagaimana dirancang pada Subbab IV.4.4. Keempat *use case* pada Gambar IV.5 direalisasikan sebagai fitur-fitur berikut.

UC01: Mengajukan Pertanyaan Teknis Kubernetes. Masukan pengguna diterima melalui `st.chat_input()` di bagian bawah antarmuka. Setiap sesi percakapan mendapatkan UUID unik yang disimpan di `st.session_state` dan digunakan sebagai kunci pengambilan riwayat percakapan dari SQLite. Manajemen multi-sesi diimplementasikan melalui *sidebar* dengan tombol “Chat Baru” untuk memulai sesi baru dan daftar sesi sebelumnya yang dapat diakses kembali. Setiap pertanyaan

diteruskan ke `agent_graph.invoke()` bersama `session_id` yang aktif.

UC02: Melihat Jawaban Sistem. Jawaban sistem dirender melalui fungsi `render_assistant_message()` di dalam `st.chat_message("assistant")`. Setiap respons diawali dengan *intent badge* yang menampilkan tipe *intent* dan *primary resource* hasil ekstraksi modul *thinker*, disertai indikator *grounding* yang berwarna berdasarkan panjang *reasoning path*: kuning untuk “Pengetahuan Parametrik LLM”, biru untuk “Sebagian informasi diambil dari Graph”, dan hijau untuk “Seluruh informasi diambil dari *Knowledge Graph*”.

UC03: Melihat Jejak Penalaran. Jejak penalaran ditampilkan melalui fungsi `render_retrieval_trace()` dalam komponen `st.expander`. Komponen ini memuat baris metrik (*Root Resource*, API Version, jumlah *node*, jumlah *edge*, dan kedalaman maksimum) serta empat tab:

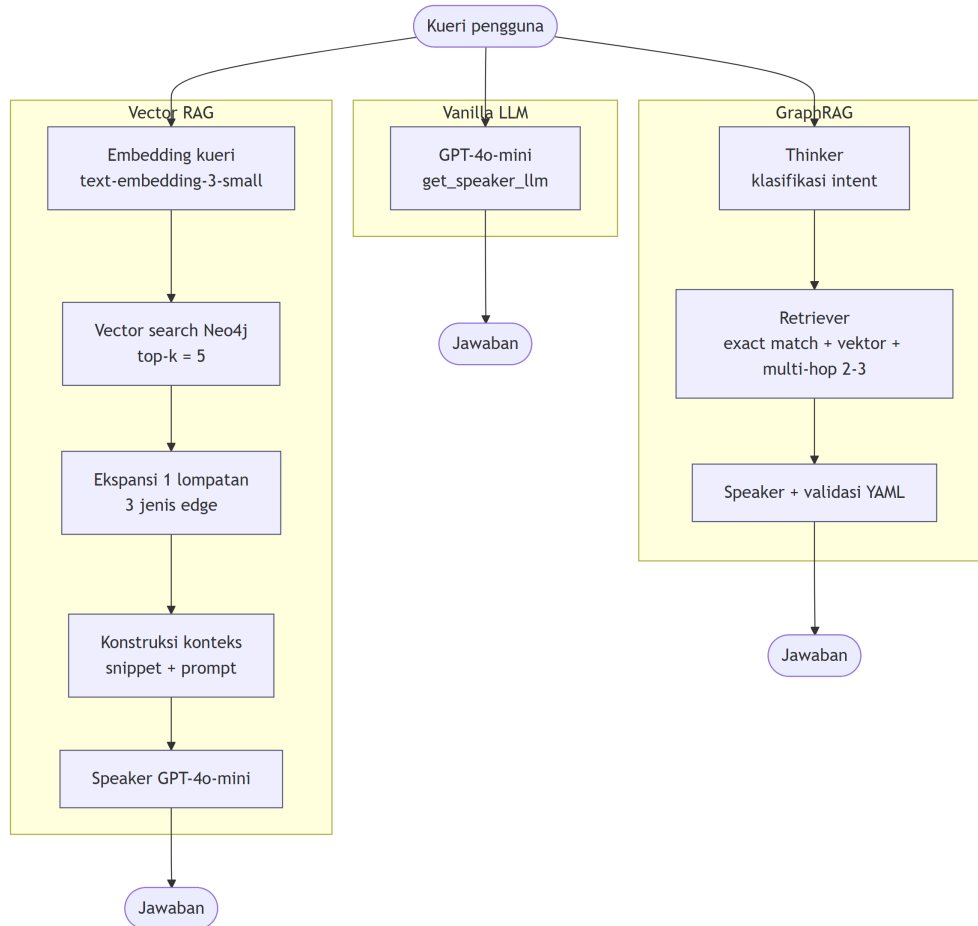
1. *Graph Visualization*: *reasoning path* dirender sebagai graf berarah dengan menggunakan modul *Graphviz* yang memiliki kode warna berdasarkan kedalaman traversal;
2. *Edge Table*: seluruh relasi traversal dalam *DataFrame* yang diurutkan berdasarkan kedalaman;
3. *Sumber Referensi*: tautan dokumentasi resmi *kubernetes.io* untuk setiap *resource* yang ditemukan; dan
4. *Keterangan Warna*: legenda pemetaan warna *node* terhadap kedalaman traversal.

UC04: Menerima Konfigurasi YAML. Ketika *intent* diklasifikasikan sebagai *generate_yaml*, blok YAML dalam respons modul *speaker* diekstraksi menggunakan ekspresi reguler di fungsi `render_response_with_yaml()`. Setiap blok YAML dirender menggunakan `st.code(language="yaml")` yang mengaktifkan tombol salin bawaan Streamlit. Apabila *YAMLValidator* mendeteksi pelanggaran pada salah satu lapisan validasi, peringatan `Warning: Potential Syntax Error` ditampilkan sebagai bagian dari respons.

V.4 Implementasi Sistem Pembanding

Kedua sistem pembanding yang dirancang pada Subbab IV.5 diimplementasikan dalam berkas `scripts/evaluate.py`. Ketiga sistem yang dievaluasi, yaitu GraphRAG beserta kedua sistem pembanding, dieksekusi melalui satu *pipeline* dengan pola *invoker* yang seragam `invoke_mode(question, session_id)`. Model yang dipilih didefinisikan melalui argumen `--mode` dan mengembalikan triple (`answer`, `reasoning_path`, `context`). Keseragaman *pipeline* ini menjamin perbedaan kom-

ponen teknis yang dirangkum pada Tabel IV.5 menjadi satu-satunya sumber perbedaan kinerja. Gambar V.13 membandingkan alur eksekusi ketiga sistem secara berdampingan.



Gambar V.13 Perbandingan alur eksekusi ketiga sistem yang dievaluasi: Vanilla LLM, Vector RAG, dan *GraphRAG*

V.4.1 Implementasi Vanilla LLM

Vanilla LLM diaktifkan melalui argumen `--mode llm` dan memanggil GPT-4o-mini secara langsung melalui fungsi `get_speaker_llm()` yang sama dengan yang digunakan modul *speaker* pada GraphRAG sehingga konfigurasi model identik antar-sistem. *Invoker* membungkus pertanyaan pengguna ke dalam satu `HumanMessage` tanpa konteks tambahan, kemudian meneruskannya ke model. Sistem tidak melakukan *retrieval* apa pun sehingga jawaban bergantung sepenuhnya pada pengetahuan parametrik model. Sejalan dengan ketiadaan *retrieval*, *invoker* mengembalikan `reasoning_path` dan `context` yang kosong.

Implementasi ini secara sengaja meniadakan seluruh komponen augmentasi yang

dimiliki GraphRAG, yaitu klasifikasi *intent*, *retrieval* berbasis graf, manajemen memori, dan validasi YAML, sebagaimana ditunjukkan pada kolom Vanilla LLM Tabel IV.5. Dengan demikian, Vanilla LLM berperan sebagai kondisi yang mengisolasi kapabilitas parametrik model tanpa pengaruh mekanisme *retrieval*.

V.4.2 Implementasi Vector RAG

Vector RAG diaktifkan melalui argumen `--mode vector` dan mengimplementasikan *retrieval* berbasis kemiripan vektor. *Invoker* menghasilkan *embedding* kueri melalui `VectorIndexManager.generate_embedding()` dengan model *text-embedding-3-small* yang sama dengan GraphRAG, kemudian mengeksekusi `SIMPLE_GRAPH_EXPAND_QUERY` terhadap *native vector index* Neo4j untuk mengambil *top-k=5 node* termirip.

Konteks disusun dengan merangkai tiap hasil menjadi *snippet* berlabel *Resource*, *Description*, dan *Related To* yang digabungkan menggunakan pemisah garis, sebagaimana ditampilkan pada Listing V.2. Nama *node* yang muncul dikumpulkan dengan deduplikasi melalui himpunan *seen* dan dikembalikan sebagai *reasoning_path* yang menjadi dasar penilaian *RetQ*. *Invoker* kemudian menyusun *prompt* bertemplate *Context*, *Question*, dan *Answer*, lalu meneruskannya ke modul *speaker* GPT-4o-mini. Implementasi ini tidak menggunakan klasifikasi *intent*, kedalaman *intent-adaptive*, traversal *multi-hop* lebih dari satu lompatan, penggabungan multi-entitas, maupun validasi YAML sehingga merepresentasikan pendekatan RAG konvensional sebagaimana dirancang pada Subbab IV.5.3.

```

1 // Mode "vector" -- Vector RAG
2 FUNCTION invoke_mode(question, session_id):
3     embedding    <- generate_embedding(question)
4     results      <- execute_query(SIMPLE_GRAPH_EXPAND_QUERY,
5                                   {embedding: embedding, top_k: 5})
6     context      <- ""
7     node_names   <- []
8     FOR EACH record IN results:
9         Collect node fullName and related fullName (deduplicated into
10            node_names)
11         Append snippet(Resource, Description, Related To) to context
12     prompt <- "Context: " + context + " Question: " + question
13     RETURN llm.invoke(prompt).content, node_names, context
14
15 // Mode "llm" -- Vanilla LLM
16 FUNCTION invoke_mode(question, session_id):
17     response <- llm.invoke(question) // no retrieval, no

```

```
    augmentation
18  RETURN response.content, [], ""
```

Listing V.2 Pseudokode *invoker* `invoke_mode` untuk mode Vector RAG dan Vanilla LLM

Hasil evaluasi komparatif ketiga sistem disajikan pada Bab VI.

BAB VI

EVALUASI

Bab ini menjawab tujuan penelitian ketiga (T3): membandingkan kinerja sistem *GraphRAG* yang dikembangkan terhadap dua *baseline* — *Vector RAG* dan *Vanilla LLM* — pada domain konfigurasi Kubernetes.

VI.1 Metode Evaluasi

Subbab ini memaparkan kerangka metodologis yang menjadi landasan seluruh analisis empiris pada Bab VI. Pertama, dipaparkan karakteristik *dataset* uji beserta prosedur validasi oleh pakar yang memastikan representativitas *dataset* tersebut (Subbab VI.1.1). Kedua, definisi operasional metrik dan dimensi evaluasi yang digunakan (Subbab VI.1.2). Ketiga, eksperimen ini mencakup perbandingan antara ketiga sistem serta menerapkan uji statistik pada *GraphRAG* (Subbab VI.1.3).

VI.1.1 Dataset Uji dan Validasi Pakar

Evaluasi menggunakan *dataset* yang terdiri dari 97 *fixture* uji yang mencakup delapan kategori pertanyaan: *conceptual* (15), *relationship* (18), *yaml_gen* (15), *followup* (12), *realworld* (24), *planning* (5), *troubleshooting* (5), dan *command* (3). Distribusi kategori ini mencerminkan realita penggunaan berdasarkan hasil wawancara dengan pakar.

Sebelum evaluasi kuantitatif dilaksanakan, dataset divalidasi secara kualitatif melalui wawancara mendalam dengan tiga praktisi DevOps/SRE yang menggunakan Kubernetes dan LLM secara aktif. Profil narasumber ditampilkan pada Tabel VI.1.

Tabel VI.1 Profil Narasumber Wawancara Validasi Dataset

Narasumber	Peran	Pengalaman Kubernetes	Frekuensi Penggunaan LLM
N1	<i>DevOps Engineer</i>	>5 tahun, klaster 50–200 <i>node</i>	Harian
N2	<i>Cloud & DevOps Engineer</i>	1–2 tahun, klaster menengah	Harian
N3	<i>Site Reliability Engineer</i>	6 bulan–1 tahun, klaster menengah	Mingguan

Wawancara dilaksanakan dalam dua segmen. Segmen pertama meminta narasumber memberikan skor realisme (skala 1–5) terhadap sepuluh sampel *fixture* yang merepresentasikan keseluruhan *fixture* yang ada. Kriteria yang digunakan adalah skor 1 berarti pertanyaan bersifat *textbook* yang tidak pernah diajukan ke LLM dalam praktik sehari-hari, sedangkan skor 5 berarti pertanyaan persis mencerminkan kebutuhan pekerjaan sehari-hari. Hasil penilaian ditampilkan pada Tabel VI.2.

Tabel VI.2 Penilaian Realisme Sampel *Fixture* oleh Pakar (Skala 1–5)

<i>Fixture</i>	N1	N2	N3	Rata-rata
<i>deployment_basic</i>	3	1	1	1,67
<i>service_types</i>	2	3	2	2,33
<i>scale_existing_deployment</i>	5	5	4	4,67
<i>ingress_service_pod</i>	3	4	2	3,00
<i>hpa_deployment_pod</i>	3	5	2	3,33
<i>cronjob_backup</i>	4	5	5	4,67
<i>networkpolicy_deny_all</i>	4	5	5	4,67
<i>statefulset_with_pvc</i>	4	5	5	4,67
<i>reject_all_docker_hub</i>	5	5	5	5,00
<i>kubectl_env_grep</i>	5	3	3	3,67
Rata-rata Keseluruhan				3,87

Penilaian menunjukkan variasi yang signifikan antar-*fixture*. *Fixture* yang bersifat definitif murni (*deployment_basic*, rata-rata 1,67) dinilai tidak realistis karena praktisi berpengalaman tidak menanyakan definisi dasar ke LLM. Sebaliknya, *fixture* berbasis skenario konkret (*statefulset_with_pvc*, *networkpolicy_deny_all*) mendapatkan skor tertinggi (4,67–5,00) karena mencerminkan kebutuhan operasional yang sebenarnya. Segmen kedua menggali tipe pertanyaan yang secara organik diajukan kepada LLM. Ketiga narasumber secara konsisten menyebutkan **debug** dan **troubleshooting** sebagai penggunaan utama, diikuti generasi YAML dengan penyesuaian konteks dan perencanaan arsitektur.

VI.1.2 Metrik dan Dimensi Evaluasi

Definisi lengkap setiap metrik beserta turunan literturnya dipaparkan di Subbab II.6. Penelitian ini mengorganisasikan metrik dalam tiga dimensi utama:

1. **Answer Quality (AnsQ)**. Metrik ini mengambil nilai rata-rata sub-metrik, yaitu *syntactic validity*, *schema compliance* (khusus *yaml_gen*), *answer relevance*, dan *faithfulness*.
2. **Retrieval Quality (RetQ)**. Metrik ini menghitung komposit dari tiga sub-metrik, yaitu $\text{RetQ} = (F1@k + \text{NDCG}@k + \text{Path Coverage})/3$. *Path Coverage* mengukur kelengkapan *edge* berbasis graf sehingga lebih diskriminatif dari hanya mengambil cakupan *node* semata.
3. **Reasoning Quality (ReaQ)**. Metrik ini menghitung komposit dua sub-metrik, yaitu $\text{ReaQ} = (\text{Hop Accuracy} + \text{Grounding Score})/2$. *Hop Accuracy* mengukur akurasi pemilihan kedalaman traversal ($d_{\text{pred}} = d_{\text{gt}}$) untuk memvalidasi langsung mekanisme *intent-adaptive depth*.

Empat metrik domain Kubernetes yang dievaluasi secara terpisah meliputi *Path Coverage*, *Hop Accuracy*, RGA (*Retrieval-Grounded Answer*), dan *syntactic validity* YAML. Adapun sub-metrik diagnostik yang terdiri atas *precision@k*, *recall@k*, *NDCG@k*, *grounding score*, serta *hallucination rate* digunakan sebagai informasi pelengkap.

VI.1.3 Protokol Eksperimen dan Perbandingan Tiga Sistem LLM

Evaluasi dilakukan terhadap tiga sistem menggunakan *dataset* yang sama (97 *fixture*), *speaker* LLM yang identik (GPT-4o-mini, *temperature*=0,1), dan seluruh metrik yang sama:

1. **Vanilla LLM**: menjawab langsung tanpa konteks apapun, mengukur kemampuan pengetahuan parametrik model.
2. **Vector RAG**: *cosine similarity* top-5 terhadap *vector index* Neo4j.
3. **GraphRAG**: sistem lengkap dengan *exact match* + *multi-hop graph traversal*

+ *intent-adaptive depth* + validasi YAML.

Setiap *fixture* dijalankan melalui `scripts/evaluate.py` dengan *session ID* yang unik per-*fixture*.

VI.2 Hasil Evaluasi

Tabel VI.3 menyajikan skor ketiga sistem untuk seluruh faktor beserta 95% CI *bootstrap*.

Tabel VI.3 Perbandingan Skor per Faktor: Vanilla LLM vs. Vector RAG vs. GraphRAG (97 *Fixture*, 95% CI *bootstrap*)

Faktor	Vanilla LLM	Vector RAG	GraphRAG	Sig
<i>Dimensi utama</i>				
AnsQ	0,5956	0,5916	0,5771	n.s.
RetQ	0,0241	0,4255	0,7259	***
ReaQ	0,3351	0,3307	0,5554	***
<i>Metrik domain Kubernetes</i>				
Path Coverage	0,0722	0,5411	0,8515	***
Hop Accuracy	0,0722	0,0000	0,3505	***
RGA	0,0206	0,2784	0,4536	***
Syntactic Validity	0,8947	0,8947	0,8947	n.s.
<i>Diagnostik</i>				
<i>Answer Relevance</i>	0,6239	0,6468	0,6747	—
<i>Faithfulness</i>	0,5193	0,4872	0,4356	—
<i>NDCG@k</i>	0,0000	0,4443	0,7492	—
<i>Grounding Score</i>	0,5981	0,6615	0,7602	—
<i>Hallucination Rate</i>	0,4019	0,3385	0,2398	—

Sig: *** $p < 0,001$; n.s. = tidak signifikan. Detail per-baseline pada Tabel VI.10.

Tabel di atas memperlihatkan pola yang konsisten. Ketiga sistem memperoleh skor AnsQ yang relatif setara, sementara perbedaan paling signifikan terdapat pada metrik yang terkait langsung dengan *retrieval* dan penalaran berbasis graf (RetQ, Path

Coverage, *Hop Accuracy*, RGA). Metrik-metrik tersebut menjadi pembeda utama antara GraphRAG dan kedua *baseline* di seluruh kategori *fixture*. Penjelasan lebih rinci mengenai tiap-tiap dimensi dan metrik domain tersebut dibahas pada subbab-subbab berikut.

VI.2.1 *Answer Quality* (AnsQ)

Pada dimensi AnsQ, ketiga sistem mencapai skor yang sebanding. Vanilla LLM memiliki skor 0,5956, Vector RAG memiliki skor 0,5916, dan GraphRAG memiliki skor 0,5771. Perbedaan antar-sistem tidak signifikan secara statistik setelah koreksi Holm ($p_{\text{Holm}} > 0,05$), sebuah hasil yang mengindikasikan bahwa kualitas teks jawaban ditentukan oleh kemampuan generatif LLM, bukan oleh mekanisme *retrieval*.

Pada sub-metrik *syntactic validity* YAML, ketiga sistem memperoleh skor identik (0,8947) karena menggunakan *speaker* LLM yang sama dengan instruksi format serupa. Skor *faithfulness* GraphRAG (0,4356) yang lebih rendah dari Vanilla LLM (0,5193) mencerminkan bahwa jawaban Vanilla LLM lebih sering menyebutkan nama *node* utama secara eksplisit. Hal tersebut merupakan konsekuensi dari mode parafrase tanpa *grounding* terhadap *retrieved context*. Sebaliknya, *answer relevance* GraphRAG (0,6747) yang lebih tinggi mengindikasikan bahwa konteks graf berhasil meningkatkan relevansi topik meskipun tanpa menyebutkan nama *node* secara literal.

Tabel VI.4 merinci skor AnsQ dan sub-metriknya untuk sistem GraphRAG.

Tabel VI.4 Sub-Metrik *Answer Quality* (AnsQ) Sistem GraphRAG (97 *Fixture*)

Sub-Metrik	Nilai	Keterangan
<i>Syntactic Validity</i> (YAML)	0,8947	17 dari 19 <i>fixture</i> yang menghasilkan YAML lolos PyYAML (15 <i>yaml_gen</i> + 4 <i>realworld</i>)
<i>Schema Compliance</i> (YAML)	0,7895	Kesesuaian skema Kubernetes v1.29
<i>Answer Relevance</i>	0,6747	Kemiripan <i>cosine</i> jawaban vs. <i>ground truth</i>
<i>Faithfulness</i>	0,4356	Proporsi node kunci disebutkan dalam jawaban
Skor AnsQ	0,5771	Rata-rata sub-metrik yang berlaku

VI.2.2 *Retrieval Quality* (RetQ)

RetQ merupakan dimensi yang paling jelas membedakan ketiga sistem. GraphRAG memperoleh RetQ 0,7259 [0,66; 0,78], Vector RAG 0,4255 [0,37; 0,48], dan Vanilla LLM 0,0241 [0,01; 0,04]. Keunggulan GraphRAG bersifat sangat signifikan pada kedua perbandingan ($p < 0,001$, Holm-Bonferroni).

Vanilla LLM mendekati nol pada RetQ karena tidak ada *retrieval* terstruktur; skor kecil yang tersisa (0,0241) berasal dari pengukuran *path coverage* yang sesekali menemukan kecocokan parsial di teks jawaban. Vector RAG mencapai RetQ 0,4255, menunjukkan bahwa ekspansi 1-hop mampu menjangkau sebagian node relevan, tetapi terbatas pada tetangga langsung *seed node*. GraphRAG dengan *multi-hop traversal* kedalaman 2–3 menjangkau *intermediate structural nodes* seperti *DeploymentSpec*, *PodTemplateSpec*, dan *Container* dalam rantai dependensi skema.

Secara konkret, *Path Coverage* GraphRAG (0,8515) berarti bahwa 85% *edge* relevan yang ditetapkan dalam *ground truth* berhasil dilalui selama traversal, dibandingkan dengan 54% pada Vector RAG. Nilai F1@k (0,5770) dan NDCG@k (0,7492) mengonfirmasi korelasi yang tinggi antara *node* yang di-*retrieve* dengan *ground truth*. Tabel VI.5 memerinci skor RetQ, komponen komposit, dan sub-metrik diagnostik untuk sistem GraphRAG.

Tabel VI.5 Sub-Metrik *Retrieval Quality* (RetQ) Sistem GraphRAG (97 Fixture)

Sub-Metrik	Nilai	Peran
<i>Komponen komposit RetQ</i>		
F1@k	0,5770	Rata-rata harmonik <i>precision</i> dan <i>recall</i>
NDCG@k	0,7492	Kualitas peringkat node relevan
<i>Path Coverage</i>	0,8515	Fraksi <i>edge</i> relevan yang dilalui traversal
Skor RetQ	0,7259	$(F1 + NDCG + PathCov)/3$
<i>Diagnostik (tidak masuk komposit)</i>		
<i>Precision@k</i>	0,6827	Fraksi node di- <i>retrieve</i> yang relevan
<i>Recall@k</i>	0,5914	Fraksi node relevan yang berhasil di- <i>retrieve</i>

VI.2.3 Reasoning Quality (ReaQ)

Pada dimensi ReaQ, GraphRAG mencapai 0,5554 [0,51; 0,61], sedangkan Vector RAG dan Vanilla LLM masing-masing 0,3307 [0,31; 0,36] dan 0,3351 [0,30; 0,37]. Keunggulan GraphRAG signifikan ($p < 0,001$ untuk kedua perbandingan). Menariknya, Vector RAG dan Vanilla LLM memperoleh ReaQ yang hampir sama, mengindikasikan bahwa penalaran berbasis graf merupakan sumber keunggulan yang berbeda dari sekadar kualitas jawaban.

Karakteristik Vector RAG yang mengembalikan sekumpulan *node* secara independen menyebabkan panjang *reasoning path*-nya sangat jarang yang sesuai dengan referensi *ground truth* dari hasil traversal graf. Hal ini secara empiris mengonfirmasi bahwa mekanisme *intent-adaptive depth* adalah komponen yang membedakan GraphRAG secara fundamental.

Grounding Score GraphRAG (0,7602) lebih tinggi dari Vector RAG (0,6615) dan Vanilla LLM (0,5981), mengindikasikan bahwa 76% istilah Kubernetes dalam jawaban GraphRAG dapat dilacak ke *retrieved context*, dibandingkan 66% untuk Vector RAG. *Hallucination Rate* GraphRAG (0,2398) secara konsekuensi lebih rendah. Tabel VI.6 merinci skor ReaQ, komponen komposit, sub-metrik diagnostik, dan RGA

untuk sistem GraphRAG.

Tabel VI.6 Sub-Metrik *Reasoning Quality* (ReaQ) Sistem GraphRAG (97 Fixture)

Sub-Metrik	Nilai	Peran
<i>Komponen komposit ReaQ</i>		
<i>Hop Accuracy</i>	0,3505	Akurasi pemilihan kedalaman: $d_{\text{pred}} = d_{\text{gt}}$
<i>Grounding Score</i>	0,7602	Proporsi istilah K8s terlacak ke <i>retrieved context</i>
Skor ReaQ	0,5554	$(\text{HopAcc} + \text{Grounding})/2$
<i>Diagnostik (tidak masuk komposit)</i>		
<i>Hallucination Rate</i>	0,2398	$1 - \text{Grounding Score}$; semakin rendah semakin baik
<i>Metrik domain</i>		
RGA (<i>Retrieval-Grounded Answer</i>)	0,4536	Jawaban benar <i>dan</i> didukung <i>retrieval</i>

VI.2.4 Metrik Domain Kubernetes

Empat metrik domain Kubernetes memberikan sudut pandang tambahan yang tidak terwakili dalam dimensi standar IR:

Path Coverage (dibahas di 6.2.2) mengukur kelengkapan *edge* antar-node, bukan sekadar keberadaan node. GraphRAG unggul signifikan ($p < 0,001$, $\Delta = +0,310$ vs. Vector RAG).

Hop Accuracy (dibahas di 6.2.3) mengukur ketepatan pemilihan kedalaman traversal. GraphRAG satu-satunya sistem yang melakukan traversal bertingkat sehingga faktor ini menjadi pembeda antara GraphRAG dengan kedua *baseline*.

RGA (*Retrieval-Grounded Answer*) mengukur proporsi jawaban yang benar *dan* didukung oleh *retrieval* yang cukup sehingga mengecualikan jawaban yang benar karena pengetahuan parametrik LLM semata. GraphRAG memperoleh RGA 0,4536 [0,36; 0,56] dibandingkan 0,2784 untuk Vector RAG dan 0,0206 untuk Vanilla LLM dengan keunggulan signifikan ($p < 0,001$). Nilai RGA Vanilla LLM yang mendekati nol mengonfirmasi bahwa jawaban LLM yang benar hampir sepenuhnya berasal

dari pengetahuan parametrik, bukan dari *retrieval* terstruktur.

Syntactic validity YAML tidak berbeda antar-sistem (p n.s.), karena ketiga sistem menggunakan *speaker* LLM yang sama. Validasi berlapis berbasis *knowledge graph* (ablasi A5) memberikan kontribusi pada dimensi ReaQ meski tidak pada validitas sintaksis itu sendiri.

Tabel VI.7 menyajikan skor per kategori *fixture* untuk sistem GraphRAG.

Tabel VI.7 Skor Evaluasi Sistem GraphRAG per Kategori *Fixture*

Kategori	<i>n</i>	AnsQ	RetQ	ReaQ	PathCov	HopAcc
<i>yaml_gen</i>	15	0,84	0,92	0,62	1,00	0,67
<i>followup</i>	12	0,52	0,89	0,61	0,89	0,58
<i>command</i>	3	0,49	0,88	0,46	1,00	0,33
<i>relationship</i>	18	0,68	0,80	0,57	0,90	0,17
<i>conceptual</i>	15	0,66	0,75	0,66	0,89	0,47
<i>planning</i>	5	0,46	0,82	0,32	0,97	0,00
<i>troubleshooting</i>	5	0,40	0,82	0,28	0,99	0,00
<i>realworld</i>	24	0,39	0,39	0,53	0,61	0,25
Keseluruhan	97	0,58	0,73	0,56	0,85	0,35

Pada per-kategori, sistem GraphRAG memperoleh RetQ dan PathCov tertinggi pada *yaml_gen* (RetQ 0,92, PathCov 1,00) dan *followup* (RetQ 0,89), mencerminkan efektivitas traversal multi-tingkat pada kueri yang secara inheren membutuhkan konteks relasional. Kategori *realworld* memperoleh RetQ terendah (0,39) karena pertanyaan berbasis konteks operasional sering membutuhkan informasi di luar cakupan skema OpenAPI statis. Analisis terhadap sumber perbedaan tersebut, beserta signifikansi statistik dan kondisi batas yang memengaruhinya, dipaparkan pada Subbab VI.3 berikut.

VI.3 Analisis Evaluasi

Bagian sebelumnya menyajikan hasil evaluasi secara deskriptif. Bagian ini menjawab pertanyaan yang lebih dalam terkait alasan pola tersebut terbentuk, keandalan perbedaannya secara statistik, dan kondisi GraphRAG lebih unggul.

VI.3.1 Analisis Internal Sistem GraphRAG

Ablation study dilakukan untuk memverifikasi bahwa setiap komponen sistem GraphRAG memberikan kontribusi yang terukur. Enam konfigurasi ablasi dirancang sehingga tiap konfigurasi menonaktifkan tepat satu komponen, dievaluasi terhadap seluruh 97 *fixture*. Tabel VI.8 menyajikan selisih skor terhadap *baseline* GraphRAG, sedangkan Tabel VI.9 menyajikan perbedaan statistiknya.

Tabel VI.8 *Ablation study*: selisih skor terhadap *baseline* GraphRAG per faktor

Ablasi	Δ AnsQ	Δ RetQ	Δ ReaQ	Δ PCov	Δ HopA	Δ RGa
A1 <i>no_phase1</i>	+0,002	−0,249	−0,078	−0,275	−0,155	−0,175
A2 <i>no_multihop</i>	−0,020	−0,601	−0,130	−0,684	−0,278	−0,392
A3 <i>depth=2 fixed</i>	−0,007	−0,030	−0,063	−0,002	−0,124	−0,031
A4 <i>depth=3 fixed</i>	+0,006	≈ 0	−0,029	+0,021	−0,052	+0,010
A5 <i>no_yaml_layer3</i>	−0,010	+0,002	−0,018	+0,008	−0,021	−0,031
A6c <i>no_multi_ent.</i>	≈ 0	−0,010	+0,018	−0,030	+0,041	−0,041

PCov = *Path Coverage*; HopA = *Hop Accuracy*.

Baseline: AnsQ 0,5771 / RetQ 0,7259 / ReaQ 0,5554 / PCov 0,8515 / HopA 0,3505 / RGA 0,4536.

$\Delta < 0$ = ablasi mengurangi skor; lihat Tabel VI.9 untuk signifikansi.

Tabel VI.9 Signifikansi Statistik *Ablation Study* per Faktor (Wilcoxon + *Bootstrap*, *p-value*)

Ablasi	AnsQ	RetQ	ReaQ	PCov	HopAcc	RGa
A1 <i>no_phase1</i>	0,591	< 0,001	< 0,001	< 0,001	< 0,001	< 0,001
A2 <i>no_multihop</i>	0,060	< 0,001	< 0,001	< 0,001	< 0,001	< 0,001
A3 <i>depth=2 fixed</i>	0,242	< 0,001	< 0,001	0,010	< 0,001	0,128
A4 <i>depth=3 fixed</i>	0,774	0,493	0,104	0,997	0,126	0,673
A5 <i>no_yaml_layer3</i>	0,133	0,250	0,007	0,841	0,079	0,090
A6c <i>no_multi_ent.</i>	0,527	0,219	0,727	0,015	0,977	0,017

PCov = *Path Coverage*; HopAcc = *Hop Accuracy*.

H_1 : *baseline* lebih tinggi dari varian ablasi. Nilai < 0,05 mengindikasikan komponen berkontribusi.

A2 (*no_multihop*) menghasilkan penurunan terbesar: RetQ turun 0,601 poin dan PathCov turun 0,684 poin ($p < 0,001$), menunjukkan bahwa *multi-hop graph traversal* adalah komponen paling krusial. Tanpa traversal *multi-hop*, sistem hanya

memperoleh *seed node* tanpa konteks dari dependensi skema yang diperlukan untuk menjawab pertanyaan yang bersifat struktural.

A1 (*no_phase1*) menurunkan RetQ sebesar 0,249 poin dan PathCov 0,275 poin ($p < 0,001$), mengonfirmasi bahwa *exact match* Fase 1 berperan penting sebagai gerbang pertama yang memastikan *seed node* yang tepat sebelum traversal dimulai.

A3 (*depth=2 fixed*) menurunkan RetQ 0,030 poin dan HopAcc 0,124 poin ($p < 0,001$), sementara **A4 (*depth=3 fixed*)** menghasilkan selisih mendekati nol (RetQ $\Delta \approx 0$, p n.s.). Kedalaman 3 sudah mendekati perilaku adaptif pada level agregat, namun analisis sensitivitas per-tipe yang dipaparkan pada Subbab V.3.2 menunjukkan bahwa tipe *followup* justru menurun tajam pada kedalaman 3, mengonfirmasi bahwa tidak ada satu nilai kedalaman yang optimal untuk seluruh jenis kueri.

A5 (*no_yaml_layer3*) menghasilkan penurunan kecil pada ReaQ ($p = 0,007$), menunjukkan kontribusi lapisan 3 validasi Neo4j pada kualitas penalaran, khususnya untuk *fixture* YAML.

A6c (*no_multi_entity*) menunjukkan penurunan PathCov dan RGA yang signifikan ($p < 0,05$) meski delta kecil, mengindikasikan kontribusi *multi-entity retrieval* pada kelengkapan dan ketepatan *retrieval*.

VI.3.2 Perbandingan Tiga Sistem

Tabel VI.10 menyajikan signifikansi statistik perbandingan GraphRAG terhadap dua *baseline* menggunakan uji Wilcoxon + *paired bootstrap* 95% CI dengan koreksi Holm-Bonferroni.

Tabel VI.10 Uji Signifikansi Statistik: GraphRAG vs. *Baseline* per Faktor ($n = 97$, Holm-Bonferroni)

Faktor	GraphRAG vs. Vector RAG			GraphRAG vs. Vanilla LLM		
	Δ	95% CI	Sig	Δ	95% CI	Sig
AnsQ	-0,014	[-0,05; +0,01]	n.s.	-0,019	[-0,06; +0,02]	n.s.
RetQ	+0,300	[+0,23; +0,37]	***	+0,702	[+0,63; +0,77]	***
ReaQ	+0,225	[+0,17; +0,28]	***	+0,220	[+0,17; +0,27]	***
<i>Path Coverage</i>	+0,310	[+0,23; +0,40]	***	+0,779	[+0,70; +0,85]	***
<i>Hop Accuracy</i>	+0,350	[+0,26; +0,45]	***	+0,278	[+0,19; +0,37]	***
RGA	+0,175	[+0,08; +0,28]	***	+0,433	[+0,34; +0,54]	***
YAML <i>Syntactic</i>	0,000	[0,00; 0,00]	n.s.	0,000	—	n.s.

*** $p < 0,001$ (Wilcoxon *signed-rank* + *paired bootstrap* 1.000 iterasi, H_1 : GraphRAG > *baseline*).

Koreksi Holm-Bonferroni per keluarga uji (7 faktor). n.s. = tidak signifikan.

Kelima faktor *retrieval-focused* (RetQ, ReaQ, PathCov, HopAcc, RGA) menunjukkan keunggulan GraphRAG yang sangat signifikan ($p < 0,001$) terhadap kedua *baseline*. Sebaliknya, AnsQ tidak signifikan di kedua perbandingan yang menunjukkan hasil yang konsisten dengan temuan 6.2.1, yaitu kedua *baseline* menggunakan LLM yang sama sehingga kualitas teks jawaban tidak dipengaruhi oleh mekanisme *retrieval*.

Rentang CI 95% yang tidak melewati nol untuk RetQ [+0,23; +0,37] vs. Vector RAG dan [+0,63; +0,77] vs. Vanilla LLM mengonfirmasi bahwa perbedaan ini konsisten dan bukan hasil variasi sampling.

Temuan penting pada Vanilla LLM: meski RetQ mendekati nol, AnsQ Vanilla LLM (0,5956) sedikit lebih tinggi dari GraphRAG. Hal ini menunjukkan bahwa GPT-4o-mini memiliki pengetahuan parametrik Kubernetes yang kuat dan cukup untuk menghasilkan jawaban dengan relevansi dan *faithfulness* tinggi tanpa *retrieval*. Implikasinya adalah keunggulan GraphRAG terletak pada **ketertelusuran** (*retrieval-grounded*) dan **kelengkapan relasional**.

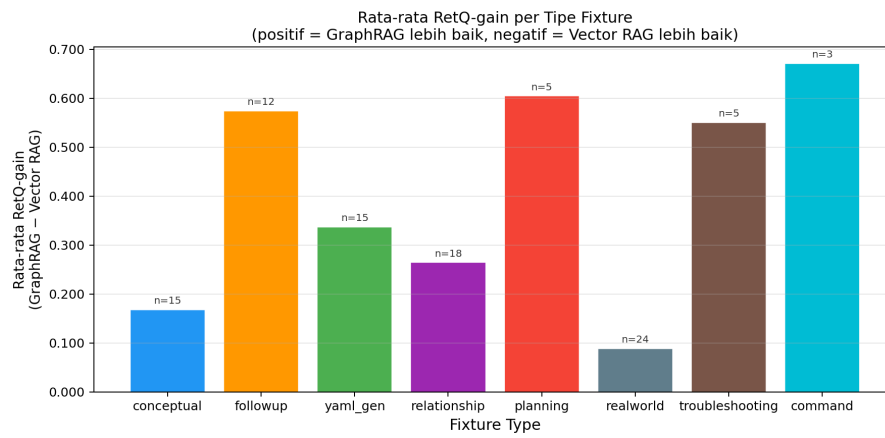
VI.3.3 Faktor Keunggulan GraphRAG

Agregasi skor seluruh *fixture* dapat menyembunyikan variasi penting. Analisis kondisi batas ini mengidentifikasi faktor penentu keunggulan GraphRAG menggunakan metrik *RetQ-gain* per *fixture*:

$$\text{RetQ-gain}_i = \text{RetQ}_{\text{GraphRAG},i} - \text{RetQ}_{\text{Vector RAG},i} \quad (\text{VI.1})$$

Nilai positif pada Persamaan VI.1 menunjukkan GraphRAG unggul, sedangkan nilai negatif berarti Vector RAG setara atau lebih baik.

Gambar VI.1 menampilkan rata-rata *RetQ-gain* untuk setiap tipe *fixture*. Berdasarkan data pada Tabel VI.11, GraphRAG mengungguli Vector RAG pada **seluruh delapan tipe fixture**, tanpa satu pun tipe yang membuat Vector RAG setara atau lebih baik.



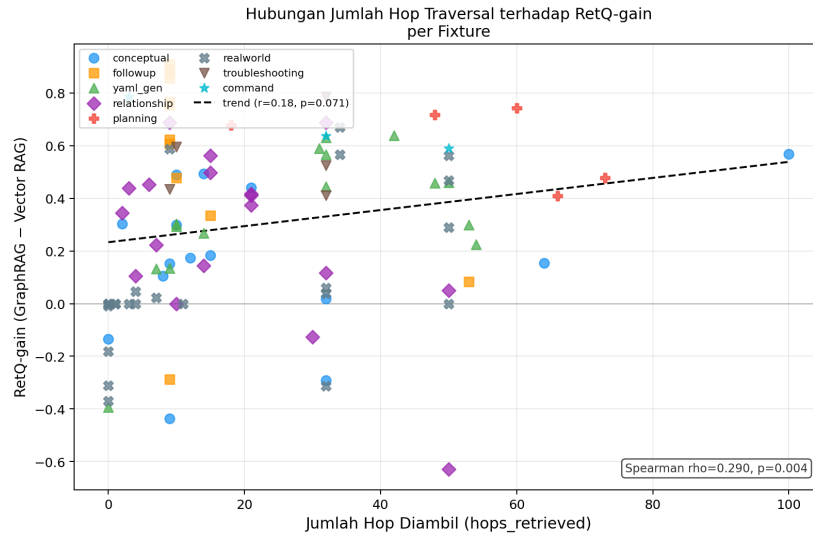
Gambar VI.1 Rata-rata *RetQ-gain* (GraphRAG – Vector RAG) per tipe *fixture*.

Tabel VI.11 Rata-rata *RetQ-gain* (GraphRAG – Vector RAG) per Tipe *Fixture*

Tipe	<i>n</i>	Rata-rata <i>RetQ-gain</i>	Karakteristik Kueri
<i>command</i>	3	+0,671	Konfigurasi operasional lini perintah
<i>planning</i>	5	+0,605	Perancangan multi-komponen bertingkat
<i>followup</i>	12	+0,574	Tindak lanjut relasi antar- <i>resource</i>
<i>troubleshooting</i>	5	+0,551	Diagnosis lintas <i>resource</i>
<i>yaml_gen</i>	15	+0,337	Generasi manifes terstruktur
<i>relationship</i>	18	+0,264	Pemetaan hubungan eksplisit
<i>conceptual</i>	15	+0,168	Penjelasan konsep tunggal
<i>realworld</i>	24	+0,089	Skenario dunia nyata (konteks luas)
Keseluruhan	97	+0,300	—

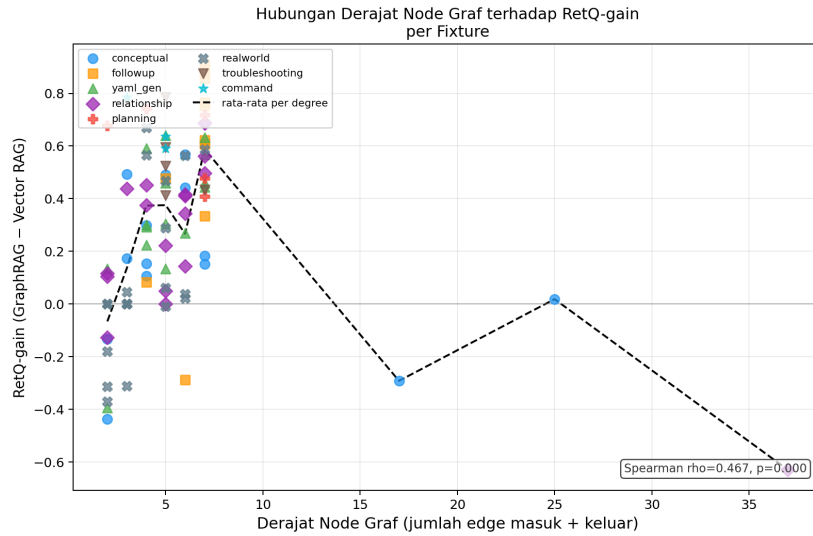
Pola yang konsisten teridentifikasi: tipe *fixture* dengan *gain* tertinggi (*command* +0,671, *planning* +0,605, *followup* +0,574) secara inheren membutuhkan pemahaman relasi antar-entitas yang melintasi beberapa tingkat hierarki. Sebaliknya, tipe *realworld* memperoleh *gain* terendah (+0,089) karena pertanyaan berbasis konteks operasional sering membutuhkan informasi di luar cakupan skema OpenAPI yang statis.

Dua faktor kuantitatif dianalisis menggunakan korelasi Spearman terhadap *RetQ-gain*: jumlah *hop* yang dilalui selama *retrieval* (*hops_retrieved*) dan derajat node primer dalam graf (*graph_degree*).



Gambar VI.2 *Scatter plot* jumlah *hop* yang dilalui vs. *RetQ-gain* per *fixture*.
Korelasi Spearman $\rho = +0,290$ ($p = 0,004$, $n = 97$).

Gambar VI.2 menunjukkan hubungan positif yang lemah namun signifikan antara jumlah *hop* yang dilalui dan *RetQ-gain* ($\rho = +0,290$, $p = 0,004$). *Fixture* yang membutuhkan traversal lebih dalam cenderung menghasilkan *gain* lebih tinggi bagi GraphRAG, karena *multi-hop traversal* menjangkau konteks relasional yang tidak bisa diperoleh ekspansi 1-*hop* milik Vector RAG. Namun, variasi pada *scatter plot* mengindikasikan bahwa kedalaman traversal saja tidak cukup menjelaskan keunggulan GraphRAG yang dibuktikan adanya *fixture* dengan hop tinggi namun *gain* rendah (pertanyaan *realworld* dengan konteks luas) dan sebaliknya. Hal ini mengonfirmasi bahwa jumlah *hop* merupakan faktor pendukung, bukan faktor penentu utama.



Gambar VI.3 Scatter plot derajat node primer vs. *RetQ-gain*.

Gambar VI.3 menunjukkan korelasi yang lebih kuat antara derajat node primer dan *RetQ-gain* ($\rho = +0,467, p < 0,001$). *Resource* Kubernetes dengan konektivitas menengah hingga tinggi dalam graf, seperti *Deployment* dan *StatefulSet* yang memiliki derajat 7, secara konsisten menghasilkan *gain* positif, karena banyaknya *edge* pada node tersebut memberikan jalur traversal yang dengan konteks yang komprehensif bagi GraphRAG. Sebaliknya, *resource* berderajat sangat rendah, seperti *ConfigMap* dan *Secret* yang berderajat 2, menunjukkan *gain* yang mendekati nol atau negatif, mengindikasikan bahwa kemiripan semantik berbasis *embedding* sudah memadai untuk resource dengan sedikit relasi. Pada derajat sangat tinggi (≥ 17), *gain* kembali menurun yang konsisten dengan temuan *depth sensitivity* bahwa traversal pada node berkonektivitas ekstrem menghasilkan banyak konteks generik yang justru menurunkan presisi. Tabel VI.12 merangkum koefisien korelasi Spearman dan nilai signifikansi untuk kedua faktor tersebut.

Tabel VI.12 Korelasi Spearman: Faktor Struktural vs. *RetQ-gain*

Faktor	ρ	<i>p</i> -value	<i>n</i>	Interpretasi
Derajat node (graph_degree)	+0,467	< 0,001	96	Sedang ***
Hop dilalui (hops_retrieved)	+0,290	0,004	97	Lemah **
<i>Multi-hop</i> flag (0/1)	+0,067	0,516	97	Tidak signifikan

$\rho > 0$: faktor berkorelasi positif dengan keunggulan GraphRAG atas Vector RAG.

*** $p < 0,001$ ** $p < 0,01$

Derajat node menunjukkan korelasi sedang yang signifikan ($\rho = +0,467$, $p < 0,001$). *Resource* dengan konektivitas tinggi dalam graf (seperti *Deployment*, *StatefulSet*) memberikan keunggulan *retrieval* yang lebih besar bagi GraphRAG karena traversal *multi-hop* dapat menjangkau lebih banyak konteks relasional yang relevan. Jumlah *hop* yang dilalui berkorelasi lemah namun signifikan ($\rho = +0,290$, $p = 0,004$), mengindikasikan bahwa traversal yang lebih dalam cenderung memberikan *gain* lebih besar, meski bukan faktor dominan tunggal.

Berdasarkan analisis di atas, kondisi batas sistem GraphRAG dirumuskan sebagai berikut.

GraphRAG memberikan keunggulan signifikan ketika:

1. kueri membutuhkan pemahaman relasi antar-*resource*; dan/atau
2. *resource* utama memiliki derajat konektivitas menengah hingga tinggi dalam graf Kubernetes (derajat 3–7).

GraphRAG tidak memberikan keunggulan bermakna ketika:

1. kueri bersifat kontekstual yang luas tanpa referensi spesifik ke skema; atau
2. *resource* utama memiliki konektivitas sangat rendah sehingga kemiripan semantik berbasis *embedding* sudah memadai.

VI.4 Validasi Pakar atas Jawaban Sistem

Selain evaluasi otomatis terhadap 97 *fixture*, penelitian ini melakukan evaluasi manusia untuk mendapatkan perspektif praktisi Kubernetes terhadap kualitas jawaban sistem GraphRAG. Keempat pakar (lembar validasi kode E.1–E.4 di Lampiran E) menguji sistem secara langsung melalui tiga skenario representatif, yaitu generasi

YAML *StatefulSet* dengan *PersistentVolumeClaim* (S1), *troubleshooting* Pod *CrashLoopBackOff* (S2), dan penjelasan relasi antara *Deployment* dan *ReplicaSet* (S3). Setiap jawaban dinilai pada empat dimensi menggunakan skala 1–5; hasilnya dirangkum pada Tabel VI.13.

Tabel VI.13 Penilaian Pakar atas Jawaban Sistem GraphRAG (Skala 1–5, $n = 4$ Pakar)

Dimensi Penilaian	S1 YAML	S2 <i>Troubleshoot</i>	S3 Relasi	Rata-rata
Relevansi	4,25	4,25	4,75	4,42
Akurasi Teknis	4,00	3,75	4,50	4,08
Kelengkapan	4,00	3,75	3,75	3,83
Kesediaan Pakai	3,25	4,00	4,50	3,92
Rata-rata skenario	3,88	3,94	4,38	—
<i>Retrieval Trace</i>				
Keterbacaan	4,00			—
Peningkatan kepercayaan	4,50			—

S1 = generasi YAML; S2 = *troubleshooting*; S3 = penjelasan relasi antar-*resource*.

Pola per skenario: S3 (penjelasan relasi) memperoleh rata-rata tertinggi (4,38) di seluruh dimensi, diikuti S2 (3,94) dan S1 (3,88). Hasil ini bersesuaian dengan temuan metrik otomatis (6.2.2, Tabel VI.11) yang menunjukkan *gain* tertinggi GraphRAG pada tipe kueri relasional.

Kelengkapan terendah (rata-rata 3,83) pada sub-skema S1 berkorelasi dengan temuan gap *path coverage* dan *required-fields* yang terdeteksi pada evaluasi (PathCov *yaml_gen* = 1,00 tetapi AnsQ *yaml_gen* = 0,84). Temuan tersebut mengindikasikan bahwa YAML yang dihasilkan memerlukan penyesuaian sebelum siap digunakan langsung dalam produksi. Semua pakar menyatakan bahwa YAML perlu sedikit modifikasi, mengonfirmasi nilai Ketersediaan Pakai S1 yang terendah (3,25).

Retrieval Trace mendapatkan penilaian kepercayaan tertinggi di seluruh item (4,50 dari 5), melebihi rata-rata skor akurasi jawaban (4,08). Tiga dari empat pakar secara

eksplisit menyebutkan *Retrieval Trace* sebagai pembeda utama dari LLM biasa: “*tidak perlu membuka dokumentasi yang sangat luas*” dan “*bisa lebih yakin dengan adanya retrieval graph*”. Nilai tinggi ini berkorelasi dengan metrik RGA (0,4536) dan mengindikasikan bahwa jawaban yang didukung oleh *retrieval* terstruktur akan lebih dipercaya oleh pengguna.

Berdasarkan seluruh hasil analisis tersebut, kesesuaian antara penilaian pakar dan metrik teridentifikasi pada tiga titik berikut:

1. Keunggulan sistem pada kueri relasional (S3 sejalan dengan skor RetQ relasi sebesar 0,80).
2. Keterbatasan pada kelengkapan YAML (skor kelengkapan S1 berbanding lurus dengan *gap path coverage* pada kategori *yaml_gen*).
3. Tingginya nilai kepercayaan terhadap *Retrieval Trace* (4,50) yang selaras dengan keunggulan metrik RGA ($p < 0,001$).

Keselarasan ini menunjukkan bahwa metrik yang dirancang dalam penelitian ini memiliki validitas konstruk yang memadai sehingga metrik tersebut berhasil mengukur aspek-aspek yang relevan dengan ekspektasi dan kebutuhan pengguna.

BAB VII

PENUTUP

VII.1 Kesimpulan

Penelitian ini telah mengimplementasikan sistem *Graph Retrieval-Augmented Generation* (GraphRAG) berbasis spesifikasi OpenAPI Kubernetes dan memverifikasi tiga kontribusi teknis utama melalui evaluasi empiris terhadap 97 *fixture* uji, *ablation study*, uji signifikansi statistik menggunakan uji Wilcoxon *signed-rank* dengan *paired bootstrap* dan koreksi Holm-Bonferroni, serta validasi pakar terhadap kualitas jawaban sistem.

Pertama, *pipeline* ekstraksi *knowledge graph* yang deterministik berbasis skema OpenAPI berhasil diimplementasikan. Graf dibangun dari swagger . json versi v1.30 menghasilkan 725 *node* definisi dan 18 jenis *edge* dalam 7 kategori relasi semantik yang khusus dirancang untuk domain Kubernetes. Seluruh *edge* diturunkan dari referensi tipe pada skema sehingga graf bersifat *reproducible* dan dapat diverifikasi secara otomatis, berbeda dengan pendekatan ekstraksi berbasis LLM yang menghasilkan representasi stokastik (Pan dkk. 2024; Wan dkk. 2025). *Ablation study* komponen *exact match* (A1) dan *multi-hop traversal* (A2) mengonfirmasi kontribusi tiap-tiap pendekatan, yaitu penghapusan *exact match* menurunkan RetQ sebesar 0,249 poin ($p < 0,001$), sedangkan penghapusan *multi-hop* menurunkan RetQ sebesar 0,601 poin ($p < 0,001$). Hasil tersebut menunjukkan bahwa *multi-hop traversal* (A2) merupakan komponen paling krusial dan setiap komponen *retrieval* berkontribusi pada hasil jawaban LLM.

Kedua, sistem GraphRAG yang memadukan mekanisme *retrieval intent-adaptive depth* dan validasi YAML berbasis *knowledge graph* berhasil dikembangkan. Analisis sensitivitas kedalaman pada $d \in \{1, 2, 3, 4, 5\}$ menunjukkan bahwa tidak ada satu nilai kedalaman tetap yang optimal untuk seluruh tipe *intent*. Ditemukan bahwa tipe *followup* mencapai RetQ optimal pada kedalaman 2 dan menurun tajam pada kedalaman 3, sementara *yaml_gen* dan *planning* optimal pada kedalaman 3. *Ablation study* A3 (kedalaman tetap $d = 2$) mengkonfirmasi bahwa kedalaman seragam

yang terlalu dangkal menurunkan RetQ sebesar 0,030 poin dan *Hop Accuracy* sebesar 0,124 poin ($p < 0,001$), sedangkan A4 (kedalaman tetap $d = 3$) menghasilkan selisih mendekati nol. Perbedaan A3 dan A4 menunjukkan bahwa perilaku adaptif tidak dapat digantikan oleh satu nilai kedalaman tetap. Kontribusi validasi YAML teridentifikasi melalui ablasi A5 yang menunjukkan bahwa penghapusan lapisan validasi Neo4j menurunkan ReaQ secara signifikan ($p = 0,007$). Lapisan ketiga ini memverifikasi keberadaan *required fields* langsung dari *knowledge graph* tanpa memerlukan *dry-run* pada kluster aktif.

Ketiga, melalui perbandingan terhadap *Vector RAG* dan *Vanilla LLM*, penelitian ini mengidentifikasi faktor-faktor keunggulan sistem GraphRAG. Pada faktor yang berkaitan dengan *retrieval* dan penalaran berbasis graf, GraphRAG unggul secara signifikan terhadap kedua *baseline* ($p < 0,001$, Holm-Bonferroni). Nilai RetQ 0,7259 dibandingkan 0,4255 (*Vector RAG*) dan 0,0241 (*Vanilla LLM*) dengan $\Delta = +0,30$ dan 95% CI $[+0,23; +0,37]$; ReaQ 0,5554 dibandingkan 0,3307 dan 0,3351; *Path Coverage* 0,8515 dengan $\Delta = +0,310$ terhadap *Vector RAG*; serta RGA (*Retrieval-Grounded Answer*) 0,4536 dibandingkan 0,2784 dan 0,0206. Di sisi lain, AnsQ tidak berbeda signifikan antar-sistem ($p_{\text{Holm}} > 0,05$) dan hasil *syntactic validity* YAML yang identik antar-model (0,8947) karena kualitas teks jawaban ditentukan oleh kemampuan generatif LLM yang digunakan. Berdasarkan analisis tersebut, keunggulan GraphRAG diidentifikasi terletak pada ketertelusuran (*retrieval-grounded*) dan kelengkapan relasional. Analisis kondisi kritis mengungkap bahwa keunggulan terbesar terjadi pada tipe *intent* relasional (*command* +0,671, *planning* +0,605, *followup* +0,574) dan pada *resource* dengan konektivitas menengah hingga tinggi dalam graf (derajat 3–7, Spearman $\rho = +0,467$, $p < 0,001$). Validasi pakar memperkuat temuan ini dengan menyatakan *Retrieval Trace* memperoleh tingkat kepercayaan tertinggi (4,50 dari 5), selaras dengan keunggulan RGA yang terukur secara statistik.

VII.2 Keterbatasan

Penelitian ini mengidentifikasi tiga keterbatasan utama yang memengaruhi hasil.

Pertama, *knowledge graph* yang dibangun merupakan *directed graph* yang hanya merepresentasikan relasi struktural-skema dari bagian *definitions* pada *swagger.json* Kubernetes v1.30. Keterbatasan ini berdampak pada tiga aspek cakupan: (a) relasi operasional seperti ikatan RBAC antara *ServiceAccount* dan *ClusterRole* tidak dapat dijangkau melalui *traversal* karena *edge* hanya ada dalam arah terbalik (*RoleBinding* $\rightarrow$ *ServiceAccount*); (b) operasi API (*paths*), *Custom Resource Definition*, dan versi Kubernetes selain v1.30 tidak tercakup; serta (c) status *runtime* pada kluster asli

Kubernetes tidak terepresentasi dalam graf yang bersifat statis.

Kedua, *retrieval* dibatasi pada maksimum tiga lompatan. Saat kueri membutuhkan informasi di luar jangkauan ini, sistem tidak mengembalikan pesan galat melainkan mengisi kesenjangan konteks menggunakan pengetahuan parametrik LLM (*silent degradation*). Kondisi ini merupakan faktor utama yang menjelaskan mengapa kategori *realworld* memperoleh RetQ terendah (0,39) dan *RetQ-gain* terendah (+0,089) di antara delapan kategori. Permasalahan ini disebabkan oleh pertanyaan berbasis konteks operasional sering membutuhkan informasi yang melampaui batas tiga lompatan dan di luar cakupan skema statis yang ada pada sistem.

Ketiga, keunggulan GraphRAG tidak menjangkau seluruh dimensi evaluasi. AnsQ tidak menunjukkan perbedaan yang signifikan dari kedua *baseline* dan *syntactic validity* YAML identik (0,8947) di ketiga sistem sehingga keunggulan terbatas pada ketertelusuran dan kelengkapan relasional. Selain itu, nilai *grounding score* 0,7602 menunjukkan bahwa sekitar 24% istilah Kubernetes dalam jawaban masih berasal dari pengetahuan parametrik LLM.

VII.3 Saran

Berdasarkan keterbatasan yang teridentifikasi, beberapa arah pengembangan lanjutan direkomendasikan.

1. Perluasan Sumber dan Cakupan *Knowledge Graph*

Pengembangan lanjutan dapat menambahkan relasi operasional seperti ikatan RBAC dan anotasi *best practice* sebagai lapisan kedua di atas graf skema yang sudah ada, serta memperluas ingest ke bagian paths dan *Custom Resource Definition* tanpa mengubah prinsip determinisme graf.

2. Integrasi Sumber Data Dinamis untuk Pertanyaan *Realworld*

Integrasi dengan *Kubernetes Watch API* atau log operasional dapat meningkatkan kemampuan sistem menjawab pertanyaan *troubleshooting* yang membutuhkan status kluster aktif. Di sisi *retrieval*, strategi kedalaman adaptif untuk kueri lintas-*resource* yang melampaui tiga lompatan dapat mengurangi ketergantungan pada *silent degradation*.

3. Pembaruan Inkremental *Knowledge Graph*

Kubernetes merilis versi baru secara berkala dengan penambahan atau perubahan skema. Pengembangan *pipeline* pembaruan inkremental yang dapat mendeteksi perubahan antar-versi swagger .json dan memperbarui *knowledge graph* tanpa rekonstruksi penuh akan meningkatkan ketahanan sistem terhadap evolusi API.

DAFTAR PUSTAKA

- Abu-Salih, Bilal. 2021. "Domain-specific knowledge graphs: A survey". *Journal of Network and Computer Applications* 185:103076. <https://doi.org/10.1016/j.jnca.2021.103076>.
- Cao, Ruisheng, Hanchong Zhang, Tiancheng Huang, Zhangyi Kang, Yuxin Zhang, Liangtai Sun, Hanqi Li, dkk. 2025. *NeuSym-RAG: Hybrid Neural Symbolic Retrieval with Multiview Structuring for PDF Question Answering*. arXiv preprint. Version 2, released 31 May 2025. arXiv: 2505.19754 [cs.CL]. <https://arxiv.org/abs/2505.19754>.
- Cloud Native Computing Foundation. 2023. *CNCF Annual Survey 2023*. Diakses pada November 22, 2025. <https://www.cncf.io/reports/cncf-annual-survey-2023/>.
- Es, Shahul, Jithin James, Luis Espinosa-Anke, dan Steven Schockaert. 2023. "RA-GAS: Automated Evaluation of Retrieval Augmented Generation". *arXiv preprint arXiv:2309.15217*.
- Gao, Yunfan, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Meng Wang, dan Haofen Wang. 2024. "Retrieval-Augmented Generation for Large Language Models: A Survey". Accessed November 25, 2025, *arXiv preprint arXiv:2312.10997*.
- Gartner. 2021. *Gartner Says Cloud Will Be the Centerpiece of New Digital Experiences*. <https://www.gartner.com/en/newsroom/press-releases/2021-11-10-gartner-says-cloud-will-be-the-centerpiece-of-new-digital-experiences>. Press Release, accessed 22 November 2025.
- Han, Haoyu, Yu Wang, Harry Shomer, Kai Guo, Jiayuan Ding, Yongjia Lei, Mahantesh Halappanavar, Ryan A. Rossi, Subhabrata Mukherjee, dan Xianfeng Tang. 2024. "Retrieval-augmented generation with graphs (GraphRAG)". *arXiv preprint arXiv:2501.00309*.

- He, Xiaoxin, Yijun Tian, Yifei Sun, Nitesh V. Chawla, Thomas Laurent, Yann LeCun, Xavier Bresson, dan Bryan Hooi. 2024. “G-Retriever: Retrieval-Augmented Generation for Textual Graph Understanding and Question Answering”. Dalam *The Twelfth International Conference on Learning Representations (ICLR 2024)*. <https://arxiv.org/abs/2402.07630>.
- Hofer, Marvin, Daniel Obraczka, Alieh Saeedi, Hanna Köpcke, dan Erhard Rahm. 2024. “Construction of Knowledge Graphs: Current State and Challenges”. *Information* 15 (8): 509.
- Ji, Ziwei, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Yejin Bang, Andrea Madotto, dan Pascale Fung. 2023. “Survey of Hallucination in Natural Language Generation”. *ACM Computing Surveys* 55 (12): 1–38.
- Kotstein, Sebastian, dan Christian Decker. 2024. “RETBERTa: a Transformer-based question answering approach for semantic search in Web API documentation”. Published online 31 January 2024, *Cluster Computing*, <https://doi.org/10.1007/s10586-023-04237-x>.
- Liu, Zhijie, Anh Nguyen, Junjie Chen, Xin Zhang, Yanjie Li, dan Yingfei Xiong. 2024. “Deep Analysis of LLM-based Code Generation: Correctness, Complexity, and Security”. *IEEE Transactions on Software Engineering*, 1–20. <https://doi.org/10.1109/TSE.2024.3392499>.
- Lu, Zhouyang, Hailin Xu, Anrui Chen, Siyuan Tang, Junyi Zhang, Yifei Feng, Wentao Pan, dan Jiangli Huang. October 2025. “HSG-RAG: Hierarchical Knowledge Base Construction for Embedded System Development”. *ACM Transactions on Design Automation of Electronic Systems* 30, no. 6 (): 1–21. <https://doi.org/10.1145/3731680>.
- Ma, Chuangtao, Yongrui Chen, Tianxing Wu, Arijit Khan, dan Haofen Wang. 2025. “Large Language Models Meet Knowledge Graphs for Question Answering: Synthesis and Opportunities”. *arXiv preprint arXiv:2505.20099*.
- Moreno-Cediel, Antonio, Eva Garcia-Lopez, Antonio Garcia-Cabot, dan David DeFitero-Dominguez. 2025. “Optimising retrieval performance in RAG systems: A new growing window semantic chunking strategy to address weak semantic boundaries”. *Knowledge-Based Systems* 331:114896. <https://doi.org/10.1016/j.knosys.2025.114896>.

- Niakšu, Olegas. 2015. “CRISP Data Mining Methodology Extension for Medical Domain”. *Baltic Journal of Modern Computing* 3 (2): 92–109. https://www.bjmc.lu.lv/fileadmin/user_upload/lu_portal/projekti/bjmc/Contents/3_2_2_Niaksu.pdf.
- Pan, Shirui, Linhao Luo, Yufei Wang, Chen Chen, Jiapu Wang, dan Xindong Wu. 2024. “Unifying Large Language Models and Knowledge Graphs: A Roadmap”. *IEEE Transactions on Knowledge and Data Engineering* 36 (7): 3580–3599. <https://doi.org/10.1109/TKDE.2024.3352100>.
- Rahman, Akond, Asif Partho, Patrick Morrison, dan Laurie Williams. 2023. “Security Misconfigurations in Open Source Kubernetes Manifests: An Empirical Study”. *ACM Transactions on Software Engineering and Methodology* 32 (5). <https://doi.org/10.1145/3579639>.
- Soldani, Jacopo, Damian Andrew Tamburri, dan Willem-Jan Van Den Heuvel. 2018. “The pains and gains of microservices: A Systematic grey literature review”. *Journal of Systems and Software* 146:215–232. <https://doi.org/10.1016/j.jss.2018.09.082>.
- The Kubernetes Authors. 2024a. *Kubernetes Concepts*. Diakses pada: 2024-05-15. <https://kubernetes.io/docs/concepts/>.
- . 2024b. “Kubernetes Documentation”. Diakses pada February 22, 2025. <https://kubernetes.io/docs/home/>.
- Wan, Yuwei, Zheyuan Chen, Ying Liu, Chong Chen, dan Michael Packianather. 2025. “Empowering LLMs by hybrid retrieval-augmented generation for domain-centric Q&A in smart manufacturing”. *Advanced Engineering Informatics* 65:103212. ISSN: 1474-0346. <https://doi.org/10.1016/j.aei.2025.103212>.
- Yu, Hui-Hung, Wei-Tsun Lin, Chih-Wei Kuan, Chao-Chi Yang, dan Kuan-Min Liao. 2025. “GraphRAG-Enhanced Dialogue Engine for Domain-Specific Question Answering: A Case Study on the Civil IoT Taiwan Platform”. *Future Internet* 17 (9): 414. <https://doi.org/10.3390/fi17090414>.
- Yu, Jun, Yintie Zhang, Yu Wu, dan Linhui Mao. 2021. “Research on the Practical Application of Visual Knowledge Graph in Technology Service Model and Intelligent Supervision”. *Journal of Physics: Conference Series* 1982:012040.

Zhao, Penghao, Hailin Zhang, Qinhan Yu, Zhengren Wang, Yunteng Geng, Fangcheng Fu, Ling Yang, Wentao Zhang, Jie Jiang, dan Bin Cui. 2024. “Retrieval-Augmented Generation for AI-Generated Content: A Survey”. *arXiv preprint arXiv:2402.19473*.

LAMPIRAN A

KODE PROGRAM

Lampiran ini menyajikan kode sumber komponen inti sistem *GraphRAG* Kubernetes yang dibahas pada Bab V. Komentar dan *docstring* pada setiap berkas dihilangkan agar *listing* lebih ringkas dan terfokus pada logika program. Kode lengkap beserta riwayat komit tersedia di repositori proyek.

A.1 Manajemen Memori Percakapan (*ZepMemoryStore*)

Kelas *ZepMemoryStore* pada berkas `src/memory/zep_store.py` menyediakan penyimpanan riwayat percakapan berbasis SQLite lokal sebagai pengganti Zep Cloud.

```
1 import sqlite3
2 import logging
3 from pathlib import Path
4
5 logger = logging.getLogger(__name__)
6
7 _DB_PATH = Path(__file__).parent.parent.parent / "data" / "
    conversation_memory.db"
8
9
10 def _get_connection() -> sqlite3.Connection:
11     _DB_PATH.parent.mkdir(parents=True, exist_ok=True)
12     conn = sqlite3.connect(str(_DB_PATH), check_same_thread=False)
13     conn.execute("""
14         CREATE TABLE IF NOT EXISTS messages (
15             id            INTEGER PRIMARY KEY AUTOINCREMENT,
16             session_id    TEXT      NOT NULL,
17             role           TEXT      NOT NULL,
18             content        TEXT      NOT NULL,
19             created_at     DATETIME DEFAULT CURRENT_TIMESTAMP
20         )
21     """)
22     conn.execute("CREATE INDEX IF NOT EXISTS idx_session ON
    messages(session_id)")
23     conn.commit()
```

```

24     return conn
25
26
27 _conn: sqlite3.Connection | None = None
28
29
30 def _db() -> sqlite3.Connection:
31     global _conn
32     if _conn is None:
33         _conn = _get_connection()
34         logger.info(f"SQLiteMemory: database di {_DB_PATH}")
35     return _conn
36
37
38 class ZepMemoryStore:
39
40     def __init__(self):
41         _db()
42         logger.info("ZepMemoryStore: menggunakan SQLite lokal (
tanpa Zep server).")
43
44     def save_memory(self, session_id: str, role: str, content: str
):
45         role = "user" if role.lower() in ("user", "human") else "
assistant"
46         try:
47             _db().execute(
48                 "INSERT INTO messages (session_id, role, content)
VALUES (?, ?, ?)",
49                 (session_id, role, content)
50             )
51             _db().commit()
52         except Exception as e:
53             logger.warning(f"SQLiteMemory: gagal menyimpan pesan:
{e}")
54
55     def get_memory(self, session_id: str, limit: int = 5) -> str:
56         try:
57             rows = _db().execute(
58                 """
59                 SELECT role, content FROM messages
60                 WHERE session_id = ?
61                 ORDER BY id DESC
62                 LIMIT ?
63                 """,

```

```

64         (session_id, limit)
65     ).fetchall()
66     rows = list(reversed(rows))
67     return "\n".join(f"{role}: {content}" for role,
content in rows)
68     except Exception as e:
69         logger.warning(f"SQLiteMemory: gagal mengambil riwayat
: {e}")
70         return ""
71
72     def add_message(self, session_id: str = None, role: str = None
,
73                     content: str = None, user_msg: str = None,
74                     ai_msg: str = None, message: str = None, **
kwargs):
75         if not session_id:
76             return
77         if user_msg and ai_msg:
78             self.save_memory(session_id=session_id, role="user",
content=user_msg)
79             self.save_memory(session_id=session_id, role="
assistant", content=ai_msg)
80             return
81         resolved_content = content or message or user_msg or
ai_msg or kwargs.get("text", "")
82         if not resolved_content:
83             return
84         resolved_role = role or ("user" if user_msg else ("
assistant" if ai_msg else "user"))
85         self.save_memory(session_id=session_id, role=resolved_role
, content=resolved_content)
86
87     def get_history(self, session_id: str = None, limit: int = 5,
**kwargs) -> str:
88         if not session_id:
89             return ""
90         return self.get_memory(session_id=session_id, limit=limit)
91
92     def add_turn(self, session_id: str, role: str, content: str):
93         self.save_memory(session_id=session_id, role=role, content
=content)
94
95     def get_context(self, session_id: str, limit: int = 5) -> str:
96         return self.get_memory(session_id=session_id, limit=limit)

```

Listing A.1 src/memory/zep_store.py — Manajemen Memori Percakapan

A.2 Pipeline Ingestion Data (SwaggerGraphBuilder)

Kelas *SwaggerGraphBuilder* pada berkas `src/ingestion/parser.py` mengimplementasikan pipeline 5-fase untuk mengubah `swagger.json` Kubernetes menjadi *knowledge graph* di Neo4j.

```
1 import json
2 from typing import Dict, Any
3 from src.graph.neo4j_client import Neo4jClient
4 from src.utils.text_utils import safe_truncate_description
5 from src.graph.vector_index import VectorIndexManager
6
7 PRIMITIVE_TYPES = {"string", "integer", "number", "boolean", "
    array", "object"}
8
9 IGNORE_LIST = {
10     "io.k8s.apimachinery.pkg.apis.meta.v1.ManagedFieldsEntry",
11     "io.k8s.apimachinery.pkg.apis.meta.v1.ObjectMeta",
12     "io.k8s.apimachinery.pkg.apis.meta.v1.StatusDetails",
13     "io.k8s.apimachinery.pkg.apis.meta.v1.Time",
14     "io.k8s.apimachinery.pkg.apis.meta.v1.MicroTime",
15     "io.k8s.apimachinery.pkg.apis.meta.v1.Duration",
16     "io.k8s.apimachinery.pkg.apis.meta.v1.RawExtension",
17     "io.k8s.apimachinery.pkg.apis.meta.v1.FieldsV1",
18     "io.k8s.apimachinery.pkg.apis.meta.v1.OwnerReference",
19     "io.k8s.apimachinery.pkg.apis.meta.v1.Patch",
20     "io.k8s.apimachinery.pkg.apis.meta.v1.StatusCause",
21     "io.k8s.apimachinery.pkg.version.Info",
22 }
23
24 CLUSTER_SCOPED_RESOURCES = {
25     "Namespace", "Node", "PersistentVolume", "ClusterRole",
26     "ClusterRoleBinding", "StorageClass", "
    CustomResourceDefinition",
27     "PriorityClass", "CSIDriver", "CSINode", "VolumeAttachment",
28     "RuntimeClass", "APIService", "MutatingWebhookConfiguration",
29     "ValidatingWebhookConfiguration", "ValidatingAdmissionPolicy",
30     "ValidatingAdmissionPolicyBinding", "CertificateSigningRequest
    ",
31     "IngressClass", "FlowSchema", "PriorityLevelConfiguration",
32     "SelfSubjectAccessReview", "SubjectAccessReview", "TokenReview
    "
33 }
34
35
36 class SwaggerGraphBuilder:
```



```

37     def __init__(self, swagger_path: str):
38         self.swagger_path = swagger_path
39         self.db = Neo4jClient()
40         self.definitions: Dict[str, Any] = {}
41
42     def load_swagger(self):
43         with open(self.swagger_path, 'r', encoding='utf-8') as f:
44             data = json.load(f)
45             self.definitions = data.get('definitions', {})
46
47     def ingest(self):
48         self.load_swagger()
49         self.db.execute_query("MATCH (n) DETACH DELETE n")
50
51         print("--> Pass 1: Creating Resource Definition Nodes...")
52         self._pass_1_create_nodes()
53
54         print("--> Pass 1.5: Generating Vector Embeddings...")
55         vector_mgr = VectorIndexManager()
56         vector_mgr.initialize()
57
58         print("--> Pass 2: Resolving Schema Relationships...")
59         self._pass_2_create_structural_edges()
60
61         print("--> Pass 2.5: Resolving Inheritance & Union Types
62         ...")
63         self._pass_2b_create_inheritance_edges()
64
65         print("--> Pass 3: Extracting YAML Configuration Patterns
66         ...")
67         self._pass_3_build_semantic_edges()
68
69     def _pass_1_create_nodes(self):
70         created_count = 0
71
72         for full_name, schema in self.definitions.items():
73             if full_name in IGNORE_LIST:
74                 continue
75
76             short_name = full_name.split(".")[1]
77             gvk_list = schema.get("x-kubernetes-group-version-kind", [])
78
79             kind = short_name
80             scope = "Namespaced"

```

```

79         if gvk_list and isinstance(gvk_list, list) and len(
gvk_list) > 0:
80             first_gvk = gvk_list[0]
81             if isinstance(first_gvk, dict) and isinstance(
first_gvk.get('kind'), str):
82                 kind = first_gvk.get('kind').strip()
83                 is_root = True
84                 scope = "Cluster" if kind in
CLUSTER_SCOPED_RESOURCES else "Namespaced"
85             else:
86                 kind, is_root, scope = "SubResource", False, "
N/A"
87         else:
88             kind, is_root, scope = "SubResource", False, "N/A"
89
90         original_desc = schema.get('description', 'No
description provided.')
91         desc = safe_truncate_description(original_desc,
hard_limit=4000)
92
93         self.db.execute_query("""
94             MERGE (d:Definition {id: $full_name})
95             SET d.name = $short_name,
96                 d.fullName = $full_name,
97                 d.kind = $kind,
98                 d.is_root = $is_root,
99                 d.scope = $scope,
100                 d.description = $desc,
101                 d.description_length = $original_length,
102                 d.was_truncated = $was_truncated,
103                 d.source = 'k8s_swagger_v1'
104         """, {
105             "full_name": full_name, "short_name": short_name,
106             "kind": kind, "is_root": is_root, "scope": scope,
107             "desc": desc, "original_length": len(original_desc
),
108             "was_truncated": len(desc) < len(original_desc)
109         })
110         created_count += 1
111
112         self.db.execute_query("MATCH (d:Definition {is_root: true
}) SET d:K8sResource")
113         print(f"    Created {created_count} nodes")
114
115     def _pass_2_create_structural_edges(self):

```

```

116         edge_count = 0
117
118         for full_name, schema in self.definitions.items():
119             if full_name in IGNORE_LIST:
120                 continue
121
122             properties = schema.get('properties', {})
123             required_fields = schema.get('required', [])
124             primitive_props = {}
125
126             for field_name, field_schema in properties.items():
127                 field_type = field_schema.get('type')
128                 ref = field_schema.get('$ref')
129                 is_array = is_map = False
130                 is_required = field_name in required_fields
131
132                 if field_type == 'array' and 'items' in
field_schema:
133                     items = field_schema['items']
134                     if isinstance(items, dict):
135                         if '$ref' in items:
136                             ref, is_array = items['$ref'], True
137                         else:
138                             primitive_props[field_name] = f"
array_of_{items.get('type', 'object')}"
139                             continue
140
141                 if field_type == 'object' and '
additionalProperties' in field_schema:
142                     add_props = field_schema['additionalProperties
']
143                     if isinstance(add_props, dict):
144                         if '$ref' in add_props:
145                             ref, is_map = add_props['$ref'], True
146                         else:
147                             primitive_props[field_name] = f"
map_of_{add_props.get('type', 'object')}"
148                             continue
149
150                 if field_type in PRIMITIVE_TYPES and not ref:
151                     if field_name not in ["kind", "id", "name", "
is_root", "description",
152                                         "source", "was_truncated
", "description_length", "fullName"]:
153                         primitive_props[field_name] = field_type

```

```

154
155         elif ref:
156             target_name = ref.split("/")[-1]
157             if target_name in IGNORE_LIST:
158                 primitive_props[field_name] = "string" if
159 "Time" in target_name else "object"
160             else:
161                 if target_name in self.definitions:
162                     self.db.execute_query("""
163                         MATCH (source:Definition {id:
164 $source_id})
165                         MATCH (target:Definition {id:
166 $target_id})
167                         MERGE (source)-[r:HAS_PROPERTY {
168 name: $field_name}]->(target)
169                         SET r.is_array = $is_array,
170                             r.is_map = $is_map,
171                             r.is_required = $is_required
172                         """, {
173                             "source_id": full_name, "target_id":
174 target_name,
175                             "field_name": field_name, "
176 is_array": is_array,
177                             "is_map": is_map, "is_required":
178 is_required
179                         })
180                 else:
181                     self.db.execute_query("""
182                         MERGE (target:Definition {id:
183 $target_id})
184                         ON CREATE SET
185                             target.name = $short_name,
186                             target.fullName = $target_id,
187                             target.kind = 'SubResource',
188                             target.is_root = false,
189                             target.source = '
190 k8s_swagger_cross_ref',
191                             target.description = 'External
192 cross-reference from Swagger $ref',
193                             target.is_cross_reference =
194 true
195                         WITH target
196                         MATCH (source:Definition {id:
197 $source_id})

```

```

186             MERGE (source)-[r:HAS_PROPERTY {
name: $field_name}]->(target)
187             SET r.is_array = $is_array,
188                 r.is_map = $is_map,
189                 r.is_required = $is_required
190             "", {
191                 "source_id": full_name, "target_id
": target_name,
192                 "short_name": target_name.split("."
)[-1],
193                 "field_name": field_name, "
is_array": is_array,
194                 "is_map": is_map, "is_required":
is_required
195             })
196             edge_count += 1
197
198             if primitive_props:
199                 self.db.execute_query(
200                     "MATCH (d:Definition {id: $full_name}) SET d
+= $primitive_props",
201                     {"full_name": full_name, "primitive_props":
primitive_props}
202                 )
203
204             print(f"    Created {edge_count} structural edges (
HAS_PROPERTY)")
205
206         def _pass_2b_create_inheritance_edges(self):
207             inheritance_count = union_count = 0
208
209             for full_name, schema in self.definitions.items():
210                 if full_name in IGNORE_LIST:
211                     continue
212
213                 for item in schema.get('allOf', []):
214                     if isinstance(item, dict) and '$ref' in item:
215                         parent_name = item['$ref'].split('/')[1]
216                         if parent_name not in IGNORE_LIST:
217                             self.db.execute_query(
218                                 "MATCH (child:Definition {id:
$child_id}), (parent:Definition {id: $parent_id}) "
219                                 "MERGE (child)-[:EXTENDS]->(parent)",
220                                 {"child_id": full_name, "parent_id":
parent_name}

```

```

221         )
222         inheritance_count += 1
223
224         for item in schema.get('oneOf', []) + schema.get('
anyOf', []):
225             if isinstance(item, dict) and '$ref' in item:
226                 option_name = item['$ref'].split('/')[-1]
227                 if option_name not in IGNORE_LIST:
228                     self.db.execute_query(
229                         "MATCH (union:Definition {id:
$union_id}), (option:Definition {id: $option_id}) "
230                         "MERGE (union)-[:ONE_OF]->(option)",
231                         {"union_id": full_name, "option_id":
option_name})
232                     )
233                     union_count += 1
234
235         print(f"    Created {inheritance_count} inheritance and {
union_count} union edges")
236
237     def _pass_3_build_semantic_edges(self):
238         semantic_rules = [
239             ("Deployment → PodTemplate",
240              "MATCH (d:Definition {kind: 'Deployment'})-[:
HAS_PROPERTY {name: 'spec'}]->(spec)"
241              "-[:HAS_PROPERTY {name: 'template'}]->(t) MERGE (d)
-[:CONTAINS_POD_TEMPLATE]->(t)"),
242             ("ReplicaSet → PodTemplate",
243              "MATCH (rs:Definition {kind: 'ReplicaSet'})-[:
HAS_PROPERTY {name: 'spec'}]->(spec)"
244              "-[:HAS_PROPERTY {name: 'template'}]->(t) MERGE (rs)
-[:CONTAINS_POD_TEMPLATE]->(t)"),
245             ("DaemonSet → PodTemplate",
246              "MATCH (ds:Definition {kind: 'DaemonSet'})-[:
HAS_PROPERTY {name: 'spec'}]->(spec)"
247              "-[:HAS_PROPERTY {name: 'template'}]->(t) MERGE (ds)
-[:CONTAINS_POD_TEMPLATE]->(t)"),
248             ("Job → PodTemplate",
249              "MATCH (j:Definition {kind: 'Job'})-[:HAS_PROPERTY {
name: 'spec'}]->(spec)"
250              "-[:HAS_PROPERTY {name: 'template'}]->(t) MERGE (j)
-[:CONTAINS_POD_TEMPLATE]->(t)"),
251             ("StatefulSet → PodTemplate",
252              "MATCH (s:Definition {kind: 'StatefulSet'})-[:
HAS_PROPERTY {name: 'spec'}]->(spec)"

```

```

253         "-[:HAS_PROPERTY {name: 'template'}]->(t) MERGE (s)
-[:CONTAINS_POD_TEMPLATE]->(t)",
254         ("CronJob → JobTemplate",
255         "MATCH (cj:Definition {kind: 'CronJob'})-[:
HAS_PROPERTY {name: 'spec'}]->(spec)"
256         "-[:HAS_PROPERTY {name: 'jobTemplate'}]->(j) MERGE (
cj)-[:CONTAINS_JOB_TEMPLATE]->(j)"),
257         ("PodSpec → Container",
258         "MATCH (p:Definition {id: 'io.k8s.api.core.v1.PodSpec
'})"
259         "-[:HAS_PROPERTY {name: 'containers'}]->(c) MERGE (p)
-[:HAS_CONTAINER]->(c)",
260         ("StatefulSet → VolumeClaimTemplates",
261         "MATCH (s:Definition {kind: 'StatefulSet'})-[:
HAS_PROPERTY {name: 'spec'}]->(spec)"
262         "-[:HAS_PROPERTY {name: 'volumeClaimTemplates'}]->(
pvc) MERGE (s)-[:CLAIMS_VOLUME]->(pvc)",
263         ("PodSpec → Volume",
264         "MATCH (p:Definition {id: 'io.k8s.api.core.v1.PodSpec
'})"
265         "-[:HAS_PROPERTY {name: 'volumes'}]->(v) MERGE (p)-[:
MOUNTS_VOLUME]->(v)",
266         ("PVC → StorageClass",
267         "MATCH (pvc:Definition {kind: 'PersistentVolumeClaim
'}), (sc:Definition {kind: 'StorageClass'})"
268         " MERGE (pvc)-[:USES_STORAGE_CLASS]->(sc)",
269         ("Container → ConfigMap",
270         "MATCH (c:Definition {id: 'io.k8s.api.core.v1.
Container'}), (cm:Definition {kind: 'ConfigMap'})"
271         " MERGE (c)-[:LOADS_CONFIGMAP]->(cm)",
272         ("PodSpec → Secret",
273         "MATCH (p:Definition {id: 'io.k8s.api.core.v1.PodSpec
'}), (s:Definition {kind: 'Secret'})"
274         " MERGE (p)-[:USES_SECRET]->(s)",
275         ("Service → Pod Selector",
276         "MATCH (svc:Definition {kind: 'Service'}), (pod:
Definition {kind: 'Pod'})"
277         " MERGE (svc)-[:SELECTS_POD]->(pod)",
278         ("Ingress → Service",
279         "MATCH (i:Definition {kind: 'Ingress'}), (s:
Definition {kind: 'Service'})"
280         " MERGE (i)-[:ROUTES_TO_SERVICE]->(s)",
281         ("RoleBinding → Role",
282         "MATCH (rb:Definition {kind: 'RoleBinding'}), (r:
Definition {kind: 'Role'})"

```

```

283         " MERGE (rb)-[:BINDS_ROLE]->(r)",
284         ("ClusterRoleBinding → ClusterRole",
285         "MATCH (rb:Definition {kind: 'ClusterRoleBinding'}),
(r:Definition {kind: 'ClusterRole'})"
286         " MERGE (rb)-[:BINDS_ROLE]->(r)",
287         ("RoleBinding → ServiceAccount",
288         "MATCH (rb:Definition {kind: 'RoleBinding'}), (sa:
Definition {kind: 'ServiceAccount'})"
289         " MERGE (rb)-[:BINDS_SERVICE_ACCOUNT]->(sa)",
290         ("ClusterRoleBinding → ServiceAccount",
291         "MATCH (rb:Definition {kind: 'ClusterRoleBinding'}),
(sa:Definition {kind: 'ServiceAccount'})"
292         " MERGE (rb)-[:BINDS_SERVICE_ACCOUNT]->(sa)",
293         ("PodSpec → ServiceAccount",
294         "MATCH (p:Definition {id: 'io.k8s.api.core.v1.PodSpec
'})", (sa:Definition {kind: 'ServiceAccount'})"
295         " MERGE (p)-[:USES_SERVICE_ACCOUNT]->(sa)",
296         ("HPA → Deployment",
297         "MATCH (h:Definition {kind: 'HorizontalPodAutoscaler
'})", (t:Definition {kind: 'Deployment'})"
298         " MERGE (h)-[:SCALES_RESOURCE]->(t)",
299         ("HPA → StatefulSet",
300         "MATCH (h:Definition {kind: 'HorizontalPodAutoscaler
'})", (t:Definition {kind: 'StatefulSet'})"
301         " MERGE (h)-[:SCALES_RESOURCE]->(t)",
302         ("HPA → ReplicaSet",
303         "MATCH (h:Definition {kind: 'HorizontalPodAutoscaler
'})", (t:Definition {kind: 'ReplicaSet'})"
304         " MERGE (h)-[:SCALES_RESOURCE]->(t)",
305         ("Pod → PodSpec (Logical EXTENDS)", ""
306         MATCH (p:Definition {kind: 'Pod'}), (spec:
Definition {id: 'io.k8s.api.core.v1.PodSpec'})
307         MERGE (p)-[:EXTENDS]->(spec)
308         """),
309         ("Deployment → DeploymentSpec (Logical EXTENDS)", ""
310         MATCH (d:Definition {kind: 'Deployment'}), (spec:
Definition {id: 'io.k8s.api.apps.v1.DeploymentSpec'})
311         MERGE (d)-[:EXTENDS]->(spec)
312         """),
313         ("Volume → VolumeSources (Logical ONE_OF)", ""
314         MATCH (v:Definition {id: 'io.k8s.api.core.v1.
Volume'})-[:HAS_PROPERTY]->(target)
315         WHERE target.name IN ['ConfigMapVolumeSource', '
EmptyDirVolumeSource',

```



```

316         'SecretVolumeSource', '
HostPathVolumeSource',
317         '
PersistentVolumeClaimVolumeSource']
318         MERGE (v)-[:ONE_OF]->(target)
319         """),
320         ("EnvFrom → EnvSources (Logical ANY_OF)", ""
321         MATCH (env:Definition {id: 'io.k8s.api.core.v1.
EnvFromSource'})-[:HAS_PROPERTY]->(target)
322         WHERE target.name IN ['ConfigMapEnvSource', '
SecretEnvSource']
323         MERGE (env)-[:ANY_OF]->(target)
324         """)
325     ]
326
327     executed = 0
328     for rule_name, query in semantic_rules:
329         try:
330             self.db.execute_query(query)
331             executed += 1
332         except Exception as e:
333             print(f"    SKIPPED {rule_name}: {str(e)[:80]}")
334
335     print(f"    Executed {executed} semantic rules")

```

Listing A.2 src/ingestion/parser.py — Pipeline Ingestion Data
(SwaggerGraphBuilder)

A.3 LangGraph Agent Pipeline

Berkas src/chatbot/graph_agent.py mendefinisikan *state machine* LangGraph yang mengorkestrasi lima node pipeline: *memory* → *thinker* → *retriever* → *speaker* → *saver*.

```

1 import json
2 import logging
3 import operator
4 from typing import TypedDict, List, Annotated, Optional
5 from langgraph.graph import StateGraph, END
6 from langchain_core.messages import AIMessage, BaseMessage
7
8 from src.chatbot.llm_factory import get_thinker_llm,
    get_speaker_llm
9 from src.chatbot.prompts import INTENT_PROMPT, RESPONSE_PROMPT
10 from src.chatbot.custom_retriever import StatefulK8sRetriever
11 from src.memory.zep_store import ZepMemoryStore

```

```

12
13 logger = logging.getLogger(__name__)
14
15 MAX_CONTEXT_CHARS = 12_000
16
17 _zep_store = None
18
19 def get_zep() -> ZepMemoryStore:
20     global _zep_store
21     if _zep_store is None:
22         _zep_store = ZepMemoryStore()
23     return _zep_store
24
25
26 class AgentState(TypedDict):
27     messages: Annotated[List[BaseMessage], operator.add]
28     question: str
29     session_id: str
30     chat_history: str
31     extracted_intent: dict
32     graph_context: str
33     reasoning_path: Optional[List[str]]
34     intent_type: Optional[str]
35     error: Optional[str]
36
37
38 def retrieve_memory_node(state: AgentState):
39     session_id = state.get("session_id", "default_session")
40     try:
41         history = get_zep().get_history(session_id=session_id,
42                                         limit=5)
43         return {"chat_history": history or "Belum ada riwayat
44         percakapan."}
45     except Exception as e:
46         logger.warning(f"Zep memory failed: {e}")
47         return {"chat_history": "Belum ada riwayat percakapan."}
48
49
50 def extract_intent_node(state: AgentState):
51     if state.get("error"):
52         return state
53
54     try:
55         llm = get_thinker_llm()
56         chain = INTENT_PROMPT | llm

```

```

55     response = chain.invoke({
56         "chat_history": state["chat_history"],
57         "question": state["question"]
58     })
59
60     raw_content = response.content.strip()
61     if raw_content.startswith("`"):
62         raw_content = raw_content.split("`")[1]
63         if raw_content.startswith("json"):
64             raw_content = raw_content[4:]
65             raw_content = raw_content.strip()
66
67     intent_data = json.loads(raw_content)
68     intent_type = intent_data.get("intent_type", "explain")
69     return {"extracted_intent": intent_data, "intent_type":
intent_type}
70
71     except json.JSONDecodeError as e:
72         logger.error(f"Thinker failed to output valid JSON: {e}")
73         return {"error": "Failed to parse search intent from user
query."}
74     except Exception as e:
75         logger.error(f"Intent extraction failed: {e}")
76         return {"error": str(e)}
77
78
79 def _make_retrieval_node(ablation_mode: str | None = None):
80     def execute_retrieval_node(state: AgentState):
81         if state.get("error"):
82             return {"graph_context": "Error in understanding
intent.", "reasoning_path": []}
83         try:
84             retriever = StatefulK8sRetriever()
85             intent_type = state.get("intent_type") or "explain"
86             graph_context, reasoning_path = retriever.
retrieve_context(
87                 state["extracted_intent"],
88                 intent_type=intent_type,
89                 ablation_mode=ablation_mode,
90             )
91             return {"graph_context": graph_context, "
reasoning_path": reasoning_path}
92         except Exception as e:
93             logger.error(f"Custom Retrieval failed: {e}")

```

```

94         return {"graph_context": "Database retrieval failed.",
95                "reasoning_path": [], "error": str(e)}
96     return execute_retrieval_node
97
98 def generate_response_node(state: AgentState):
99     try:
100         _ERROR_STRINGS = (
101             "Database retrieval failed.",
102             "Error in understanding intent.",
103         )
104         raw_ctx = state.get("graph_context") or ""
105         if state.get("error") or any(e in raw_ctx for e in
106            _ERROR_STRINGS):
107             return {"messages": [AIMessage(content=(
108                 "Maaf, saya tidak dapat menarik konteks dari
109                 Knowledge Graph saat ini. "
110                 "Mohon perjelas spesifikasi resource yang Anda
111                 cari."
112             ))]}
113
114         llm = get_speaker_llm()
115         chain = RESPONSE_PROMPT | llm
116
117         raw_context = raw_ctx
118         if len(raw_context) > MAX_CONTEXT_CHARS:
119             raw_context = raw_context[:MAX_CONTEXT_CHARS] + "\n...
120             [context truncated]"
121
122         chat_history = state["chat_history"]
123         intent_type = state.get("intent_type") or "explain"
124         if intent_type == "followup" and chat_history.strip() in (
125             "", "Belum ada riwayat percakapan."):
126             intent_type = "explain"
127
128         response = chain.invoke({
129             "chat_history": chat_history,
130             "retrieved_data": raw_context,
131             "question": state["question"],
132             "intent_type": intent_type
133         })
134         return {"messages": [AIMessage(content=response.content)]}
135     except Exception as e:
136         logger.error(f"Response Gen failed: {e}")

```

```

132         return {"messages": [AIMessage(content="Terjadi error saat
        membuat respons.")]}
133
134
135 def save_memory_node(state: AgentState):
136     session_id = state.get("session_id", "default_session")
137     try:
138         user_msg = state["question"]
139         ai_msg = state["messages"][-1].content if state["messages"
    ] else ""
140         if user_msg and ai_msg:
141             get_zep().add_message(session_id=session_id, user_msg=
    user_msg, ai_msg=ai_msg)
142     except Exception as e:
143         logger.warning(f"Failed to save memory: {e}")
144     return {}
145
146
147 def create_agent_graph(ablation_mode: str | None = None):
148     workflow = StateGraph(AgentState)
149     workflow.add_node("memory", retrieve_memory_node)
150     workflow.add_node("thinker", extract_intent_node)
151     workflow.add_node("retriever", _make_retrieval_node(
    ablation_mode))
152     workflow.add_node("speaker", generate_response_node)
153     workflow.add_node("saver", save_memory_node)
154
155     workflow.set_entry_point("memory")
156     workflow.add_edge("memory", "thinker")
157     workflow.add_edge("thinker", "retriever")
158     workflow.add_edge("retriever", "speaker")
159     workflow.add_edge("speaker", "saver")
160     workflow.add_edge("saver", END)
161
162     return workflow.compile()

```

Listing A.3 src/chatbot/graph_agent.py — LangGraph Agent Pipeline

A.4 Mekanisme Retrieval Bertingkat (*StatefulK8sRetriever*)

Kelas *StatefulK8sRetriever* pada berkas src/chatbot/custom_retriever.py mengimplementasikan *cascading retrieval* dengan kontrol kedalaman berbasis *intent*.

```

1 import json
2 import logging
3 from src.graph.neo4j_client import Neo4jClient
4 from src.graph.vector_index import VectorIndexManager

```

```

5 from src.graph.queries import (
6     EXACT_MATCH_QUERY,
7     SCHEMA_DEPS_QUERY,
8     PATH_EDGES_QUERY,
9     HYBRID_VECTOR_GRAPH_QUERY,
10 )
11
12 logger = logging.getLogger(__name__)
13
14 _DEPTH_BY_INTENT = {
15     "explain": 2,
16     "followup": 2,
17     "generate_yaml": 3,
18     "trace_relationship": 3,
19     "planning": 3,
20 }
21 _DEFAULT_DEPTH = 3
22
23 _MULTI_ENTITY_INTENTS = {"planning", "generate_yaml"}
24
25
26 class StatefulK8sRetriever:
27     def __init__(self):
28         self.db = Neo4jClient()
29         self.vector_mgr = VectorIndexManager()
30
31     def retrieve_context(
32         self,
33         intent_data: dict,
34         intent_type: str = "explain",
35         max_depth: int | None = None,
36         ablation_mode: str | None = None,
37     ) -> tuple[str, list[str]]:
38         if ablation_mode is not None and ablation_mode.startswith(
39             'depth_'):
40             try:
41                 depth = int(ablation_mode.split('_', 1)[1])
42             except (IndexError, ValueError):
43                 depth = max_depth if max_depth is not None \
44                     else _DEPTH_BY_INTENT.get(intent_type,
45 _DEFAULT_DEPTH)
46             else:
47                 depth = max_depth if max_depth is not None \
48                     else _DEPTH_BY_INTENT.get(intent_type,
49 _DEFAULT_DEPTH)

```

```

47
48     primary = intent_data.get("primary_resource", "")
49     related = intent_data.get("related_concepts", [])
50
51     try:
52         if ablation_mode == 'no_phase1':
53             root_name = None
54         else:
55             root_name = self._exact_match(primary)
56
57         if root_name:
58             if ablation_mode == 'no_multihop':
59                 record = {"RootResource": root_name, "
SchemaDependencies": []}
60             else:
61                 record = self._schema_deps(root_name, depth)
62         else:
63             search_query = f"{primary} {' '.join(related)}
Kubernetes"
64             embedding = self.vector_mgr.generate_embedding(
search_query)
65             record = self._vector_deps(embedding, depth)
66             if record:
67                 root_name = record.get("RootResource", "")
68                 if ablation_mode == 'no_multihop':
69                     record = {
70                         "RootResource": root_name,
71                         "Description": record.get("Description
", ""),
72                         "SchemaDependencies": [],
73                     }
74
75             if not record:
76                 return "Tidak ada skema Kubernetes yang relevan di
dalam Knowledge Graph.", []
77
78             deps = record.get("SchemaDependencies") or []
79             record["SchemaDependencies"] = [d for d in deps if d
is not None]
80
81             if ablation_mode == 'no_multihop':
82                 reasoning_path = []
83             else:
84                 reasoning_path = self._build_reasoning_path(
root_name, depth)

```

```

85
86         graph_context = json.dumps(record, indent=2,
ensure_ascii=False)
87
88         effective_multi_entity = (
89             set() if ablation_mode == 'no_multi_entity' else
 MULTI_ENTITY_INTENTS
90         )
91         if intent_type in effective_multi_entity and related:
92             for extra_resource in related[:2]:
93                 try:
94                     extra_root = self._exact_match(
extra_resource)
95                     if not extra_root:
96                         continue
97                     extra_record = self._schema_deps(
extra_root, depth)
98                     if not extra_record:
99                         continue
100                     extra_deps = extra_record.get("
SchemaDependencies") or []
101                     extra_record["SchemaDependencies"] = [d
for d in extra_deps if d is not None]
102                     extra_path = self._build_reasoning_path(
extra_root, depth)
103                     graph_context += "\n" + json.dumps(
extra_record, indent=2, ensure_ascii=False)
104                     seen_steps = set(reasoning_path)
105                     for step in extra_path:
106                         if step not in seen_steps:
107                             reasoning_path.append(step)
108                             seen_steps.add(step)
109                     except Exception as ex:
110                         logger.warning(f"[Retriever] Multi-entity
failed for '{extra_resource}': {ex}")
111
112                     return graph_context, reasoning_path
113
114             except Exception as e:
115                 logger.error(f"[Retriever] Graph traversal failed: {e}
")
116                 return f"Error retrieving context from Neo4j: {str(e)}
", []
117
118     def _exact_match(self, primary: str) -> str | None:

```



```

119         if not primary:
120             return None
121         rows = self.db.execute_query(EXACT_MATCH_QUERY, {"
primary_resource": primary})
122         return rows[0]["name"] if rows else None
123
124     def _schema_deps(self, root_name: str, max_depth: int) -> dict
| None:
125         cypher = SCHEMA_DEPS_QUERY.format(max_depth=max_depth)
126         rows = self.db.execute_query(cypher, {"root_name":
root_name})
127         return dict(rows[0]) if rows else None
128
129     def _vector_deps(self, embedding: list, max_depth: int) ->
dict | None:
130         cypher = HYBRID_VECTOR_GRAPH_QUERY.format(max_depth=
max_depth)
131         rows = self.db.execute_query(cypher, {"embedding":
embedding})
132         return dict(rows[0]) if rows else None
133
134     def _build_reasoning_path(self, root_name: str, max_depth: int
) -> list[str]:
135         if not root_name:
136             return []
137         try:
138             cypher = PATH_EDGES_QUERY.format(max_depth=max_depth)
139             rows = self.db.execute_query(cypher, {"root_name":
root_name})
140             seen, path = set(), []
141             for row in rows:
142                 edge = f"{row['parent']} -[{row['rel_type']}]> {
row['child']}"
143                 if edge not in seen:
144                     seen.add(edge)
145                     path.append(edge)
146             return path
147         except Exception as e:
148             logger.warning(f"[Retriever] Could not build reasoning
path: {e}")
149         return []

```

Listing A.4 src/chatbot/custom_retriever.py — Mekanisme Retrieval Bertingkat

A.5 Validasi YAML Tiga Lapis (*YAMLValidator*)

Kelas *YAMLValidator* pada berkas `src/validation/yaml_validator.py` mengimplementasikan validasi YAML tiga lapis: sintaksis PyYAML, kepatuhan skema *kubernetes-validate*, dan verifikasi *required fields* dari *knowledge graph*.

```
1 import logging
2 import yaml
3 from src.graph.neo4j_client import Neo4jClient
4 from src.graph.queries import REQUIRED_FIELDS_QUERY
5
6 logger = logging.getLogger(__name__)
7
8
9 class YAMLValidator:
10
11     def __init__(self):
12         self.db = Neo4jClient()
13
14     def validate(self, yaml_string: str, kind: str) -> dict:
15         result = {
16             "valid": True,
17             "syntax_errors": [],
18             "schema_errors": [],
19             "missing_fields": [],
20         }
21
22         try:
23             data = yaml.safe_load(yaml_string)
24         except yaml.YAMLError as e:
25             result["valid"] = False
26             result["syntax_errors"].append(str(e))
27             return result
28
29         if not isinstance(data, dict):
30             result["valid"] = False
31             result["syntax_errors"].append("YAML root must be a
mapping, got: " + type(data).__name__)
32             return result
33
34         try:
35             import kubernetes_validate
36             kubernetes_validate.validate(data, "1.29", strict=
False)
37         except ImportError:
```

```

38         logger.warning("kubernetes-validate not installed;
skipping schema validation layer.")
39     except Exception as e:
40         result["valid"] = False
41         path = "".join(str(p) for p in getattr(e, "path", []))
42         msg = getattr(e, "message", str(e))
43         result["schema_errors"].append(f"{path}: {msg}" if
path else msg)
44
45     try:
46         rows = self.db.execute_query(REQUIRED_FIELDS_QUERY, {"
kind": kind})
47         required_fields = {r["field_name"] for r in rows}
48     except Exception as e:
49         logger.warning(f"Could not fetch required fields from
graph: {e}")
50         required_fields = set()
51
52     if required_fields:
53         yaml_keys = set(self._flatten_keys(data))
54         missing = required_fields - yaml_keys
55         if missing:
56             result["valid"] = False
57             result["missing_fields"] = sorted(missing)
58
59     return result
60
61     def _flatten_keys(self, obj, prefix: str = "") -> list[str]:
62         keys = []
63         if isinstance(obj, dict):
64             for k, v in obj.items():
65                 full_key = f"{prefix}.{k}" if prefix else k
66                 keys.append(full_key)
67                 keys.extend(self._flatten_keys(v, full_key))
68         elif isinstance(obj, list):
69             for item in obj:
70                 keys.extend(self._flatten_keys(item, prefix))
71         return keys

```

Listing A.5 src/validation/yaml_validator.py — Validasi YAML Tiga Lapis

A.6 Query Cypher Neo4j

Berkas src/graph/queries.py mendefinisikan seluruh konstanta query Cypher yang digunakan oleh sistem sebagai *single source of truth*. Query ini mencakup

exact match, *traversal multi-hop*, *vector search*, dan *validasi required fields*.

```
1 EXACT_MATCH_QUERY = ""
2 MATCH (d:Definition)
3 WHERE d.name = $primary_resource
4     OR d.name ENDS WITH ('.' + $primary_resource)
5 RETURN d.name AS name, d.kind AS kind, d.description AS
6     description
7 ORDER BY
8     CASE WHEN d.name = $primary_resource THEN 0 ELSE 1 END
9 LIMIT 1
10 ""
11
12 SCHEMA_DEPS_QUERY = ""
13 MATCH (root:Definition {{name: $root_name}})
14 OPTIONAL MATCH (root)-[r:HAS_PROPERTY*1..{max_depth}]->(child:
15     Definition)
16
17 WITH root, r, child
18
19 WITH root,
20     CASE
21         WHEN child IS NOT NULL AND r IS NOT NULL THEN {{
22             path_depth:      size(r),
23             relation_type:    type(last(r)),
24             yaml_field:       last(r).name,
25             is_array:         coalesce(last(r).is_array, false),
26             child_resource:    child.name,
27             child_description: substring(child.description, 0,
28                                     150)
29         }}
30         ELSE null
31     END AS dep
32
33 RETURN root.name      AS RootResource,
34        root.kind      AS RootKind,
35        root.description AS RootDescription,
36        1.0             AS VectorSimilarityScore,
37        collect(dep)    AS SchemaDependencies
38 ""
39
40 _ALL_EDGE_TYPES = (
41     "HAS_PROPERTY|SCALES_RESOURCE|CONTAINS_POD_TEMPLATE|
42     CONTAINS_JOB_TEMPLATE"
43     "|BINDS_ROLE|BINDS_SERVICE_ACCOUNT|EXTENDS|HAS_CONTAINER"
```

```

40     " | CLAIMS_VOLUME | MOUNTS_VOLUME | USES_STORAGE_CLASS |
LOADS_CONFIGMAP"
41     " | USES_SECRET | SELECTS_POD | ROUTES_TO_SERVICE |
USES_SERVICE_ACCOUNT"
42     " | ONE_OF | ANY_OF"
43 )
44
45 PATH_EDGES_QUERY = ""
46 MATCH p = (root:Definition {{name: $root_name}})
47         -[:{all_edges}*1..{max_depth}]->(leaf:Definition)
48 WITH p
49 LIMIT 500
50 WITH [i IN range(0, size(nodes(p))-2) | {{
51     parent:    nodes(p)[i].name,
52     child:     nodes(p)[i+1].name,
53     rel_type:  type(relationships(p)[i]),
54     depth:     i + 1
55 }}] AS edges
56 UNWIND edges AS edge
57 RETURN DISTINCT edge.parent    AS parent,
58                 edge.child     AS child,
59                 edge.rel_type  AS rel_type,
60                 edge.depth     AS depth
61 ORDER BY edge.depth ASC, edge.parent ASC
62 LIMIT 50
63 """.replace("{all_edges}", _ALL_EDGE_TYPES)
64
65 HYBRID_VECTOR_GRAPH_QUERY = ""
66 CALL db.index.vector.queryNodes('definition_description_vector',
67     1, $embedding)
68 YIELD node AS root, score
69
70 OPTIONAL MATCH (root)-[r:HAS_PROPERTY*1..{max_depth}]->(child:
71     Definition)
72
73 WITH root, score, r, child
74
75 WITH root, score,
76     CASE
77         WHEN child IS NOT NULL AND r IS NOT NULL THEN {{
78             path_depth:      size(r),
79             relation_type:    type(last(r)),
80             yaml_field:       last(r).name,
81             is_array:         coalesce(last(r).is_array, false),
82             child_resource:    child.name,

```

```

81         child_description: substring(child.description, 0,
150)
82     }}
83     ELSE null
84 END AS dep
85
86 RETURN root.name      AS RootResource,
87        root.kind      AS RootKind,
88        root.description AS RootDescription,
89        score          AS VectorSimilarityScore,
90        collect(dep)   AS SchemaDependencies
91 ""
92
93 REQUIRED_FIELDS_QUERY = ""
94 MATCH (d:Definition {name: $kind})-[r:HAS_PROPERTY {is_required:
true}]->(p)
95 RETURN p.name AS field_name
96 ""
97
98 ALL_FIELDS_QUERY = ""
99 MATCH (d:Definition {name: $kind})-[r:HAS_PROPERTY]->(p)
100 RETURN p.name AS field_name, r.is_required AS is_required
101 ""

```

Listing A.6 src/graph/queries.py — Query Cypher Neo4j

A.7 Fungsi Metrik Evaluasi

Potongan berikut dari scripts/evaluate.py menampilkan tiga fungsi komputasi metrik inti: `compute_ansq()`, `compute_retq()`, dan `compute_reaq()`. Bagian *orchestrator* (mode invoker, checkpoint, dan summary printing) dihilangkan untuk singkatnya; implementasi lengkap tersedia di repositori proyek.

```

1 import re
2 import math
3
4 _SCOPE_Q_KEYWORDS = [
5     "scope", "scoped", "namespaced", "cluster-scoped", "cluster-
wide",
6     "namespace-level", "cluster-level", "lingkup", "cakupan",
7     "bisa diakses lintas", "seluruh cluster", "non-namespaced",
8 ]
9
10 _K8S_TERM_RE = re.compile(
11     r'\b(?:'
12     r'[A-Z][a-z]+(?:[A-Z][a-zA-Z]+)+'

```

```

13     r'|Deployment|StatefulSet|DaemonSet|ReplicaSet|CronJob|Ingress
14     |
15     r'|ConfigMap|Secret|Namespace|ServiceAccount|Endpoints|Pod|
16     Service'
17     r'|Node|ResourceQuota|LimitRange|NetworkPolicy|StorageClass'
18     r'|Role|ClusterRole|RoleBinding|ClusterRoleBinding'
19     r'|HorizontalPodAutoscaler|PersistentVolume|
20     PersistentVolumeClaim'
21     r'|apiVersion|kubectl|namespace[sd]?|pod[sd]?|deployment[sd]?|
22     service[sd]?'
23     r'|configmap[sd]?|secret[sd]?|ingress(?:es)?|statefulset[sd]?|
24     daemonset[sd]?'
25     r'|replicaset[sd]?|cronjob[sd]?|job[sd]?|persistentvolume(?:
26     claim)?[sd]?'
27     r'|clusterrole(?:binding)?[sd]?|rolebinding[sd]?|
28     serviceaccount[sd]?'
29     r'|networkpolicy|hpa|pvc|pv|rbac'
30     r')\b'
31 )
32
33
34
35
36
37
38 def _token_f1(pred: str, gold: str) -> float:
39     pred_tokens = set(pred.lower().split())
40     gold_tokens = set(gold.lower().split())
41     if not pred_tokens or not gold_tokens:
42         return 0.0
43     common = pred_tokens & gold_tokens
44     precision = len(common) / len(pred_tokens)
45     recall = len(common) / len(gold_tokens)
46     return 2 * precision * recall / (precision + recall) if (
47         precision + recall) > 0 else 0.0
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99

```

```

100 def _cosine_similarity(embedder, text1: str, text2: str) -> float:
101     import numpy as np
102     e1 = embedder.embed_query(text1)
103     e2 = embedder.embed_query(text2)
104     v1, v2 = np.array(e1), np.array(e2)
105     denom = np.linalg.norm(v1) * np.linalg.norm(v2)
106     return float(np.dot(v1, v2) / denom) if denom > 0 else 0.0
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149

```

```

150 def _effective_type(fixture_type: str, ground_truth: dict) -> str:
151     if fixture_type == "realworld":
152         return ground_truth.get("realworld_subtype", "realworld")
153

```

```

50     return fixture_type
51
52
53 def _extract_yaml_block(text: str) -> str:
54     if "```yaml" in text:
55         parts = text.split("```yaml", 1)
56         if len(parts) > 1:
57             yaml_lines = parts[1].split("```")[0].strip().
splitlines()
58             return "\n".join(yaml_lines).strip()
59     if "```" in text:
60         parts = text.split("```", 1)
61         if len(parts) > 1:
62             yaml_lines = parts[1].split("```")[0].strip().
splitlines()
63             return "\n".join(yaml_lines).strip()
64     return text.strip()
65
66
67 def compute_ansq(
68     answer: str,
69     ground_truth: dict,
70     fixture_type: str,
71     embedder=None,
72     ablation_mode: str | None = None,
73 ) -> dict:
74     scores = {}
75     fixture_type = _effective_type(fixture_type, ground_truth)
76
77     if fixture_type == "yaml_gen":
78         yaml_candidate = _extract_yaml_block(answer)
79         try:
80             import yaml
81             yaml.safe_load(yaml_candidate)
82             scores["syntactic_validity"] = 1.0
83         except Exception:
84             scores["syntactic_validity"] = 0.0
85     else:
86         scores["syntactic_validity"] = None
87
88     if fixture_type == "yaml_gen":
89         yaml_candidate = _extract_yaml_block(answer)
90         try:
91             import yaml
92             import kubernetes_validate

```



```

93         data = yaml.safe_load(yaml_candidate)
94         if isinstance(data, dict):
95             kubernetes_validate.validate(data, "1.29", strict=
False)
96             scores["schema_compliance"] = 1.0
97         else:
98             scores["schema_compliance"] = 0.0
99     except ImportError:
100         scores["schema_compliance"] = None
101     except Exception:
102         scores["schema_compliance"] = 0.0
103 else:
104     scores["schema_compliance"] = None
105
106 gt_answer = ground_truth.get("answer", "")
107 if embedder is not None and gt_answer:
108     scores["answer_relevance"] = _cosine_similarity(embedder,
answer, gt_answer)
109 else:
110     scores["answer_relevance"] = _token_f1(answer, gt_answer)
111
112 def _node_matches(node: str, answer_lower: str) -> bool:
113     nl = node.lower()
114     return nl in answer_lower or nl + "s" in answer_lower or
nl + "es" in answer_lower
115
116 faith_source = ground_truth.get("key_nodes") or ground_truth.
get("relevant_nodes", [])
117 gt_nodes = [n.split(".")[1] for n in faith_source]
118 hit = sum(1 for n in gt_nodes if _node_matches(n, answer.lower
()))
119 scores["faithfulness"] = hit / len(gt_nodes) if gt_nodes else
1.0
120
121 if fixture_type == "yaml_gen" and ablation_mode is not None
and ablation_mode != "no_yaml_layer3":
122     yaml_candidate = _extract_yaml_block(answer)
123     try:
124         import yaml as _yaml
125         _data = _yaml.safe_load(yaml_candidate)
126         if isinstance(_data, dict):
127             from src.validation.yaml_validator import
YAMLValidator
128             _kind = _data.get("kind", "")

```

```

129         _vresult = YAMLValidator().validate(yaml_candidate
, _kind)
130         scores["layer3_compliance"] = None if _vresult["
syntax_errors"] \
131             else (1.0 if not _vresult["missing_fields"]
else 0.0)
132         else:
133             scores["layer3_compliance"] = None
134         except Exception:
135             scores["layer3_compliance"] = None
136         else:
137             scores["layer3_compliance"] = None
138
139         applicable = [v for v in scores.values() if v is not None]
140         scores["ansq_score"] = sum(applicable) / len(applicable) if
applicable else 0.0
141         return scores
142
143
144 def compute_retq(reasoning_path: list, ground_truth: dict) -> dict
:
145     _RELATION_RE = re.compile(r"-\[([^\]]+)\]->?")
146
147     def _node_tokens(step: str) -> list:
148         cleaned = _RELATION_RE.sub(" ", step)
149         return [t for t in cleaned.split() if t]
150
151     expected_nodes = set(n.split(".")[1] for n in ground_truth.
get("relevant_nodes", []))
152     retrieved_nodes, seen_nodes = [], set()
153     for step in reasoning_path:
154         for tok in _node_tokens(step):
155             if tok not in seen_nodes:
156                 seen_nodes.add(tok)
157                 retrieved_nodes.append(tok)
158     retrieved_set = set(retrieved_nodes)
159
160     intersection = retrieved_set & expected_nodes
161     precision = len(intersection) / len(retrieved_set) if
retrieved_set else 0.0
162     recall = len(intersection) / len(expected_nodes) if
expected_nodes else 1.0
163     f1 = 2 * precision * recall / (precision + recall) if (
precision + recall) > 0 else 0.0
164

```

```

165 expected_path = ground_truth.get("expected_path", [])
166 graph_coverage = (
167     len([p for p in expected_path
168         if any(p.split(" -")[0] in step for step in
reasoning_path)])
169     / len(expected_path) if expected_path else 1.0
170 )
171
172 k = len(retrieved_nodes)
173 dcg = sum(
174     (1.0 / math.log2(i + 2))
175     for i, node in enumerate(retrieved_nodes)
176     if node in expected_nodes
177 )
178 ideal_hits = min(len(expected_nodes), k) if k > 0 else 0
179 idcg = sum(1.0 / math.log2(i + 2) for i in range(ideal_hits))
180 ndcg_at_k = dcg / idcg if idcg > 0 else (1.0 if not
expected_nodes else 0.0)
181
182 if expected_path:
183     matched_edges = sum(
184         1 for ep in expected_path
185         if any(ep in step or step in ep for step in
reasoning_path)
186     )
187     edge_coverage = matched_edges / len(expected_path)
188 else:
189     edge_coverage = 1.0
190
191 retq_score = (precision + recall + f1 + graph_coverage +
ndcg_at_k + edge_coverage) / 6
192
193 return {
194     "precision_at_k": precision,
195     "recall_at_k": recall,
196     "f1_at_k": f1,
197     "graph_coverage": graph_coverage,
198     "ndcg_at_k": ndcg_at_k,
199     "edge_coverage": edge_coverage,
200     "retq_score": retq_score,
201 }
202
203
204 def compute_reaq(
205     reasoning_path: list,

```

```

206     answer: str,
207     ground_truth: dict,
208     fixture_type: str,
209     graph_context: str = "",
210     fixture_scope: str = "",
211     question: str = "",
212     k8s_vocabulary: set = None,
213     cgg_mode: bool = False,
214 ) -> dict:
215     fixture_type = _effective_type(fixture_type, ground_truth)
216     expected_path = ground_truth.get("expected_path", [])
217     multi_hop = ground_truth.get("multi_hop", False)
218
219     if expected_path and multi_hop:
220         matched = sum(
221             1 for ep in expected_path
222             if any(ep.split(" -")[0] in step for step in
223 reasoning_path)
224         )
225         hop_accuracy = matched / len(expected_path)
226     else:
227         hop_accuracy = 1.0 if reasoning_path else 0.0
228
229     multi_hop_success = 1.0 if (multi_hop and reasoning_path) or (
230 not multi_hop) else 0.0
231
232     question_lower = question.lower()
233     scope_relevant = (
234         any(kw in question_lower for kw in _SCOPE_Q_KEYWORDS)
235         or fixture_scope == "Cluster"
236     )
237
238     if not scope_relevant:
239         scope_accuracy = 1.0
240     else:
241         _NAMESPACED_POSITIVE = ["namespaced", "dalam namespace", "
242 di namespace",
243                                 "namespace-scoped", "namespace
244 tertentu"]
245         _NAMESPACED_NEGATIVE = ["cluster-scoped", "cluster-wide",
246 "seluruh cluster",
247                                 "tidak terikat namespace", "non-
248 namespaced"]
249         _CLUSTER_POSITIVE = ["cluster-scoped", "cluster-wide",
250 "seluruh cluster",

```

```

244         "tidak terikat namespace", "non-
namespaced", "lintas namespace"]
245     _CLUSTER_NEGATIVE = ["namespaced", "dalam namespace", "
namespace-scoped"]
246
247     answer_lower = answer.lower()
248     if fixture_scope == "Namespaced":
249         correct_hit = any(kw in answer_lower for kw in
_NAMESPACED_POSITIVE)
250         wrong_hit = any(kw in answer_lower for kw in
_NAMESPACED_NEGATIVE)
251         scope_accuracy = 1.0 if (correct_hit and not wrong_hit
) else (0.0 if wrong_hit else 0.5)
252         elif fixture_scope == "Cluster":
253             correct_hit = any(kw in answer_lower for kw in
_CLUSTER_POSITIVE)
254             wrong_hit = any(kw in answer_lower for kw in
_CLUSTER_NEGATIVE)
255             scope_accuracy = 1.0 if (correct_hit and not wrong_hit
) else (0.0 if wrong_hit else 0.5)
256         else:
257             scope_accuracy = 1.0
258
259     if cgg_mode:
260         from src.validation.cgg_validator import
cgg_grounding_score
261         grounding_score, hallucination_rate = cgg_grounding_score(
answer, graph_context)
262     else:
263         answer_terms = set(t.lower() for t in _K8S_TERM_RE.findall
(answer))
264         if answer_terms:
265             if k8s_vocabulary:
266                 grounded = sum(1 for t in answer_terms if t in
k8s_vocabulary)
267             else:
268                 ctx_lower = graph_context.lower()
269                 grounded = sum(1 for t in answer_terms if t in
ctx_lower)
270                 hallucination_rate = 1.0 - grounded / len(answer_terms
)
271         else:
272             hallucination_rate = 0.0
273         grounding_score = 1.0 - hallucination_rate
274

```

```

275     reaq_score = (hop_accuracy + multi_hop_success +
276                 scope_accuracy + grounding_score) / 4
277
278     return {
279         "hop_accuracy":      hop_accuracy,
280         "multi_hop_success": multi_hop_success,
281         "scope_accuracy":    scope_accuracy,
282         "hallucination_rate": hallucination_rate,
283         "grounding_score":   grounding_score,
284         "reaq_score":        reaq_score,
285     }

```

Listing A.7 scripts/evaluate.py — Fungsi Metrik Evaluasi (AnsQ, RetQ, ReaQ)

LAMPIRAN B

FIXTURE EVALUASI PAKAR

Lampiran ini menyajikan 10 *fixture* evaluasi yang digunakan dalam validasi pakar (*expert evaluation*) sebagaimana diuraikan pada Subbab VI.4. Setiap *fixture* mewakili satu pertanyaan beserta *ground truth* yang mencakup *relevant nodes* dan *expected path* di *knowledge graph*. *Field* answer dan context dikecualikan dari tampilan ini untuk mempertahankan objektivitas penilaian. Tabel B.1 merangkum distribusi 10 *fixture* tersebut.

Tabel B.1 Ringkasan 10 Fixture Evaluasi Pakar

No	ID	Tipe	Resource Utama	Scope	M-H
1	deployment_basic	konseptual	Deployment	Namespaced	Ya
2	service_types	konseptual	Service	Namespaced	Ya
3	scale_existing_deployment	lanjutan	Deployment	Namespaced	Tidak
4	ingress_service_pod	relasional	Ingress	Namespaced	Ya
5	hpa_deployment_pod	relasional	HorizontalPodAutoScaler	Namespaced	Ya
6	cronjob_backup	generasi YAML	CronJob	Namespaced	Ya
7	networkpolicy_deny_all	generasi YAML	NetworkPolicy	Namespaced	Tidak
8	statefulset_with_pvc	generasi YAML	StatefulSet	Namespaced	Ya
9	pi_want_to_reject_all_docker	skenario nyata	Namespace	Cluster	Ya
10	kubectl_find_pods_with_env	perintah	Pod	Namespaced	Tidak

B.1 Fixture B.1 — deployment_basic (Konseptual)

Fixture ini menguji pemahaman sistem terhadap mekanisme *RollingUpdate* Deployment: bagaimana Pod diperbarui saat *image* berubah dan apakah terjadi *downtime*.

```
1 {
2   "id": "deployment_basic",
3   "type": "conceptual",
4   "question": "Apa yang terjadi pada Pod yang sedang berjalan
5     ketika saya update image Deployment ke versi baru? Apakah ada
6     downtime?",
7   "resource": "io.k8s.api.apps.v1.Deployment",
8   "scope": "Namespaced",
9   "multi_hop": true,
10  "ground_truth": {
11    "relevant_nodes": [
12      "io.k8s.api.apps.v1.Deployment",
13      "io.k8s.api.apps.v1.DeploymentCondition",
14      "io.k8s.api.apps.v1.DeploymentSpec",
15      "io.k8s.api.apps.v1.DeploymentStatus",
16      "io.k8s.api.apps.v1.DeploymentStrategy",
17      "io.k8s.api.apps.v1.RollingUpdateDeployment",
18      "io.k8s.api.core.v1.PodSpec",
19      "io.k8s.api.core.v1.PodTemplateSpec",
20      "io.k8s.apimachinery.pkg.apis.meta.v1.LabelSelector"
21    ],
22    "expected_path": [
23      "Deployment -[CONTAINS_POD_TEMPLATE]-> PodTemplateSpec",
24      "Deployment -[EXTENDS]-> DeploymentSpec",
25      "Deployment -[HAS_PROPERTY]-> DeploymentSpec",
26      "Deployment -[HAS_PROPERTY]-> DeploymentStatus",
27      "DeploymentSpec -[HAS_PROPERTY]-> DeploymentStrategy",
28      "DeploymentSpec -[HAS_PROPERTY]-> LabelSelector",
29      "DeploymentSpec -[HAS_PROPERTY]-> PodTemplateSpec",
30      "DeploymentStatus -[HAS_PROPERTY]-> DeploymentCondition",
31      "DeploymentStrategy -[HAS_PROPERTY]->
32      RollingUpdateDeployment",
33      "PodTemplateSpec -[HAS_PROPERTY]-> PodSpec"
34    ]
35  }
36 }
```

Listing B.1 fixture-deployment-basic.json — Konseptual: Pembaruan Image Deployment

B.2 Fixture B.2 — service_types (Konseptual)

Fixture ini menguji kemampuan sistem membandingkan dua mekanisme *expose* aplikasi: Service tipe LoadBalancer versus Ingress, beserta pertimbangan *trade-off*-nya.

```
1 {
2   "id": "service_types",
3   "type": "conceptual",
4   "question": "Kapan sebaiknya menggunakan Service tipe
5     LoadBalancer dibanding Ingress untuk expose aplikasi ke
6     internet? Apa perbedaan dan trade-off-nya?",
7   "resource": "io.k8s.api.core.v1.Service",
8   "scope": "Namespaced",
9   "multi_hop": true,
10  "ground_truth": {
11    "relevant_nodes": [
12      "io.k8s.api.core.v1.LoadBalancerStatus",
13      "io.k8s.api.core.v1.Pod",
14      "io.k8s.api.core.v1.PodSpec",
15      "io.k8s.api.core.v1.PodStatus",
16      "io.k8s.api.core.v1.Service",
17      "io.k8s.api.core.v1.ServicePort",
18      "io.k8s.api.core.v1.ServiceSpec",
19      "io.k8s.api.core.v1.ServiceStatus",
20      "io.k8s.api.core.v1.SessionAffinityConfig",
21      "io.k8s.api.networking.v1.Ingress",
22      "io.k8s.api.networking.v1.IngressRule",
23      "io.k8s.api.networking.v1.IngressSpec",
24      "io.k8s.apimachinery.pkg.apis.meta.v1.Condition"
25    ],
26    "expected_path": [
27      "Ingress -[HAS_PROPERTY]-> IngressSpec",
28      "IngressSpec -[HAS_PROPERTY]-> IngressRule",
29      "Pod -[EXTENDS]-> PodSpec",
30      "Pod -[HAS_PROPERTY]-> PodSpec",
31      "Pod -[HAS_PROPERTY]-> PodStatus",
32      "Service -[HAS_PROPERTY]-> ServiceSpec",
33      "Service -[HAS_PROPERTY]-> ServiceStatus",
34      "Service -[SELECTS_POD]-> Pod",
35      "ServiceSpec -[HAS_PROPERTY]-> ServicePort",
36      "ServiceSpec -[HAS_PROPERTY]-> SessionAffinityConfig",
37      "ServiceStatus -[HAS_PROPERTY]-> Condition",
38      "ServiceStatus -[HAS_PROPERTY]-> LoadBalancerStatus"
39    ]
40  }
41 }
```

39 }

Listing B.2 fixture-service-types.json — Konseptual: Service
LoadBalancer vs. Ingress

B.3 Fixture B.3 — scale_existing_deployment (Lanjutan)

Fixture bertipe *followup* ini mengukur kemampuan sistem memanfaatkan konteks percakapan sebelumnya (pembuatan Deployment nginx) untuk menjawab pertanyaan lanjutan tentang perubahan jumlah replika.

```
1 {
2   "id": "scale_existing_deployment",
3   "type": "followup",
4   "context_question": "Buat YAML Deployment nginx dengan 3 replika",
5   "question": "Sekarang ubah jumlah replikanya menjadi 5",
6   "resource": "io.k8s.api.apps.v1.Deployment",
7   "scope": "Namespaced",
8   "multi_hop": false,
9   "ground_truth": {
10    "relevant_nodes": [
11      "io.k8s.api.apps.v1.Deployment",
12      "io.k8s.api.apps.v1.DeploymentCondition",
13      "io.k8s.api.apps.v1.DeploymentSpec",
14      "io.k8s.api.apps.v1.DeploymentStatus",
15      "io.k8s.api.apps.v1.DeploymentStrategy",
16      "io.k8s.api.core.v1.PodSpec",
17      "io.k8s.api.core.v1.PodTemplateSpec",
18      "io.k8s.apimachinery.pkg.apis.meta.v1.LabelSelector"
19    ],
20    "expected_path": [
21      "Deployment -[CONTAINS_POD_TEMPLATE]-> PodTemplateSpec",
22      "Deployment -[EXTENDS]-> DeploymentSpec",
23      "Deployment -[HAS_PROPERTY]-> DeploymentSpec",
24      "Deployment -[HAS_PROPERTY]-> DeploymentStatus",
25      "DeploymentSpec -[HAS_PROPERTY]-> DeploymentStrategy",
26      "DeploymentSpec -[HAS_PROPERTY]-> LabelSelector",
27      "DeploymentSpec -[HAS_PROPERTY]-> PodTemplateSpec",
28      "DeploymentStatus -[HAS_PROPERTY]-> DeploymentCondition",
29      "PodTemplateSpec -[HAS_PROPERTY]-> PodSpec"
30    ],
31    "expected_yaml_keys": ["spec.replicas"]
32  }
```

33 }

Listing B.3 fixture-scale-existing-deployment.json — Lanjutan:
Skalakan Replika Deployment

B.4 Fixture B.4 — ingress_service_pod (Relasional)

Fixture ini menguji penelusuran relasi multi-hop: alur traffic dari Ingress melalui Service hingga ke Pod, dan identifikasi titik kegagalan pada error 502.

```
1 {
2   "id": "ingress_service_pod",
3   "type": "relationship",
4   "question": "Saya punya Ingress dengan host app.example.com yang
      diarahkan ke Service frontend-svc port 80. Pod saya statusnya
      Running tapi request dari luar masih dapat response 502 Bad
      Gateway. Bagaimana alur traffic dari Ingress sampai ke Pod dan
      di mana biasanya masalah terjadi?",
5   "resource": "io.k8s.api.networking.v1.Ingress",
6   "scope": "Namespaced",
7   "multi_hop": true,
8   "ground_truth": {
9     "relevant_nodes": [
10      "io.k8s.api.core.v1.LoadBalancerStatus",
11      "io.k8s.api.core.v1.Pod",
12      "io.k8s.api.core.v1.PodSpec",
13      "io.k8s.api.core.v1.PodStatus",
14      "io.k8s.api.core.v1.Service",
15      "io.k8s.api.core.v1.ServicePort",
16      "io.k8s.api.core.v1.ServiceSpec",
17      "io.k8s.api.core.v1.ServiceStatus",
18      "io.k8s.api.core.v1.SessionAffinityConfig",
19      "io.k8s.api.core.v1.TypedLocalObjectReference",
20      "io.k8s.api.networking.v1.HTTPIngressPath",
21      "io.k8s.api.networking.v1.HTTPIngressRuleValue",
22      "io.k8s.api.networking.v1.Ingress",
23      "io.k8s.api.networking.v1.IngressBackend",
24      "io.k8s.api.networking.v1.IngressLoadBalancerIngress",
25      "io.k8s.api.networking.v1.IngressLoadBalancerStatus",
26      "io.k8s.api.networking.v1.IngressRule",
27      "io.k8s.api.networking.v1.IngressServiceBackend",
28      "io.k8s.api.networking.v1.IngressSpec",
29      "io.k8s.api.networking.v1.IngressStatus",
30      "io.k8s.api.networking.v1.IngressTLS",
31      "io.k8s.apimachinery.pkg.apis.meta.v1.Condition"
32    ],
```

```

33     "expected_path": [
34         "HTTPIngressPath -[HAS_PROPERTY]-> IngressBackend",
35         "HTTPIngressRuleValue -[HAS_PROPERTY]-> HTTPIngressPath",
36         "Ingress -[HAS_PROPERTY]-> IngressSpec",
37         "Ingress -[HAS_PROPERTY]-> IngressStatus",
38         "Ingress -[ROUTES_TO_SERVICE]-> Service",
39         "IngressBackend -[HAS_PROPERTY]-> IngressServiceBackend",
40         "IngressBackend -[HAS_PROPERTY]-> TypedLocalObjectReference"
41     ],
42     "IngressLoadBalancerStatus -[HAS_PROPERTY]->
IngressLoadBalancerIngress",
43     "IngressRule -[HAS_PROPERTY]-> HTTPIngressRuleValue",
44     "IngressSpec -[HAS_PROPERTY]-> IngressBackend",
45     "IngressSpec -[HAS_PROPERTY]-> IngressRule",
46     "IngressSpec -[HAS_PROPERTY]-> IngressTLS",
47     "IngressStatus -[HAS_PROPERTY]-> IngressLoadBalancerStatus",
48     "Pod -[EXTENDS]-> PodSpec",
49     "Pod -[HAS_PROPERTY]-> PodSpec",
50     "Pod -[HAS_PROPERTY]-> PodStatus",
51     "Service -[HAS_PROPERTY]-> ServiceSpec",
52     "Service -[HAS_PROPERTY]-> ServiceStatus",
53     "Service -[SELECTS_POD]-> Pod",
54     "ServiceSpec -[HAS_PROPERTY]-> ServicePort",
55     "ServiceSpec -[HAS_PROPERTY]-> SessionAffinityConfig",
56     "ServiceStatus -[HAS_PROPERTY]-> Condition",
57     "ServiceStatus -[HAS_PROPERTY]-> LoadBalancerStatus"
58 ],
59 "key_nodes": [
60     "io.k8s.api.networking.v1.Ingress",
61     "io.k8s.api.networking.v1.IngressSpec",
62     "io.k8s.api.networking.v1.IngressRule",
63     "io.k8s.api.core.v1.Service"
64 ]
65 }

```

Listing B.4 fixture-ingress-service-pod.json — Relasional: Alur Traffic
Ingress→Service→Pod

B.5 Fixture B.5 — hpa_deployment_pod (Relasional)

Fixture ini menguji kemampuan sistem menjelaskan siklus kerja HPA: alur dari pembacaan metrik CPU hingga pembaruan spec.replicas di Deployment, dan diagnosis ketika HPA tidak berfungsi.

```
1 {
```

```

2  "id": "hpa_deployment_pod",
3  "type": "relationship",
4  "question": "Saya sudah set HPA dengan
    targetCPUUtilizationPercentage 50% untuk Deployment saya, tapi
    meskipun CPU usage sudah di atas 80% selama beberapa menit,
    jumlah Pod tidak bertambah. Bagaimana HPA seharusnya mengatur
    jumlah Pod dan apa yang mungkin menyebabkan HPA tidak bekerja?"
5  ,
6  "resource": "io.k8s.api.autoscaling.v2.HorizontalPodAutoscaler",
7  "scope": "Namespaced",
8  "multi_hop": true,
9  "ground_truth": {
10     "relevant_nodes": [
11         "io.k8s.api.apps.v1.Deployment",
12         "io.k8s.api.apps.v1.ReplicaSet",
13         "io.k8s.api.apps.v1.StatefulSet",
14         "io.k8s.api.autoscaling.v2.CrossVersionObjectReference",
15         "io.k8s.api.autoscaling.v2.HorizontalPodAutoscaler",
16         "io.k8s.api.autoscaling.v2.HorizontalPodAutoscalerBehavior",
17         "io.k8s.api.autoscaling.v2.HorizontalPodAutoscalerCondition"
18     ],
19     "io.k8s.api.autoscaling.v2.HorizontalPodAutoscalerSpec",
20     "io.k8s.api.autoscaling.v2.HorizontalPodAutoscalerStatus",
21     "io.k8s.api.autoscaling.v2.MetricSpec",
22     "io.k8s.api.autoscaling.v2.MetricStatus",
23     "io.k8s.api.autoscaling.v2.ResourceMetricSource",
24     "io.k8s.api.core.v1.PodSpec",
25     "io.k8s.api.core.v1.PodTemplateSpec"
26 ],
27 "expected_path": [
28     "Deployment -[CONTAINS_POD_TEMPLATE]-> PodTemplateSpec",
29     "Deployment -[EXTENDS]-> DeploymentSpec",
30     "Deployment -[HAS_PROPERTY]-> DeploymentSpec",
31     "Deployment -[HAS_PROPERTY]-> DeploymentStatus",
32     "HorizontalPodAutoscaler -[HAS_PROPERTY]->
HorizontalPodAutoscalerSpec",
33     "HorizontalPodAutoscaler -[HAS_PROPERTY]->
HorizontalPodAutoscalerStatus",
34     "HorizontalPodAutoscaler -[SCALES_RESOURCE]-> Deployment",
35     "HorizontalPodAutoscaler -[SCALES_RESOURCE]-> ReplicaSet",
36     "HorizontalPodAutoscaler -[SCALES_RESOURCE]-> StatefulSet",
37     "HorizontalPodAutoscalerBehavior -[HAS_PROPERTY]->
HPAScalingRules",
38     "HorizontalPodAutoscalerSpec -[HAS_PROPERTY]->
CrossVersionObjectReference",

```

```

37     "HorizontalPodAutoscalerSpec -[HAS_PROPERTY]->
HorizontalPodAutoscalerBehavior",
38     "HorizontalPodAutoscalerSpec -[HAS_PROPERTY]-> MetricSpec",
39     "HorizontalPodAutoscalerStatus -[HAS_PROPERTY]->
HorizontalPodAutoscalerCondition",
40     "HorizontalPodAutoscalerStatus -[HAS_PROPERTY]->
MetricStatus",
41     "MetricSpec -[HAS_PROPERTY]-> ContainerResourceMetricSource"
,
42     "MetricSpec -[HAS_PROPERTY]-> ResourceMetricSource",
43     "PodTemplateSpec -[HAS_PROPERTY]-> PodSpec"
44 ],
45 "key_nodes": [
46     "io.k8s.api.autoscaling.v2.HorizontalPodAutoscaler",
47     "io.k8s.api.autoscaling.v2.HorizontalPodAutoscalerSpec",
48     "io.k8s.api.autoscaling.v2.CrossVersionObjectReference",
49     "io.k8s.api.apps.v1.Deployment"
50 ]
51 }
52 }

```

Listing B.5 fixture-hpa-deployment-pod.json — Relasional: HPA dan Penskalaan Otomatis

B.6 Fixture B.6 — cronjob_backup (Generasi YAML)

Fixture generasi YAML ini mengukur kemampuan sistem menghasilkan manifes CronJob yang valid dengan jadwal *cron* yang tepat dan konfigurasi *job template* yang benar.

```

1 {
2   "id": "cronjob_backup",
3   "type": "yaml_gen",
4   "question": "Buatkan YAML CronJob untuk menjalankan backup
database setiap hari jam 2 pagi",
5   "resource": "io.k8s.api.batch.v1.CronJob",
6   "scope": "Namespaced",
7   "multi_hop": true,
8   "ground_truth": {
9     "relevant_nodes": [
10      "io.k8s.api.batch.v1.CronJob",
11      "io.k8s.api.batch.v1.CronJobSpec",
12      "io.k8s.api.batch.v1.CronJobStatus",
13      "io.k8s.api.batch.v1.JobSpec",
14      "io.k8s.api.batch.v1.JobTemplateSpec",
15      "io.k8s.api.batch.v1.PodFailurePolicy",

```

```

16     "io.k8s.api.batch.v1.SuccessPolicy",
17     "io.k8s.api.core.v1.ObjectReference",
18     "io.k8s.api.core.v1.PodTemplateSpec",
19     "io.k8s.apimachinery.pkg.apis.meta.v1.LabelSelector"
20 ],
21 "expected_path": [
22     "CronJob -[CONTAINS_JOB_TEMPLATE]-> JobTemplateSpec",
23     "CronJob -[HAS_PROPERTY]-> CronJobSpec",
24     "CronJob -[HAS_PROPERTY]-> CronJobStatus",
25     "CronJobSpec -[HAS_PROPERTY]-> JobTemplateSpec",
26     "CronJobStatus -[HAS_PROPERTY]-> ObjectReference",
27     "JobSpec -[HAS_PROPERTY]-> LabelSelector",
28     "JobSpec -[HAS_PROPERTY]-> PodFailurePolicy",
29     "JobSpec -[HAS_PROPERTY]-> PodTemplateSpec",
30     "JobSpec -[HAS_PROPERTY]-> SuccessPolicy",
31     "JobTemplateSpec -[HAS_PROPERTY]-> JobSpec"
32 ],
33 "required_fields": ["apiVersion", "kind", "metadata", "spec"],
34 "expected_yaml_keys": [
35     "spec.schedule",
36     "spec.jobTemplate",
37     "spec.jobTemplate.spec.template"
38 ]
39 }
40 }

```

Listing B.6 fixture-cronjob-backup.json — Generasi YAML: CronJob Backup Database

B.7 Fixture B.7 — networkpolicy_deny_all (Generasi YAML)

Fixture ini menguji generasi NetworkPolicy dengan *empty podSelector* untuk memblokir semua *ingress traffic* ke namespace—pola keamanan yang umum tetapi memerlukan pemahaman semantik podSelector: {}.

```

1 {
2   "id": "networkpolicy_deny_all",
3   "type": "yaml_gen",
4   "question": "Buatkan YAML NetworkPolicy untuk memblokir semua
5     traffic masuk ke namespace production",
6   "resource": "io.k8s.api.networking.v1.NetworkPolicy",
7   "scope": "Namespaced",
8   "multi_hop": false,
9   "ground_truth": {
10     "relevant_nodes": [
11       "io.k8s.api.networking.v1.NetworkPolicy",

```

```

11     "io.k8s.api.networking.v1.NetworkPolicyEgressRule",
12     "io.k8s.api.networking.v1.NetworkPolicyIngressRule",
13     "io.k8s.api.networking.v1.NetworkPolicyPeer",
14     "io.k8s.api.networking.v1.NetworkPolicyPort",
15     "io.k8s.api.networking.v1.NetworkPolicySpec",
16     "io.k8s.apimachinery.pkg.apis.meta.v1.LabelSelector",
17     "io.k8s.apimachinery.pkg.apis.meta.v1.
LabelSelectorRequirement"
18 ],
19 "expected_path": [
20     "LabelSelector -[HAS_PROPERTY]-> LabelSelectorRequirement",
21     "NetworkPolicy -[HAS_PROPERTY]-> NetworkPolicySpec",
22     "NetworkPolicyEgressRule -[HAS_PROPERTY]-> NetworkPolicyPeer
",
23     "NetworkPolicyEgressRule -[HAS_PROPERTY]-> NetworkPolicyPort
",
24     "NetworkPolicyIngressRule -[HAS_PROPERTY]->
NetworkPolicyPeer",
25     "NetworkPolicyIngressRule -[HAS_PROPERTY]->
NetworkPolicyPort",
26     "NetworkPolicySpec -[HAS_PROPERTY]-> LabelSelector",
27     "NetworkPolicySpec -[HAS_PROPERTY]-> NetworkPolicyEgressRule
",
28     "NetworkPolicySpec -[HAS_PROPERTY]->
NetworkPolicyIngressRule"
29 ],
30 "required_fields": ["apiVersion", "kind", "metadata", "spec"],
31 "expected_yaml_keys": ["spec.podSelector", "spec.policyTypes"]
32 }
33 }

```

Listing B.7 fixture-networkpolicy-deny-all.json — Generasi YAML:
NetworkPolicy Deny-All

B.8 Fixture B.8 — statefulset_with_pvc (Generasi YAML)

Fixture ini menguji generasi StatefulSet dengan volumeClaimTemplates—field yang spesifik untuk StatefulSet dan tidak ada di Deployment—untuk menguji kemampuan sistem membedakan kedua *workload* tersebut.

```

1 {
2   "id": "statefulset_with_pvc",
3   "type": "yaml_gen",
4   "question": "Buat YAML StatefulSet untuk database MySQL dengan
PersistentVolumeClaim 10Gi",
5   "resource": "io.k8s.api.apps.v1.StatefulSet",

```



```

6  "scope": "Namespaced",
7  "multi_hop": true,
8  "ground_truth": {
9    "relevant_nodes": [
10     "io.k8s.api.apps.v1.StatefulSet",
11     "io.k8s.api.apps.v1.StatefulSetSpec",
12     "io.k8s.api.apps.v1.StatefulSetStatus",
13     "io.k8s.api.apps.v1.StatefulSetUpdateStrategy",
14     "io.k8s.api.core.v1.Container",
15     "io.k8s.api.core.v1.PersistentVolumeClaim",
16     "io.k8s.api.core.v1.PersistentVolumeClaimSpec",
17     "io.k8s.api.core.v1.PodSpec",
18     "io.k8s.api.core.v1.PodTemplateSpec",
19     "io.k8s.api.storage.v1.StorageClass",
20     "io.k8s.apimachinery.pkg.apis.meta.v1.LabelSelector"
21   ],
22   "expected_path": [
23     "LabelSelector -[HAS_PROPERTY]-> LabelSelectorRequirement",
24     "PersistentVolumeClaim -[HAS_PROPERTY]->
25 PersistentVolumeClaimSpec",
26     "PersistentVolumeClaim -[HAS_PROPERTY]->
27 PersistentVolumeClaimStatus",
28     "PersistentVolumeClaim -[USES_STORAGE_CLASS]-> StorageClass"
29   ],
30   "PersistentVolumeClaimSpec -[HAS_PROPERTY]->
31 VolumeResourceRequirements",
32   "PodSpec -[HAS_CONTAINER]-> Container",
33   "PodSpec -[HAS_PROPERTY]-> Container",
34   "PodSpec -[MOUNTS_VOLUME]-> Volume",
35   "PodTemplateSpec -[HAS_PROPERTY]-> PodSpec",
36   "StatefulSet -[CLAIMS_VOLUME]-> PersistentVolumeClaim",
37   "StatefulSet -[CONTAINS_POD_TEMPLATE]-> PodTemplateSpec",
38   "StatefulSet -[HAS_PROPERTY]-> StatefulSetSpec",
39   "StatefulSet -[HAS_PROPERTY]-> StatefulSetStatus",
40   "StatefulSetSpec -[HAS_PROPERTY]-> LabelSelector",
41   "StatefulSetSpec -[HAS_PROPERTY]-> PersistentVolumeClaim",
42   "StatefulSetSpec -[HAS_PROPERTY]-> PodTemplateSpec",
43   "StatefulSetSpec -[HAS_PROPERTY]-> StatefulSetUpdateStrategy"
44   ],
45   "StatefulSetUpdateStrategy -[HAS_PROPERTY]->
46 RollingUpdateStatefulSetStrategy"
47 ],
48 "required_fields": ["apiVersion", "kind", "metadata", "spec"],
49 "expected_yaml_keys": [
50   "apiVersion",

```

```

45     "kind",
46     "metadata",
47     "spec",
48     "spec.volumeClaimTemplates"
49 ],
50 "key_nodes": [
51     "io.k8s.api.apps.v1.StatefulSet",
52     "io.k8s.api.apps.v1.StatefulSetSpec",
53     "io.k8s.api.core.v1.PersistentVolumeClaim"
54 ]
55 }
56 }

```

Listing B.8 fixture-statefulset-with-pvc.json — Generasi YAML:
StatefulSet dengan PVC

B.9 Fixture B.9 — pi_want_to_reject_all_docker (Skenario Nyata)

Fixture bertipe *realworld* ini berasal dari pertanyaan Stack Overflow. Pertanyaan tentang kebijakan pembatasan *docker registry* menguji kemampuan sistem mengarahkan pengguna ke konsep *Admission Controller* yang tidak memiliki representasi langsung sebagai node di KG.

```

1 {
2   "id": "pi_want_to_reject_all_docker",
3   "type": "realworld",
4   "question": "I want to reject all docker registries except my
5               own one. I'm looking for some kind of policies for docker
6               registries and their images. For example my registry name is
7               registry.my.company.com. How can I enforce this at the cluster
8               level?",
9   "resource": "io.k8s.api.core.v1.Namespace",
10  "scope": "Cluster",
11  "multi_hop": true,
12  "ground_truth": {
13    "relevant_nodes": [
14      "io.k8s.api.core.v1.Namespace",
15      "io.k8s.api.core.v1.NamespaceCondition",
16      "io.k8s.api.core.v1.NamespaceSpec",
17      "io.k8s.api.core.v1.NamespaceStatus"
18    ],
19    "expected_path": [
20      "Namespace -[HAS_PROPERTY]-> NamespaceSpec",
21      "Namespace -[HAS_PROPERTY]-> NamespaceStatus",
22      "NamespaceStatus -[HAS_PROPERTY]-> NamespaceCondition"
23    ]
24  }
25 }

```

```

20     "key_nodes": [
21         "io.k8s.api.core.v1.Namespace"
22     ]
23 }
24 }

```

Listing B.9 fixture-pi-reject-docker-registry.json — Skenario Nyata:
Kebijakan Registry Docker

B.10 Fixture B.10 — kubectl_find_pods_with_env (Perintah)

Fixture bertipe *command* ini menguji kemampuan sistem memberikan perintah `kubectl` yang tepat untuk mencari *environment variable* di seluruh Pod—pertanyaan operasional yang membutuhkan pemahaman struktur `Container.env` dan `envFrom`.

```

1 {
2     "id": "kubectl_find_pods_with_env",
3     "type": "command",
4     "question": "Bagaimana cara mencari tahu di Pod mana saja
5         environment variable DATABASE_URL digunakan di seluruh
6         namespace cluster Kubernetes saya?",
7     "resource": "io.k8s.api.core.v1.Pod",
8     "scope": "Namespaced",
9     "multi_hop": false,
10    "ground_truth": {
11        "relevant_nodes": [
12            "io.k8s.api.core.v1.ConfigMap",
13            "io.k8s.api.core.v1.Container",
14            "io.k8s.api.core.v1.EnvFromSource",
15            "io.k8s.api.core.v1.EnvVar",
16            "io.k8s.api.core.v1.Pod",
17            "io.k8s.api.core.v1.PodSpec",
18            "io.k8s.api.core.v1.PodStatus",
19            "io.k8s.api.core.v1.Secret",
20            "io.k8s.api.core.v1.ServiceAccount",
21            "io.k8s.api.core.v1.Volume",
22            "io.k8s.apimachinery.pkg.apis.meta.v1.LabelSelector"
23        ],
24        "expected_path": [
25            "Container -[HAS_PROPERTY]-> ContainerPort",
26            "Container -[HAS_PROPERTY]-> EnvFromSource",
27            "Container -[HAS_PROPERTY]-> EnvVar",
28            "Container -[HAS_PROPERTY]-> VolumeMount",
29            "Container -[LOADS_CONFIGMAP]-> ConfigMap",
30            "Pod -[EXTENDS]-> PodSpec",
31            "Pod -[HAS_PROPERTY]-> PodSpec",

```

```

30     "Pod -[HAS_PROPERTY]-> PodStatus",
31     "PodSpec -[HAS_CONTAINER]-> Container",
32     "PodSpec -[HAS_PROPERTY]-> Container",
33     "PodSpec -[HAS_PROPERTY]-> Volume",
34     "PodSpec -[MOUNTS_VOLUME]-> Volume",
35     "PodSpec -[USES_SECRET]-> Secret",
36     "PodSpec -[USES_SERVICE_ACCOUNT]-> ServiceAccount",
37     "PodStatus -[HAS_PROPERTY]-> ContainerStatus"
38 ]
39 }
40 }

```

Listing B.10 fixture-kubect1-find-pods-with-env.json — Perintah:
Pencarian Environment Variable

LAMPIRAN C

HASIL EVALUASI KUANTITATIF PER *FIXTURE*

Lampiran ini menyajikan nilai metrik lengkap untuk seluruh 97 *fixture* evaluasi pada sistem *GraphRAG* Kubernetes. Tabel C.1 menampilkan metrik *Answer Quality* (AnsQ); Tabel C.2 menampilkan metrik *Retrieval Quality* (RetQ); Tabel C.3 menampilkan metrik *Reasoning Quality* (ReaQ); dan Tabel C.4 menampilkan metrik domain Kubernetes (Path Coverage, Hop Accuracy, RGA, lapisan kegagalan YAML, dan *intent* yang terdeteksi). Kolom **Sint.** dan **Skema** pada Tabel C.1 hanya relevan untuk *fixture* bertipe `yaml`; nilai “—” menunjukkan bahwa metrik tersebut tidak diukur untuk tipe *fixture* yang bersangkutan.

Singkatan tipe *fixture*: *cmd* = perintah, *cncpt* = konseptual, *flwp* = lanjutan, *plan* = perencanaan, *rlwld* = skenario nyata, *rel* = relasional, *trbl* = *troubleshooting*, *yaml* = generasi YAML.

C.1 Metrik *Answer Quality* (AnsQ)

Tabel C.1 Nilai Metrik AnsQ per *Fixture* (97 *Fixture*)

ID Fixture	Tipe	Sint.	Skema	Relevansi	Faithf.	AnsQ
kubect1 _export _namespace _res..	cmd	—	—	0,7932	0,5714	0,6823
kubect1 _find _pods _with _env	cmd	—	—	0,7659	0,0233	0,3946
kubect1 _force _delete _pod	cmd	—	—	0,8005	0,0116	0,4061
configmap _usage	cncpt	—	—	0,7743	0,3333	0,5538
daemonset _purpose	cncpt	—	—	0,7847	0,3333	0,5590
deployment _basic	cncpt	—	—	0,7962	0,5000	0,6481
hpa _target	cncpt	—	—	0,5467	0,6667	0,6067
ingress _purpose	cncpt	—	—	0,6670	0,2500	0,4585
job _cronjob	cncpt	—	—	0,7998	0,6000	0,6999

ID Fixture	Tipe	Sint.	Skema	Relevansi	Faithf.	AnsQ
namespace _quota	cncpt	—	—	0,6591	0,0000	0,3295
persistent _volume _concept	cncpt	—	—	0,6568	0,6000	0,6284
pod _spec	cncpt	—	—	0,6882	1,0000	0,8441
required _fields _container	cncpt	—	—	0,6442	1,0000	0,8221
scope _accuracy _node	cncpt	—	—	0,7307	0,6667	0,6987
secret _types	cncpt	—	—	0,8498	0,6667	0,7583
service _types	cncpt	—	—	0,8210	0,5000	0,6605
statefulset _storage	cncpt	—	—	0,7816	0,5000	0,6408
storageclass _concept	cncpt	—	—	0,8943	1,0000	0,9471
add _env _from _configmap	flwp	—	—	0,5846	0,1667	0,3756
add _env _from _secret	flwp	—	—	0,5683	0,3333	0,4508
add _hpa _to _deployment	flwp	—	—	0,6388	0,1250	0,3819
add _liveness _probe	flwp	—	—	0,6432	0,2727	0,4580
add _pvc _to _statefulset	flwp	—	—	0,6577	0,6667	0,6622
add _readiness _probe	flwp	—	—	0,6136	1,0000	0,8068
add _resource _limits	flwp	—	—	0,5920	0,6667	0,6293
add _resource _limits _deployment	flwp	—	—	0,5680	0,2000	0,3840
change _service _type	flwp	—	—	0,6108	0,5000	0,5554
expose _with _ingress	flwp	—	—	0,6774	0,3333	0,5054
scale _existing _deployment	flwp	—	—	0,6749	0,5000	0,5875

ID Fixture	Tipe	Sint.	Skema	Relevansi	Faithf.	AnsQ
update _image _version	flwp	—	—	0,6314	0,2222	0,4268
plan _autoscale _ha	plan	—	—	0,7491	0,0909	0,4200
plan _cronjob _batch	plan	—	—	0,6806	0,2308	0,4557
plan _namespace _isolation	plan	—	—	0,8075	0,1818	0,4947
plan _redis _persistent	plan	—	—	0,7980	0,1136	0,4558
plan _webapp _with _db	plan	—	—	0,7562	0,2059	0,4810
configmap _volume _mount	rlwld	0.00	0.00	0,5501	0,7500	0,3250
deployment _vs _statefulset _co..	rlwld	—	—	0,8769	0,5000	0,6885
hpa _custom _metrics _yaml	rlwld	1.00	0.00	0,6457	0,2500	0,4739
pbased _on _a _hrefhttpsstackov..	rlwld	—	—	0,4141	0,0000	0,2071
pcopied _from _here _a _hrefhttp..	rlwld	—	—	0,3424	0,0000	0,1712
pheres _a _simplified _version _..	rlwld	—	—	0,6675	1,0000	0,8338
pi _am _quite _new _to _tektonp	rlwld	—	—	0,3901	0,0000	0,1950
pi _am _wondering _if _it _is	rlwld	—	—	0,6959	1,0000	0,8480
pi _cant _find _documentation _o..	rlwld	—	—	0,4701	0,0000	0,2350
pi _have _a _command _to _run	rlwld	—	—	0,4113	0,0000	0,2057

ID Fixture	Tipe	Sint.	Skema	Relevansi	Faithf.	AnsQ
pi _have _a _google _kubernetes _..	rlwld	—	—	0,5042	0,0000	0,2521
pi _have _a _tekton _codepipelin..	rlwld	—	—	0,4149	0,0000	0,2075
pi _want _to _pass _a _certificate	rlwld	—	—	0,2790	0,0000	0,1395
pi _want _to _reject _all _docker	rlwld	—	—	0,4162	0,0000	0,2081
pid _like _to _confirm _informat..	rlwld	—	—	0,6837	1,0000	0,8418
pim _trying _to _use _a _container	rlwld	—	—	0,4171	1,0000	0,7086
pim _using _argocd _and _i _want	rlwld	1.00	0.00	0,3478	0,0000	0,3369
pkubectl _provides _a _nice _way..	rlwld	—	—	0,4637	1,0000	0,7318
precodespec _container s _image..	rlwld	—	—	0,2611	0,0000	0,1306
pthis _eks _cluster _has _a _priv..	rlwld	—	—	0,4660	0,0000	0,2330
pusing _kubernetes _v1157 _in _m..	rlwld	—	—	0,4032	0,0000	0,2016
pwhen _using _istio _with _kuber..	rlwld	—	—	0,3748	0,0000	0,1874
pwith _kubectl _i _know _i _can	rlwld	—	—	0,4646	1,0000	0,7323
serviceaccount _pod _binding	rlwld	0.00	0.00	0,4963	0,2500	0,1866
anyof _intorstring	rel	—	—	0,6946	0,3333	0,5140
cronjob _job _pod	rel	—	—	0,8030	1,0000	0,9015

ID Fixture	Tipe	Sint.	Skema	Relevansi	Faithf.	AnsQ
deep _deployment _container _re..	rel	—	—	0,8302	0,6000	0,7151
deep _statefulset _container _p..	rel	—	—	0,8327	0,8000	0,8164
deployment _pod _relation	rel	—	—	0,7282	0,3333	0,5308
extends _hpa _metric	rel	—	—	0,7226	0,6667	0,6946
hpa _deployment _pod	rel	—	—	0,8285	0,2500	0,5392
ingress _service _pod	rel	—	—	0,8139	0,5000	0,6570
namespace _quota _relation	rel	—	—	0,8737	0,6667	0,7702
networkpolicy _pod _selector	rel	—	—	0,8204	0,3333	0,5769
oneof _volume _source	rel	—	—	0,6429	0,2000	0,4214
pod _namespace _relation	rel	—	—	0,7960	1,0000	0,8980
pvc _storageclass	rel	—	—	0,7636	0,6667	0,7151
rbac _binding	rel	—	—	0,8464	1,0000	0,9232
secret _usage	rel	—	—	0,6972	0,6000	0,6486
service _selector	rel	—	—	0,6998	0,5000	0,5999
serviceaccount _token _binding	rel	—	—	0,8713	0,3333	0,6023
statefulset _pvc _pv	rel	—	—	0,8216	0,6667	0,7441
crashloopbackoff _oomkilled	trbl	—	—	0,8392	0,0115	0,4254
deployment _rollout _stuck	trbl	—	—	0,6680	0,0345	0,3513
imagepullbackoff _registry _se..	trbl	—	—	0,7667	0,0233	0,3950
pod _pending _no _resources	trbl	—	—	0,7459	0,0233	0,3846

ID Fixture	Tipe	Sint.	Skema	Relevansi	Faithf.	AnsQ
service _no _endpoints	trbl	—	—	0,8237	0,0769	0,4503
clusterrole _read _pods	yaml	1.00	1.00	0,7542	0,5000	0,8136
clusterrolebinding _yaml	yaml	1.00	1.00	0,8875	1,0000	0,9719
configmap _env	yaml	1.00	1.00	0,6302	1,0000	0,9076
cronjob _backup	yaml	1.00	1.00	0,6676	0,2500	0,7294
daemonset _fluentd	yaml	1.00	1.00	0,8328	0,3333	0,7915
deployment _3 _replicas	yaml	1.00	1.00	0,7607	0,3333	0,7735
deployment _liveness _probe	yaml	1.00	1.00	0,6128	1,0000	0,9032
ingress _rules	yaml	1.00	1.00	0,8141	0,3333	0,7869
networkpolicy _deny _all	yaml	1.00	1.00	0,7993	0,5000	0,8248
pod _configmap _volume	yaml	1.00	1.00	0,7745	0,6000	0,8436
pvc _dynamic	yaml	1.00	1.00	0,8274	0,5000	0,8319
rolebinding _dev	yaml	1.00	1.00	0,8796	1,0000	0,9699
service _nodeport	yaml	1.00	1.00	0,8096	0,3333	0,7857
statefulset _with _pvc	yaml	1.00	1.00	0,6373	0,6667	0,8260
yaml _layer3 _required _fields	yaml	1.00	1.00	0,6644	0,5000	0,7911

C.2 Metrik *Retrieval Quality* (RetQ)

Tabel C.2 Nilai Metrik RetQ per *Fixture* (97 *Fixture*)

ID Fixture	Tipe	P@k	NDCG@k	PathCov	F1@k	RetQ
kubectl _export _namespace _res..	cmd	1,0000	1,0000	1,0000	0,7273	0,9091
kubectl _find _pods _with _env	cmd	1,0000	1,0000	1,0000	0,4912	0,8304
kubectl _force _delete _pod	cmd	1,0000	1,0000	1,0000	0,6769	0,8923

ID Fixture	Type	P@k	NDCG@k	PathCov	F1@k	RetQ
configmap_usage	cncpt	0,0000	0,0000	0,0000	0,0000	0,0000
daemonset_purpose	cncpt	1,0000	1,0000	1,0000	1,0000	1,0000
deployment_basic	cncpt	1,0000	1,0000	1,0000	0,9412	0,9804
hpa_target	cncpt	1,0000	1,0000	1,0000	1,0000	1,0000
ingress_purpose	cncpt	0,2241	0,9508	0,9286	0,3611	0,7468
job_cronjob	cncpt	1,0000	1,0000	1,0000	0,9091	0,9697
namespace_quota	cncpt	0,3846	0,6181	1,0000	0,5556	0,7246
persistent_volume _concept	cncpt	0,1628	0,4252	1,0000	0,2800	0,5684
pod_spec	cncpt	0,7143	0,6836	1,0000	0,3960	0,6932
required_fields _container	cncpt	0,1429	0,1773	0,6333	0,1379	0,3162
scope_accuracy _node	cncpt	1,0000	1,0000	1,0000	0,9677	0,9892
secret_types	cncpt	0,0000	0,0000	1,0000	0,0000	0,3333
service_types	cncpt	1,0000	1,0000	0,8333	0,8696	0,9010
statefulset _storage	cncpt	1,0000	1,0000	1,0000	1,0000	1,0000
storageclass _concept	cncpt	1,0000	1,0000	1,0000	0,8571	0,9524
add_env_from _configmap	flwp	1,0000	1,0000	0,7500	0,8000	0,8500
add_env_from _secret	flwp	1,0000	1,0000	1,0000	0,8889	0,9630
add_hpa_to _deployment	flwp	0,5000	0,6479	0,1818	0,2857	0,3718
add_liveness _probe	flwp	1,0000	1,0000	0,8182	0,8421	0,8868
add_pvc_to _statefulset	flwp	1,0000	1,0000	1,0000	1,0000	1,0000
add_readiness _probe	flwp	1,0000	1,0000	1,0000	0,8889	0,9630
add_resource _limits	flwp	1,0000	1,0000	1,0000	0,8889	0,9630

ID Fixture	Type	P@k	NDCG@k	PathCov	F1@k	RetQ
add_resource _limits _deployment	flwp	1,0000	1,0000	0,9000	0,8889	0,9296
change_service _type	flwp	1,0000	1,0000	1,0000	1,0000	1,0000
expose_with _ingress	flwp	0,2245	1,0000	1,0000	0,3667	0,7889
scale_existing _deployment	flwp	1,0000	1,0000	1,0000	1,0000	1,0000
update_image _version	flwp	1,0000	1,0000	1,0000	0,9412	0,9804
plan_autoscale _ha	plan	0,4921	0,9595	1,0000	0,6458	0,8685
plan_cronjob _batch	plan	0,2157	0,8833	1,0000	0,3438	0,7423
plan_namespace _isolation	plan	0,5000	0,5028	0,9091	0,6207	0,6775
plan_redis _persistent	plan	0,9762	0,9839	1,0000	0,9535	0,9791
plan_webapp_with _db	plan	0,5345	0,9385	0,9429	0,6739	0,8518
configmap_volume _mount	rlwld	0,0357	0,0903	1,0000	0,0625	0,3843
deployment_vs _statefulset_co..	rlwld	1,0000	1,0000	0,9412	0,4103	0,7838
hpa_custom _metrics_yaml	rlwld	1,0000	1,0000	1,0000	1,0000	1,0000
pbased_on_a _hrefhttpsstackov..	rlwld	1,0000	1,0000	1,0000	0,4912	0,8304
pcopied_from _here_a _hrefhttp..	rlwld	0,0000	0,0000	0,0000	0,0000	0,0000
pheres_a _simplified _version_..	rlwld	1,0000	1,0000	1,0000	1,0000	1,0000
pi_am_quite_new _to_tektonp	rlwld	0,0000	0,0000	0,0000	0,0000	0,0000

ID Fixture	Tipe	P@k	NDCG@k	PathCov	F1@k	RetQ
pi _am _wondering _if _it _is	rlwld	0,0000	0,0000	1,0000	0,0000	0,3333
pi _cant _find _documentation _o..	rlwld	0,0000	0,0000	0,0000	0,0000	0,0000
pi _have _a _command _to _run	rlwld	1,0000	1,0000	1,0000	0,6769	0,8923
pi _have _a _google _kubernetes _..	rlwld	0,0000	0,0000	0,0000	0,0000	0,0000
pi _have _a _tekton _codepipelin..	rlwld	0,1429	0,0979	0,0714	0,0952	0,0882
pi _want _to _pass _a _certificate	rlwld	0,0000	0,0000	1,0000	0,0000	0,3333
pi _want _to _reject _all _docker	rlwld	0,0000	0,0000	0,0000	0,0000	0,0000
pid _like _to _confirm _informat..	rlwld	0,2000	0,2372	1,0000	0,2857	0,5076
pim _trying _to _use _a _container	rlwld	1,0000	1,0000	1,0000	1,0000	1,0000
pim _using _argocd _and _i _want	rlwld	0,0000	0,0000	0,0000	0,0000	0,0000
pkubectl _provides _a _nice _way..	rlwld	0,0000	0,0000	1,0000	0,0000	0,3333
precodespec _containers _image..	rlwld	1,0000	1,0000	1,0000	0,6769	0,8923
pthis _eks _cluster _has _a _priv..	rlwld	0,0000	0,0000	0,0000	0,0000	0,0000
pusing _kubernetes _v1157 _in _m..	rlwld	0,0000	0,0000	1,0000	0,0000	0,3333
pwhen _using _istio _with _kuber..	rlwld	0,0000	0,0000	1,0000	0,0000	0,3333

ID Fixture	Type	P@k	NDCG@k	PathCov	F1@k	RetQ
pwith _kubectl _i _know _i _can	rlwld	0,0000	0,0000	0,0000	0,0000	0,0000
serviceaccount _pod _binding	rlwld	0,1071	0,3489	0,6000	0,1765	0,3751
anyof _intorstring	rel	0,0690	0,6364	1,0000	0,1250	0,5871
cronjob _job _pod	rel	1,0000	1,0000	1,0000	0,7500	0,9167
deep _deployment _container _re..	rel	1,0000	1,0000	0,9697	0,4324	0,8007
deep _statefulset _container _p..	rel	1,0000	1,0000	0,9787	0,5000	0,8262
deployment _pod _relation	rel	1,0000	1,0000	1,0000	1,0000	1,0000
extends _hpa _metric	rel	1,0000	1,0000	1,0000	0,6230	0,8743
hpa _deployment _pod	rel	1,0000	1,0000	1,0000	0,6230	0,8743
ingress _service _pod	rel	1,0000	1,0000	0,9565	0,9767	0,9778
namespace _quota _relation	rel	1,0000	1,0000	0,6000	0,8000	0,8000
networkpolicy _pod _selector	rel	1,0000	1,0000	1,0000	0,7692	0,9231
oneof _volume _source	rel	0,0682	0,0686	0,6102	0,0667	0,2485
pod _namespace _relation	rel	1,0000	1,0000	1,0000	0,6718	0,8906
pvc _storageclass	rel	1,0000	1,0000	1,0000	1,0000	1,0000
rbac _binding	rel	1,0000	1,0000	1,0000	1,0000	1,0000
secret _usage	rel	0,1071	0,4212	0,5000	0,1818	0,3677
service _selector	rel	1,0000	1,0000	1,0000	0,4167	0,8056
serviceaccount _token _binding	rel	1,0000	1,0000	0,5000	0,7500	0,7500
statefulset _pvc _pv	rel	1,0000	1,0000	1,0000	0,5091	0,8364
crashloopbackoff _oomkilled	trbl	1,0000	1,0000	0,9911	0,4870	0,8260

ID Fixture	Type	P@k	NDCG@k	PathCov	F1@k	RetQ
deployment _rollout _stuck	trbl	1,0000	1,0000	1,0000	0,4324	0,8108
imagepullbackoff _registry _se..	trbl	1,0000	1,0000	1,0000	0,4912	0,8304
pod _pending _no _resources	trbl	1,0000	1,0000	1,0000	0,4912	0,8304
service _no _endpoints	trbl	1,0000	1,0000	0,9767	0,4082	0,7950
clusterrole _read _pods	yaml	0,1020	1,0000	1,0000	0,1852	0,7284
clusterrolebinding _yaml	yaml	0,9091	1,0000	1,0000	0,9524	0,9841
configmap _env	yaml	0,0000	0,0000	1,0000	0,0000	0,3333
cronjob _backup	yaml	1,0000	1,0000	1,0000	1,0000	1,0000
daemonset _fluentd	yaml	1,0000	1,0000	1,0000	1,0000	1,0000
deployment _3 _replicas	yaml	1,0000	1,0000	1,0000	1,0000	1,0000
deployment _liveness _probe	yaml	1,0000	1,0000	1,0000	0,9831	0,9944
ingress _rules	yaml	0,4286	1,0000	1,0000	0,6000	0,8667
networkpolicy _deny _all	yaml	1,0000	1,0000	1,0000	1,0000	1,0000
pod _configmap _volume	yaml	1,0000	1,0000	1,0000	0,6769	0,8923
pvc _dynamic	yaml	1,0000	1,0000	1,0000	1,0000	1,0000
rolebinding _dev	yaml	1,0000	1,0000	1,0000	1,0000	1,0000
service _nodeport	yaml	1,0000	1,0000	1,0000	1,0000	1,0000
statefulset _with _pvc	yaml	0,9762	1,0000	1,0000	0,9880	0,9960
yaml _layer3 _required _fields	yaml	1,0000	1,0000	1,0000	1,0000	1,0000

C.3 Metrik Reasoning Quality (ReaQ)

Tabel C.3 Nilai Metrik ReaQ per *Fixture* (97 *Fixture*)

ID Fixture	Tipe	HopAcc	d_gt	Δd	Grounding	ReaQ
kubect1 _export _namespace _res..	cmd	1,0000	3	0	0,8333	0,9167
kubect1 _find _pods _with _env	cmd	0,0000	111	-79	0,4000	0,2000
kubect1 _force _delete _pod	cmd	0,0000	111	-61	0,5000	0,2500
configmap _usage	cncpt	0,0000	2	7	1,0000	0,5000
daemonset _purpose	cncpt	1,0000	8	0	1,0000	1,0000
deployment _basic	cncpt	0,0000	10	-1	1,0000	0,5000
hpa _target	cncpt	1,0000	21	0	1,0000	1,0000
ingress _purpose	cncpt	0,0000	14	50	0,8000	0,4000
job _cronjob	cncpt	1,0000	10	0	1,0000	1,0000
namespace _quota	cncpt	0,0000	5	7	0,6667	0,3333
persistent _volume _concept	cncpt	0,0000	12	88	0,5556	0,2778
pod _spec	cncpt	0,0000	93	-61	1,0000	0,5000
required _fields _container	cncpt	0,0000	30	2	1,0000	0,5000
scope _accuracy _node	cncpt	1,0000	14	0	1,0000	1,0000
secret _types	cncpt	1,0000	0	0	0,6667	0,8333
service _types	cncpt	0,0000	12	-2	0,7500	0,3750
statefulset _storage	cncpt	1,0000	15	0	0,7500	0,8750
storageclass _concept	cncpt	1,0000	2	0	0,7500	0,8750
add _env _from _configmap	flwp	0,0000	12	-3	0,7500	0,3750
add _env _from _secret	flwp	1,0000	9	0	0,6000	0,8000
add _hpa _to _deployment	flwp	0,0000	22	-13	0,6667	0,3333
add _liveness _probe	flwp	0,0000	11	-2	0,6667	0,3333

ID Fixture	Type	HopAcc	d_gt	Δd	Grounding	ReaQ
add_pvc_to_statefulset	flwp	1,0000	15	0	0,5714	0,7857
add_readiness_probe	flwp	1,0000	9	0	0,6667	0,8333
add_resource_limits	flwp	1,0000	9	0	0,5000	0,7500
add_resource_limits_deployment	flwp	0,0000	10	-1	0,6667	0,3333
change_service_type	flwp	1,0000	10	0	0,4000	0,7000
expose_with_ingress	flwp	0,0000	10	43	0,7500	0,3750
scale_existing_deployment	flwp	1,0000	9	0	0,6667	0,8333
update_image_version	flwp	1,0000	9	0	0,6667	0,8333
plan_autoscale_ha	plan	0,0000	34	39	0,5000	0,2500
plan_cronjob_batch	plan	0,0000	11	49	0,6250	0,3125
plan_namespace_isolation	plan	0,0000	11	7	0,8333	0,4167
plan_redis_persistent	plan	0,0000	47	1	0,5714	0,2857
plan_webapp_with_db	plan	0,0000	35	31	0,6364	0,3182
configmap_volume_mount	rlwld	0,0000	3	29	0,6667	0,3333
deployment_vs_statefulset_co..	rlwld	0,0000	34	-25	1,0000	0,5000
hpa_custom_metrics_yaml	rlwld	1,0000	50	0	0,5000	0,7500
pbased_on_a_hrefhttpsstackov..	rlwld	0,0000	111	-79	1,0000	0,5000
pcopied_from_here_a_hrefhttp..	rlwld	0,0000	3	-2	1,0000	0,5000

ID Fixture	Tip	HopAcc	d_gt	Δd	Grounding	ReaQ
pheres_a _simplified _version ...	rlwld	1,0000	34	0	0,0000	0,5000
pi_am_quite_new _to_tektonp	rlwld	0,0000	111	-111	1,0000	0,5000
pi_am_wondering _if_it_is	rlwld	1,0000	0	0	1,0000	1,0000
pi_cant_find _documentation _o..	rlwld	0,0000	3	8	1,0000	0,5000
pi_have_a _command_to_run	rlwld	0,0000	111	-61	1,0000	0,5000
pi_have _a_google _kubernetes ...	rlwld	0,0000	19	-19	1,0000	0,5000
pi_have _a_tekton _codepipelin..	rlwld	0,0000	14	-7	1,0000	0,5000
pi_want_to_pass _a_certificate	rlwld	0,0000	0	3	0,7500	0,3750
pi_want_to _reject_all _docker	rlwld	0,0000	3	-3	1,0000	0,5000
pid_like _to_confirm _informat..	rlwld	0,0000	1	3	0,5000	0,2500
pim_trying_to _use_a_container	rlwld	1,0000	34	0	0,5000	0,7500
pim_using_argocd _and_i_want	rlwld	0,0000	3	47	1,0000	0,5000
pkubectl_provides _a_nice_way..	rlwld	1,0000	0	0	0,5000	0,7500
precodespec _containers _image..	rlwld	0,0000	111	-61	0,5000	0,2500
pthis_eks _cluster_has _a_priv..	rlwld	0,0000	1	-1	1,0000	0,5000

ID Fixture	Type	HopAcc	d_gt	Δd	Grounding	ReaQ
pusing _kubernetes _v1157 _in _m..	rlwld	1,0000	0	0	1,0000	1,0000
pwhen _using _istio _with _kuber..	rlwld	0,0000	0	1	1,0000	0,5000
pwith _kubectl _i _know _i _can	rlwld	0,0000	3	1	0,5000	0,2500
serviceaccount _pod _binding	rlwld	0,0000	5	27	1,0000	0,5000
anyof _intorstring	rel	0,0000	1	29	1,0000	0,5000
cronjob _job _pod	rel	0,0000	10	-4	1,0000	0,5000
deep _deployment _container _re..	rel	0,0000	33	-24	1,0000	0,5000
deep _statefulset _container _p..	rel	0,0000	47	-32	1,0000	0,5000
deployment _pod _relation	rel	1,0000	32	0	0,7500	0,8750
extends _hpa _metric	rel	0,0000	50	-29	1,0000	0,5000
hpa _deployment _pod	rel	0,0000	50	-29	1,0000	0,5000
ingress _service _pod	rel	0,0000	23	-2	0,7500	0,3750
namespace _quota _relation	rel	0,0000	5	-2	1,0000	0,5000
networkpolicy _pod _selector	rel	0,0000	9	-5	1,0000	0,5000
oneof _volume _source	rel	0,0000	59	-9	1,0000	0,5000
pod _namespace _relation	rel	0,0000	111	-61	1,0000	0,5000
pvc _storageclass	rel	1,0000	14	0	1,0000	1,0000
rbac _binding	rel	1,0000	7	0	1,0000	1,0000
secret _usage	rel	0,0000	4	28	1,0000	0,5000
service _selector	rel	0,0000	42	-32	1,0000	0,5000
serviceaccount _token _binding	rel	0,0000	4	-2	1,0000	0,5000

ID Fixture	Tipe	HopAcc	d_gt	Δd	Grounding	ReaQ
statefulset _pvc _pv	rel	0,0000	46	-31	1,0000	0,5000
crashloopbackoff _oomkilled	trbl	0,0000	112	-80	0,5000	0,2500
deployment _rollout _stuck	trbl	0,0000	32	-23	0,6667	0,3333
imagepullbackoff _registry _se..	trbl	0,0000	111	-79	0,6000	0,3000
pod _pending _no _resources	trbl	0,0000	111	-79	0,5000	0,2500
service _no _endpoints	trbl	0,0000	43	-33	0,5000	0,2500
clusterrole _read _pods	yaml	0,0000	4	50	0,2500	0,1250
clusterrolebinding _yaml	yaml	0,0000	9	1	0,6667	0,3333
configmap _env	yaml	1,0000	0	0	0,6667	0,8333
cronjob _backup	yaml	1,0000	10	0	0,5000	0,7500
daemonset _fluentd	yaml	1,0000	31	0	0,5000	0,7500
deployment _3 _replicas	yaml	1,0000	32	0	0,6667	0,8333
deployment _liveness _probe	yaml	1,0000	32	0	0,5000	0,7500
ingress _rules	yaml	0,0000	21	32	0,7500	0,3750
networkpolicy _deny _all	yaml	1,0000	9	0	0,7500	0,8750
pod _configmap _volume	yaml	0,0000	111	-61	0,6667	0,3333
pvc _dynamic	yaml	1,0000	14	0	0,4000	0,7000
rolebinding _dev	yaml	1,0000	7	0	0,6667	0,8333
service _nodeport	yaml	1,0000	42	0	0,5000	0,7500
statefulset _with _pvc	yaml	0,0000	46	2	0,5000	0,2500
yaml _layer3 _required _fields	yaml	1,0000	32	0	0,6667	0,8333

C.4 Metrik Domain Kubernetes

Tabel C.4 Metrik Domain Kubernetes per *Fixture* (Path Coverage, Hop Accuracy, RGA, YAML, *Intent*)

ID Fixture	Tipe	PathCov	HopAcc	RGA	YAML Fail	Intent
kubectl _export _namespace _res..	cmd	1,0000	1,0000	1.00	---	explain
kubectl _find _pods _with _env	cmd	1,0000	0,0000	0.00	---	explain
kubectl _force _delete _pod	cmd	1,0000	0,0000	0.00	---	generate _yaml
configmap _usage	cncpt	0,0000	0,0000	0.00	---	explain
daemonset _purpose	cncpt	1,0000	1,0000	0.00	---	explain
deployment _basic	cncpt	1,0000	0,0000	1.00	---	explain
hpa _target	cncpt	1,0000	1,0000	1.00	---	explain
ingress _purpose	cncpt	0,9286	0,0000	0.00	---	planning
job _cronjob	cncpt	1,0000	1,0000	1.00	---	explain
namespace _quota	cncpt	1,0000	0,0000	0.00	---	planning
persistent _volume _concept	cncpt	1,0000	0,0000	1.00	---	planning
pod _spec	cncpt	1,0000	0,0000	1.00	---	explain
required _fields _container	cncpt	0,6333	0,0000	1.00	---	explain
scope _accuracy _node	cncpt	1,0000	1,0000	1.00	---	explain
secret _types	cncpt	1,0000	1,0000	1.00	---	explain
service _types	cncpt	0,8333	0,0000	1.00	---	explain
statefulset _storage	cncpt	1,0000	1,0000	1.00	---	explain
storageclass _concept	cncpt	1,0000	1,0000	1.00	---	explain
add _env _from _configmap	flwp	0,7500	0,0000	0.00	---	followup
add _env _from _secret	flwp	1,0000	1,0000	0.00	---	followup
add _hpa _to _deployment	flwp	0,1818	0,0000	0.00	---	followup

ID Fixture	Type	PathCov	HopAcc	RGA	YAML Fail	Intent
add _liveness _probe	flwp	0,8182	0,0000	0.00	---	followup
add _pvc _to _statefulset	flwp	1,0000	1,0000	1.00	---	followup
add _readiness _probe	flwp	1,0000	1,0000	1.00	---	followup
add _resource _limits	flwp	1,0000	1,0000	1.00	---	followup
add _resource _limits _deployment	flwp	0,9000	0,0000	0.00	---	followup
change _service _type	flwp	1,0000	1,0000	1.00	---	followup
expose _with _ingress	flwp	1,0000	0,0000	0.00	---	generate _yaml
scale _existing _deployment	flwp	1,0000	1,0000	1.00	---	followup
update _image _version	flwp	1,0000	1,0000	0.00	---	followup
plan _autoscale _ha	plan	1,0000	0,0000	0.00	---	planning
plan _cronjob _batch	plan	1,0000	0,0000	0.00	---	planning
plan _namespace _isolation	plan	0,9091	0,0000	0.00	---	planning
plan _redis _persistent	plan	1,0000	0,0000	0.00	---	planning
plan _webapp _with _db	plan	0,9429	0,0000	0.00	---	planning
configmap _volume _mount	rlwld	1,0000	0,0000	1.00	syntactic	explain
deployment _vs _statefulset _co..	rlwld	0,9412	0,0000	1.00	---	explain
hpa _custom _metrics _yaml	rlwld	1,0000	1,0000	0.00	schema	planning
pbased _on _a _hrefhttpsstackov..	rlwld	1,0000	0,0000	0.00	---	explain

ID Fixture	Tip	PathCov	HopAcc	RGA	YAML Fail	Intent
pcopied _from _here _a _hrefhttp..	rlwld	0,0000	0,0000	0.00	---	explain
pheres _a _simplified _version ...	rlwld	1,0000	1,0000	1.00	---	generate _yaml
pi _am _quite _new _to _tektonp	rlwld	0,0000	0,0000	0.00	---	planning
pi _am _wondering _if _it _is	rlwld	1,0000	1,0000	1.00	---	explain
pi _cant _find _documentation _o..	rlwld	0,0000	0,0000	0.00	---	planning
pi _have _a _command _to _run	rlwld	1,0000	0,0000	0.00	---	planning
pi _have _a _google _kubernetes ...	rlwld	0,0000	0,0000	0.00	---	planning
pi _have _a _tekton _codepipelin..	rlwld	0,0714	0,0000	0.00	---	explain
pi _want _to _pass _a _certificate	rlwld	1,0000	0,0000	0.00	---	generate _yaml
pi _want _to _reject _all _docker	rlwld	0,0000	0,0000	0.00	---	planning
pid _like _to _confirm _informat..	rlwld	1,0000	0,0000	1.00	---	explain
pim _trying _to _use _a _container	rlwld	1,0000	1,0000	0.00	---	planning
pim _using _argocd _and _i _want	rlwld	0,0000	0,0000	0.00	schema	planning
pkubectl _provides _a _nice _way..	rlwld	1,0000	1,0000	0.00	---	explain
precodespec _containers _image..	rlwld	1,0000	0,0000	0.00	---	generate _yaml

ID Fixture	Type	PathCov	HopAcc	RGA	YAML Fail	Intent
pthis _eks _cluster _has _a _priv..	rlwld	0,0000	0,0000	0.00	---	planning
pusing _kubernetes _v1157 _in _m..	rlwld	1,0000	1,0000	0.00	---	generate _yaml
pwhen _using _istio _with _kuber..	rlwld	1,0000	0,0000	0.00	---	planning
pwith _kubectl _i _know _i _can	rlwld	0,0000	0,0000	0.00	---	explain
serviceaccount _pod _binding	rlwld	0,6000	0,0000	0.00	syntactic	explain
anyof _intorstring	rel	1,0000	0,0000	0.00	---	explain
cronjob _job _pod	rel	1,0000	0,0000	1.00	---	explain
deep _deployment _container _re..	rel	0,9697	0,0000	1.00	---	explain
deep _statefulset _container _p..	rel	0,9787	0,0000	1.00	---	explain
deployment _pod _relation	rel	1,0000	1,0000	0.00	---	trace _relati..
extends _hpa _metric	rel	1,0000	0,0000	1.00	---	explain
hpa _deployment _pod	rel	1,0000	0,0000	0.00	---	explain
ingress _service _pod	rel	0,9565	0,0000	1.00	---	trace _relati..
namespace _quota _relation	rel	0,6000	0,0000	1.00	---	explain
networkpolicy _pod _selector	rel	1,0000	0,0000	0.00	---	explain
oneof _volume _source	rel	0,6102	0,0000	0.00	---	trace _relati..
pod _namespace _relation	rel	1,0000	0,0000	1.00	---	trace _relati..
pvc _storageclass	rel	1,0000	1,0000	1.00	---	trace _relati..
rbac _binding	rel	1,0000	1,0000	1.00	---	trace _relati..
secret _usage	rel	0,5000	0,0000	1.00	---	explain

ID Fixture	Type	PathCov	HopAcc	RGA	YAML Fail	Intent
service _selector	rel	1,0000	0,0000	1.00	---	explain
serviceaccount _token _binding	rel	0,5000	0,0000	0.00	---	explain
statefulset _pvc _pv	rel	1,0000	0,0000	1.00	---	explain
crashloopbackoff _oomkilled	trbl	0,9911	0,0000	0.00	---	explain
deployment _rollout _stuck	trbl	1,0000	0,0000	0.00	---	explain
imagepullbackoff _registry _se..	trbl	1,0000	0,0000	0.00	---	explain
pod _pending _no _resources	trbl	1,0000	0,0000	0.00	---	explain
service _no _endpoints	trbl	0,9767	0,0000	0.00	---	explain
clusterrole _read _pods	yaml	1,0000	0,0000	1.00	none	generate _yaml
clusterrolebinding _yaml	yaml	1,0000	0,0000	1.00	none	generate _yaml
configmap _env	yaml	1,0000	1,0000	1.00	none	generate _yaml
cronjob _backup	yaml	1,0000	1,0000	0.00	none	generate _yaml
daemonset _fluentd	yaml	1,0000	1,0000	0.00	none	generate _yaml
deployment _3 _replicas	yaml	1,0000	1,0000	0.00	none	generate _yaml
deployment _liveness _probe	yaml	1,0000	1,0000	1.00	none	generate _yaml
ingress _rules	yaml	1,0000	0,0000	0.00	none	generate _yaml
networkpolicy _deny _all	yaml	1,0000	1,0000	1.00	none	generate _yaml
pod _configmap _volume	yaml	1,0000	0,0000	1.00	none	generate _yaml
pvc _dynamic	yaml	1,0000	1,0000	1.00	none	generate _yaml
rolebinding _dev	yaml	1,0000	1,0000	1.00	none	generate _yaml
service _nodeport	yaml	1,0000	1,0000	0.00	none	generate _yaml
statefulset _with _pvc	yaml	1,0000	0,0000	1.00	none	generate _yaml

ID Fixture	Tipe	PathCov	HopAcc	RGA	YAML Fail	Intent
yaml _layer3 _required _fields	yaml	1,0000	1,0000	1.00	none	generate _yaml

LAMPIRAN D

PANDUAN DAN HASIL WAWANCARA PRAKTISI

Lampiran ini menyajikan panduan dan ringkasan hasil wawancara semi-terstruktur dengan tiga praktisi Kubernetes (N1–N3) yang digunakan untuk memvalidasi permasalahan pada Subbab III.1 sekaligus memvalidasi realisme dataset uji pada Bab VI. Identitas narasumber dianonimkan; profil ringkasnya disajikan pada Tabel VI.1.

D.1 Panduan Wawancara

Wawancara disusun dalam beberapa segmen dengan tujuan menggali konteks narasumber dan tipe permasalahan nyata yang dihadapi saat memakai asisten LLM untuk pekerjaan Kubernetes.

1. Segmen Konteks Narasumber.

- a. Bagaimana peran Anda saat ini dan bagaimana Kubernetes masuk ke pekerjaan harian Anda?
- b. Seberapa sering Anda mengajukan pertanyaan terkait Kubernetes ke asisten LLM?

2. Segmen Tipe Pertanyaan Nyata.

- a. Dalam satu minggu terakhir, pertanyaan apa saja yang Anda ajukan? (3–5 contoh konkret)
- b. Konteks tambahan apa yang biasanya Anda lampirkan (misalnya *log*, *YAML*, pesan kesalahan)?
- c. Adakah pertanyaan yang Anda ketahui sering gagal dijawab LLM sehingga tidak Anda ajukan? Apa yang membuat Anda skeptis?

3. Segmen Hambatan dan Harapan.

- a. Hal apa yang paling membuat frustrasi saat bertanya Kubernetes ke LLM?
- b. Jika ada *chatbot* Kubernetes ideal, pertanyaan seperti apa yang harus dapat dijawabnya dengan baik dan belum ada solusinya saat ini?

D.2 Ringkasan Hasil Wawancara

D.2.1 Konteks dan Frekuensi Penggunaan

Ketiga narasumber menggunakan Kubernetes secara aktif dalam pekerjaan harian dengan peran *DevOps Engineer*, *Cloud Engineer*, dan *Site Reliability Engineer*. Asisten LLM digunakan secara rutin sebagai alat bantu, dengan intensitas bervariasi dari penggunaan harian hingga sekitar 15–20 kueri per hari untuk keperluan diskusi pendekatan, pencarian penjelasan, serta perencanaan.

D.2.2 Tipe Pertanyaan dan Konteks yang Dilampirkan

Pertanyaan yang diajukan mencakup penjelasan perilaku objek Kubernetes, penye-telan *HorizontalPodAutoscaler*, definisi *resource*, penelusuran koneksi antar*Pod*, pendataan *service*, serta penelusuran pemakaian *environment variable*. Konteks yang umum dilampirkan meliputi berkas manifes YAML (dengan kredensial sensitif yang telah disaring), deskripsi peran dan kondisi, pesan kesalahan, serta variabel dan token konfigurasi.

D.2.3 Keterbatasan LLM yang Dirasakan

Narasumber secara konsisten menyoroti halusinasi sebagai masalah utama, yaitu jawaban yang gagal dijalankan, berbeda antarmodel, merujuk versi *service* yang tidak lagi kompatibel, serta membuat asumsi yang tidak sesuai keinginan pengguna. Pada skenario generatif, hasil penyusunan konfigurasi YAML dirasakan tidak konsisten antara penyusunan dalam satu *prompt* dan penyusunan bertahap. Berkas konfigurasi yang panjang juga menyulitkan karena penjelasan kerap tidak tuntas.

D.2.4 Harapan terhadap Sistem Ideal

Harapan narasumber mencakup asisten yang aman dan tidak berasumsi keliru, penelusuran *bottleneck* berbasis pemantauan, serta dukungan menyeluruh dari perencanaan, penyusunan manifes, hingga pengujian. Sebagian harapan ini menuntut akses terhadap status operasional klaster (*runtime*) sehingga berada di luar cakupan penelitian yang berfokus pada representasi skema statis OpenAPI.

LAMPIRAN E

LEMBAR VALIDASI PAKAR

Lampiran ini menyajikan lembar validasi yang telah ditandatangani oleh lima pakar. Peran masing-masing pakar dirangkum pada Tabel E.1 berikut.

Tabel E.1 Peran Lima Pakar dalam Proses Validasi

Kode	Nama	Validasi <i>Fixture</i>	Evaluasi Jawaban
E.1	Ahmad Afzalulhaq	—	✓
E.2	Raden Dizi Assyafadi	✓	✓
E.3	Zaky Hassani	—	✓
E.4	Muhamad Iqbal Ramadh-an	✓	✓
E.5	Muhammad Faiq Dhiya Ul Haq	✓	—

Jumlah validator *fixture*: $n = 3$ (E.2, E.4, E.5); jumlah evaluator jawaban: $n = 4$ (E.1–E.4).

E.1 Lembar Validasi E.1 — Ahmad Afzalulhaq

LEMBAR PERNYATAAN VALIDATOR

Implementasi *Graph Retrieval Augmented Generation* untuk Meningkatkan Presisi Retrieval dan Validitas Sintaksis pada Konfigurasi Kubernetes

Tugas Akhir

A. Identitas Penelitian

Nama Mahasiswa	Jihan Aurelia
NIM	18222001
Program Studi	Sistem Teknologi Informasi
Institusi	Institut Teknologi Bandung (ITB)
Dosen Pembimbing	Dr. Ir. Dimitri Mahayana, M.Eng.

B. Identitas Validator

Nama Lengkap	Ahmad Afzalulhaq
Jabatan / Posisi	Anggota Tim IT Solutions
Institusi / Perusahaan	PT. Sharing Vision Indonesia
Lama Pengalaman Kubernetes	2 tahun

C. Pernyataan

Saya yang bertanda tangan di bawah ini menyatakan bahwa saya telah berpartisipasi sebagai validator dalam penelitian Tugas Akhir tersebut di atas. Saya telah mengikuti sesi pengujian sistem chatbot berbasis GraphRAG untuk domain dokumentasi Kubernetes dan memberikan penilaian serta masukan yang tertuang dalam instrumen evaluasi yang telah disediakan oleh peneliti.

Dengan ini saya menyatakan bahwa:

1. Partisipasi saya dalam penelitian ini bersifat sukarela.
2. Hasil evaluasi yang saya berikan dapat digunakan sebagai data penelitian.
3. Kutipan dari hasil evaluasi dapat digunakan secara anonim dalam laporan penelitian.

Bandung, 05 Juni 2026

Validator,



(Ahmad Afzalulhaq)

E.2 Lembar Validasi E.2 — Raden Dizi Assyafadi

LEMBAR PERNYATAAN VALIDATOR

Implementasi *Graph Retrieval Augmented Generation* untuk Meningkatkan Presisi Retrieval dan Validitas Sintaksis pada Konfigurasi Kubernetes

Tugas Akhir

A. Identitas Penelitian

Nama Mahasiswa	Jihan Aurelia
NIM	18222001
Program Studi	Sistem Teknologi Informasi
Institusi	Institut Teknologi Bandung (ITB)
Dosen Pembimbing	Dr. Ir. Dimitri Mahayana, M.Eng.

B. Identitas Validator

Nama Lengkap	Raden Dizi Assyafadi
Jabatan / Posisi	DevOps Engineer
Institusi / Perusahaan	GDP Labs
Lama Pengalaman Kubernetes	1 tahun

C. Pernyataan

Saya yang bertanda tangan di bawah ini menyatakan bahwa saya telah berpartisipasi sebagai validator dalam penelitian Tugas Akhir tersebut di atas. Saya telah mengikuti sesi pengujian sistem chatbot berbasis GraphRAG untuk domain dokumentasi Kubernetes dan memberikan penilaian serta masukan yang tertuang dalam instrumen evaluasi yang telah disediakan oleh peneliti.

Dengan ini saya menyatakan bahwa:

1. Partisipasi saya dalam penelitian ini bersifat sukarela.
2. Hasil evaluasi yang saya berikan dapat digunakan sebagai data penelitian.
3. Kutipan dari hasil evaluasi dapat digunakan secara anonim dalam laporan penelitian.

Bandung, 05 Juni 2026

Validator,



(Raden Dizi Assyafadi)

E.3 Lembar Validasi E.3 — Zaky Hassani

LEMBAR PERNYATAAN VALIDATOR

Implementasi *Graph Retrieval Augmented Generation* untuk Meningkatkan Presisi Retrieval dan Validitas Sintaksis pada Konfigurasi Kubernetes

Tugas Akhir

A. Identitas Penelitian

Nama Mahasiswa	Jihan Aurelia
NIM	18222001
Program Studi	Sistem Teknologi Informasi
Institusi	Institut Teknologi Bandung (ITB)
Dosen Pembimbing	Dr. Ir. Dimitri Mahayana, M.Eng.

B. Identitas Validator

Nama Lengkap	Zaky Hassani
Jabatan / Posisi	Kepala Tim IT Solutions
Institusi / Perusahaan	PT. Sharing Vision Indonesia
Lama Pengalaman Kubernetes	> 5 tahun

C. Pernyataan

Saya yang bertanda tangan di bawah ini menyatakan bahwa saya telah berpartisipasi sebagai validator dalam penelitian Tugas Akhir tersebut di atas. Saya telah mengikuti sesi pengujian sistem chatbot berbasis GraphRAG untuk domain dokumentasi Kubernetes dan memberikan penilaian serta masukan yang tertuang dalam instrumen evaluasi yang telah disediakan oleh peneliti.

Dengan ini saya menyatakan bahwa:

1. Partisipasi saya dalam penelitian ini bersifat sukarela.
2. Hasil evaluasi yang saya berikan dapat digunakan sebagai data penelitian.
3. Kutipan dari hasil evaluasi dapat digunakan secara anonim dalam laporan penelitian.

Bandung, 05 Juni 2026

Validator,



(Zaky Hassani)

E.4 Lembar Validasi E.4 — Muhamad Iqbal Ramadhan

LEMBAR PERNYATAAN VALIDATOR

Implementasi *Graph Retrieval Augmented Generation* untuk Meningkatkan Presisi Retrieval dan Validitas Sintaksis pada Konfigurasi Kubernetes
Tugas Akhir

A. Identitas Penelitian

Nama Mahasiswa	Jihan Aurelia
NIM	18222001
Program Studi	Sistem Teknologi Informasi
Institusi	Institut Teknologi Bandung (ITB)
Dosen Pembimbing	Dr. Ir. Dimitri Mahayana, M.Eng.

B. Identitas Validator

Nama Lengkap	Muhamad Iqbal Ramadhan
Jabatan / Posisi	DevOps Engineer
Institusi / Perusahaan	Accenture
Lama Pengalaman Kubernetes	Lebih dari 5 tahun

C. Pernyataan

Saya yang bertanda tangan di bawah ini menyatakan bahwa saya telah berpartisipasi sebagai validator dalam penelitian Tugas Akhir tersebut di atas. Saya telah mengikuti sesi pengujian sistem chatbot berbasis GraphRAG untuk domain dokumentasi Kubernetes dan memberikan penilaian serta masukan yang tertuang dalam instrumen evaluasi yang telah disediakan oleh peneliti.

Dengan ini saya menyatakan bahwa:

1. Partisipasi saya dalam penelitian ini bersifat sukarela.
2. Hasil evaluasi yang saya berikan dapat digunakan sebagai data penelitian.
3. Kutipan dari hasil evaluasi dapat digunakan secara anonim dalam laporan penelitian.

Bandung, 05 Juni 2026
Validator,



(Muhamad Iqbal Ramadhan)

E.5 Lembar Validasi E.5 — Muhammad Faiq Dhiya Ul Haq

LEMBAR PERNYATAAN VALIDATOR

Implementasi *Graph Retrieval Augmented Generation* untuk Meningkatkan Presisi Retrieval dan Validitas Sintaksis pada Konfigurasi Kubernetes

Tugas Akhir

A. Identitas Penelitian

Nama Mahasiswa	Jihan Aurelia
NIM	18222001
Program Studi	Sistem Teknologi Informasi
Institusi	Institut Teknologi Bandung (ITB)
Dosen Pembimbing	Dr. Ir. Dimitri Mahayana, M.Eng.

B. Identitas Validator

Nama Lengkap	Muhammad Faiq Dhiya Ul Haq
Jabatan / Posisi	DevOps Engineer
Institusi / Perusahaan	Accenture
Lama Pengalaman Kubernetes	1 – 2 tahun

C. Pernyataan

Saya yang bertanda tangan di bawah ini menyatakan bahwa saya telah berpartisipasi sebagai validator dalam penelitian Tugas Akhir tersebut di atas. Saya telah mengikuti sesi pengujian sistem chatbot berbasis GraphRAG untuk domain dokumentasi Kubernetes dan memberikan penilaian serta masukan yang tertuang dalam instrumen evaluasi yang telah disediakan oleh peneliti.

Dengan ini saya menyatakan bahwa:

1. Partisipasi saya dalam penelitian ini bersifat sukarela.
2. Hasil evaluasi yang saya berikan dapat digunakan sebagai data penelitian.
3. Kutipan dari hasil evaluasi dapat digunakan secara anonim dalam laporan penelitian.

Bandung, 05 Juni 2026

Validator,



(Muhammad Faiq Dhiya Ul Haq)